Python

9 novembre 2026

Indice

I Linguaggio **5**

1 Introduzione **7**
1.1 Esecuzione . . . 7
1.2 Ottenere aiuto . . . 9
1.3 Strutturazione del codice . . . 9
1.3.1 Moduli . . . 9
1.3.1.1 Nome e namespace . . . 9
1.3.1.2 Importazione dei moduli . . . 9
1.3.1.3 Path di ricerca dei moduli . . . 10
1.3.2 Pacchetti . . . 10
1.3.2.1 Struttura e `__init__.py` . . . 10
1.3.2.2 `__init__.py`, `__all__` e `import *` da pacchetto 12
1.4 Virtual environments e `uv` . . . 12
1.4.1 Inizializzazione di un progetto . . . 12
1.4.2 Contenuto directory root . . . 12
1.4.3 Shell nel virtual environment . . . 13
1.4.4 Esecuzione di cli e script . . . 13
1.4.4.1 Da dentro progetto . . . 13
1.4.4.2 Dall'esterno . . . 13
1.4.5 Aggiunta/rimozione dipendenze . . . 14

2 Dati **15**
2.1 Introduzione . . . 15
2.2 Tipi nativi . . . 16
2.3 Classificazione dei tipi di base . . . 16
2.4 Numeri . . . 17
2.5 Date e ore . . . 18
2.6 Sequenze: stringhe, liste e tuple . . . 18
2.6.1 Operatore di slice (selezione da sequenza) e indici . . . 18
2.6.2 Stringhe . . . 19
2.6.2.1 Stringhe valide . . . 19
2.6.2.2 Formattazione . . . 19
2.6.2.3 Elaborazioni varie . . . 19
2.6.2.4 Classe con formato . . . 19
2.6.3 Liste . . . 20
2.6.3.1 Definizione . . . 20
2.6.3.2 Manipolazione e modifica . . . 20
2.6.3.3 Metodi utili . . . 20

2.6.3.4 Liste nested 20
2.6.3.5 List comprehensions 20
2.6.4 Tuple 21
2.6.5 Sequence unpacking 21
2.7 Classi mapping e set 21
2.7.1 Dict 21
2.7.1.1 Dict comprehension 21
2.7.2 Sets 21
2.7.2.1 Operatori/metodi utili 22
2.7.2.2 Set comprehension 22

3 Controllo del flusso **23**
3.1 Costrutti condizionali: `if` e `while` 23
3.1.1 `if` 23
3.1.2 `match` 23
3.1.3 `while` 23
3.1.4 `break`, `continue` ed `else` 23
3.2 Condizioni e test logici 24
3.2.1 Test verità 24
3.2.2 Operatori booleani 24
3.2.3 Comparazioni 24
3.2.4 Comparazioni concatenate 25
3.2.5 Check appartenenza: `in` e `not in` 25
3.2.6 Comparare sequenze e altri tipi 25
3.3 Looping su oggetti: `for` 26
3.3.1 Looping in sequenze (stringhe, liste, tuple) 26
3.3.2 Looping nei dict 26
3.3.3 Looping sui set 26
3.3.4 L'utilizzo di `range` 26

4 Funzioni **27**
4.1 Definizione 27
4.2 Chiamata 28
4.2.1 Valutazione dei valori di default 28
4.2.2 Valore ritornato 28
4.2.3 Unpacking di liste/tuple/dict in chiamata 28
4.3 Regole di scope 28
4.3.1 Namespace 29
4.3.2 Attributi 29
4.3.3 Scoping 29
4.4 Funzioni anonime (`lambda`) 30
4.5 Programmazione funzionale 30
4.5.1 Funzioni classiche 30
4.5.1.1 `map` (= `Map`/`lappy`) 30
4.5.1.2 `itertools.starmap` 31
4.5.1.3 `functools.reduce` 31
4.5.1.4 `itertools.accumulate` 31
4.5.1.5 `filter` 31
4.5.2 Partialling 31
4.5.2.1 `functools.partial` 31

4.5.2.2 `functools.partialmethod` 31
4.5.3 Function factory 32
4.5.4 Composizione di funzioni 32
4.5.5 Decorators 32
4.5.5.1 Definizione e utilizzo 32
4.5.5.2 Esempi semplici 32
4.5.5.3 Decorator con parametri 33
4.5.5.4 Decoratori già disponibili 33
4.5.6 Single e multiple dispatch 33
4.5.6.1 Single dispatch (function overloading) 33
4.5.6.2 Multiple dispatch 33
4.6 Altri argomenti 34
4.6.1 Type annotation/type hint 34
4.6.1.1 Sintassi 34
4.6.1.2 Checking 34
4.6.1.3 Tipi utilizzabili 35
4.6.1.4 Creazione di alias 35
4.6.1.5 Tipo `Callable` per funzioni 35
4.6.2 `atexit` 35

5 Input/Output **37**
5.1 Accesso al filesystem 37
5.1.1 Ottenere/cambiare directory di lavoro 37
5.1.2 Listing di directory, glob files 37
5.1.3 Creazione/rimozione di file/directory 37
5.1.4 File e directory temporanei 37
5.1.5 Paths e metodi utili 37
5.2 Salvataggio/importazione file 37
5.2.1 Testo semplice 37
5.2.1.1 Lettura 38
5.2.1.2 Scrittura 38
5.2.2 `csv`/`tsv` 38
5.2.2.1 Lettura 38
5.2.2.2 Scrittura 38
5.2.3 JSON 38
5.2.3.1 Scrittura 39
5.2.3.2 Lettura 39
5.2.3.3 Pretty printing 39
5.2.3.4 Formati custom 39
5.2.4 pickle 39
5.3 File di configurazione 39
5.4 Esecuzione di programmi esterni 40
5.4.1 `run` 40
5.4.2 `Popen` 40
5.5 Parsing opzioni da linea di comando 41
5.5.1 `sys.argc` e `sys.argv` 41
5.5.2 Modulo `argparse` 41
5.6 Accesso variabili d'ambiente 42

6 Object Oriented Programming **43**
6.1 Introduzione . . . 43
6.1.1 Fasi di sviluppo . . . 43
6.1.2 Classi, oggetti etc . . . 43
6.1.3 Quando utilizzare programmazione ad oggetti . . . 44
6.2 Classi . . . 44
6.2.1 Definizione, istanziazione e scoping . . . 44
6.2.1.1 Definizione . . . 44
6.2.1.2 Istanziazione e scoping . . . 45
6.2.2 Metodi . . . 45
6.2.2.1 Definizione e chiamata . . . 45
6.2.2.2 Inizializzatore custom . . . 45
6.2.3 Condivisione dati . . . 46
6.2.4 Data hiding . . . 46
6.3 Ereditarietà . . . 46
6.3.1 Ereditarietà singola . . . 46
6.3.2 Ereditarietà multipla . . . 47
6.4 Amenità/utilità per le classi . . . 47
6.4.1 Funzioni builtin utili . . . 47
6.4.2 Decoratori utili per metodi . . . 47
6.4.2.1 `property` . . . 47
6.4.2.2 `cached_property` . . . 48
6.4.2.3 `classmethod` . . . 48
6.4.2.4 staticmethod . . . 48
6.4.2.5 `abstractmethod` . . . 49
6.4.3 Type hint . . . 49
6.5 Polimorfismo . . . 49
6.6 Abstract Base Class . . . 50
6.6.1 Definizione . . . 50
6.6.2 Creazione di classi conformi ad ABC del linguaggio . . . 50
6.7 Dunder methods e applicazioni . . . 50
6.7.1 Iteratori . . . 51
6.7.1.1 Funzionamento . . . 51
6.7.1.2 Implementazione mediante classe . . . 52
6.7.1.3 Implementazione mediante espressioni generatrici 52
6.7.1.4 Implementazione mediante generatori . . . 52
6.7.1.5 Funzioni e operatori utili su iteratori . . . 53
6.7.1.6 Il modulo itertools . . . 53
6.7.2 Context Managers . . . 54
6.7.2.1 Implementazione mediante classe . . . 54
6.7.2.2 Implementazione mediante generatore . . . 55
6.8 Classi builtin notevoli . . . 55
6.8.1 defaultdict . . . 55
6.8.2 `Counter` . . . 55
6.8.3 Enum & Flag . . . 55
6.8.3.1 Enum . . . 55
6.8.3.2 Flag . . . 56
6.8.4 Named tuples (`collections.namedtuple` e `typing.NamedTuple`) 56
6.8.5 `dataclass` . . . 56
6.8.5.1 `field`: dati mutabili, valori default, parametri . 57

6.8.5.2 Eseguire codice post inizializzazione: `post_init` 57
6.8.5.3 Freezing di una dataclass . . . 57
6.8.5.4 Altri parametri utili del decoratore . . . 57
6.8.6 Queues . . . 58
6.8.6.1 Queues (FIFO queues) . . . 58
6.8.6.2 Stacks (LIFO queues) . . . 58
6.8.6.3 Heaps (priority queues) . . . 58
6.9 Altri argomenti . . . 59
6.9.1 Estensione di classi base . . . 59
6.9.2 Manager Object . . . 59

7 Eccezioni **61**
7.1 Sollevare eccezioni . . . 62
7.2 Gestire eccezioni . . . 62
7.2.1 `try` . . . 62
7.2.2 Tips . . . 63
7.2.3 Eccezioni particolari . . . 63
7.3 Eccezioni custom . . . 64
7.3.1 Definizione e utilizzo . . . 64
7.4 Warnings . . . 64
7.4.1 Sollevare warnings . . . 65
7.4.2 Ignorare warnings . . . 65

8 Devtools **67**
8.1 Debugging . . . 67
8.2 Testing . . . 69
8.2.1 Introduzione . . . 69
8.2.1.1 Gergo . . . 69
8.2.1.2 Tipologie di testing . . . 69
8.2.1.3 Good testing practice . . . 69
8.2.1.4 Fasi di un test . . . 70
8.2.2 `pytest` . . . 70
8.2.2.1 Setup . . . 70
8.2.2.2 Esecuzione dei test . . . 70
8.2.2.3 Contenuto file di test . . . 71
8.2.2.4 Fixture . . . 71
8.2.2.5 Marking di test . . . 72
8.2.2.6 Esecuzione condizionale di test . . . 72
8.2.2.7 Failure accettate/expected . . . 72
8.2.2.8 Parametrizzazione . . . 73
8.2.3 Testing con dati/input randomici . . . 73
8.2.4 Doctest . . . 73
8.2.4.1 Skippare test . . . 74
8.2.4.2 Normalizzare spazi bianchi . . . 74
8.2.5 `unittest` . . . 74
8.3 Linting/formattazione (`ruff`) . . . 74
8.3.1 Linting . . . 74
8.3.2 Formattazione . . . 75
8.4 Timing/temporizzazione . . . 75
8.5 Profiling . . . 75

8.6 Scrittura di plugin . . . 76
8.7 Documentazione . . . 76
8.7.1 Setup . . . 77
8.7.2 Doc-writing e reStructuredText . . . 77
8.7.3 Building . . . 77
8.7.4 Setup di readthedocs . . . 78

II Scientific Stack 79

9 Numpy 81
9.1 L'`ndarray` . . . 81
9.1.1 Creazione e copia . . . 81
9.1.2 Tipi (`dtype`): coercizione e testing . . . 81
9.1.3 Forma, dimensioni e reshape . . . 82
9.1.3.1 Forma e dimensioni . . . 82
9.1.3.2 Reshaping . . . 82
9.1.3.3 Ravelling/flattening . . . 83
9.2 Indexing . . . 83
9.2.1 Array unidimensionali . . . 83
9.2.2 Array multidimensionali . . . 83
9.2.3 Subarray come viste vs copie . . . 84
9.2.4 Assegnazione e unicità degli indici . . . 84
9.3 Elaborazioni di array . . . 84
9.3.1 Inserimento/rimozione elementi (`insert`, `delete`) . . . 84
9.3.2 Aritmetica vettorizzata . . . 84
9.3.3 Operazioni insiemistiche . . . 84
9.3.4 Concatenazione (`concatenate`, `vstack`, `hstack`) . . . 84
9.3.5 Splitting (`split`, `vsplit`, `hsplit`) . . . 85
9.3.6 Ripetizione/binding (`repeat`, `tile`) . . . 85
9.3.7 Sorting/ordine (`sort`, `argsort`) . . . 85
9.4 Universal functions . . . 85
9.4.1 Introduzione . . . 85
9.4.2 Metodi delle ufunction (`reduce`, `accumulate`, `outer`) . . . 86
9.4.3 Creazione di ufunctions . . . 87
9.5 Broadcasting . . . 87
9.6 Altri argomenti . . . 89
9.6.1 Lavorare con booleani . . . 89
9.6.2 Array di stringhe . . . 89

10 Sympy 91

11 Polars 93
11.1 Introduzione . . . 93
11.1.1 Tipi di dati . . . 93
11.1.2 Espressioni e contesti . . . 94
11.1.3 Expression expansion e selettori . . . 94
11.2 Data I/O . . . 95
11.2.1 Importazione dati . . . 95
11.2.1.1 Da file . . . 95

11.2.1.2 Da pandas 95
11.2.2 Esportazione dati 95
11.2.2.1 Su file 95
11.2.2.2 Verso pandas 95
11.3 `Series` 95
11.3.1 Creazione 95
11.3.2 Attributi 95
11.3.3 Indexing e selezione 96
11.3.4 Elaborazione vettorizzata 96
11.3.5 Coercizione di tipo (`cast`) 96
11.3.6 Valori condizionali a-la `ifelse` (`pl.when`) 96
11.3.7 Applicazione di una funzione a tutti gli elementi (`map`) . . 96
11.3.8 Sostituzione di valori (`replace`) 96
11.3.9 Test di appartenenza (`in`, `isin`) 96
11.3.10 Dati mancanti 96
11.3.11 Duplicati 96
11.3.12 Sorting 97
11.3.13 Discretizzazione/creazione di classi (`cut`, `qcut`) 97
11.3.14 Dummy variables 97
11.3.14.1 Utilizzo base (`get_dummies`) 97
11.3.14.2 Variabili a risposta multipla 97
11.3.15 Stringhe 97
11.3.15.1 Paste 97
11.3.15.2 Collapse 97
11.3.15.3 Formattazione varia 97
11.3.15.4 Lunghezza ed sottostringhe 97
11.3.15.5 Splitting 98
11.3.15.6 Ricerca/rimpiazzo 98
11.3.15.7 Estrazione 98
11.3.16 Date/ore 98
11.3.16.1 Parsing da stringa 98
11.3.16.2 Creazione da componenti 98
11.3.16.3 Estrazione componenti 98
11.3.16.4 Esportazione a stringa 98
11.3.16.5 Intervalli di tempo/differenze date 98
11.3.16.6 Creazione di range di date 98
11.3.17 Dati categorici (`Enum` e `Categorical`) 98
11.3.18 Liste, array, structs 99
11.3.18.1 Liste 99
11.3.18.2 Array 100
11.3.18.3 Structs 100
11.4 `DataFrame` 100
11.4.1 Creazione 100
11.4.2 Contenuto 100
11.4.3 Copia di df e di struttura (`clone`, `clear`) 101
11.4.4 Selezione 101
11.4.4.1 Colonne (`.select`) 101
11.4.4.2 Righe (`.filter`/`.remove`) 101
11.4.4.3 Righe e colonne 101
11.4.4.4 Singoli elementi (`.item`) 101

11.4.5 Inserimento colonna in un punto (`.insert_column`) . . . 101
11.4.6 Assegnazione (`with_columns`) . . . 101
11.4.6.1 Elaborazioni particolari . . . 102
11.4.7 Rimozione righe/colonne (`.filter`/`.drop`) . . . 102
11.4.8 Rinominare colonne . . . 102
11.4.9 Coercizione di tipi (`.cast`) . . . 102
11.4.10 Applicazione di funzioni . . . 102
11.4.10.1 All'intero dataframe . . . 102
11.4.10.2 A colonne . . . 103
11.4.10.3 A righe . . . 103
11.4.11 Ciclo su righe/colonne . . . 103
11.4.11.1 Righe . . . 103
11.4.11.2 Colonne . . . 103
11.4.12 Merge (`.join`) . . . 103
11.4.12.1 Due dataframe . . . 103
11.4.12.2 Molteplici dataframe . . . 104
11.4.12.3 Merge approssimato (`.join_asof`) . . . 104
11.4.13 Binding (`pl.concat`) . . . 104
11.4.14 Aggiunta di righe parziali ad un dataframe . . . 104
11.4.15 Reshape . . . 104
11.4.15.1 Porre in long (`.unpivot`) . . . 104
11.4.15.2 Porre in wide (`.pivot`) . . . 105
11.4.16 Trasporre (`.transpose`) . . . 105
11.4.17 Test di appartenenza (`in`) . . . 105
11.4.18 Dati mancanti . . . 105
11.4.18.1 Identificazione . . . 105
11.4.18.2 Rimozione . . . 105
11.4.18.3 Imputazione . . . 105
11.4.19 Duplicati . . . 105
11.4.19.1 Identificazione (`.duplicated`) . . . 105
11.4.19.2 Rimozione (`.unique`) . . . 106
11.4.20 Sorting . . . 106
11.4.21 Id univoco di riga . . . 106
11.4.22 Windows functions . . . 106
11.4.23 folds (elaborazioni orizzontali) . . . 106
11.5 Descrittive . . . 107
11.5.1 Univariate . . . 107
11.5.2 Bivariate . . . 107
11.5.2.1 Tabelle di contingenza . . . 107
11.5.2.2 Covarianza e correlazione . . . 107
11.5.2.3 Tabelle pivot . . . 108
11.5.2.4 Stratificate . . . 108

12 Matplotlib **111**
12.1 Introduzione . . . 111
12.2 Salvataggio figura . . . 111
12.3 Impostazione layout figura ed esempi . . . 113
12.3.1 Layout standard . . . 113
12.3.2 Layout custom . . . 113
12.4 Fine tuning . . . 114

12.4.1 Ticks e subticks . . . 114
12.4.2 Spines e grid . . . 114
12.4.3 Gestire la sovrapposizione di elementi diversi (`zorder`) . . 115
12.4.4 Legenda . . . 115
12.4.5 Plot con doppio asse delle y . . . 115
12.4.6 Padding dei subplots (spazio bianco bordi) . . . 115
12.5 Configurazioni . . . 116
12.5.1 Ottenimento e modifica . . . 116
12.5.2 Ripristino impostazioni default . . . 116
12.5.3 Cambiare stile . . . 116
12.6 Grafici utili . . . 116
12.6.1 Linee . . . 117
12.6.2 Diagramma a barre . . . 117
12.6.3 Istogramma . . . 117
12.6.4 Scatterplot . . . 117
12.6.5 Matrice di scatterplot . . . 117
12.6.6 Boxplot . . . 117
12.6.7 Correlation matrix . . . 117

III Cookbook 119

13 Espressioni regolari 121

14 Logging 123
14.1 Uso del logger di default . . . 123
14.1.1 Setup . . . 123
14.2 Logger custom . . . 124
14.2.1 Setup degli handler . . . 124
14.2.2 Impostare la formattazione di un handler . . . 124
14.2.3 impostare livello di log generale e di un handler . . . 125
14.2.4 Filtrare log . . . 125

15 Algebra lineare 127
15.1 Vettori, matrici e dimensioni . . . 127
15.1.1 Creazione . . . 127
15.1.2 Funzioni utilità per creazione . . . 127
15.2 Operazioni . . . 127
15.2.1 Somma . . . 127
15.2.2 Trasposizione . . . 127
15.2.3 Prodotti . . . 128
15.3 Misc . . . 128

16 Probabilità e simulazione 129
16.1 Funzionalità native . . . 129
16.1.1 Combinatoria . . . 129
16.1.2 Numeri casuali (`random`) . . . 129
16.2 Numeri casuali (`numpy`) . . . 129
16.2.1 Creazione del generatore . . . 130
16.3 Variabili casuali (`scipy.stats`) . . . 130

16.3.1 Funzioni e parametri principali . . . 130
16.3.2 Uso interattivo rapido . . . 132
16.3.3 Freezing di una distribuzione . . . 132
16.3.4 Uso del generatore di `numpy` . . . 132
16.4 Altri argomenti . . . 132
16.4.1 Bootstrap CI . . . 132
16.4.2 Esecuzione parallela . . . 132

17 Test statistici **133**
17.1 Setup . . . 133
17.2 Medie . . . 133
17.3 Proporzioni . . . 133
17.3.1 McNemar . . . 133
17.3.2 Q di Cochrane . . . 133
17.4 Tassi . . . 133
17.4.1 Comparazione 2 tassi . . . 133
17.5 Varianze . . . 133
17.5.1 Test di Bartlett . . . 134
17.5.2 Test di Levene . . . 134
17.5.3 Test di Fligner . . . 134
17.6 Sopravvivenza . . . 134
17.6.1 Logrank test . . . 134
17.7 Agreement . . . 134
17.7.1 Cohen's K . . . 134
17.7.2 Fleiss K . . . 134
17.7.3 Lin coefficient . . . 134
17.8 Reliability/consistency . . . 134
17.8.1 Cronbach α . . . 134
17.8.2 ICC . . . 134
17.9 Multiplicity . . . 135
17.10Test simulativi . . . 135

18 Misc cookbook **137**
18.1 Tabella trial . . . 137
18.2 Calendario . . . 137
18.3 Tcl/Tk . . . 137
18.4 Telegram . . . 137

Parte I

Linguaggio

Capitolo 1

Introduzione

Contents

1.1 Esecuzione 7
1.2 Ottenere aiuto 9
1.3 Strutturazione del codice 9
1.3.1 Moduli 9
1.3.2 Pacchetti 10
1.4 Virtual environments e `uv` 12
1.4.1 Inizializzazione di un progetto 12
1.4.2 Contenuto directory root 12
1.4.3 Shell nel virtual environment 13
1.4.4 Esecuzione di cli e script 13
1.4.5 Aggiunta/rimozione dipendenze 14

1.1 Esecuzione

Vi sono due modi per utilizzare l'interprete classico:

- in via batch, alternativamente:
  - creando un file con estensione `.py`, che contenga istruzioni python valide a seconda della versione, ed eseguendolo attraverso `python file.py`
  - creando un file senza estensione con le seguenti sha-bang, dargli i permessi di esecuzione e porlo nel path degli eseguibili

    ```
    #!/usr/bin/env python
    #!/usr/bin/env python3
    ```

- in modalità interattiva:
  - entrando nell'interprete mediante `python` o `python3`
  - editando un file `.py` in Emacs, questi va in `python-mode`. Far partire un processo python in un buffer con `C-c C-p` e passando all'interprete i comandi descritti in tabella 1.1.

Sequenza	Comando	Descrizione
C-c C-r	python-shell-send-region	invia la regione selezionata
C-c C-e	python-shell-send-statement	manda la regione selezionata o lo statement della linea
C-c C-s	python-shell-send-string	invia una riga da specificare
C-c C-c	python-shell-send-buffer	invia tutto il buffer corrente
C-c C-l	python-shell-send-file	tipo `source` di bash
C-c <	python-indent-shift-left	indenta a sinistra
C-c >	python-indent-shift-right	indenta a destra
C-c C-d	python-describe-at-point	descrivi la cosa
C-c C-f	python-eldoc-at-point	??
C-c C-j	imenu	muoversi a definizioni
C-c C-v	python-check	usa qualche tester da impostare con python-check-command
C-c C-t c	python-skeleton-class	introduci una classe da template
C-c C-t d	python-skeleton-def	introduci una funzione da template
C-c C-t f	python-skeleton-for	introduci un for da template
C-c C-t i	python-skeleton-if	introduci un if da template
C-c C-t m	python-skeleton-import	introduci un import da template
C-c C-t t	python-skeleton-try	introduci un try da template
C-c C-t w	python-skeleton-while	introduci un while da template

Tabella 1.1: Comandi `python-mode` di emacs

1.2 Ottenere aiuto

`help` fa uso di docstring e si usa alternativamente come

```
help()        # help di sistema (moduli, keyword, simboli, topics)
help(oggetto) # help specifico di un oggetto - docstring
```

Si può accedere alla documentazione anche dalla shell mediante `pydoc`

```
pydoc nome  # equivale a help(nome) dall'interprete
```

1.3 Strutturazione del codice

Le definizioni:

modulo file `.py` che può definire classi, funzioni e variabili globali importabili da un altro modulo mediante `import`;

pacchetto una directory contenente moduli con scopi affini e un file `__init__.py` (eseguito se si fa l'import del pacchetto per configurazioni/inizializzazioni).

1.3.1 Moduli

1.3.1.1 Nome e namespace

Ogni modulo ha:

- un *nome*, contenuto nella variabile globale `__name__` che solitamente coincide con:
  - `"nomefile"` senza l'estensione `.py` se il modulo è importato mediante `import`
  - `"__main__"`, qualora il modulo sia eseguito come script attraverso `python nomefile.py`
- un *namespace*: ogni modulo ha il suo che viene utilizzato da tutte le funzioni definite in esso. Le funzioni di un modulo per riferirsi ad altri oggetti definiti nello stesso, possono evitare di precedere il nome del modulo all'oggetto richiesto.

La presenza di un nome del modulo fa sì che si possa eseguire codice a seconda che il modulo sia importato o eseguito come script. Ad esempio nel file `miomodulo.py` dopo tutte le definizioni possiamo dare

```
if __name__ == "__main__":
    checks()
```

in questo caso i `checks()` verranno eseguite solamente se eseguiamo il modulo mediante `python miomodulo.py` (altrimenti mediante `import` il nome del modulo è impostato a `miomodulo` e i `checks` non vengono eseguiti).

1.3.1.2 Importazione dei moduli

I moduli possono importare altri moduli per ottenerne funzionalità in due modi differenti per la gestione del namespace. Supponiamo di aver creato un file `mymodule.py` nella directory corrente che contiene la funzione `do_complicated_stuff` e la variabile `strange_var`. Possiamo comandare:

- `import mymodule <as abbreviazione>`

  Viene creato un oggetto-modulo chiamato `mymodule` che contiene quanto definite; per utilizzale

  ```
  mymodule.do_complicated_stuff()
  mymodule.strange_var
  ```

  Specificando l'abbreviazione si crea un alias per il `mymodule` (verosimilmente più piccola e veloce da diteggiare)

- `from mymodule import do_complicated_stuff <as abbrev>`
  `from mymodule import strange_var <as abbrev>`

  Si importano solamente la funzione (o la costante) ed è possibile riferirvisi in seguito con un più veloce `do_complicated_stuff` senza specificare il nome del modulo di provenienza.

- `from mymodule import *`

  Si importa tutto quello che è definito nel modulo (non è considerato buona pratica)

1.3.1.3 Path di ricerca dei moduli

Quando viene richiesta l'importazione di un modulo di nome `spam.py` mediante `import spam` (o simili), Python cerca nell'ordine:

1. nei moduli builtin distribuiti col linguaggio;
2. nella lista di directory specificate in `sys.path`, nell'ordine specificato:

   ```
   import sys
   sys.path
   ```

   La variabile contiene solitamente la directory corrente al primo posto (come stringa vuota), facendole assumere priorità tra quelle del `sys.path`. Se si desidera aggiungere una directory come prima si può usare

   ```
   sys.path.insert(0,'/path/to/mod_directory')
   ```

1.3.2 Pacchetti

1.3.2.1 Struttura e `__init__.py`

Al minimo, un pacchetto è una directory contenente un file `__init__.py` e i file `.py` che costituiscono i moduli.
I files `__init__.py` sono necessari per far si che Python tratti la directory come un pacchetto;

- nel caso più semplice (considerata best practice) può essere un file vuoto;
- alternativamente si possono eseguire inizializzazioni o impostare la variabile `__all__` di cui si parla in seguito

Struttura semplice Ad esempio, con una siffatta situazione:

```
mypackage/
|-- __init__.py
|-- module_a.py
+-- module_b.py
```

L'utilizzo (in uno script al di fuori della directory `mypackage`) può avvenire come:

```
import mypackage.module_a
```

Struttura con subpackages In situazioni più complesse si possono prevedere subpackage, ossia subdirectory con moduli affini e un `__init__.py` per ogni directory/subdirectory. Ad esempio per un pacchetto che gestisce l'audio (di nome `sound`):

```
sound/                          Top-level package
      __init__.py               Initialize the sound package
      formats/                  Subpackage for file format conversions
              __init__.py
              wavread.py
              wavwrite.py
              aiffread.py
              aiffwrite.py
              auread.py
              auwrite.py
              ...
      effects/                  Subpackage for sound effects
              __init__.py
              echo.py
              surround.py
              reverse.py
              ...
      filters/                  Subpackage for filters
              __init__.py
              equalizer.py
              vocoder.py
              karaoke.py
              ...
```

References da esterno L'importazione di funzionalità può essere avvenire con `import` o `from`:

- con `import` possiamo importare *pacchetti o moduli*. In `import a.b.c` ogni item tranne l'ultimo deve essere un pacchetto (directory con `__init__.py`); `c` può essere un pacchetto o un modulo

```
# modulo versione 1: con import è necessaria poi la full reference
import sound.effects.echo
sound.effects.echo.echofilter(input, output, delay=0.7, atten=4)

# (consigliabile usare as sinonimo)
```

- con `from` possiamo importare *moduli e funzioni*, controlla che l'oggetto sia definito nel pacchetto, se sì lo importa, se no ipotizza sia un modulo e lo cerca

```
# modulo vrs 2: con from si può usare la relative reference
from sound.effects import echo
echo.echofilter(input, output, delay=0.7, atten=4) # from allows relative ref

# funzione
from sound.effects.echo import echofilter
echofilter(input, output, delay=0.7, atten=4)
```

Intra-package references Quando i pacchetti sono strutturati in sottopacchetti si possono usare sia referenza assoluta che relativa. Ad esempio

- se `sound.filters.vocoder` necessita del modulo `sound.effects.echo` nelle sue prime linee si può usare alternativamente

```
from sound.effects import echo
from ..effects import echo
```

- se `sound.effects.surround` necessita `sound.effects.echo` `sound.formats` o `sound.filters.equalizer`

```
from . import echo
from .. import formats
from ..filters import equalizer
```

Gli import relativi si basano sul nome del modulo corrente: dato che il nome del modulo main è sempre `__main__`, i file `.py` (moduli) che debbono essere eseguiti come modulo main (script) debbono utilizare sempre import assoluti.

1.3.2.2 `__init__.py`, `__all__` e `import *` da pacchetto

Se in `sounds/effects/__init__.py` si imposta

```
__all__ = ["echo", "surround", "reverse"]
```

e si comanda

```
from sound.effects import *
```

verranno importati `echo.py`, `surround.py` e `reverse.py`.

1.4 Virtual environments e uv

I virtual environments fanno si che interprete Python e singoli pacchetti possano essere installati in un maniera isolata per una data applicazione/progetto. Il tool moderno per fare questo è uv.

1.4.1 Inizializzazione di un progetto

Dopo l'installazione di `uv` per la creazione di un progetto/environment isolato:

```
uv init --library mylib # solo libreria python
uv init --package mylib # libreria + script da commandline
```

Ad esempio la creazione di un pacchetto crea la seguente struttura

```
$ uv init --package mypkg
Initialized project `mypkg` at `/tmp/mypkg`

$ tree mypkg/
mypkg
|-- pyproject.toml
|-- README.md
`-- src
    `-- mypkg
        `-- __init__.py
```

1.4.2 Contenuto directory root

Dopo un utilizzo (es `uv run`) viene inizializzata la seguente directory root di default

```
$ ls -la mypkg
totale 44
drwxr-xr-x  5 l    users  4096 25 giu 06.05 .
drwxrwxrwt 22 root root  12288 25 giu 06.18 ..
drwxr-xr-x  6 l    users  4096 25 giu 06.04 .git
-rw-r--r--  1 l    users   109 25 giu 06.04 .gitignore
-rw-r--r--  1 l    users   349 25 giu 06.04 pyproject.toml
-rw-r--r--  1 l    users     5 25 giu 06.04 .python-version
-rw-r--r--  1 l    users     0 25 giu 06.04 README.md
drwxr-xr-x  3 l    users  4096 25 giu 06.04 src
-rw-r--r--  1 l    users   126 25 giu 06.05 uv.lock
drwxr-xr-x  4 l    users  4096 25 giu 06.05 .venv
```

Il progetto viene gia posto sotto git e vengono creati alcuni file

- `pyproject.toml` contiene i metadati del progetto
- `.python-version` fissa la versione di python utilizzata
- la cartella `.venv` contiene il virtual environment (eseguibile python e pacchetti/dipendenze)
- `uv.lock` contiene le versioni specifiche sulle dipendenze per garantire riproducibilità; dovrebbe essere sotto git ma *non* editato a mano

1.4.3 Shell nel virtual environment

Per una shell Python che abbia il progetto a disposizione

```
$ uv run python
Python 3.13.13 (main, Apr  7 2026, 18:19:01) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import mypkg
>>> mypkg.main()
Hello from mypkg!
```

1.4.4 Esecuzione di cli e script

1.4.4.1 Da dentro progetto

cli L'unica differenza del `-package` dalla `-library` è la presenza nel file toml della sezione

```
[project.scripts]
mypkg = "mypkg:main"
```

che permette di eseguire la funzione main in `__init__.py` come script

```
$ uv run mypkg
Using CPython 3.13.13 interpreter at: /run/current-system/sw/bin/python3.13
Creating virtual environment at: .venv
      Built mypkg @ file:///tmp/mypkg
Installed 1 package in 1ms
Hello from mypkg!
```

script Se si ha nel progetto uno script (es in `scripts/example.py`), per eseguirlo facendo uso dell'environment

```
$ uv run scripts/example.py arg1 arg2 ..
```

1.4.4.2 Dall'esterno

Provato vari metodi ma il meglio (più indipendente da sistema operativo e si basa solo su uv) è creare uno script esterno in `PATH` che si posizioni nella directory del progetto che chiami

- `uv run` per entry point specificati in `pyproject.toml`
- `uv run -script` per altri file (eg residenti nella subdirectory `scripts`)

Qualcosa del tipo:

```
#!/usr/bin/env bash

cd  ~/src/pypkg/myuvpkg && uv run --quiet mypkg $*
# cd  ~/src/pypkg/myuvpkg && uv run --quiet scripts/mioscriptsimple.py
```

L'approccio di sopra può creare problemi se specifichiamo path relativi da linea di comando; per usi personali si può modificare lo script bash affinché passi la vecchia `PWD` al codice python, che la gestisca opportunamente. Un esempio è `winston_sends`:

```
#!/usr/bin/env bash

OLDPWD=$(pwd)
cd ~/src/pypkg/winston && uv run --quiet winston_sends $OLDPWD $*
```

1.4.5 Aggiunta/rimozione dipendenze

Di sotto le operazioni più comuni, per maggiori info qui.

Aggiunta dipendenze Per aggiungere:

- dipendenze per l'uso della libreria/package (necessarie per l'utilizzo)

  ```
  uv add pandas
  ```

- tool per il suo sviluppo del pacchetto (non per il successivo utilizzo)

  ```
  uv add --dev pytest ruff mypy
  ```

I comandi andranno a modificare il file `pyproject.toml` opportunamente per garantire riproducibilità anche altrove/in futuro.

Rimozione dipendenze Per rimuovere

```
uv remove pandas
```

Aggiornamento pacchetto Per aggiornare un pacchetto all'ultima versione compatibile lasciando il resto intatto

```
uv lock --upgrade-package pandas
```

Capitolo 2

Dati

Contents

2.1 Introduzione 15
2.2 Tipi nativi 16
2.3 Classificazione dei tipi di base 16
2.4 Numeri 17
2.5 Date e ore 18
2.6 Sequenze: stringhe, liste e tuple 18
2.6.1 Operatore di slice (selezione da sequenza) e indici . . 18
2.6.2 Stringhe 19
2.6.3 Liste 20
2.6.4 Tuple 21
2.6.5 Sequence unpacking 21
2.7 Classi mapping e set 21
2.7.1 Dict 21
2.7.2 Sets 21

2.1 Introduzione

Assegnazione Gli oggetti del linguaggio vengono creati mediante l'**assegnazione**, che avviene attraverso l'uso di `=`, e non vi è necessità di dichiarare precedentemente il tipo della variabile (dinamically typed):

```
message = 'Hi friend'
pi = 3.1415926535897932
```

Keyword linguaggio I nomi non utilizzabili come identificatori sono:

```
and       del       from      not       while
as        elif      global    or        with
assert    else      if        pass      yield
break     except    import    print
class     in        raise     nonlocal
continue  finally   is        return
def       for       lambda    try
```

Eliminazione oggetti La rimozione del binding ad aree di memoria (pre intervento del garbage collector) avviene mediante la keyword `del`

```
a = 1
del a
```

Operazioni su oggetti Di ogni oggetto è possibile:

- conoscere la **classe di appartenenza** mediante `type` o `isinstance`

  ```
  >>> type(1)
  <class 'int'>
  >>> isinstance(1, int)
  True
  ```

- **listare dati e metodi disponibili** (ai quali si accede mediante l'operatore punto), derivanti dalla classe di appartenenza mediante `dir`

  ```
  >>> dir(1)
  ['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delatt
  ```

- ottenere un **identificatore univoco** (della singola istanzazione), ottenuto mediante la funzione `id` applicata all'oggetto (questa non restituisce altro che l'indirizzo in memoria dell'oggetto)

  ```
  >>> a = 1
  >>> id(a)
  10758864
  ```

2.2 Tipi nativi

I tipi più di base che il python mette a disposizione sono:

- `bool`: variabili booleane come `True` o `False`
- `int`: gli interi
- `float`: numeri con virgola mobile
- `str`: stringhe di caratteri unicode (non modificabili) compresi tra virgolette
- `bytes`: per la manipolazione di binario

A partire da questi tipi di base si possono creare oggetti composti tra i quali:

- `set`: insiemi di elementi non ordinati
- `list` sono sequenze ordinate e modificabili di elementi
- `tuple` sono sequenze ordinate e non modificabili di elementi
- `dict`: sono array coppie chiave - valore

Infine altri tipi builtin (che servono soprattutto nell'ottica della programmazione) sono:

- `type`: la classe di un oggetto è essa stessa un oggetto di classe `type`
- `None`: serve per indicare un valore vuoto ed ha classe `NoneType`. Non presenta attributi.
- funzioni
- classi
- moduli

Il nome del singolo tipo (ad esempio `int`) serve generalmente, se utilizzato come funzione[1], per **coercire da un tipo all'altro**:

```
>>> int(1.1)
1
>>> float(1)
1.0
>>> str(123)
'123'
```

2.3 Classificazione dei tipi di base

I tipi di base possono essere classificati a seconda di:

- **storage** model: quanti oggetti base possono essere contenuti in un oggetto? in base a questo distinguiamo
  - *oggetti scalari*: contengono un singolo oggetto base
  - *oggetti container*: contengono molteplici oggetti singoli. Il fatto che contengano molteplici oggetti pone poi che questi debbano essere o meno della stessa tipologia. In python tutti i tipi container base possono esser formati da oggetti base di tipo diverso.
- **update** model: una volta creato l'oggetto può esser modificato? Distinguiamo oggetti *mutabili* e *immutabili*
- **access** model: come si accede ad un singolo elemento facente parte dell'oggetto? distinguiamo
  - accesso *direct*: caratteristico di alcuni oggetti atomici per i quali non si pone problemi di accesso particolare
  - accesso *sequence*: accesso mediante indice numerico
  - accesso *mapping*: accesso mediante key alfanumerica

Tabella 2.1 sintetizza la classificazione seguendo i criteri presentati.

[1] Questo perchè il nome di una classe usato come funzione, serve come costruttore.

Data type	Storage	Update	Access
Numbers	Scalar	Immutable	Direct
Strings	Scalar	Immutable	Sequence
Lists	Container	Mutable	Sequence
Tuples	Container	Immutable	Sequence
Dictionaries	Container	Mutable	Mapping

Tabella 2.1: Classificazione dei tipi

2.4 Numeri

Vi sono tre tipi numerici: interi, floating point e complessi; i booleani sono un sottotipo di intero. Tutti i tipi numerici ad eccezione dei complessi supportano le seguenti operazioni, le quali hanno maggior priorità che gli operatori di comparazione

```
x + y    somma
x - y    differenza
x * y    prodotto
x / y    quoziente
x // y   parte intera della divisione
x % y    resto della divisione
divmod(x, y)  la coppia (x // y, x % y)
x ** y       elevamento a potenza
pow(x, y)    elevamento a potenza
abs(x)   valore assoluto
int(x)   x convertito a intero
float(x) x convertito a virgola mobile
complex(re, im)    numero complesso con re parte reale e im immaginaria
c.conjugate()      conjugate of the complex number c
```

`int` e `float` supportano

```
math.trunc(x)     parte intera
round(x[, n])     x arrotondato a n digits (se omesso default a 0)
math.floor(x)     maggiore intero <= x
math.ceil(x)      minor intero >= x
```

Infine l'unico metodo che sembra veramente utile per i `float` è `is_integer` (gli int non hanno metodi di interesse soprattutto in ambito reale)

```
>>> f1 = 2.0
>>> f1.is_integer()
True

>>> f2 = 1.2
>>> f2.is_integer()
False
```

Per operazioni aggiungive vedere i moduli `math` e `cmath`

2.5 Date e ore

il modulo `datetime` mette a disposizione le classi di base per date, ore, dateore etc

```
>>> import datetime as dt

>>> # definire date/ore
>>> birth = dt.date(1983, 11, 4) # year, month, day
>>> birth_time = dt.time(15, 50, 37) # hour, minute, second
>>> birth_dt = dt.datetime(1983, 11, 4, 15, 50, 37) # tutto

>>> # parsing/importazione da stringhe
>>> dt.datetime.strptime("11/11/2011", "%d/%m/%Y") # custom
datetime.datetime(2011, 11, 11, 0, 0)
>>> dt.datetime.strptime("11/11/2011 12:12:12", "%d/%m/%Y %H:%M:%S") #  custom
datetime.datetime(2011, 11, 11, 12, 12, 12)
>>> dt.date.fromisoformat('2019-12-04') #  iso
datetime.date(2019, 12, 4)
>>> dt.datetime.fromisoformat('2011-11-04T00:05:23') #  iso
datetime.datetime(2011, 11, 4, 0, 5, 23)

>>> # oggi/ora
>>> oggi = dt.date.today()  # oggi: data
>>> ora = dt.datetime.now() # adesso: datetime

>>> # accesso a singoli elementi
>>> [birth.year, birth.month, birth.day]
[1983, 11, 4]
>>> [birth_time.hour, birth_time.minute, birth_time.second]
[15, 50, 37]
>>> [birth_dt.year, birth_dt.month, birth_dt.day,
...  birth_dt.hour, birth_dt.minute, birth_dt.second]
[1983, 11, 4, 15, 50, 37]

>>> # convertire datetime in date
>>> ora.date()
datetime.date(2026, 11, 9)

>>> # differenze di date
>>> diff = oggi - birth
>>> diff
datetime.timedelta(days=15711)
>>> diff.days / 365.25
43.01437371663244

>>> # differenza di tempi
>>> a = dt.datetime(2000, 11, 11, 2, 50, 30)
>>> b = dt.datetime(2001, 11, 13, 2, 50, 0)
>>> diff_t = b - a
>>> diff_t
```

Operatore	Funzione
`seq[:::]`	accedi ad elementi specifici di `seq`
`seq[ind1:ind2]`	da `ind1` incluso sino a `ind2` escluso
`seq[ind1:ind2:ind3]`	da `ind1` incluso a `ind2` escluso, facendo passi di `ind3`
`seq * expr`	ripeti la sequenza `expr` volte
`seq1 + seq2`	concatena le due sequenze
`obj in seq`	testa se l'oggetto `obj` è presente nella sequenza `seq`
`obj not in seq`	contrario del precedente test

Tabella 2.2: Operatori comuni per le sequenze

```
datetime.timedelta(days=366, seconds=86370)
>>> diff_t.total_seconds() # secondi di differenza totali
31708770.0

>>> # far scorrere il tempo
>>> un_giorno = dt.timedelta(days = 1)
>>> ieri = oggi - un_giorno
>>> ieri
datetime.date(2026, 11, 8)
>>> domani = oggi + un_giorno
>>> domani
datetime.date(2026, 11, 10)

>>> # esportazione/formattazione a stringa
>>> ora.strftime("%d-%m-%Y") #  custom
'09-11-2026'
>>> ora.strftime("%d-%m-%Y - %H:%M:%S") #  custom
'09-11-2026 - 07:54:24'
>>> oggi.isoformat() # iso
'2026-11-09'
>>> ora.isoformat() # iso
'2026-11-09T07:54:24.025641'
```

2.6 Sequenze: stringhe, liste e tuple

Introduciamo prima gli operatori e le funzioni builtin che funzionano con tutte le sequenze per affrontare le peculiarità di ognuna in sezione separata.
Gli operatori presentati in tabella 2.2 si applicano a tutte le sequenze. Altre funzioni di utilità per tutte le sequenze[2] sono quelle di tabella 2.3 Agli iterabili (di cui le sequenze fanno parte) si possono applicano le funzioni di tabella 2.4 per coercirli a sequenze.

2.6.1 Operatore di slice (selezione da sequenza) e indici

Le parentesi `[]` (operatore di slice) servono per effettuare estrazione da una sequenza. Le sequenze hanno indici che, alternativamente

[2] A parte len, reversed e sum queste si applicano agli iteratori in genere

Funzione	Attività
`enumerate(iter)`	ritorna un oggetto enumerato
`len(seq)`	ritorna la lunghezza della sequenza
`max(iter)`	ritorna il massimo del'iterabile
`min(iter)`	ritorna il minimo del'iterabile
`reversed(seq)`	ritorna un iteratore che attraversa la sequenza in ordine inverso
`sorted(iter)`	ritorna una lista ordinata dell'iterabile fornito
`sum(seq)`	somma della sequenza

Tabella 2.3: Operatori per la coercizione a sequenze

Funzione	Attività
`list(iter)`	converti l'iterabile ad una lista
`str(iter)`	converti l'iterabile ad una stringa
`tuple(iter)`	converti l'iterabile ad una tuple

Tabella 2.4: Operatori per la coercizione a sequenze

- vanno da 0 (indice del primo elemento) a `n-1` dove `n` è la lunghezza della sequenza, restituita da `len` (analogamente a quanto avviene nel C).
- vanno da `-n` a `-1` per riferirsi agli oggetti dal primo all'ultimo con indici negativi

```
len = 6
 +---+---+---+---+---+---+
 | P | y | t | h | o | n |
 +---+---+---+---+---+---+
   0   1   2   3   4   5
  -6  -5  -4  -3  -2  -1
```

La sintassi dello slicing prevede al massimo tre indici

```
seq[partenza:termine:passo]
```

- indice di partenza: specifica da che elemento partire, se mancante (o `None`) si intende di partire dall'inizio della sequenza
- indice di termine: specifica sino a quale elemento (escluso) terminare con l'estrazione. Se mancante (o `None`) si intende il termine della sequenza.
- indice di passo: specifica lo step dell'estrazione: se mancante lo step è di singola unità

2.6.2 Stringhe

La creazione avviene normalmente mediante assegnazione; la stringa può essere ugualmente inclusa tra apici singoli o doppi. Vediamo alcuni metodi comuni

2.6.2.1 Stringhe valide

```
>>> stringa = "asdomar"
>>> stringa_multilinea = """
... bla bla bla
... qua qua qua
... za za za
... """
>>> stringa_multilinea2 = (
...     "Queste stringhe tra parentesi, senza virgola, "
...     "saranno concatenate "
...     "come avviene nel C, "
...     "sono utili per spezzare."
... )
>>> print(stringa_multilinea2)
Queste stringhe tra parentesi, senza virgola, saranno concatenate come avviene nel C,
```

2.6.2.2 Formattazione

```
>>> import datetime

>>> s = "test"
>>> n = 13
>>> p = 2/3
>>> a_mill = 1000000
>>> dt = datetime.datetime.now()
>>> the_format = ".3f"
>>> address = "Via Tal dei tali"
>>> name = "Mario"

>>> # per formattazione. Quello messo a dx di : dipende dal tipo
>>> f"{s} {n}"          # standard..
'test 13'
>>> f"{p:.2f}"          # float con numero decimali
'0.67'
>>> f"{p:.0f}"          # rimuovere decimali da un float
'1'
>>> f"{p:{the_format}}" # formato nested
'0.667'
>>> f"{p:.2%}"          # percentuali con due decimali
'66.67%'
>>> f"{p:e}"            # notazione scientifica
'6.666667e-01'
>>> f"{a_mill:,}"       # re di migliaia
'1,000,000'
>>> f"{a_mill:,.2f}"    # separatore di migliaia e specifica dec
'1,000,000.00'
>>> f"{dt:%d/%m/%Y}"            # date
'09/11/2026'
>>> f"{dt:%Y-%m-%d (%H:%M:%S)}" # datetime
'2026-11-09 (07:54:24)'
```

```
>>> f"today is a {dt:%A of %B}" # giorno della settimana e mese
'today is a Monday of November'

>>> # per debugging: a sinistra di = una espressione, a destra il risultato
>>> f"{s = }, {n = }"
"s = 'test', n = 13"
>>> f"{n % 2 = }" # qualsiasi espressione a sinistra viene valutata
'n % 2 = 1'
>>> f"{isinstance(n, int) = }" # e il rispettivo valore restituito
'isinstance(n, int) = True'

>>> # padding/allineamento di stringhe e numeri
>>> f"{s:8}"  # allineato a sinistra in campo da 8
'test    '
>>> f"{s:>8}"  # allineato a destra
'    test'
>>> f"{s:*<8}"  # allineato a sx riempiendo con *
'test****'
>>> f"{s:_^8}"  # centrato riempiendo con _
'__test__'
>>> f'{s:_^{n}}' # uso di una variabile per n. di padding
'____test_____'
>>> f'{address:20}{name:10}' # tabulating without libs
'Via Tal dei tali    Mario     '
>>> f"{n:06}" # aggiungere zeri prima di un numero
'000013'
```

2.6.2.3 Elaborazioni varie

```
>>> # Aggiunta di zero
>>> # ----------------
>>> "123".zfill(4)     # Aggiunta di 0 iniziali
'0123'

>>> # Rimozione spazi
>>> # ----------------
>>> "  test di strip ".strip()  # finali e iniziali
'test di strip'
>>> "   test ".lstrip()         # solo iniziali
'test '
>>> " test   ".rstrip()         # solo finali
' test'

>>> # Join di iterabili contenenti stringhe
>>> # ----------------
>>> ", ".join(["a", "b", "c"])
'a, b, c'

>>> # Ricerca/rimpiazzo
>>> # ----------------
>>> "aiuola".find("a")   # Ricerca da sx (prima occorrenza)
```

```
0
>>> "aiuola".find("u")
2
>>> "aiuola".find("x")
-1
>>> "aiuola".rfind("a")  #Ricerca da destra (ultima occorrenza)
5
>>> "prova".replace("a", "e")    # Rimpiazzo
'prove'

>>> # Splitting
>>> # ----------------
>>> "amicici".partition("c")
('ami', 'c', 'ici')
>>> "amicici".rpartition("c")
('amici', 'c', 'i')
>>> "amicici".split("c")
['ami', 'i', 'i']
>>> "linea1\nlinea2".splitlines()
['linea1', 'linea2']

>>> # Checks
>>> # ------
>>> "PROVA".islower()            # Lower/upper case
False
>>> "PROVA".isupper()
True
>>> "BRGLCU83S04H223C".isalnum() # alfanumerico/alfabetico
True
>>> "BRGLCU83S04H223C".isalpha()
False
>>> "prova".startswith("f")      # inizia/termina con
False
>>> "prova".endswith("a")
True

>>> # Coercizioni
>>> # -----------
>>> "PROVA".lower()
'prova'
>>> "prova".upper()
'PROVA'
>>> "uno due tre prova".capitalize()
'Uno due tre prova'
>>> "uno due tre prova".title()
'Uno Due Tre Prova'
```

2.6.2.4 Classe con formato

```
>>> class MyClass:
...     def __format__(self, format_spec) -> str:
```

```
...         return "object"
...
>>> class MyClass2:
...     def __format__(self, format_spec) -> str:
...         match format_spec:
...             case 'upper':
...                 return "OBJECT"
...             case 'lower':
...                 return "object"
...             case _:
...                 raise ValueError(f"{format_spec} not accepted")
...
>>> obj1 = MyClass()
>>> obj2 = MyClass2()

>>> f"{obj1}"          # classe con formato
'object'
>>> f"{obj2:upper}"    # classe con formato e parametro
'OBJECT'
>>> f"{obj2:lower}"    #
'object'
```

2.6.3 Liste

Le **liste** una sequenza di valori, non necessariamente dello stesso tipo, separati da virgole e racchiusi tra parentesi quadre. Le liste sono modificabili

2.6.3.1 Definizione

La definizione di una lista avviene come segue

```
>>> squares = [1, 2, 4, 9, 16, 25]
>>> letters = ['a','b','c','d']
>>> squares
[1, 2, 4, 9, 16, 25]
>>> letters
['a', 'b', 'c', 'd']
```

2.6.3.2 Manipolazione e modifica

Le liste si comportano similmente alle stringhe per ciò che riguarda il **subset**:

```
>>> squares[0]
1
>>> squares[-1]
25
>>> squares[-3:]
[9, 16, 25]
```

e operatori (concatenazione, moltiplicazione)

```
>>> squares + squares
[1, 2, 4, 9, 16, 25, 1, 2, 4, 9, 16, 25]
```

```
>>> squares*2
[1, 2, 4, 9, 16, 25, 1, 2, 4, 9, 16, 25]
```

A differenza delle stringhe sono modificabili:

```
>>> squares[0] = 0 # modifica di un valore
>>> squares
[0, 2, 4, 9, 16, 25]
>>> letters[1:2] = []  # eliminazione di valori
>>> letters
['a', 'c', 'd']
```

Le liste sono oggetti e hanno **metodi** per le operazioni più classiche. Si veda `help(list)`.

2.6.3.3 Metodi utili

```
>>> l = []
>>> a = ["a", "c", "d"]

>>> # Aggiunta elementi
>>> # -----------------
>>> l.append(1)        # un elemento in coda
>>> a.insert(1, "b")   # un elemento prima dell'indice specificato
>>> l.extend([1,2,3]) # elementi da un iterabile in coda

>>> # Rimozione elementi
>>> # ------------------
>>> a.remove("c")      # rimuove la prima occorrenza di un elemento
>>> x = a.pop(1)       # rimuove il valore all'indice specificato e lo ritorna
>>> l.clear()          # rimuove tutti gli elementi

>>> # Ricerca elementi
>>> # ----------------
>>> ["a", "x", "c", "x"].index("x") # indice di un elemento (prima occorrenza)
1
>>> [1,2,1,3].count(1)              # conta le occorrenze di un dato elemento
2

>>> # Ordinamento
>>> # -----------
>>> x = ["w", "j", "a" ]
>>> x.sort()    # ordina
>>> x.reverse() # inverti
```

2.6.3.4 Liste nested

Le liste possono essere nested, e nel caso il subsetting necessita di una parentesi graffa per ogni livello:

```
>>> a = [1, 2, 3]
>>> b = ['a', 'b', 'c']
>>> c = [a, b, 3]
```

```
>>> c
[[1, 2, 3], ['a', 'b', 'c'], 3]
>>> c[0]
[1, 2, 3]
>>> c[1]
['a', 'b', 'c']
>>> c[2]
3
>>> c[1][2]
'c'
```

2.6.3.5 List comprehensions

Sono un trick del linguaggio per creare liste in maniera concisa.

Definizione La versione più generale è

```
newlist = [expression for item1 in iterable1 if condition1
                      for item2 in iterable2 if condition2
                      ...                    ...
                      for itemN in iterableN if conditionN
           if conditionN+1
           if conditionN+2
           ...
           if conditionN+M
]
```

con:

1. `expression` è una espressione contenente `item1..itemN`;
2. seguita da uno statement `for` (obbligatorio) che si riferisca ad un iterabile (volendo filtrato mediante `if`);
3. al quale seguono 0+ più ulteriori statement `for` con altrettanti iterabili (per ciclare su altro, eventualmente filtrando con `if`);
4. al quale seguono 0+ statement `if` (opzionalmente per selezionare gli elementi da porre nell'output).

Gli `iterable*` non necessariamente debbono essere della stessa lunghezza, perché sono iterate da sinistra a destra, non in parallelo: per ogni elemento in `iterable1`, viene fatto loop su `iterable2` e tutte le rimanenti a cascata

Esempi

```
>>> ## Esempio base
>>> doubled = [x * 2 for x in range(10)]
>>> doubled
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

>>> ## Esempio con if
>>> combs = [(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y]
```

```
>>> # che equivale a
>>> combs = []
>>> for x in [1,2,3]:
...     for y in [3,1,4]:
...         if x != y:
...             combs.append((x, y))
...

>>> X = [1, 2, 3]
>>> Y = [4, 5, 6]
>>> res = [[x, y, x*y]
...         for x in X if x > 1
...         for y in Y if y < 6
...         if y % x == 0]
```

Espressioni più complesse e trick utili a seguire

```
>>> vec = [-4, -2, 0, 2, 4]

>>> ## Selezionare gli elementi >= 0
>>> [x for x in vec if x >= 0]
[0, 2, 4]

>>> ## Applicare una funzione a tutti gli elementi di una lista
>>> [abs(x) for x in vec]
[4, 2, 0, 2, 4]

>>> ## utilizzo di più di un if: stampa dei numeri divisibili per 2 e per 5
>>> ## tra 0 e 100
>>> [y for y in range(100) if y % 2 == 0 if y % 5 == 0]
[0, 10, 20, 30, 40, 50, 60, 70, 80, 90]

>>> ## Le espressioni possono essere molto generali;
>>> ## if .. else con list comprehension
>>> [">=0" if i>=0 else "<0" for i in vec]
['<0', '<0', '>=0', '>=0', '>=0']

>>> ## creiamo una lista composta da tuple
>>> [(x, x**2) for x in range(6)]
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]

>>> ## esegui un metodo su ogni elemento
>>> fruit = ['banana', 'strawberry', 'passion fruit']
>>> [y.capitalize() for y in fruit]
['Banana', 'Strawberry', 'Passion fruit']
```

List comprehensions nested Espressione di una list comprehension può esser qualunque cosa, quindi anche una list comprehension. Questo può essere utile per creare liste di liste, che possono avere applicazione.

```
>>> ## Calcolo tabelline del 7 e dell'8 (nb:cicla per prima è la lista
>>> ## esterna, poi quella interna)
```

```
>>> nl = [[i*j for j in range(1, 11)] for i in range(7, 9)]
>>> nl
[[7, 14, 21, 28, 35, 42, 49, 56, 63, 70], [8, 16, 24, 32, 40, 48, 56, 64, 72, 80]]

>>> ## Trasposizione di matrice creata mediante lista di liste
>>> matrix = [
...             [1, 2,   3,  4],
...             [5, 6,   7,  8],
...             [9, 10, 11, 12]
...           ]
>>> t = [[row[i] for row in matrix] for i in range(4)]
>>> t
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

2.6.4 Tuple

Le **tuple** sono insiemi di valori separati da virgole (buona norma porle tra parentesi tonde per chiarezza) e non modificabili una volta create

```
>>> ## Definizione
>>> atuple = ('robots', 77, 93, 'try')
>>> atuple
('robots', 77, 93, 'try')

>>> ## subset
>>> atuple[:3]
('robots', 77, 93)

>>> ## tuple nested
>>> a = (1, 2)
>>> b = ('x', 'y')
>>> c = (a, b)
>>> c
((1, 2), ('x', 'y'))

>>> ## modifica diretta? no: da errore
>>> atuple[1] = 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> ## tuttavia è possibile creare tuple che contengono oggetti modificabili
>>> btuple = ([1,2], "abc")
>>> btuple[0][0] = 3
>>> btuple
([3, 2], 'abc')

>>> ## Metodi utili, uguali a quelli delle liste
>>> atuple.count(93)
1
>>> atuple.index("try")
3
```

2.6.5 Sequence unpacking

Consiste nell'assegnare gli elementi di una sequenza (posta come *rvalue*) a variabili separate (*lvalue*)

```
>>> ## Esempio con lista
>>> a, b, c = [1, 2, 'goodbye']

>>> ## Esempio con tupla
>>> t = (12345, 54321, 'hello!')
>>> t1, t2, t3 = t

>>> ## Esempio con stringa
>>> x, y, z = 'cia'
```

2.7 Classi mapping e set

2.7.1 Dict

Sono un insieme non ordinato di coppie `key:value`, con `key` univoche all'interno di un dict (e immutabili), utile per memorizzare un dato e ritornarlo attraverso la sua chiave. Come chiave può essere data qualsiasi cosa[3]:

```
>>> ## Definizione, accesso e modifica
>>> ## ------------------------------
>>> d = {'planet':'earth', 'region':'europe', 'prefix':39, 'number_list': [1,2,3]}
>>> d
{'planet': 'earth', 'region': 'europe', 'prefix': 39, 'number_list': [1, 2, 3]}
>>> d2 = {1: "a", 2: "b"} # esempio con chiave numerica
>>> f = dict(sape = 4139, guido = 4127, jack = 4098) # Metodo alternativo: funzione d
>>> g = dict([('sape', 4139), ('guido', 4127), ('jack', 4098)])

>>> ## Accesso
>>> ## -------
>>> d['prefix']          # lettura (es d2[1] per chiave numerica)
39
>>> d.get('planet')      # ottiene un valore data la chiave
'earth'
>>> d["asdomar"]         # se fallisce accesso, solleva eccezione
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'asdomar'
>>> d.get("asd")         # se fallisce accesso, ritorna None
>>> d.get("asd", "foo")  # se fallisce accesso, ritorna il secondo
'foo'
>>> d.setdefault("asd","foo")  # se fallisce l'accesso, ritorna il secondo e lo impos
'foo'
```

[3] La chiave deve essere hashabile, ossia trasformabile in un intero che viene utilizzato per lookup: ad esempio le stringhe possono essere trasformate in interi sulla base dei caratteri, le tuple combinando le sue componenti. Il contenuto della chiave deve essere immodificabile; per questo motivo le tuple sono ammesse come chiave, le liste no.

```
>>> d["asd"]              # ora asd è impostato
'foo'

>>> ## Modifica/inserimento
>>> ## ---------------------
>>> d['prefix'] = 45    # scrittura valore esistente
>>> d['foo'] = "bar"    # nuovo inserimento

>>> ## cancellazione
>>> ## -------------
>>> del d2[1]           # cancellazione di un elemento
>>> g.clear()           # rimozione di tutti gli item


>>> ## ottenere le chiavi/valori come iterabili (es per un for)
>>> ## ---------------------------------------------------------
>>> d.keys()    # ottenere le chiavi
dict_keys(['planet', 'region', 'prefix', 'number_list', 'asd', 'foo'])
>>> 'foo' not in d.keys()
False
>>> d.values() # ottenere valori come iterabile
dict_values(['earth', 'europe', 45, [1, 2, 3], 'foo', 'bar'])
>>> d.items() # coppia (indice, valore) come iterabile per unpacking
dict_items([('planet', 'earth'), ('region', 'europe'), ('prefix', 45), ('number_list', [1, 2, 3
```

2.7.1.1 Dict comprehension

Hanno sintassi analoga a quella delle liste e possono tornare talvolta utili come forma di mapping

```
>>> pow2 = {x: x**2 for x in (2, 4, 6)}
>>> pow2[2]
4
```

2.7.2 Sets

Sono una collezione di elementi senza ordine e duplicati. Si usano tipicamente per:

- verificare l'appartenenza di un qualcosa ad un insieme (mediante `in`);
- per eliminare duplicati di altre strutture dati
- per effettuare operazioni insiemistiche

Si definiscono mediante parentesi graffe o la funzione `set`.

```
>>> basket = {'apple', 'orange', 'apple', 'pear'}
>>> basket ## elementi doppi vengono eliminati
{'orange', 'apple', 'pear'}

>>> ## definizione mediante set: dalla sequenza fornita sono eliminati i doppi
>>> a = set('abracadabra')                  ## stringa
```

```
>>> a
{'b', 'c', 'd', 'a', 'r'}
>>> a = set(['asd', 'foo', 'bar', 'asd'])    ## lista
>>> a
{'foo', 'bar', 'asd'}
>>> a = set( ('asd', 'foo', 'bar', 'asd') )  ## tuple
>>> a
{'foo', 'bar', 'asd'}

>>> ## emptyset: si usa set, non due graffe (usate per dict vuoto)
>>> empty = set()

>>> ## Test appartenenza
>>> 'orange' in basket
True
>>> 'strawberry' in basket
False

>>> ## aggiunta, rimozione, azzeramento
>>> empty.add(1)
>>> empty.remove(1)
>>> empty
set()
>>> empty.add(1)
>>> empty.add(2)
>>> empty.clear()
```

2.7.2.1 Operatori/metodi utili

```
>>> ## Applicazione operatori numerici e logici
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a
{'b', 'c', 'd', 'a', 'r'}
>>> b
{'c', 'z', 'a', 'l', 'm'}

>>> ## Operazioni insiemistiche
>>> a - b # lettere in a ma non in b
{'b', 'r', 'd'}
>>> a | b # lettere in a o in b
{'b', 'c', 'z', 'a', 'd', 'r', 'l', 'm'}
>>> a & b # lettere sia in a che in b
{'c', 'a'}
>>> a ^ b # xor: lettere in a o in b ma non in entrambi
{'z', 'b', 'l', 'r', 'm', 'd'}

>>> ## Test insiemistici
>>> # intersezione
>>> {1,2}.isdisjoint({3,4})
True
```

```
>>> {1,2}.isdisjoint({2,3})
False
>>> # sottinsieme e sovrainsieme
>>> a = {1, 2}
>>> b = {1}
>>> a.issuperset(b)
True
>>> b.issubset(a)
True
```

2.7.2.2 Set comprehension

Le comprehension sono ammesse anche nei set

```
>>> a = {x for x in 'abracadabra' if x not in 'abc'}
>>> a
{'r', 'd'}
```

Capitolo 3

Controllo del flusso

Contents

3.1 Costrutti condizionali: `if` e `while` 23
3.1.1 `if` 23
3.1.2 `match` 23
3.1.3 `while` 23
3.1.4 `break`, `continue` ed `else` 23
3.2 Condizioni e test logici 24
3.2.1 Test verità 24
3.2.2 Operatori booleani 24
3.2.3 Comparazioni 24
3.2.4 Comparazioni concatenate 25
3.2.5 Check appartenenza: `in` e `not in` 25
3.2.6 Comparare sequenze e altri tipi 25
3.3 Looping su oggetti: `for` 26
3.3.1 Looping in sequenze (stringhe, liste, tuple) 26
3.3.2 Looping nei dict 26
3.3.3 Looping sui set 26
3.3.4 L'utilizzo di `range` 26

3.1 Costrutti condizionali: `if` e `while`

3.1.1 `if`

Ha la seguente sintassi con `<>` a indicare un contenuto obbligatorio, `[]` uno facoltativo e `*` la possibilità di ripetizione:

```
if <condizione>:
    <istruzioni>
[elif <condizione>:
     <istruzioni>]*
[else:
     <istruzioni>]
```

3.1.2 `match`

Simile allo switch di altri linguaggi, `match` prende una espressione e la compara ad una casistica di altre espresioni (poste dopo `case`) per eseguire azioni specificate:

```
def http_error(status):
    match status:
        case 400:
            return "Bad request"
        case 401 | 403 | 404:
            return "Not allowed"
        case 418:
            return "I'm a teapot"
        case _:
            return "Something's wrong with the internet"
```

Si notano che si possono specificare match multipli con `|`, mentre `_` matcha sempre e quindi può essere usato come caso default (eventualmente) quando nessun altro caso ha matchato.

3.1.3 `while`

python ha solo l'istruzione `while` per implementare l'iterazione nel senso di altri linguaggi come il Pascal o il C:

```
while <condizione>:
    <istruzioni>
[else:
    <istruzioni>]
```

3.1.4 `break`, `continue` ed `else`

Sia in `while` che `for` (che vedremo poi):

- `continue` permette di saltare le rimanenti istruzioni del ciclo passando all'iterazione successiva;
- `break` termina il ciclo;
- se è presente la clausola `else` le sue istruzioni vengono eseguite qualora il ciclo termini normalmente, mentre non vengono eseguite se il ciclo termina a causa dell'istruzione break.

3.2 Condizioni e test logici

3.2.1 Test verità

Un oggetto può essere testato come `True`/`False` in un `if` o in un `while`: in generale un oggetto è considerato `True` a meno che, alternativamente

- la sua classe definisce il metodo `__bool__`, che restituisce `False`

- il metodo `__len__` ritorna qualcosa, se chiamato sull'oggetto

Quindi:

- `None` è valutato `False`
- i numeri restituiscono tutti `True` ad eccezione dello 0 (`int` o `float` che sia) che è `False`
- stringa, lista, tuple, set e dict **vuoti restituiscono** `False`, altrimenti (con almeno un elemento) viene restituito `True` (indipendentemente dal contenuto)

Mediante una funzioni del genere possiamo testare la valutazione booleana di oggetti di natura diversa:

```
>>> def is_it_true(anything):
...     if anything:
...         print("yes, it's true")
...     else:
...         print("no, it's false")
...
```

3.2.2 Operatori booleani

Le comparazioni possono esser elaborate mediante operatori booleani `and` `or` e `not`.
È sempre meglio aggiungere le parentesi per indirizzare ordine/priorità di valutazione. In assenza tra gli operatori `not` ha la priorità maggiore, `or` la minore. Pertanto:

```
A and not B or C
```

equivale a

```
(A and (not B)) or C
```

3.2.3 Comparazioni

Vi sono otto operatori di comparazione, hanno tutti la stessa priorità (che è superiore a quella degli operatori booleani

```
<        minore
<=       minore o uguale
>        maggiore
>=       maggiore o uguale
==       equal
!=       not equal
is       due oggetti sono identici
is not   due oggetti sono diversi
```

Alcune regole:

- oggetti di tipo differente (esclusi numeri) son sempre diversi

- oggetti non identici di una stessa classe sono diversi, a meno che forniscano un metodo `__eq__` che li battezzi come uguali
- istanze di una classe non possono essere ordinate tra loro a meno che la classe non definisca `__lt__` ed `__eq__` (si può definire volendo `__lt__`, `__le__`, `__gt__`, `__ge__`)
- il funzionamento di `is` e `is not` non può esser modificato ed è supportato da iterabili o oggetti che implementano il metodo `__contains__`

3.2.4 Comparazioni concatenate

Le comparazioni possono essere concatenate, come in:

```
a < b == c
```

che equivale a in pratica

```
(a < b) and (b == c)
```

3.2.5 Check appartenenza: `in` e `not in`

`in` e `not in` controllano se un valore è presente o meno in una sequenza

3.2.6 Comparare sequenze e altri tipi

Oggetti di tipo sequenza possono esser comparati con oggetti del medesimo tipo; la comparazione avviene in maniera ricorsiva, con ordinamento lessicografico (in base a unicode). Vi sono alcune peculiarità:

- se tutti gli item di due sequenze sono uguali, le sequenze sono considerate uguali
- se una sequenza costituisce l'inizio di un'altra sequenza più lunga, la sequenza più corta è considerata minore

La comparazione restituisce un valore `True` o `False`; ad esempio nei seguenti casi viene restituito sempre `True`:

```
>>> (1, 2, 3)              < (1, 2, 4)
True
>>> [1, 2, 3]              < [1, 2, 4]
True
>>> 'ABC' < 'C' < 'Pascal' < 'Python'
True
>>> (1, 2, 3, 4)           < (1, 2, 4)
True
>>> (1, 2)                 < (1, 2, -1)
True
>>> (1, 2, 3)             == (1.0, 2.0, 3.0)
True
>>> (1, 2, ('aa', 'ab'))   < (1, 2, ('abc', 'a'), 4)
True
```

3.3 Looping su oggetti: `for`

Nel python serve per iterare sugli elementi di una sequenza (come lista stringa o tuple) e presenta questa sintassi

```
for <variabile> in <sequenza>:
    <istruzioni>
[else:
     <istruzioni>]
```

Il funzionamento interno del `for` verrà spiegato nella sezione di OOP; qui si riassume l'utilizzo standard coi dati più comuni.

3.3.1 Looping in sequenze (stringhe, liste, tuple)

Nelle sequenze possiamo:

- fare loop sul singolo elemento (ponendo la sequenza nel `for` direttamente)
- ottenere un progressivo numerico indice e il contenuto della sequenza con `enumerate`
- fare loop sull'oggetto ordinato con `sorted`
- fare loop sull'oggetto ordinato in maniera decrescente con `reversed`

Gli esempi presentano liste, ma il funzionamento è speculare anche per stringhe e tuple:

```
>>> games = ['monopoli', 'risiko', 'dnd']

>>> for g in games:
...     print(g)
...
monopoli
risiko
dnd
>>> for i, g in enumerate(games):
...     print(i, g)
...
0 monopoli
1 risiko
2 dnd
>>> for g in sorted(games):
...     print(g)
...
dnd
monopoli
risiko
>>> for g in reversed(games):
...     print(g)
...
dnd
risiko
monopoli
```

3.3.2 Looping nei dict

Di un dict possiamo volere le chiavi (usiamo l'oggetto direttamente o ne chiamiamo/esplicitiamo il metodo `keys`), i contenuti (risp `values`) o tutti e due (`items`):

```
>>> knights = {"gallahad": "the pure", "ciro": "the brave"}

>>> for k in knights:
...     print(k)
...
gallahad
ciro
>>> for k in knights.keys():
...     print(k)
...
gallahad
ciro
>>> for v in knights.values():
...     print(v)
...
the pure
the brave
>>> for k, v in knights.items():
...     print(k, v)
...
gallahad the pure
ciro the brave
```

3.3.3 Looping sui set

La forma più semplice, si pone il set nel for

```
>>> S = {2, 3, 5, 7}
>>> for i in S:
...     print(i)
...
2
3
5
7
```

3.3.4 L'utilizzo di `range`

Se è necessario iterare su una sequenza di interi la funzione `range` torna utile

```
>>> range(5)              # 0, 1, 2, 3, 4
range(0, 5)
>>> range(5, 10)          # 5, 6, 7, 8, 9
range(5, 10)
>>> range(0, 10, 3)       # 0, 3, 6, 9
range(0, 10, 3)
```

```
>>> range(-10, -100, -30)  # -10, -40, -70
range(-10, -100, -30)
```

Capitolo 4

Funzioni

Contents

4.1 Definizione 27
4.2 Chiamata 28
4.2.1 Valutazione dei valori di default 28
4.2.2 Valore ritornato 28
4.2.3 Unpacking di liste/tuple/dict in chiamata 28
4.3 Regole di scope 28
4.3.1 Namespace 29
4.3.2 Attributi 29
4.3.3 Scoping 29
4.4 Funzioni anonime (`lambda`) 30
4.5 Programmazione funzionale 30
4.5.1 Funzioni classiche 30
4.5.2 Partialling 31
4.5.3 Function factory 32
4.5.4 Composizione di funzioni 32
4.5.5 Decorators 32
4.5.6 Single e multiple dispatch 33
4.6 Altri argomenti 34
4.6.1 Type annotation/type hint 34
4.6.2 `atexit` 35

4.1 Definizione

Avviene mediante `def`, seguita dal nome della funzione e ponendo tra tonde i parametri:

```
def nome_funzione(arg1, arg2 = default2, *args, **kwargs):
    """
    Stringa di documentazione disponibile in nome_funzione.__doc__
    """
    codice
    ...
```

Gli argomenti devono essere specificati nell'ordine di cui sopra, e sono:

1. *positional argument* (`arg1`) son definiti con solamente il nome dell'argomento. In chiamata è obbligatorio specificarli inserendo alternativamente `valore` o `nome = valore` (in questo secondo caso sarà possibile seguire un ordine di argomenti diverso da quello dato in definizione);

2. *keyword argument* (`arg2`) son definiti con un valore di default: in sede di chiamata non sono obbligatori da specificare (e si possono specificare sia con `valore` che `nome=valore`);

3. *arbitrary argument list*: se si specifica `*args` in definizione, la variabile `args` memorizzerà una tupla con gli argomenti posizionali della chiamata (specificati mediante il solo `valore`) esclusi i valori associati ad altri argomenti posizionali: ad esempio sopra prima viene riempito `arg1`, poi `args`);

4. *arbitrary keyword argument list*: se si specifica `**kwargs` in definizione, la variabile `kwargs` memorizzerà un dizionario con i keyword argument (`nome=valore`) specificati in sede di chiamata (esclusi quelli che matchano keyworded argument specificati, es `arg2`)

Parametri speciali `/*` in definizione Gli argomenti possono esser passati alle funzioni sia per posizione che utilizzando la keyword. Si può (leggibilità/performance) in definizione imporre che la chiamata di alcuni possa avvenire in un modo, nell'altro o in entrambi, specificando gli argomenti opzionali `/` e `*`

```
def f(pos1, pos2, /, pos_or_kwd, *, kwd1, kwd2):
      -----------    ----------     ----------
        |             |                  |
        |        Positional or keyword   |
        |                                  +-- Keyword only
        +-- Positional only
```

Esempio definizione

```
>>> def args_demo(arg1, x=1, *args, **kwargs):
...     print("arg1 = ", arg1)
...     print("x = ", x)
...     print("args = ", args)
...     print("kwargs = ", kwargs)
...
```

4.2 Chiamata

In chiamata si:

- vanno specificati tutti i parametri posizionali (senza default);

- i posizionali vanno posti prima dei keyworded;

- i posizionali posti senza `nome=` verranno associati nell'ordine dato in definizione; se si specificano tutti `nome=valore` l'ordine può differire dalla definizione

```
>>> args_demo("asd", "baz", 2, [1,2,3], kw1=1, kw2=5)
arg1 =  asd
x =  baz
args =  (2, [1, 2, 3])
kwargs =  {'kw1': 1, 'kw2': 5}
>>> args_demo(1, 2, 3, foo = 6, baz = 7)
arg1 =  1
x =  2
args =  (3,)
kwargs =  {'foo': 6, 'baz': 7}
```

4.2.1 Valutazione dei valori di default

Questa:

- avviene nel punto in cui la funzione è stata *definita*, non quando viene chiamata, e il valore valutato viene conservato per le future chiamate

```
>>> x = 5

>>> def f(arg = x):
...     print(arg)
...
>>> x = 6
>>> f()
5
```

- se il valore di default è modificabile (lista, dict, classe) e viene modificato entro la funzione, la versione *cambiata* sarà conservata per le prossime chiamate (cosa raramente voluta)

```
>>> # qui un intero (1 valutato in definizione) è immodificabile quindi
>>> # eventuali entro la funzione non intaccano le prossime chiamate
>>> def f(x = 1):
...     print(x)
...     x = 2
...
>>> f()
1
>>> f()
1

>>> # qui L è una lista modificabile. Se la modifichiamo entro funzione il valore
>>> # verrà ritrovato nelle prossime chiamate
>>> def f(x, L = []): # una lista è modificabile
...     L.append(x)
...     return L
...
```

```
>>> f(1)
[1]
>>> f(2)
[1, 2]
>>> f(3)
[1, 2, 3]
```

Se non si vuole che il valore di default sia condiviso tra successive chiamate scrivere una funzione come la seguente

```
def f(x, L = None):
    if L is None:
        L = []
    L.append(x)
    return L
```

4.2.2 Valore ritornato

Se:

- si utilizza l'istruzione `return`, viene restituito dalla funzione alla chiamante un determinato dato fornito come argomento di `return`;
- se non si specifica nulla (o `return` senza argomenti) viene restituito `None`.

4.2.3 Unpacking di liste/tuple/dict in chiamata

Si può usare `*` (per lista, tuple) o `**` (dict) prima del nome della variable per spacchettare gli argomenti. Un esempio

```
>>> def foo(x,y,z):
...     print("x=" + str(x))
...     print("y=" + str(y))
...     print("z=" + str(z))
...
>>> l = [1,2,3] # esempio con lista
>>> foo(*l)
x=1
y=2
z=3

>>> d = {'z':'foo','x':'bar','y':'baz'} # esempio con dict
>>> foo(**d)
x=bar
y=baz
z=foo
```

4.3 Regole di scope

Vediamo le regole con cui Python cerca i valori delle variabili/nomi forniti in programmazione.

4.3.1 Namespace

Un *namespace* è abbinamento di nomi ad oggetti presenti in memoria; sono *creati in diversi momenti* e hanno *differente durata*. I principali sono:

- i nomi **builtin**: creato all'avvio dell'interprete e dura sino al termine dell'esecuzione;
- i nomi di un **modulo** (`__main__` o moduli importati): è reso disponibile quando il modulo viene caricato (e anche esso generalmente dura sino al termine dell'esecuzione);
- i nomi di **funzione**: creato alla chiamata, viene distrutto quando la funzione ritorna o solleva una eccezione non gestita[1].

Per ottenere la lista di nomi di un namespace si utilizza `dir`:

```
>>> import sys

>>> dir(__builtins__) ## nomi builtin del linguaggio
['__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__
>>> dir()             ## nomi nel modulo globale (workspace)
['MyClass', 'MyClass2', 'S', 'X', 'Y', '__builtins__', '__doc__', '__name__', 'a', 'a_mill', 'a
>>> dir(sys)          ## nomi di un modulo importato
['__breakpointhook__', '__displayhook__', '__doc__', '__excepthook__', '__interactivehook__', '

>>> # nomi in una funzione
>>> def f():
...     a = 1
...     b = 2
...     print(dir())
...
>>> f()
['a', 'b']
```

4.3.2 Attributi

Qualsiasi nome che segue un punto: ad esempio in `z.real`, `real` è un attributo di `z` (qualsiasi cosa sia).
Gli attributi possono essere *read-only* o *scrivibili*; nel secondo caso:

- è possibile assegnarvi qualcosa mediante `x.attributo = valore`;
- è possibile eliminarli mediante `del x.attributo`, che rimuove attributo da `x`.

4.3.3 Scoping

All'interno della chiamata di una funzione, la ricerca di un nome avviene nel seguente ordine:

1. nel namespace della *funzione corrente*;

[1] Le funzioni ricorsive hanno un namespace per ogni chiamata

2. nella funzione *enclosing* (quella all'interno del quale la funzione corrente è stata definita[2]); dopodichè la *enclosing della enclosing*, e così via;
3. il *namespace del modulo* corrente, sia esso main o importato
4. il namespace delle *builtin* functions.

Eccezioni: `nonlocal` e `global` Alcuni casi particolari:

- `nonlocal` serve per dichiarare nomi che devono essere presi non dal namespace corrente ma da quelli enclosing

```
>>> a = 1

>>> def scope_local():
...     a = 2
...     print(a)
...
>>> scope_local()
2

>>> def scope_nonlocal_read():
...     a = 3
...     def nested():
...         nonlocal a
...         print(a)
...     nested()
...
>>> scope_nonlocal_read()
3

>>> def scope_nonlocal_write():
...     a = 3
...     def nested():
...         nonlocal a
...         a = 4
...     nested()
...     print(a)
...
>>> scope_nonlocal_write()
4
```

- se un nome è dichiarato mediante `global` la lettura e scrittura da tale variabile va a inficiare il namespace del modulo, indipendentemente da dove essa sia stata effettuata

```
>>> a = 1

>>> def scope_global():
...     global a
...     print(a)
```

[2] Questo è quello che permette le *function factory*.

```
...
>>> scope_global()
1
```

4.4 Funzioni anonime (lambda)

Le funzioni anonime:

- sono create mediante la keyword `lambda`. dove i parametri di input sono separati dal valore restituito da `:`
  Ad esempio la seguente ritorna la somma dei due argomenti passati:

  ```
  lambda a, b: a + b
  ```

  Possono essere assegnate (volendo) comportandosi come normali funzioni ma questo è considerato bad practice da taluni

  ```
  >>> x = lambda a, b : a * b
  >>> x(5, 6)
  30
  ```

- possono essere usate dove è necessaria una funzione (il suo nome). Ad esempio anche con:

  ```
  >>> (lambda a, b: a+b)(1, 2)
  3
  ```

- sono limitate a *una singola espressione* come corpo

4.5 Programmazione funzionale

Le funzioni sono first class object (quindi possono essere date in input e ritornate come output) e ciò rende possibile la programmazione funzionale. In questa si possono sfruttare caratteristiche del linguaggio come:

- funzioni builtin tipiche della pf: `map`, `filter` etc
- funzioni anonime definite mediante `lambda`
- per i dati analizzati: *iterabili* e *iteratori* (quindi anche generatori ed espressioni generatrici) e le funzioni utili su di essi (es `enumerate`, `sorted`, `any`, `all`, `zip`)

A livello di pacchetti

- `itertools`: iteratori comuni e funzioni per elaborarli
- `functools`: high order functions (funzioni che processano funzioni modificandole)
- `operator`: funzioni che corrispondono agli operatori del python e possono aiutare in un approccio funzionale (evitandoci di scrivere funzioni triviali per effettuare singole operazioni)

4.5.1 Funzioni classiche

4.5.1.1 `map` (= `Map`/`lappy`)

La builtin

```
map(f, iterA, iterB, ...)
```

restituisce un iteratore sulla sequenza che applica la funzione con argomenti presi dagli iterabili passati:

```
f(iterA[0], iterB[0]), f(iterA[1], iterB[1]),  ....
```

Se applicata con

- un singolo iterabile (e una funzione che prende in input un solo argomento) è di fatto equivalente a `lapply` di R

  ```
  >>> a = ['sentence', 'fragment']
  >>> list(map(str.upper, a))
  ['SENTENCE', 'FRAGMENT']
  ```

- due/più iterabili (uno per ciascun argomento della funzione passata) è equivalente a `Map`

  ```
  >>> b = ["foo", "bar"]
  >>> list(map(lambda x, y: f"{x} + {y}", a, b))
  ['sentence + foo', 'fragment + bar']
  ```

4.5.1.2 `itertools.starmap`

Applicata ad un singolo iterabile crea un iteratore che valuta la funzione utilizzando argomenti ottenuti dall'iterabile; da utilizzare al posto di `map` quando gli input sono già stati raggruppati (iterabile di iterabili) o zippati nell'iterabile passato

```
>>> from itertools import starmap

>>> def custom_f(a, b, c):
...     return(a * b - c)
...
>>> x = (2,5,2)
>>> y = (3,2,1)
>>> z = (10,3,2)

>>> # qui applichiamo la funzione prendendo sequenzialmente argomenti da x, poi y e
>>> # poi da z
>>> list(starmap(custom_f, [x, y, z]))
[8, 5, 28]

>>> # qui prendiamo in parallelo da tutti e tre
>>> list(starmap(custom_f, zip(x, y, z)))
[-4, 7, 0]
```

4.5.1.3 `functools.reduce`

Applica una funzione che prende in input due parametri e ne restituisce uno agli elementi di una sequenza:

- la funzione è applicata ai primi due, il risultato applicato insieme al terzo elemento, e così via
- se `initializer` è presente viene piazzato prima della sequenza nel calcolo e funge da default se la sequenza è vuota;

```
>>> from functools import reduce
>>> from operator import add

>>> reduce(add, [1,2,3])
6
>>> reduce(add, [1,2,3], 5)
11
```

4.5.1.4 `itertools.accumulate`

A differenza di `functools.reduce` (che da il risultato finale) restituisce un iteratore sui risultati parziali:

```
>>> from itertools import accumulate
>>> from operator import add

>>> cumsum = accumulate([1,2,3], add)
>>> list(cumsum)
[1, 3, 6]
```

4.5.1.5 `filter`

```
filter(predicate, iter)
```

ritorna un iteratore su tutti gli elementi che rispettano un `predicate`, ossia una funzione che restituisce `True` o `False`. Per funzionare con filter, il predicato deve prendere in input un singolo parametro

```
>>>  # funzione per selezionare i pari (x%2==0 è True)
>>> list(filter(lambda x: x%2==0, range(10)))
[0, 2, 4, 6, 8]
```

Su iteratori funzionalità analoghe si hanno con `filterfalse` e `takewhile` di `itertools`

4.5.2 Partialling

Consiste nel creare una copia di una funzione con uno o più argomenti settati ad un default

4.5.2.1 functools.partial

Per parzializzare funzioni

```
>>> from functools import partial

>>> def spam(a, b, c, d):
...     print(a, b, c, d)
...
>>> s1 = partial(spam, 1)        # a = 1
>>> s1(4, 5, 6)
1 4 5 6

>>> s2 = partial(spam, d = 42)   # d = 42
>>> s2(4, 5, 6)
4 5 6 42

>>> s3 = partial(spam, 1, 2, d = 42)   # a = 1, b = 2, d = 42
>>> s3(5)
1 2 5 42
```

4.5.2.2 functools.partialmethod

Per parzializzare metodi

```
from functools import partialmethod

class Cell:

    def __init__(self):
        self._alive = False

    @property
    def alive(self):
        return self._alive

    # general method ...
    def set_state(self, state):
        self._alive = bool(state)

    # ... and partialled methods
    set_alive = partialmethod(set_state, True)
    set_dead = partialmethod(set_state, False)


c = Cell()
c.alive # False

c.set_alive()
c.alive # True
```

4.5.3 Function factory

Le function factory fanno uso della capacità del linguaggio di ritornare una funzione che fa uso del namespace della chiamante per la risoluzione di alcune free variable. Vediamo una implementazione di una function factory che crea funzioni potenze in base al parametro passatogli

```
>>> def make_power(n):
...     def power(x):
...         print(x ** n)
...     return power
...
>>> square = make_power(2)
>>> square(3) # restituisce 9
9

>>> # con funzione anonima
>>> def lambda_power(n):
...     return lambda x: print(x ** n)
...
>>> squarel = lambda_power(2)
>>> squarel(3)
9
```

4.5.4 Composizione di funzioni

Sfruttando le function factory per comporre diverse funzioni in una sola si può definire la seguente

```
>>> def compose(*funs):
...     """
...     Return a new function which is composition in math sense
...     compose(f,g,...)(x) = f(g(...(x)))
...     """
...     def worker(data, funs = funs):
...         result = data
...         for f in reversed(funs):
...             result = f(result)
...         return result
...     return worker
...
>>> def add2(x):
...     return x+2
...
>>> def mul2(x):
...     return x*2
...
>>> f = compose(add2, mul2) # 2x + 2
>>> g = compose(mul2, add2) # 2*(x+2)

>>> X = [1,2,3]
>>> [f(x) for x in X]
```

```
[4, 6, 8]
>>> [g(x) for x in X]
[6, 8, 10]
```

Alternativamente, `functools.reduce` può essere utilizzata per comporre una lista di funzioni, ad esempio:

```
>>> import functools

>>> def compose(f, g):
...     return lambda x: f(g(x))
...
>>> funcs = [lambda s: s + "a",
...          lambda s: s + "j",
...          lambda s: s + "e",
...          lambda s: s + "j",
...          lambda s: s + "e"]
>>> # questo è necessario se compose è definita
>>> # applicando la funzione sinistra come seconda (come in math) invece
>>> # di prima, come pensiamo a livello di logica sopra
>>> funcs.reverse()

>>> worker = functools.reduce(compose, funcs)
>>> worker("brazorv_")
'brazorv_ajeje'
```

4.5.5 Decorators

Un decorator è una funzione, solitamente[3], che wrappa un'altra funzione modificandone il comportamento standard in qualche modo

4.5.5.1 Definizione e utilizzo

La definizione è del tipo:

```
def decorator(fun):
    def worker:
       # function definition using fun
    return worker
```

Una volta definito possiamo usarlo

- prima definendo la funzione da decorare e poi decorandola

  ```
  def undecorated():
      # do something

  dec = decorator(undecorated)  # dec is decorated
  ```

- mediante una sintassi compatta equivalente dove viene prefisso il nome del decoratore alla definizione della funzione da decorare

[3] Decorator può essere qualsiasi callable, ovvero un oggetto che presenta il metodo `__call__`

```
@decorator
def undecorated():    # undecorated is actually decorated
    # do something
```

- i decorator possono essere concatenati, facendo si che ad una funzione base vengano applicati più decoratori contemporaneamente (aggiungendo feature in maniera pulita).

```
@decorator1
@decorator2
def undecorated():    # undecorated is actually double-decorated
    # do something
```

4.5.5.2 Esempi semplici

Caching/Memoization Per renderla più veloce una funzione che viene utilizzata più volte si puo possono salvare i risultati in una cache che verrà consultata alla prossima chiamata prima (facendo eseguire la funzione decorata solo se i risultati non siano già disponibili. Vediamo un esempio con la sequenza di fibonacci

```
>>> def memoize(f):
...     cache = {}
...     def helper(x):
...         if x not in cache:
...             cache[x] = f(x)
...         return cache[x]
...     return helper
...

>>> @memoize
... def fib(n):
...     if n in (0,1):
...        return n
...     else:
...        return fib(n - 1) + fib(n - 2)
...

>>> fib(1)
1
>>> fib(1) # using cache, not computing
1
```

La libreria standard fornisce il decoratore `cache` in `functools` proprio per questo

```
from time import sleep
from functools import cache


@cache
def sleep_identity(x):
```

```
    # long task
    sleep(5)
    return x


sleep_identity(1)  # first time take 5 seconds
sleep_identity(1)  # second time (same arg) is immediate
```

Decorator multipli Se vogliamo aggiungere sia la memoizzazione che il logging alla funzione `fib` di cui prima, definiamo prima i due decorator `memoize` e `trace`, dopodichè decoriamo la definizione di `fib`

```
>>> def trace(f):
...     def helper(x):
...         call_str = f"{f.__name__}({x})"
...         print(f"Calling {call_str} ...")
...         result = f(x)
...         print(f"... returning from {call_str} = {result}")
...         return result
...     return helper
...
>>> @memoize
... @trace
... def fib(n):
...     # stesso codice di prima
...     if n in (0,1):
...        return n
...     else:
...        return fib(n - 1) + fib(n - 2)
...
>>> fib(3)
Calling fib(3) ...
Calling fib(2) ...
Calling fib(1) ...
... returning from fib(1) = 1
Calling fib(0) ...
... returning from fib(0) = 0
... returning from fib(2) = 1
... returning from fib(3) = 2
2
>>> fib(1)
1
>>> fib(4)
Calling fib(4) ...
... returning from fib(4) = 3
3
```

4.5.5.3 Decorator con parametri

I decoratori visti sin'ora non si prestano ad essere configurati (ossia ad utilizzare parametri. Il seguente codice

```
@decorator(params)
def func_name():
    ''' Function implementation'''
```

è equivalente a

```
func_name = (decorator(params))(undecorated_implementation)
```

L'interpretazione avviene da sinistra a destra: viene chiamato il decoratore col suo parametro; questo deve ritornare una funzione (`inner`, di sotto) che elabora la funzione passatagli in sede di chiamata (restituendone una versione modificata `wrapper`).
Un esempio di decoratore con parametro (temporizza il codice, il parametro `n` serve per scegliere il numero di esecuzioni della funzione

```
def timeit(n=1):     # la funzione esterna prende in input i parametri del decoratore
    def inner(f):    # l'intermedia è la funzione utilizzata in chiamata (come in precedenza)
        def wrapper(*args, **kwargs):    # la piu interna i parametri della chiamata
            start_time = _time.perf_counter()
            for i in range(n):
                rval = f(*args, **kwargs)
            stop_time = _time.perf_counter()
            elapsed = stop_time - start_time
            function_name = f.__name__
            msg_start = f"{n} calls of" if n > 1 else "Function"
            msg_end = f" (mean: {elapsed/n:.4f} secs)" if n > 1 else "."
            msg = f"{msg_start} {function_name} [{args=} {kwargs=}] " \
                f"took {elapsed:.2f} secs{msg_end}"
            print(msg)
            return rval
        return wrapper
    return inner


# esempio di utilizzo per temporizzare tre esecuzioni ad ogni chiamata
@timeit(n=3)
def test2():
     time.sleep(2)

test2()
```

I decoratori con parametri vanno sempre chiamati con aperta e chiusa direi, anche se si vogliono utilizzare i valori di default

```
@timeit()
def test3:
    pass
```

4.5.5.4 Decoratori già disponibili

Alcuni utili sono

- `functools.cache` memorizza i risultati di una funzione in un dict e controlla alla seconda chiamata che non sia già disponibile (tipo memoize usata sotto)
- ...

4.5.6 Single e multiple dispatch

4.5.6.1 Single dispatch (function overloading)

Le funzioni con single dispatch sono tipo le S3 di R, che stabiliscono come comportarsi (evitando `if`/`elif`) sulla base dell'input (un parametro) fornito. Per definire:

- una *funzione generica* `gen` la si decora con `functools.singledispatch`: dopodiché si definiranno gli handler per tipo di dato decorandoli con `@gen.register`
- un metodo di classe come generico si usa (`functools.singledispatchmethod`) (qui tralasciata)

```
>>> from functools import singledispatch

>>> @singledispatch
... def f(arg, verbose = False):
...     print("Unhandled arg")
...
>>> @f.register
... def _(arg: int, verbose = False):
...     print(arg, "is integer")
...     if verbose:
...         print("f call was verbose too")
...

>>> @f.register
... def _(arg: str, verbose = True):
...     print(arg, "is a string")
...     if verbose:
...         print("f call was verbose too")
...

>>> f(3)                        # tipi conosciuti
3 is integer
>>> f("foo")
foo is a string
f call was verbose too
>>> f("bar", verbose = False)
bar is a string
>>> f({'planet':'earth', 'region':'europe', 'prefix':39}) # tipo non conosciuto da de
Unhandled arg
```

La definizione si può veder scritta anche come segue, dove il tipo di arg è passato a f.register

```
>>> @f.register(list)
... def _(arg, verbose = True):
...      print(arg * 2)
...
>>> f([1,2,3])
[1, 2, 3, 1, 2, 3]
```

Questa definizione è necessaria, utilizzata in stack, se vogliamo registrare uno stesso metodo per dati differenti

```
>>> @f.register(float)
... @f.register(bool)
... def _(arg, verbose = False):
...      print(arg * 2)
...
>>> f(2.5)
5.0
>>> f(True)
2
```

4.5.6.2 Multiple dispatch

Consiste nel programmare una funzione per comportarsi diversamente a seconda dei tipi (di più di un parametro) passati.
Si implementa mediante `multimethod` (dall'omonimo pacchetto) che viene applicato ripetutamente alla generica e ai vari handler specifici.
Un esempio che riproduce il funzionamento di `*` con numeri e stringhe:

```
>>> from multimethod import multimethod

>>> @multimethod
... def test(a, b): # default senza specificare il tipo
...     print("boo")
...
>>> @multimethod
... def test(a: int, b: int): # particolarizzando il tipo
...     return a * b
...
>>> @multimethod
... def test(a: str, b: int):
...     return a * b
...
>>> test(2, 3)
6
>>> test("a", 3)
'aaa'
>>> test("e", "qui?")
boo
```

Un esempio col metodo di una classe: definiamo `do` come l'interfaccia pubblica al multimethod sottostante per gestire i parametri passati in inizializzazione:

```
from dataclasses import dataclass
from multimethod import multimethod

@dataclass
class Test:
    x: int|str
    y: int|str|None = None

    def do(self):
        self._dispatch(self.x, self.y)

    @multimethod
    def _dispatch(self, a, b): # default case
        print("dunno")

    @multimethod
    def _dispatch(self, a: int, b: None):
        print("int x none")

    @multimethod
    def _dispatch(self, a: str, b: None):
        print("str x none")

    @multimethod
    def _dispatch(self, a: int, b: str):
        print("int x str")

    @multimethod
    def _dispatch(self, a: str, b: str):
        print("str x str")


Test(1).do() # int x none
Test("test").do() # str x none
Test(1, "test").do() # int x str
Test("test", "test").do() # str x str
```

4.6 Altri argomenti

4.6.1 Type annotation/type hint

Nel Python:

- ogni cosa è un oggetto/variabile
- ogni oggetto/variabile è l'istanza di (almeno) una classe (e di fatto un puntatore a essa)
- formalmente ogni classe (builtin o creata mediante `class`), è un oggetto instanza della classe `type`. Sostanzialmente spesso e volentieri `class` e `type` sono usati come sinonimi anche se non lo sono

- il tipo si può investigare mediante l'omonima funzione `type`

```
>>> type(0)   # passo una istanza
<class 'int'>
>>> type(int) # passo la relativa classe (che è un type)
<class 'type'>
```

- l'interprete non ci impedisce di passare dati del tipo erroneo ad una funzione che si aspetta altro
- possiamo però fornire informazioni sul tipo in sede di
  - definizione di variabili
  - definizione di parametri di funzioni
  - valori di ritorno di funzioni

  Queste informazioni verranno *ignorate* dall'interprete (Python rimane un linguaggio dinamico con variabili che hanno un tipo *non* stabilito a priori/che può variare nel corso dell'esecuzione del programma) ma potranno essere segnalate da strumenti di analisi codice per potenziali bug (se ad esempio passiamo ad una funzione oggetti di tipo)

Per maggiori info su type hint: `https://docs.python.org/3/library/typing.html`.

4.6.1.1 Sintassi

```
>>> # formati per variabile: definizione e inizializzazione in unico colpo
>>> x: int = 4

>>> # spezzato
>>> x: int
>>> x = "4"  # qua viene dato errore nei programmi giusti, non nell'interprete

>>> # funzioni: type hint di parametri e tipo restituito
>>> # z è un parametro opzionale (può essere int al massimo) di default a None
>>> def repeat(x: str, y: int = 2, z: int | None = None) -> None:
...     if z is None:
...         print(x * y)
...     else:
...         print(x * y * z)
...
>>> repeat(x = 'foo')
foofoo
```

4.6.1.2 Checking

`ty` è un programma che controlla i tipi (altri `mypy` e `pyright`):

```
# installazione
uv add --dev ty

# check di un file
```

```
uv run ty check file.py

# controllo di directory
uv run ty check

# Nel mio emacs, in un file python
C-c C-v [Enter]
```

Se ad esempio programmiamo una funzione del genere

```
def main() -> bool:
   return "True"
```

ci viene segnalato

```
error[invalid-return-type]: Return type does not match returned value
 --> ty_test.py:1:15
  |
1 | def main() -> bool:
  |               ---- Expected `bool` because of return type
2 |     return "True"
  |            ^^^^^^ expected `bool`, found `Literal["True"]`
  |
```

4.6.1.3 Tipi utilizzabili

Per i tipi base

- usare direttamente `str`, `int`, `float`, `bool`, ma anche `None` (per indicare che la funzione non ritorna nulla).
- per i tipi compositi: sotto alcuni esempi di assegnazione corretta

```
# --------------
# Tipi specifici
# --------------
x: list[str] = ["foo", "bar"]    # lista di sole stringhe
x: set[int] = {6, 7}             # insieme di interi
x: dict[str, float] = {"field": 2.0}  #  dict: fornire tipo di key e value
# tuple dimensione fissa specifica il tipo di ogni elemento
x: tuple[int, str, float] = (3, "yes", 7.5)
# tuple dimensione variabile: si usa un tipo e l'ellissi
x: tuple[int, ...] = (1, 2, 3)


# --------------
# Tipi generici
# --------------
from typing import Iterable, Mapping, MutableMapping

x: Iterable[str] = ...        # iterabile (lista,tuple, set, altro) di stringhe
x: Mapping[int, str] = {3: 'yes', 4: 'no'} # dict-like non mutabili (con metodo __get
                              # se si da x[5] = 'foo' il checker si lamenta
```

```
x: MutableMapping[int, str] = {3: 'yes', 4: 'no'} # dict-like mutabili (con metodo __setitem__)
                             # x[5] = 'foo' # qui mypy non si lamenta

# ---------------------------
# tipi molteplici/alternativi
# ---------------------------
x: list[int | str] = [3, 5, "test", "fun"]

# -----------------------------------------------------
# Dati che possono essere di un tipo specificato o None
# -----------------------------------------------------
# Es sotto, dopo la definizione, test(True) fa lamentare il checker

from typing import Optional
def test(x: Optional[str] = None) -> bool:
    return True
```

4.6.1.4 Creazione di alias

La digitazione di tipi complessi più volte può essere evitata mediante la creazione di alias, fatti semplicemente mediante assegnazione preceduta da type. Ad esempio

```
type Vector = list[float]

def scale(scalar: float, vector: Vector) -> Vector:
    return [scalar * num for num in vector]

# utile anche per rendere i tipi più leggibili/compatti/riutilizzabili
from collections.abc import Sequence
type ConnectionOptions = dict[str, str]
type Address = tuple[str, int]
type Server = tuple[Address, ConnectionOptions]
def broadcast_message(message: str, servers: Sequence[Server]) -> None:
    ...
```

4.6.1.5 Tipo `Callable` per funzioni

Per passare funzioni ad altre funzioni, se vogliamo descriverne il tipo, dobbiamo utilizzare `Callable`

```
from typing import Callable

def repeat(x: str, y: int = 2) -> None:
     print(x * y)

# variabile:  x può essere una funzione di tipo def xx(str, int): -> None
x: Callable[[str, int], None] = repeat
```

Per funzioni generatrici che restituiscono un iteratore di int si può fare così

```
def gen(n: int) -> Iterator[int]:
    i = 0
```

```
    while i < n:
        yield i
        i += 1
```

4.6.2 atexit

Similmente a `on.exit` di R possiamo registrare funzioni che vengano eseguite prima dell'uscita da una funzione mediante il decoratore `atexit.register`

```
import atexit

# General before exit actions in exit_from_main
@atexit.register
def before_exit_from_main():
    print("Before exiting from main")


def my_fun():
    # function specific act
    @atexit.register
    def before_exit_from_function():
        print("Before exiting from my_fun")

    print("Within my_fun")


print("In main")
my_fun()
```

Che una volta fatto eseguire farà

```
l@ambrogio:~/src/tests/python$ python atexit.py
In main
Within my_fun
Before exiting from my_fun
Before exiting from main
```

Se

- necessario registrare una chiamata ad una funzione con parametri utilizzare esplicitamente `atexit.register`

  ```
  def goodbye(name, adjective):
      print(f'Goodbye {name}, it was {adjective} to meet you.')

  atexit.register(goodbye, 'Donny', 'nice')
  ```

- si registrano più funzioni (es nell'ordine A, B, and C), `atexit` le eseguirà in ordine inverso rispetto alla registrazione (C, B, A).

- necessario togliere una registrazione utilizzare `atexit.unregister`

Capitolo 5

Input/Output

Contents

5.1 Accesso al filesystem 37
5.1.1 Ottenere/cambiare directory di lavoro 37
5.1.2 Listing di directory, glob files 37
5.1.3 Creazione/rimozione di file/directory 37
5.1.4 File e directory temporanei 37
5.1.5 Paths e metodi utili 37
5.2 Salvataggio/importazione file 37
5.2.1 Testo semplice 37
5.2.2 `csv`/`tsv` 38
5.2.3 JSON 38
5.2.4 pickle 39
5.3 File di configurazione 39
5.4 Esecuzione di programmi esterni 40
5.4.1 `run` 40
5.4.2 `Popen` 40
5.5 Parsing opzioni da linea di comando 41
5.5.1 `sys.argc` e `sys.argv` 41
5.5.2 Modulo `argparse` 41
5.6 Accesso variabili d'ambiente 42


```
>>> import polars as pl
>>> pl.Config.set_ascii_tables(True) # altrimenti la stampa di df da problemi
<class 'polars.config.Config'>
```

5.1 Accesso al filesystem

Oggigiorno si usa `pathlib`, per alcune cose marginali rimasto `os` e `shutil`.

```
>>> from pathlib import Path
>>> import os
>>> import shutil
```

5.1.1 Ottenere/cambiare directory di lavoro

```
>>> # ottenere la directory
>>> os.getcwd()
'/home/l/.sintesi/sintesi_cs'
>>> Path.cwd()
PosixPath('/home/l/.sintesi/sintesi_cs')

>>> # cambio directory
>>> os.chdir('/tmp')
```

5.1.2 Listing di directory, glob files

```
>>> d = Path("/home/l/.sintesi")

>>> # listing di directory
>>> list(d.iterdir()) # paths del contenuto (iteratore)
[PosixPath('/home/l/.sintesi/sintesi_cs'), PosixPath('/home/l/.sintesi/sintesi_biosta

>>> # glob/matching nome file
>>> pdf1 = sorted(d.glob("*/*.pdf"))  # cerca nella sottodirectory figlie
>>> pdf2 = sorted(d.glob("**/*.pdf")) # cerca in tutto l'albero
>>> pdf3 = sorted(d.rglob("*.pdf"))   # stessa cosa di sopra ma più compatta
```

5.1.3 Creazione/rimozione di file/directory

```
>>> # Creazione path/file
>>> d = Path('/tmp/barbaz/whahaa/yeah')

>>> # Creazione directory
>>> d.mkdir(parents = True) # parents se vogliamo creare tutto l'albero

>>> # creazione in maniera safe
>>> try:
...     d.mkdir(parents = True)
... except FileExistsError:
...     pass
...

>>> # Rimozione directory con tutto il contenuto
>>> rm = Path('/tmp/barbaz')
>>> # rm.rmdir() # erore: cartella deve essere vuota...
>>> shutil.rmtree(rm) # vedere dopo per shutil

>>> # Creazione rimozione di file
>>> f = Path('/tmp/asdomar.txt')
>>> f.touch()
>>> f.unlink()
```

5.1.4 File e directory temporanei

```
>>> from tempfile import TemporaryFile
>>> from tempfile import NamedTemporaryFile
>>> from tempfile import TemporaryDirectory


>>> # File con path (è quello che si vuole molto probabilmente)
>>> # ---------------------------------------------------------
>>> with NamedTemporaryFile() as f:
...     print(f.file)  # file object
...     print(f.name)  # path
...     # fai cose con f
...
<_io.BufferedRandom name='/tmp/tmpxtfl4k5a'>
/tmp/tmpxtfl4k5a
>>> # qui il file non c'è più

>>> # File senza nome
>>> # ---------------
>>> with TemporaryFile('w+t') as f:
...    f.write("Hello World\n")
...    f.write("Testing\n")
...    # ritorna ad inizio file e leggi quanto scritto
...    f.seek(0)
...    data = f.read()
...
12
8
0
>>> # qui il file non c'è più ma ..
>>> data
'Hello World\nTesting\n'

>>> # Directory
>>> # ---------
>>> with TemporaryDirectory() as dirname:
...     print("dirname is: ", dirname)
...     ## use the directory
...
dirname is:  /tmp/tmpznnvp138
>>> # qui directory e suo contenuto non c'è più
```

Il mode è `w+t` per il testo o `w+b` per binario: si è posto 'w' per permettere sia lettura che scrittura che è utile qua, dato che chiudere il file implicherebbe distruggerlo.

5.1.5 Paths e metodi utili

```
>>> f = Path("/usr/bin/python")
>>> d = Path("~/.sintesi")
```

```
>>> f2 = Path("/etc/apt/sources.list")
>>> f3 = Path("Makefile")
>>> subdir = "sintesi_cs"

>>> ## questi path possono esser usati dove è accettato un os.PathLike
>>> ## alternativamente per ottenere la rappresentazione in stringa

>>> str(d)
'~/.sintesi'
>>> str(f)
'/usr/bin/python'

>>> # sostituire la tilde per l'amor del cielo in paths
>>> de = d.expanduser()

>>> # test esistenza
>>> f.exists()
True
>>> d.exists() # attenzione a ~
False
>>> de.exists()
True

>>> # test tipologia
>>> f.is_file()
True
>>> d.is_dir() # attenzione a ~
False
>>> de.is_dir()
True

>>> # links
>>> f.is_symlink()
True
>>> f.readlink()  # follow just one redirection
PosixPath('python3')
>>> os.path.realpath(f) # follow all redirections
'/usr/bin/python3.13'

>>> # estrarre parti di un path
>>> f2.parent  # directory del file
PosixPath('/etc/apt')
>>> f2.name  # nome file
'sources.list'
>>> f2.stem  # nome file senza estensione
'sources'
>>> f2.suffix # estensione - per tar.gz usare suffixes
'.list'
>>> Path(f2.parent) / f2.stem # fpath_sans_ext (only one extension)
PosixPath('/etc/apt/sources')
```

```
>>> # controllare l'estensione (mediante glob)
>>> f2.match("*.list")
True

>>> # creazione di path mediante modifica di esistente
>>> d / subdir # concatenazione, alternativamente d.joinpath(subdir)
PosixPath('~/.sintesi/sintesi_cs')
>>> f.with_name("foo.list") # nome cambiato
PosixPath('/usr/bin/foo.list')
>>> f.with_stem("ssd")      # stem cambiato, stessa estensione
PosixPath('/usr/bin/ssd')
>>> f.with_suffix(".deb")   # estensione cambiata, stesso stem
PosixPath('/usr/bin/python.deb')
>>> f2.relative_to("/etc")  # path relativo, escludendo quanto passato
PosixPath('apt/sources.list')
>>> f3.absolute() # crea l'assoluto a partire da un relativo
PosixPath('/tmp/Makefile')
>>> f3.resolve()  # stessa cosa ma fixa anche symlink
PosixPath('/tmp/Makefile')
```

5.2 Salvataggio/importazione file

5.2.1 Testo semplice

La funzione `open` prende in input un path e un mode (`r` per la lettura, `w` per la scrittura, `a` per l'appending) restituisce un file object, quando si è finito di operare chiamare il metodo `close`:

```
f = open(file = '/tmp/test.txt', mode = 'r')
f.operazioni
f.close()
```

o meglio/più compattamente

```
with open(file = '/tmp/test.txt', mode = 'r') as f:
    f.operazioni
```

in questo secondo caso il file viene chiuso automaticamente all'uscita dal blocco anche in caso di eccezioni (warning, error).
È possibile anche utilizzare oggetti `pathlib.Path`, che hanno il metodo `open`:

```
from pathlib import Path
p = Path("/etc/motd")
with p.open() as f:
    lines = f.readlines()
print(lines)
```

5.2.1.1 Lettura

Alcuni metodi:

- `read` legge tutto il file e lo restituisce come stringa. Quando si raggiunge la fine del file, una chiamata successiva a read restituisce una stringa vuota

```
with open(file = '/tmp/test.txt', mode = 'r') as f:
    whole_file = f.read()
```

- `readline` legge una singola riga; anche qui quando si giunge alla fine del file restituisce una stringa vuota. `readlines` legge tutte le righe e le restituisce come lista

```
with open(file = '/tmp/test.txt', mode = 'r') as f:
    lines = f.readlines()
```

- se si vuole processare linea per linea si può ciclare su f come segue, dato che il file è un iteratore sulle righe

```
with open(file = '/tmp/test.txt', mode = 'r') as f:
    for line in f:
        # process line, es per stampa a video
        print(line)
```

5.2.1.2 Scrittura

Per scrittura su file si può usare `write` o `writelines` a seconda che l'input sia una stringa o una lista di stringhe; (alternativamente `print`

```
# scrittura di stringa (o """...""")
with open(file = '/tmp/test.txt', mode = 'w') as f:
    f.write("test") # write restituisce il numero di caratteri scritti
    # print("test", file = f) # alternativamente

# scrittura di una lista di linee
L = ["line1", "line2"]
with open(file = '/tmp/test.txt', mode = 'w') as f:
    f.writelines(L)
```

5.2.2 csv/tsv

Il modulo `csv` contiene le funzioni `reader` e `writer` per leggere formati testuali tabulari di vario tipo (anche separati da tab).
L'apertura/chiusura del file avviene come qualsiasi file di testo, come visto prima.

5.2.2.1 Lettura

Per importare un file e processarlo una riga alla volta `csv.DictReader` restituisce ciascuna riga come dict (altrimenti se può bastare utilizzare `csv.reader` che ritorna una lista)

```
# CSV esempio
#
# dir,repo,link   # attenzione a non lasciare spazi
# ~/.av/, https://lbraglia@bitbucket.org/lbraglia,
# ~/.configs/, https://lbraglia@github.com/lbraglia/.configs, ~/configs/
# ~/.dnd/, https://lbraglia@github.com/lbraglia/.dnd, ~/dnd
```

Python	JSON
`dict`	object
`list`, `tuple`	array
`str`	string
`int`, `float`	number
`True`	true
`False`	false
`None`	null

Tabella 5.1: Encoding JSON

```
import csv
with open('setup.csv') as csvfile:
    reader = csv.DictReader(csvfile, quoting=csv.QUOTE_NONNUMERIC)
    for row in reader:
        dir = row['dir']    # attenzione a non lasciare spazi
        repo = row['repo']
        link = row['link']
        ...
```

5.2.2.2 Scrittura

Quello che si fa è creare un writer in cui si impostano le formattazioni di delimitatore, carattere di quoting e così via, per poi utilizzarlo per scrivere le righe. Un esempio con un dataset separato da tab

```
import csv
with open(file = '/tmp/dataset.tab', mode = 'w') as f:
    dataset = csv.writer(f,
                         delimiter = '\t',
                         quotechar = '"',
                         quoting = csv.QUOTE_NONNUMERIC)
    header = ['x', 'y', 'z']
    data   = [ 1,   2,  'a']
    dataset.writerow(header)
    dataset.writerow(data)
```

5.2.3 JSON

L'omonimo modulo `json` di python permette l'interfaccia. Le funzioni principali sono `dump` e `load` per scrivere su file, o `dumps` e `loads` (dump-s) per scrivere e leggere stringhe. Qua vediamo l'interfaccia con i file.

5.2.3.1 Scrittura

L'encoding dei tipi base (e loro combinazioni) segue tabella 5.1

```
>>> import json

>>> data = {
...    'name' : ['ACME', "FOO", "BAR"],
```

JSON	Python
object	`dict`
array	`list`
string	`str`
number (int)	`int`
number (real)	`float`
true	`True`
false	`False`
null	`None`

Tabella 5.2: Decoder JSON

```
...   'shares' : [100, 200, 300],
...   'price' : (321, 9, 8912)
... }

>>> with open("/tmp/data.json", "w") as f:
...     json.dump(data, f)
...
```

5.2.3.2 Lettura

Il decoding dei tipi di base segue tabella 5.2.

```
>>> with open("/tmp/data.json", "r") as f:
...     data2 = json.load(f)
...
>>> print(data2)
{'name': ['ACME', 'FOO', 'BAR'], 'shares': [100, 200, 300], 'price': [321, 9, 8912]}
```

5.2.3.3 Pretty printing

Utilizzare `json.dumps` con l'opzione indent

```
pretty_json = json.dumps(json_dict, indent=4)
print(pretty_json)
```

5.2.3.4 Formati custom

Per tipi di dati più elaborati (es classi custom, numeri complessi) vedere alcuni esempi qui: `https://realpython.com/python-json/`

5.2.4 pickle

Può salvare in binario qualsiasi struttura dati. Un esempio con un `dict` contenenti dati di varia natura a seguire.

```
>>> import pickle
>>> import pylbmisc as lb
>>> from pprint import pp

>>> # data to be saved
```

```
>>> a_dataset = lb.datasets.load().to_pandas()
>>> original = {i: a_dataset for i in range(2)}
>>> original["asd"] = 34
>>> original["qwe"] = [1, 2, 3]
>>> original["fun"] = lb.datasets.load

>>> # # display
>>> # pp(original)

>>> # save
>>> fpath = "/tmp/pickle_test.pkl"
>>> with open(fpath, "ab") as f:
...     pickle.dump(original, f)
...
>>> # load data
>>> with open(fpath, "rb") as f:
...     the_copy = pickle.load(f)
...
>>> # # display the copy
>>> # pp(the_copy)
```

5.3 File di configurazione

Per

- la lettura/scrittura di file `.ini` utilizzare la libreria `configparser`
- la lettura (solamente, per ora) di file `toml` (evoluzione degli `.ini`) è disponibile la libreria `tomllib`

```
import configparser

# Scrittura
data = {
    'customer': "ajeje",
    'acronym': "brazorv",
    'title': "un titolo",
    'created': "2020-01-01",
    'url': "lbraglia.altervista.org"
}
fpath = "/tmp/asd.ini"
configs = configparser.ConfigParser()
configs["project"] = data # project è la sezione del file .ini
with open(fpath, 'w') as f:
    configs.write(f)

# Lettura
configs = configparser.ConfigParser()
configs.read(fpath)
print(configs["project"]["url"])
```

```
# modifica
configs["project"]["url"] = "lbraglia.github.io"
```

5.4 Esecuzione di programmi esterni

Si usa la library `subprocess`

5.4.1 run

Se si vuole **attendere il termine del processo** utilizzare `run` (funzione di convenienza consigliata). Si specificano eventuali comandi composti da più token come lista

```
>>> import subprocess

>>> # Comandi semplici (prendi in input da comando esterno)
>>> # ------------------------------------------------------

>>> # eseguire un comando senza prendere risultati in input
>>> # in caso di parametri del comando dare una lista ["ls", "-l"]
>>> cmd = "ls -l -a -r -t".split(" ")
>>> rval = subprocess.run(cmd)  # risultati stampati a video
>>> rval  # ma non disponibili nell'output
CompletedProcess(args=['ls', '-l', '-a', '-r', '-t'], returncode=0)

>>> # salvando risultati (e standard error) in formato testuale
>>> child = subprocess.run(["ls"], capture_output=True, text=True)
>>> child.returncode
0
>>> child.stdout.split("\n")
['asd.ini', 'babel-GU5wYb', 'babel-stable-399', 'babel-stable-813', 'babel-vHYXE9', '
>>> child.stderr
''
```

5.4.2 Popen

Utilizzare **altrimenti** `Popen`, classe più generale che rappresenta un processo esterno quando la si istanzia si sta lanciando un progrmama esterno.

```
cmd = subprocess.Popen(args, stdin=None, stdout=None, stderr=None,
                       shell=False)
```

dove

- `args`: lista del comando da eseguire;
- `stdin`, `stdout`, `stderr`: come gestire input, output, and error del processo esterno. Di default `None` ereditano dal processo genitore. Utilizzare `subprocess.PIPE` per mandarli in pipe. In particolare `stdin=subprocess.PIPE` permette di inviare dati in input, mentre `stdout=subprocess.PIPE` cattura l'output;

- `text=True` fa si che input/output sian trattati come stringa, non bytes;
- `shell`: se settato a `True`, il comando è eseguito attraverso una shell (utile per eseguire comandi di shell complessi ma meno sicuro);
- `cwd`: setta la cwd del processo prima di eseguirlo
- `env`: setta environment variables

Di default *non attende* il ritorno del figlio; si può attendere se chiamiamo il metodo `comunicate`.
Alcuni esempi per replicare `subprocess.run` di sopra:

- prendere in input l'output/error fornito dal programma esterno

```
>>> ls = subprocess.Popen(["ls"],
...                       stdout=subprocess.PIPE,
...                       stderr=subprocess.PIPE,
...                       text=True)
>>> output, err = ls.communicate()
>>> output
'asd.ini\nbabel-GU5wYb\nbabel-stable-399\nbabel-stable-813\nbabel-vHYXE9\ncorrmatrix.eps\n

>>> ls = subprocess.Popen(["ls", "/nonexistentdir"],
...                       stdout=subprocess.PIPE,
...                       stderr=subprocess.PIPE,
...                       text=True)
>>> output, err = ls.communicate()
>>> err
"ls: impossibile accedere a '/nonexistentdir': File o directory non esistente\n"
```

- inviare input ad un programma esterno a python e riprenderne l'output

```
>>> text_input = "first line\nsecond line"
>>> nl = subprocess.Popen(["nl"],
...                       stdin=subprocess.PIPE,
...                       stdout=subprocess.PIPE,
...                       text=True)
>>> output, err = nl.communicate(input=text_input)
>>> output  # le linee a cui è stato aggiunto il numerino da nl
'     1\tfirst line\n     2\tsecond line\n'
```

- concatenare programmi esterni, es `ls | nl | python` e prendere in input il risultato

```
>>> # ls non prende in input da nulla e butta nel pipe
>>> ls = subprocess.Popen(["ls"], stdout=subprocess.PIPE, text=True)
>>> # nl prende in input da ls e butta nel pipe
>>> nl = subprocess.Popen(["nl"], stdin=ls.stdout, stdout=subprocess.PIPE, text=True)
>>> ls.stdout.close() # di a ls di terminare
>>> output, err = nl.communicate()  # e ad nl di fare il suo
>>> output.split("\n")
['     1\tasd.ini', '     2\tbabel-GU5wYb', '     3\tbabel-stable-399', '     4\tbabel-sta
```

- dare input con python, concatenare programmi esterni e prendere il risultato. Ad esempio `python lista parole | sort | nl | python`

```
>>> # ls non prende in input da nulla e butta nel pipe
>>> sort = subprocess.Popen(["sort"],
...                         stdin=subprocess.PIPE,
...                         stdout=subprocess.PIPE,
...                         text=True)
>>> nl = subprocess.Popen(["nl"], stdin=sort.stdout, stdout=subprocess.PIPE,
...                       text=True)
>>> sort.stdin.write("a\nx\nb\nz\nc")
9
>>> sort.stdin.close()
>>> output, err = nl.communicate()
>>> output.split("\n")
['     1\ta', '     2\tb', '     3\tc', '     4\tx', '     5\tz', '']
```

5.5 Parsing opzioni da linea di comando

Mi sono fatto anche `pylbmisc.utils.argparser` (basato su argparse) ma per la roba più rapida sotto basta e avanza.

5.5.1 `sys.argc` e `sys.argv`

Nel template

```
#!/usr/bin/env python3
import sys

print(sys.argv)
```

Per la chiamata senza parametri

```
l@m740n:~/pytests$ ./argparse_sysargv
['./argparse_sysargv']

l@m740n:~/pytests$ python argparse_sysargv
['argparse_sysargv']
```

Per la chiamata con parametri

```
l@m740n:~/pytests$ ./argparse_sysargv asd foo bar baz
['./argparse_sysargv', 'asd', 'foo', 'bar', 'baz']

l@m740n:~/pytests$ python argparse_sysargv asd foo bar baz
['argparse_sysargv', 'asd', 'foo', 'bar', 'baz']
```

5.5.2 Modulo `argparse`

Una dimostrazione veloce, per approfondimenti vedere la documentazione. Sia il template

```
#!/usr/bin/env python3
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("infile")
parser.add_argument("outfile")
args = parser.parse_args()
print(args)
print(args.infile)
print(args.outfile)
```

Per l'invocazione senza argomenti da errore (sono attesi)

```
l@m740n:~/pytests$ ./argparse_tests
usage: argparse_tests [-h] infile outfile
argparse_tests: error: the following arguments are required: infile, outfile

l@m740n:~/pytests$ python argparse_tests
usage: argparse_tests [-h] infile outfile
argparse_tests: error: the following arguments are required: infile, outfile
```

Per l'invocazione con parametri

```
l@m740n:~/pytests$ ./argparse_tests asd foo
Namespace(infile='asd', outfile='foo')
asd
foo

l@m740n:~/pytests$ python argparse_tests asd foo
Namespace(infile='asd', outfile='foo')
asd
foo
```

5.6 Accesso variabili d'ambiente

Si usa `os.environ`. Se adottiamo un template

```
#!/usr/bin/python

import os

# print(os.environ) # un po too much
print(os.environ["SHELL"])
print(os.environ["BROWSER"])

# roba settata al volo
p1 = int(os.environ["PARAM1"])
p2 = int(os.environ["PARAM2"])
print(p1 + p2)
```

Si ha

```
l@m740n:~/pytests$ export PARAM1=1; export PARAM2=2 ;  python envvars.py
/bin/bash
librewolf
3
```

Capitolo 6

Object Oriented Programming

Contents

6.1 Introduzione 43
6.1.1 Fasi di sviluppo 43
6.1.2 Classi, oggetti etc 43
6.1.3 Quando utilizzare programmazione ad oggetti 44
6.2 Classi 44
6.2.1 Definizione, istanziazione e scoping 44
6.2.2 Metodi 45
6.2.3 Condivisione dati 46
6.2.4 Data hiding 46
6.3 Ereditarietà 46
6.3.1 Ereditarietà singola 46
6.3.2 Ereditarietà multipla 47
6.4 Amenità/utilità per le classi 47
6.4.1 Funzioni builtin utili 47
6.4.2 Decoratori utili per metodi 47
6.4.3 Type hint 49
6.5 Polimorfismo 49
6.6 Abstract Base Class 50
6.6.1 Definizione 50
6.6.2 Creazione di classi conformi ad ABC del linguaggio . 50
6.7 Dunder methods e applicazioni 50
6.7.1 Iteratori 51
6.7.2 Context Managers 54
6.8 Classi builtin notevoli 55
6.8.1 defaultdict 55
6.8.2 `Counter` 55
6.8.3 Enum & Flag 55
6.8.4 Named tuples (`collections.namedtuple` e `typing.NamedTuple`) 56

6.8.5 `dataclass` 56
6.8.6 Queues 58
6.9 Altri argomenti 59
6.9.1 Estensione di classi base 59
6.9.2 Manager Object 59

La programmazione orientata agli oggetti risolve problemi mediante l'interazione tra oggetti.

6.1 Introduzione

6.1.1 Fasi di sviluppo

Distinguiamo le fasi, idealmente sequenziali di:

- **object oriented analysis**: processo di guardare/descrivere un problema e identificarne i requisiti (oggetti nonché le interazioni tra loro).

  Example 6.1.1. Nella realizzazione di un sito i vistatori (in italic le azioni, in grassetto gli oggetti):

  - *rivedere* la nostra **history**
  - *apply* for a **job**
  - *browse*, *compare* and *order* **products**

- **object oriented design**: conversione dei requirements in una specifica di implementazione (implementabile mediante qualsiasi linguaggio di programmazione ad oggetti)

- **object oriented programming** conversione del design in un programma funzionante

6.1.2 Classi, oggetti etc

Le

- classi sono una descrizione di un oggetto, con le sue proprietà (dati) e metodi (funzioni/attività svolte)

- classi di maggiore complessità posson essere create mediante

  - *ereditarietà*: quando una classe eredita proprietà e metodi da un'altra classe più generale (e li specializza facendone l'*override* o ne aggiunge altri). L'ereditarietà si presta per modellare classi/oggetti che tra loro sono in relazione "è uno/una".
  - *composizione* (quando un oggetto/classe è una collezione di altri oggetti/classi che ne rappresentano le componenti). La composizione si presta per modellare classi/oggetti che tra loro sono in relazione "ha uno/una".

6.1.3 Quando utilizzare programmazione ad oggetti

Nella modellazione di un problema se abbiamo a che fare con entità che hanno bisogno *sia di dati che di comportamenti* allora probabilmente è necessaria una classe. Se invece

- sono/necessitano di solo dati può bastare un tipo base (o una dataclass)
- si sta parlando di un comportamento/algoritmo, dovrebbe bastare una funzione

Uno stesso problema poi può essere modellato come serie di funzioni o mediante l'utilizzo di classi. Nella scelta tra le due alternative considerare:

- la leggibilità delle *api* all'atto dell'utilizzo (le funzioni, con nomi di verbi, possono essere più dirette in questo)
- la chiarezza del codice nelle due *implementazioni* (es classi sono abbastanza self-documenting)

In merito alle relazioni tra concetti se abbiamo bisogno di modellare ereditarietà abbiamo bisogno necessariamente di classi.

6.2 Classi

L'implementazione della oop in Python si basa su trick di scope (cfr sezione 4.3). La definizione di una classe pone semplicemente un altro namespace all'interno di quello nel quale ci si trova.

6.2.1 Definizione, istanziazione e scoping

6.2.1.1 Definizione

La definizione di una classe fa uso di:

- la keyword `class` associato ad un nome di classe (ed eventualmente tra parentesi la classe dalla quale si eredita, in assenza viene posto `object` di default);
- una stringa di documentazione opzionale: porre qui
  - la *documentazione generale della classe*
  - la descrizione dei parametri della init
  - esempi di utilizzo tipici della classe

  Porre invece nelle docstring dei metodi, parametri ed esempi ad essi riferiti
- vari `statement` di assegnazione di variabili e `def` di funzioni (i *metodi* della classe)

```
class NomeClasse:
    """
    Documentation
    """
```

```
    <statement-1>
    .
    .
    .
    <statement-N>
```

Una volta valutata la definizione viene creato un *oggetto classe*, ossia un *namespace* contenuto in quello dove è stato definito, che supporta:

1. riferirsi ai suoi attributi in lettura o scrittura (nel Python una classe può esser modificata);
2. creare istanze/oggetti

```
>>> b = "ajeje"

>>> # definizione di classe

>>> class firstClass:
...     """La mia prima classe"""
...     # attributi della classe condivisi tra istanze
...     a = 0
...     b = "miao"
...     # metodi della classe
...     def get_a(self):
...         return self.a
...     def set_a(self, x):
...         self.a = x
...     def test_scoping(self):
...         print(b)
...

>>> ## accesso ad attributi in lettura/scrittura
>>> firstClass.a = 1          ## modifica di un attributo di una classe, volendo
>>> print(firstClass.__doc__) ## stampa la docstring della classe
La mia prima classe
```

6.2.1.2 Istanziazione e scoping

La creazione di oggetti appartenenti alla classe si fa usando il nome della classe come se fosse una funzione.
In merito allo **scoping**, una volta istanziato un oggetto, la ricerca dei nomi entro una classe (a parte le variabili locali ad un metodo) continua nel modulo dove la classe è definita (sia esso `__main__` o importato).

```
>>> ## Istanziazione e creazione di oggetti
>>> x = firstClass()  ## istanziamento usando il costuttore di default
>>> x.a               ## accesso ai dati della classe
1
>>> x.get_a()
1
>>> x.set_a(3)
```

```
>>> x.get_a()
3

>>> ## Scoping rules: sotto stampa "ajeje", non "miao", per il quale avremmo
>>> ## dovuto usare self.b in test_scoping
>>> x.test_scoping()
ajeje
```

6.2.2 Metodi

6.2.2.1 Definizione e chiamata

I metodi di una classe presentano una definizione particolare:

- il primo argomento, obbligatorio, serve nel codice della definizione di funzione per riferirsi all'oggetto istanziato: come nome `self` è arbitrario ma standard/preferibile;
- gli altri sono parametri normali da passare in chiamata del metodo

Internamente, chiamare:

- `object.function()` è equivalente a chiamare `Classe.function(object)`
- `object.function(x, y, z)` equivale a `Classe.function(object, x, y, z)`

6.2.2.2 Inizializzatore custom

In Python vi è sia un costruttore `__new__` che un inizializzatore `__init__` ma di solito si usa solo il secondo.
Se si desidera specificare un inizializzatore diverso da quello di default (che non assegna nulla all'oggetto istanziato), si definisce una funzione di nome `__init__` che si occupa di inizializzare valori forniti dall'utente in sede di istanziazione:

```
>>> class Complex:
...     ## inizializzatore con valori di default
...     def __init__(self, realpart = 0, imagpart = 0):
...         self.r = realpart
...         self.i = imagpart
...     def value(self):
...         return('' + str(self.r)+ '+' + str(self.i) + 'i')
...
>>> x = Complex()
>>> x.value()
'0+0i'
>>> y = Complex(1, 2)
>>> y.value()
'1+2i'
```

6.2.3 Condivisione dati

I dati di una classe iniziano ad esistere dal momento in cui vengon assegnati, sia ciò in definizione della classe o usando un metodo nell'oggetto istanza. Pertanto:

- argomenti **inizializzati nella definizione della classe** sono *comuni* a tutti gli oggetti generati;
- argomenti **inizializzati mediante un metodo** sono *caratteristici* dell'oggetto istanziato.

```
>>> class Dog:
...     kind = 'canine'         # class variable shared by all instances
...     def __init__(self, name):
...         self.name = name    # instance variable unique to each instance
...
>>> f = Dog('Fido')
>>> b = Dog('Buddy')
>>> f.kind                  # shared by all dogs
'canine'
>>> b.kind                  # shared by all dogs
'canine'
>>> f.name                  # unique to fido
'Fido'
>>> b.name                  # unique to buddy
'Buddy'
```

Attenzione a quando come *dato di classe vi è un tipo mutabile*: se metodi modificano il dato comune (e non è spesso quello che si vuole) questo sarà condiviso tra le istanze.

```
>>> # Versione erronea: i tricks son condiviso da tutti i cani
>>> class Dog:
...     tricks = []
...     def __init__(self, name):
...         self.name = name
...     def add_trick(self, trick):
...         self.tricks.append(trick)
...
>>> fido = Dog('Fido')
>>> buddy = Dog('Buddy')
>>> fido.add_trick('roll over')
>>> buddy.add_trick('play dead')
>>> fido.tricks
['roll over', 'play dead']
>>> # peraltro la versione aggiornata può essere acceduta anche dalla classe
>>> Dog.tricks
['roll over', 'play dead']

>>> # Versione corretta: i tricks sono creati nell'inizializzatore
>>> # e specifici per la singola istanza
>>> class Dog:
...     def __init__(self, name):
...         self.name = name
...         self.tricks = []    # ok
...     def add_trick(self, trick):
...         self.tricks.append(trick)
```

```
...
>>> fido = Dog('Fido')
>>> buddy = Dog('Buddy')
>>> fido.add_trick('roll over')
>>> buddy.add_trick('play dead')
>>> fido.tricks
['roll over']
```

Se invece viene cambiato un dato immodificabile questa modifica avrà effetto solo per il singolo oggetto/istanza

```
>>> class Test:
...     value = "default"
...     def change(self):
...         self.value = "changed"
...

>>> x = Test()
>>> y = Test()
>>> x.change()

>>> x.value
'changed'
>>> y.value
'default'
>>> Test.value
'default'
```

6.2.4 Data hiding

Nel Python non vi è un concetto di *data hiding*; di un oggetto istanziato si può accedere ai valori/funzioni senza problemi:

- se si desidera **celare un elemento** gli si può dare un nome che inizi con "_" (per convenzione, considerato come "privato", da non utilizzare direttamente)
- per **incrementare la celatura**, dare nome che inizia con __: nell'oggetto istanziato verrà preceduto dal nome della classe ed underscore (*name mangling*):

  ```
  >>> class Test:
  ...     __name = "default"
  ...     def print_name(self):
  ...         # definition/programming access is simple as usual
  ...         print(self.__name)
  ...

  >>> x = Test()

  >>> # direct access is difficult
  >>> x._Test__name
  ```

```
'default'
>>> Test._Test__name
'default'

>>> # programming access
>>> x.print_name()
default
```

6.3 Ereditarietà

L'ereditarietà serve per modellare relazioni di tipo "è un/una" dove alle informazioni della classe base (chiamata *parent* o *superclass*) ne aggiungiamo altre della classe derivata (chiamata *subclass* o *children*).
L'ereditarietà singola può essere utilizzata per estendere classi base con comportamenti aggiuntivi e creando classi funzionali per particolari applicazioni.

6.3.1 Ereditarietà singola

La sintassi per definire una classe che eredita da un'altra è

```
class NomeClasseDerivata(NomeClasseBase):
    <statement-1>
    .
    .
    .
    <statement-N>
```

Si ha che:

- nella ricerca nomi, nel caso nel caso non vengano trovati nella classe derivata si procederà nella classe base (e ricorsivamente nelle classi ad essa *parent*);
- le classi derivate possono eventualmente **specializzare i metodi** (sia funzioni speciali tipo `__init__` che standar), ridefinendoli con il medesimo nome della classe base. Nella specializzazione possiamo eseguire il metodo della classe base riferendoci a questa mediante `super`

```
>>> class Greeting:
...     def greet(self, n=1):
...         print("hi " * n)
...
>>> # nessuna specializzazione, si usa quello della classe parent
>>> class Standard(Greeting):
...      pass
...
>>> # completo overwrite
>>> class Leaving(Greeting):
...     def greet(self):
...         print("goodbye")
...
>>> # modifica incrementale
```

```
>>> class Afternoon(Greeting):
...     def greet(self):
...         # utilizzo della funzione base
...         super().greet(n=3)
...         # peculiarità di questa classe
...         print("in the afternoon")
...
>>> std = Standard()
>>> std.greet()
hi

>>> lea = Leaving()
>>> lea.greet()
goodbye

>>> aft = Afternoon()
>>> aft.greet()
hi hi hi
in the afternoon
```

6.3.2 Ereditarietà multipla

È solitamente sconsigliata per evitare complessità. La definizione avviene come segue:

```
class DerivedClassName(Base1, Base2, Base3):
    <statement-1>
    .
    .
    .
    <statement-N>
```

dove la classe eredita attributi e funzioni da tutte e tre le classi di base.
Per la ricerca la risoluzione di nomi funziona prima in profondità per `Base1` (cercando anche nelle classi dalle quali questa eredita), poi passando a `Base2` (e quindi in profondità) e a `Base3`(e in profondità), evitando di cercare due volte nella stessa classe se vi sono sovrapposizioni nell'albero genealogico.

Mixin Una applicazione di ereditarietà multipla che può essere a volte utile è quando nella definizione di una classe, ad una classe base aggiungiamo una classe che estende funzionalità specifiche

```
>>> class Contact:
...     def __init__(self, name, email):
...         self.name = name
...         self.email = email
...
>>> class MailSender:
...     def send_email(self, message):
...         print(f"sending mail to {self.email} with message '{message}'")
...
```

```
>>> class MailableContact(Contact, MailSender):
...     pass
...
>>> l = MailableContact("luca", "lbraglia@asd.foo")
>>> l.send_email("ciao")
sending mail to lbraglia@asd.foo with message 'ciao'

>>> # Onesto si può anche fare una cosa come sotto, che non richiede che l'oggetto ab
>>> # memorizzato proprio in self.email la email, ma questa viene passata in
>>> # chiamata di self.send_email(self.email, message) dove necessario
>>> class MailSender2:
...     def send_email(self, mail, message):
...         print(f"sending mail to {email} with message '{message}'")
...
```

Alternativa a questa struttura:

- scrivere una *funzione esterna* alla classe e chiamarla quando necessario
- fare *composition*: ad esempio invece di ereditare da `MailSender`, prevedere un parametro di `Contact` cui passare `MailSender`, che venga memorizzato all'interno di `Contact`

6.4 Amenità/utilità per le classi

6.4.1 Funzioni builtin utili

In runtime possiamo voler controllare che:

- un oggetto sia istanza di una classe o di una sua parent (in maniera diretta/indiretta/virtuale), mediante `isinstance`. Si può fornire come secondo argomento anche una tupla o una `UnionType` (classi separate da |)con le classi ricercate e sarà restituito `True` se appartiene ad almeno una di esse

```
>>> # utilizzo con singole classi
>>> isinstance(l, MailableContact)
True
>>> isinstance(l, Contact)
True
>>> isinstance(l, MailSender)
True
>>> isinstance(l, list)
False

>>> # utilizzo con gruppi di classi
>>> isinstance(l, (Contact, list)) # tupla
True
>>> isinstance(l, dict | list)     # UnionType
False
```

- una classe sia sottoclasse (diretta, indiretta, virtuale) di un'altra mediante `issubclass`. Una classe è considerata sottoclasse di se stessa. Nuovamente si possono fornire `tuple` o `UnionType` per indicare un insieme di classi

```
>>> # utilizzo con singole classi
>>> issubclass(MailableContact, Contact)
True
>>> issubclass(Contact, MailableContact)
False

>>> # utilizzo con gruppi di classi
>>> isinstance(Contact, (dict, list))
False
```

6.4.2 Decoratori utili per metodi

6.4.2.1 `property`

Mediante `property` si possono implementare metodi *getter* e *setter* di dati:

- il getter espone una funzione accedibile come attributo: può essere utile per valori calcolati/derivati e limitare l'accesso alla sola lettura.
- il setter permette che all'assegnazione di un attributo di un oggetto venga eseguita una funzione (es per validazione input)

Si utilizza a questo scopo la funzione `property` (tipicamente come decoratore) che formalmente prende in input 4 parametri: una funzione getter, una setter, una deleter e un commento. Un esempio di utilizzo schietto

```
>>> # ----------------------...--------
>>> # 1 Implementazione senza decoratori
>>> # --------------------------..------

>>> class PropertyClass:
...     # getter
...     def _get_internal(self):
...         print("Ritorno il valore interno.")
...         return self._internal
...     # setter
...     def _set_internal(self, value):
...         if not isinstance(value, int):
...             raise ValueError("Il valore deve esser intero.")
...         print(f"Imposto il valore interno a {value}")
...         self._internal = value
...     # deleter
...     def _del_internal(self):
...         print("Rimuovo il valore interno.")
...         del self._internal
...     # l'attributo esterno che mappa self._internal sarà x
...     x = property(
```

```
...         _get_internal, _set_internal, _del_internal, "Accessor to _internal prope
...     )
...

>>> # help(PropertyClass) mostra x tra i dati di classe con 'Accessor to ..'
>>> o = PropertyClass()
>>> o.x = 1  # uso del setter
Imposto il valore interno a 1
>>> o.x = "a"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 9, in _set_internal
ValueError: Il valore deve esser intero.
>>> o.x  # uso del getter
Ritorno il valore interno.
1
>>> del o.x  # uso del deleter
Rimuovo il valore interno.
```

Solitamente le properties sono definite solamente con il primo argomento (getter) o al massimo il secondo (setter, se si vuole concedere un accesso a dati mascherati, se pur mediato). La funzione deleter è spesso lasciata vuota perché raramente gli attributi vengono cancellati; se l'utente da `del` su un attributo property senza deleter viene sollevata eccezione.
Un esempio di utilizzo come decoratore, consigliabile per leggibilità:

```
>>> # --------------------------------
>>> # 2 Implementazione con decoratori
>>> # --------------------------------


>>> class PropertyClass2:
...     # il getter usa il nome pubblico x. Inoltre contiene la stringa di
...     # documentazione dell'attributo. Infine non vi sarà una assegnazione x =
...     # property(getter, setter, deletere, doc) al termine della classe
...     @property
...     def x(self):
...         """Accessor to _internal property"""
...         print("Ritorno il valore interno.")
...         return self._internal
...     # la setter e deleter vengono definite con un metodo/decoratore della
...     # getter e con il medesimo nome
...     @x.setter
...     def x(self, value):
...         if not isinstance(value, int):
...             raise ValueError("Il valore deve esser intero.")
...         print(f"Imposto il valore interno a {value}")
...         self._internal = value
...     @x.deleter
...     def x(self):
...         print("Rimuovo il valore interno.")
```

```
...         del self._internal
...

>>> # l'utilizzo dopo è uguale
>>> o = PropertyClass2()
>>> o.x = 1  # uso del setter
Imposto il valore interno a 1
>>> # o.x = "a"
>>> o.x  # uso del getter
Ritorno il valore interno.
1
>>> del o.x  # uso del deleter
Rimuovo il valore interno.
```

Quando utilizzare delle property Alcune applicazioni:

- i getter possono esser utilizzati per effettuare calcoli al volo sulla base di dati interni (es bmi a partire da peso e altezza)
- usare property se si vuole mediare l'accesso a dati:
  - per motivi di sicurezza/integrità (es controllo sul tipo sopra)
  - per opportunità (es si potrebbe effettuare caching dei risultati se il modo per ottenerli è dispendioso)
  - se si vogliono far partire altre automazioni esterne quando un valore è impostato (es se il valore di webpage è impostato, loggati al server e caricalo)

6.4.2.2 `cached_property`

Tipo property ma con cache quindi calcola il valore una volta dopodiché lo riutilizza. Utile per calcoli lunghi, il cui risultato non cambia lungo il tempo di vita dell'istanza

```
>>> from time import sleep
>>> from functools import cached_property

>>> class Report:
...     @cached_property
...     def summary(self):
...         sleep(1)
...         return 1
...
>>> r1 = Report()
>>> # r1.summary # it takes 1 second
>>> # r1.summary # it takes 0 second
```

6.4.2.3 `classmethod`

È una funzione che

- viene eseguita dalla classe, non dalla sua istanza.

- come primo argomento viene dato `cls` si riferisce alla classe (mentre `self` tipicamente si riferisce all'oggetto/istanza)

Un utilizzo tipico è la creazione di oggetti *non* mediante l'inizializzatore default della classe

```
>>> class User:
...     def __init__(self, name):
...         self.name = name
...     @classmethod
...     def from_email(cls, email):
...         return cls(email.split("@")[0])
...
>>> l1 = User("lbraglia")
>>> l2 = User.from_email("lbraglia@gmail.com")
```

6.4.2.4 staticmethod

Sono metodi che:

- non ricevono un primo argomento implicito (come `self`, `cls`) e NON utilizzano dunque i dati della classe. Sono funzioni "standard" che però vivono nel namespace di una classe.
- sono utilizzati per funzioni in qualche modo relate alla classe ma non necessitano dei dati dell'oggetto in lettura ne scrittura. Utilizzare un metodo static segnala agli altri sviluppatori che la funzione è indipendente dai dati dell'oggetto
- possono essere utili per funzioni di aiuto legate alla classe.

```
>>> class Temperature:
...     def __init__(self, celsius):
...        self.celsius = self.validate_celsius(celsius)
...     @staticmethod
...     def validate_celsius(celsius):
...         if celsius >= -273.15:
...             return celsius
...         else:
...            raise ValueError("strange celsius")
...

>>> yesterday_t = Temperature(100)
>>> today_t = Temperature(-300)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in __init__
  File "<stdin>", line 9, in validate_celsius
ValueError: strange celsius

>>> # staticmethod vive nel namespace della classe quindi lo possiamo usare
>>> # anche cosi
>>> Temperature.validate_celsius(132)
132
```

6.4.2.5 `abstractmethod`

Approfondito nella sezione di ABC, serve per richiedere alle classi figlie di implementare un metodo a pena di non poter istanziare

6.4.3 Type hint

`self` non va caratterizzato (nei parametri, se restituito si usa `Self`), poi spesso le funzioni di classi modificano dati internamente non restituendo nulla quindi si userà `None` come dato restituito

```
from typing import Self

class BankAccount:
    def __init__(self, account_name: str, initial_balance: int = 0) -> None:
        self.account_name = account_name
        self.balance = initial_balance

    def deposit(self, amount: int) -> None:
        self.balance += amount

    def withdraw(self, amount: int) -> None:
        self.balance -= amount

    def returnme(self) -> Self:
        return self
```

Le classi definite dall'utente sono tipi validi da usare per le annotazioni

```
account: BankAccount = BankAccount("Alice", 400)
def transfer(src: BankAccount, dst: BankAccount, amount: int) -> None:
    src.withdraw(amount)
    dst.deposit(amount)
```

Funzioni che accettano una classe, accetteranno anche classi derivate senza problemi:

```
class BankAccountForYoungs(BankAccount):
     ....

timmysba = BankAccountForYoungs("Timmy", 2)
transfer(account, timmysba, 100)  # type checks!
```

6.5 Polimorfismo

Una delle maggiori ragioni per l'uso dell'ereditarietà è creare polimorfismo nel codice, ossia il fatto che metodi accomunati dal nome funzionano in maniera differente a seconda dell'oggetto/classe di appartenenza, senza richiedere al client che le evoca di conoscere i dettagli.
Ad esempio un file mp3 e un file ogg sono entrambi audio file ma l'attività `play` funzionerà in maniera diversa a seconda del formato; sotto una implementazione di un media player che controlla la corrispondenza del formato alla classe

```
>>> class AudioFile:
...     def __init__(self, filename):
...         if not filename.endswith(self.ext):
...             raise Exception("Invalid file format")
...         self.filename = filename
...
>>> class MP3(AudioFile):
...     ext = "mp3"
...     def play(self):
...         print(f"playing mp3 {self.filename}")
...
>>> class Ogg(AudioFile):
...     ext = "ogg"
...     def play(self):
...         print(f"playing ogg {self.filename}")
...
>>> mp3 = MP3("foo.mp3")
>>> mp3.play()
playing mp3 foo.mp3

>>> ogg = Ogg("foo.ogg")
>>> ogg.play()
playing ogg foo.ogg

>>> ogg2 = Ogg("foo.wav")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 4, in __init__
Exception: Invalid file format
```

In Python a differenza di altri linguaggi il polimorfismo è reso meno imprescindibile dal **duck typing**. Ad esempio il seguente codice può essere utilizzato come sopra avendo però definizione diversa/non essendo classe derivata di `AudioFile`

```
>>> class Flac:
...    def __init__(self, filename):
...       if not filename.endswith(".flac"):
...           raise Exception("Invalid file format")
...       self.filename = filename
...    def play(self):
...        print(f"playing flac {self.filename}")
...
>>> fl1 = Flac("foo.flac")
>>> fl1.play()
playing flac foo.flac

>>> fl2 = Flac("foo.ogg")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 4, in __init__
Exception: Invalid file format
```

Ossia se è l'interfaccia pubblica comune a essere quella importante ai fini del polimorfismo allora il duck typing come sopra può bastare; questo limita la necessità di ereditarietà (che rimane pur tuttavia utile per condividere codice) e con essa delle complessità portate dall'ereditarietà multipla.

Verso abstract base class Solo perché un oggetto soddisfa una particolare interfaccia come sopra non significa che funzionerà in maniera corretta in tutte le circostanze. Ad esempio una classe che incapsuli una simulazione di partita di schacchi potrebbe essere modellata con un metodo `play`, ma se forniamo un oggetto istanziato al media player di sopra non funzionerà/sarà quello che vogliamo, sebbene rispetti l'interfaccia.

6.6 Abstract Base Class

Servono per definire classi struttura da utilizzare necessariamente come parent di classi di interesse:

- richiedere che certe interfacce siano implementate dalle figlie: in sede di definizione si richiedono metodi/funzioni decorando con `abstractmethod` funzioni tipicamente con `pass`
- fornire codice standard come nelle classi base (sotto l'init e jump sono funzioni standard)

Tipicamente:

- si possono definire ABC se si intende fornire codice/librerie che deve essere distribuito per modifiche; partiamo da queste
- si definiscono classi che rispettano ABC fornite dal linguaggio, facendo sì che gli oggetti derivanti si integrino bene nella sintassi di Python

6.6.1 Definizione

```
>>> from abc import ABC, abstractmethod

>>> class Animal(ABC):
...     def __init__(self, name):
...         self.name = name
...     @abstractmethod    # richiede che speak sia implementato dalle figlie
...     def speak(self) -> str:
...         pass
...     def jump(self):
...         print(f"{self.name} makes an incredible jump")
...

>>> class Dog(Animal):  # Must inherit!
...     def speak(self) -> None:
...         print(f"{self.name} says 'Woof'")
...
>>> class Cat(Animal):
```

```
...     pass
...
>>> # this is ok. Even if Dog.speak does not return a string
>>> fido = Dog("Fido")
>>> fido.speak()
Fido says 'Woof'

>>> # This fails: no speak is implemented in Cat, which is required by Animal
>>> mimi = Cat("Mimi")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't instantiate abstract class Cat without an implementation for abstrac

>>> # This fails for the same reason before
>>> generic = Animal("Boo")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't instantiate abstract class Animal without an implementation for abst
```

6.6.2 Creazione di classi conformi ad ABC del linguaggio

La maggior parte delle ABC fornite dal linguaggio si trovano nel modulo `collections.abc`. La più semplice è l'ABC `Container`; una classe che lo rispetta permette di utilizzare l'operatore `in`.

```
>>> from collections.abc import Container

>>> # metodi richiesti dalla classe
>>> Container.__abstractmethods__
frozenset({'__contains__'})

>>> # controlliamo alcuni classi che vi appartengono/rispettano le specifiche
>>> issubclass(list, Container)
True
>>> issubclass(dict, Container)
True

>>> # Per controllare cosa va implementato vedere la documentazione in html ma
>>> # help(Container.__contains__) dice che si aspetta solo un argomento x che
>>> # verosimilmente sarà il valore da controllare.
>>> # Utilizziamo per impleentare un container che ci dice se un input è intero è dis

>>> class OddContainer:
...     def __contains__(self, x):
...         if not isinstance(x, int) or not x % 2:
...             return False
...         return True
...
```

Da notare che

1. non abbiamo subclassato `Container` e tuttavia viene riconosciuta come una subclass

```
>>> issubclass(OddContainer, Container)
True
```

2. possiamo dopo istanza controllare che appartiene ai Container

```
>>> dispari = OddContainer()
>>> isinstance(dispari, Container)
True
```

3. infine la presenza di contains (ergo l'appartenenza ai container) sblocca l'uso di `in`

```
>>> print(3 in dispari)
True
```

6.7 Dunder methods e applicazioni

Metodi speciali che iniziano e terminanon con doppio "_" e servono per migliorare l'integrazione delle classi da noi scritte con il linguaggio/ambiente (a parte `__init__`, già introdotta).
Un buon punto di partenza per molte classi custom è implementare `__init__`, `__repr__` e i metodi che rendono gli oggetti facilmente utilizzabili (pensare al Python che si vuole leggere/scrivere idiomaticamente quando si utilizzano queste classi):

- `__repr__`: ritorna una rappresentazione dell'oggetto per programmatori, idealmente il codice che serve per ricrearlo; viene utilizzata nella valutazione a terminale e da `repr`.
  `__str__` ritorna una rapporesentazione user-friendly/per gli utenti dell'oggetto; è utilizzata da `str(obj)` e `print(obj)` (se non definita si utilizzerà `__repr__`)

```
>>> class ReprStrTest:
...     def __init__(self, value):
...         self._value = value
...     def __repr__(self):
...        return f"ReprStrTest({self._value!r})"  # !r handles strings nicely
...     def __str__(self):
...         return f"an interesting value: {self._value}"
...
>>> x = ReprStrTest(3)

>>> x          # printa il valore dato da __repr__
ReprStrTest(3)
>>> repr(x)    # chiama __repr__
'ReprStrTest(3)'
>>> print(x)   # printa il valore dato da __str__
an interesting value: 3
>>> str(x)     # chiama __str__
```

```
'an interesting value: 3'

>>> y = ReprStrTest("hi")

>>> y          # printa il valore dato da __repr__
ReprStrTest('hi')
>>> repr(y)    # chiama __repr__
"ReprStrTest('hi')"
```

- `__len__`: ritorna la lunghezza dell'oggetto ed è utilizzata da `len`

```
>>> class LenTest:
...     def __init__(self, *args):
...         self._data = list(args)
...     def __len__(self):
...         return len(self._data)
...
>>> lentest3 = LenTest(1,4,2)
>>> len(lentest3)
3
```

- `__bool__`: restituisce il valore di verità, viene utilizzata da `bool`, e sblocca l'uso dell'`if` con l'oggetto; se assente viene utilizzata `__len__`, se disponibile

```
>>> # una classe senza __bool__ usa __len__
>>> lentest0 = LenTest()
>>> bool(lentest0)
False
>>> bool(lentest3)
True
```

- `__eq__` definisce l'uguaglianza tra due oggetti (e sblocca `==` e `!=`) mentre `__lt__` l'inferiorità (`<`)
  - accettano il `self` e un altro parametro con cui si ottiene l'istanza dell'oggetto
  - se definite entrambe e si decora la classe con `functools.total_ordering`, vengono generate tutte le funzioni derivanti e sbloccati tutti i confronti

```
>>> from functools import total_ordering

>>> @total_ordering
... class EqTest:
...     def __init__(self, value):
...         self._value = value
...     def __eq__(self, other):
...         return self._value == other._value
...     def __lt__(self, other):
...         return self._value < other._value
...
```

```
>>> a = EqTest(1)
>>> b = EqTest(1)
>>> c = EqTest(2)

>>> a == b
True
>>> a == c
False
>>> a < b
False
>>> a <= b
True
```

- `__add__` permette di sbloccare l'addizione tra due oggetti (similmente si avranno `__sub__` per la sottrazione, `__mul__` per la moltiplicazione).

```
>>> class AddTest:
...     def __init__(self, *args):
...         self._data = list(args)
...     def __add__(self, other):
...         all_data = self._data + other._data
...         return AddTest(*all_data)
...     def __repr__(self):
...         return f"AddTest({self._data!r})"
...
>>> a = AddTest(1,2,3)
>>> b = AddTest(4,5,6)
>>> a+b
AddTest([1, 2, 3, 4, 5, 6])
```

- `__contains__` sblocca l'uso di `in`:

```
>>> # wrapping a list
>>> class ContainsTest:
...     def __init__(self, *args):
...         self._data = list(args)
...     def __contains__(self, value):
...         return value in self._data
...
>>> x = ContainsTest(1,2,3)
>>> 1 in x
True
>>> 4 in x
False

>>> # wrapping a dict
>>> class ContainsTest2:
...     def __init__(self, **kwargs):
...         self._data = dict(kwargs)
...     def __contains__(self, key):
...         return key in self._data.keys()
```

```
...
>>> y = ContainsTest2(foo=1,bar=2,baz=3)
>>> "foo" in y
True
>>> 4 in y
False
```

- `__getitem__` permette l'utilizzo di indexing in lettura. Python chiama `oggetto.__getitem__(indice)` quando si da `oggetto[indice]`.

```
>>> # wrapping a list
>>> class GetitemTest:
...     def __init__(self, *args):
...         self._data = list(args)
...     def __getitem__(self, index):
...         return self._data[index]
...
>>> x = GetitemTest(1, 2, 3)
>>> x[0]
1
>>> x[2]
3
```

```
>>> # wrapping a dict
>>> class GetitemTest2:
...     def __init__(self, **kwargs):
...         self._data = dict(kwargs)
...     def __getitem__(self, key):
...         return self._data[key]
...
>>> y = GetitemTest2(foo=1, bar=2, baz=3)
>>> y["foo"]
1
>>> y["baz"]
3
```

- `__setitem__` permette l'indexing in scrittura. Python chiama `oggetto.__setitem__(key, value)` quando si da `oggetto[key] = value`

```
>>> # wrapping di lista
>>> class SetItemTest:
...     def __init__(self, *args):
...         self._data = list(args)
...     def __setitem__(self, index, value):
...         self._data[index] = value
...     def __repr__(self):
...         return f"SetItemTest({self._data!r})"
...

>>> x = SetItemTest(1, 2, 3)
>>> x[0] = 10
```

```
>>> x
SetItemTest([10, 2, 3])

>>> # wrapping di dict
>>> class SetItemTest2:
...     def __init__(self, **kwargs):
...         self._data = dict(kwargs)
...     def __setitem__(self, key, value):
...         self._data[key] = value
...     def __repr__(self):
...         return f"SetItemTest2({self._data!r})"
...

>>> y = SetItemTest2(foo=1, bar=2, baz=3)
>>> y["foo"] = 10
>>> y["qux"] = 4
>>> y
SetItemTest2({'foo': 10, 'bar': 2, 'baz': 3, 'qux': 4})
```

6.7.1 Iteratori

Il `for` del python è conciso e flessibile, si pensi a:

```
>>> List = [1, 2, 3]
>>> Set = (1, 2, 3)
>>> Dict = {'one': 1, 'two': 2}
>>> String = "asd"

>>> for element in List:
...     print(element)
...
1
2
3
>>> for element in Set:
...     print(element)
...
1
2
3
>>> for key in Dict:
...     print(key)
...
one
two
>>> for char in String:
...     print(char)
...
a
s
d
```

Quello che lo rende flessibile è il fatto di gestire in maniera standard quelli che in Python sono chiamati iterabili, ossia oggetti passibili di iterazione.

6.7.1.1 Funzionamento

Si ha che:

- *iterabile* è un oggetto che presenta un metodo `__iter__`, il quale restituisce un iteratore, oggetto che rappresenta un flusso di dati dell'iterabile considerato;
- un *iteratore* è un oggetto che rappresenta un flusso di dati e mediante il metodo `__next__` li ritorna un elemento alla volta (oppure ritorna `StopIteration` se non ve ne sono altri).

Ora vi sono due funzioni di utilità che servono per implementare il protocollo del for:

- `iter` prende in input un oggetto arbitrario e cerca di restituire un iteratore sui suoi dati (chiamando `__iter__`), oppure `TypeError` se non possibile;
- sull'iteratore possiamo utilizzare `next` (che chiama il `__next__`)

```
>>> s = 'abc'
>>> it = iter(s)
>>> it
<str_ascii_iterator object at 0x7f3127ec6890>
>>> next(it)
'a'
>>> next(it)
'b'
>>> next(it)
'c'
>>> next(it)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration

>>> iter(123) # questo da errore perché un intero non è iterabile
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'int' object is not iterable
```

Quindi complessivamente, lo statement `for`:

1. chiama innanzitutto `iter()` sull'oggetto che è in ciclo (il contenitore) ottenendone un iteratore;
2. chiama via via `next()` sull'iteratore permettendo cosi il processo di iterazione;
3. quando non vi sono altri elementi sui quali iterare, `next()` solleva l'eccezione `StopIteration` e `for` termina.

6.7.1.2 Implementazione mediante classe

Spesso si vuole creare classi/strutture di dati che siano iterabili; per farlo occorre:

- definire una classe con metodo `__iter__` (iterabile) che ritorni un oggetto con un metodo `__next__` senza parametri (rendendo l'oggetto ritornato un iteratore).
- caso classico: una classe ha sia `__next__` che `__iter__`. In tal caso è sufficiente che `__iter__` ritorni `self`.

```
>>> class Reverse:
...     """Iterator for looping over a sequence backwards."""
...     def __init__(self, data):
...         self.data = data
...         self.index = len(data)
...     def __iter__(self):
...         return self
...     def __next__(self):
...         if self.index == 0:
...             raise StopIteration
...         self.index = self.index - 1
...         return self.data[self.index]
...
>>> for char in Reverse('spam'):
...     print(char)
...
m
a
p
s
```

6.7.1.3 Implementazione mediante espressioni generatrici

Generatori semplici possono essere ottenuti mediante le espressioni generatrici, espressioni poste tra tonde (invece che quadre) che utilizzano una sintassi simile alle list comprehension. Alcuni esempi:

```
>>> squares = ((i*i for i in range(10)))
>>> type(squares)
<class 'generator'>
>>> sum(squares)
285

>>> data = 'golf'
>>> reversed = (data[i] for i in range(len(data) - 1, -1, -1))
>>> list(reversed)
['f', 'l', 'o', 'g']
```

Alcuni confronti:

- le espressioni generatrici sono compatte ma meno versatili rispetto alla definizione di un generatore

- rispetto a una list comprehension, le espressioni generatrici ritornano un iteratore che calcola il valore al bisogno; le list comprehension sono inutili o meno efficienti se si lavora con iteratori che ritornano uno stream infinito di valori o un numero molto alto

6.7.1.4 Implementazione mediante generatori

I generatori sono funzioni che creano degli iteratori:

- l'unica differenza dalle funzioni classiche è che usano `yield` quando vogliono ritornare dei dati;
- tutto quello che è possibile fare mediante la definizione di iteratori mediante classi è possibile farlo anche con i generatori; quello che rende questi ultimi interessanti è il fatto che i metodi `__iter__` e `__next__` sono generati automaticamente e complessivamente la programmazione è molto più chiara/compatta.

```
>>> def reverse(data):
...     for index in range(len(data) - 1, -1, -1):
...         yield data[index]
...
>>> # crea un iterabile e iteratore, ossia un oggetto che ha sia iter che next
>>> r = reverse('golf')
>>> r.__iter__
<method-wrapper '__iter__' of generator object at 0x7f3127f04740>
>>> r.__next__
<method-wrapper '__next__' of generator object at 0x7f3127f04740>

>>> for char in r:
...     print(char, end = ' ')
...
f l o g
```

Vediamo la differenza di funzionamento rispetto a una funzione classica:

- nel chiamare una funzione classica viene creato un namespace che contiene i dati e al `return` questo viene distrutto; una chiamata successiva alla stessa riparte con un namespace nuovo. I generatori possono essere pensati come funzioni dove il namespace non viene gettato all'uscita ma è disponibile alla successiva chiamata
- alla chiamata un generatore non ritorna un singolo valore: invece ritorna un oggetto generatore che supporta il protocollo degli iteratori. Quando si esegue `yield` il generatore restituisce l'espressione, ma a differenza di `return` l'esecuzione della funzione si sospende e le variabili locali sono preservate; alla prossima chiamata di `__next__` la funzione riprenderà l'esecuzione da capo.

6.7.1.5 Funzioni e operatori utili su iteratori

Funzioni Alcune funzioni builtin:

- su iteratori con dati logici `all` e `any` ritornano `True` rispettivamente se tutti gli elementi sono `True` o almeno uno lo è
- su iteratori con dati confrontabili, `max`, `min` ritornano l'elemento maggiore o minore, `sorted` ordina l'iteratore
- `enumerate` restituisce un oggetto involucro con un id progressivo
- `zip` prende in input più iterabili e restituisce un iteratore che genera tuple con un elemento di ciascun oggetto di partenza alla volta. Nel caso gli iterabili abbiano lunghezza diversa verrà prodotto
- per la programmazione funzionale sono utili `filter`, `map`

```
>>> all([True, True])
True
>>> any([False, False])
False
>>> max([1, 2, 10])
10

>>> A = [1,2,3]
>>> B = "letters"

>>> for a, b in zip(A, B):
...     print(a, b)
...
1 l
2 e
3 t
```

Operatori `in` e `not in`: la sintassi `X in iterator` ritorna `True` se `X` è ritrovato nell'iteratore

6.7.1.6 Il modulo itertools

Contiene funzioni per la creazione e gestione di iteratori

Creazione di nuovi iteratori

```
itertools.count()                ## 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ...
itertools.cycle([1, 2, 3, 4, 5]) ## 1, 2, 3, 4, 5, 1, 2, 3, 4, 5  ...
itertools.repeat('abc')          ## abc, abc, abc, abc, abc, abc  ...
itertools.chain(['a', 'b', 'c'], (1, 2, 3)) ##  a, b, c, 1, 2, 3
itertools.islice(range(10), 2, 8)    ##  2, 3, 4, 5, 6, 7
```

Selezione

```
itertools.filterfalse(predicate, iterable)
```

Crea un iteratore che elimina elementi da un iterabile laddove un predicato ad essi applicato è falso. Lo applichiamo ad una lista come esempio:

```
>>> import itertools

>>> def selector(x):
...       return x < 5
...
>>> res = itertools.filterfalse(selector, [1, 4, 6, 4, 1])
>>> list(res)
[6]
```

`itertools.compress(data, selectors)`

crea un iteratore che filtra gli elementi di `data` ritornando solo quelli che valuno `True` in `selectors`

```
>>> res = itertools.compress('ABCDEF', [1,0,1,0,1,1])
>>> list(res)
['A', 'C', 'E', 'F']
```

Grouping Vediamo:

`itertools.groupby(iter, key_func = None)`

si ha:

- `iter` un iterabile
- `key_func` è una funzione che restituisce un id per ogni elemento dell'iterabile

`groupby`

- assume che il contenuto di `iter` sia ordinato per chiave
- mette assieme tutti gli elements dell'iterable con stessa chiave, e ritorna uno stream di tuple di due elementi, con primo elemento la chiave e secondo elemento l'iteratore su tutti gli elementi con tale chiave

```
city_list = [('Decatur', 'AL'), ('Huntsville', 'AL'), ('Selma', 'AL'),
             ('Anchorage', 'AK'), ('Nome', 'AK'),
             ('Flagstaff', 'AZ'), ('Phoenix', 'AZ'), ('Tucson', 'AZ'),
             ...
            ]

def get_state(city_state):
    return city_state[1]

itertools.groupby(city_list, get_state) =>
  ('AL', iterator-1),
  ('AK', iterator-2),
  ('AZ', iterator-3), ...

where
iterator-1 =>
  ('Decatur', 'AL'), ('Huntsville', 'AL'), ('Selma', 'AL')
```

```
iterator-2 =>
  ('Anchorage', 'AK'), ('Nome', 'AK')
iterator-3 =>
  ('Flagstaff', 'AZ'), ('Phoenix', 'AZ'), ('Tucson', 'AZ')
```

Combinazioni e permutazioni

```
>>> list(itertools.combinations([1, 2, 3, 4, 5], 2))
[(1, 2), (1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5), (3, 4), (3, 5), (4, 5)]
>>> list(itertools.combinations_with_replacement([1, 2, 3, 4, 5], 2))
[(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 2), (2, 3), (2, 4), (2, 5), (3, 3), (3, 4), (3, 5)

>>> list(itertools.permutations([1, 2, 3, 4, 5]))
[(1, 2, 3, 4, 5), (1, 2, 3, 5, 4), (1, 2, 4, 3, 5), (1, 2, 4, 5, 3), (1, 2, 5, 3, 4), (1, 2, 5,
>>> list(itertools.permutations([1, 2, 3, 4, 5], 2))
[(1, 2), (1, 3), (1, 4), (1, 5), (2, 1), (2, 3), (2, 4), (2, 5), (3, 1), (3, 2), (3, 4), (3, 5)
```

Prodotto cartesiano

```
>>> list(itertools.product('ABCD', 'xy'))
[('A', 'x'), ('A', 'y'), ('B', 'x'), ('B', 'y'), ('C', 'x'), ('C', 'y'), ('D', 'x'), ('D', 'y')
```

6.7.2 Context Managers

Un context manager è un oggetto che fornisce informazioni contestuali o esecuzione automatica ad una azione ed è quello che funziona col protocollo/keyword `with`. Il più conosciuto è `open` grazie al quale `f` sarà automaticamente chiusa all'uscita dal manager (il blocco):

```
with open('file.txt') as f:
    contents = f.read()
```

Un context manager può essere implementato mediante classe o mediante generatore; la classe è meglio se vi è un tot di dati/logica da incapsulare, la funzione è meglio se abbiamo a che fare con casi più semplici.

6.7.2.1 Implementazione mediante classe

Per duplicare `open` semplicemente si programma:

```
class CustomOpen(object):
    def __init__(self, filename):
        self.file = open(filename)

    def __enter__(self):
        return self.file

    def __exit__(self, ctx_type, ctx_value, ctx_traceback):
        self.file.close()

with CustomOpen('file') as f:
    contents = f.read()
```

I due metodi speciali utilizzati da with sono `__enter__` e `__exit__`. Il funzionamento:

- `CustomOpen` è inizializzata con `__init__`
- `__enter__` è chiamata e qualsiasi cosa ritorni è assegnato a `f`
- quando il blocco di `with` finisce, viene chiamata `__exit__`

6.7.2.2 Implementazione mediante generatore

Bisogna utilizzare `contextmanager` dalla libreria `contextlib`:

```
from contextlib import contextmanager

@contextmanager
def custom_open(filename):
    f = open(filename)
    try:
        yield f
    finally:
        f.close()

with custom_open('file') as f:
    contents = f.read()
```

Il funzionamento:

- `custom_open` è eseguita fino a `yield`
- ritorna il controllo allo statement with; ciò che è stato dato da `yield` viene assegnato ad `f` (nel pezzo `as f`)
- la clausola `finally` assicura che `close` sia chiamata che vi sia stata una eccezione all'interno del blocco `with` o meno

6.8 Classi builtin notevoli

Si pongono qua alcuni tipi di dati lievemente più complessi di quelli base; alcuni debbono o possono esser tipicamente utilizzati per il subclassing.

6.8.1 defaultdict

Per evitare che il lookup di una chiave in un dict fallisca possiamo `setdefault` fornendo il valore di volta in volta; se invece vogliamo risolvere questo problema una volta soltanto dobbiamo creare un `defaultdict`, ossia un dict che in caso di fallimento del lookup restituisce un valore standard/default.
Il primo argomento nell'istanziazione deve essere una funzione/callable che restituisce il valore default; interessante l'utilizzo con listeche permette di aggiungere elementi ad una lista indicizzata da una key di dict

```
>>> from collections import defaultdict

>>> # creazione
>>> d0 = defaultdict(int)          # default: intero 0
>>> d1 = defaultdict(lambda x: -1) # default: intero -1
>>> dl = defaultdict(list)         # default: lista vuota

>>> # utilizzo
>>> d0["foo"] = "bar"
>>> d0["foo"]         # valore presente
'bar'
>>> d0["asd"]         # valore assente restituito il default
0

>>> dl["one"].append("a") # valore assente: restituisce il default a cui viene appeso
>>> dl["one"].append("b") # valore presente: appende al valore restituito
>>> dl["two"].append("c")
>>> dl
defaultdict(<class 'list'>, {'one': ['a', 'b'], 'two': ['c']})
```

6.8.2 Counter

Sono versioni speciali di `dict` che servono per contare frequenze di occorrenze: se istanziati con argomento ne fanno il conteggio, altrimenti funzionano come un dict

```
>>> from collections import Counter
>>> import random

>>> # initialization 1: use a counter on elements of a list
>>> d4 = [1, 2, 3, 4]
>>> rolls = random.choices(d4, k = 200) # list of 200 d4 rolls
>>> c1 = Counter(rolls)
>>> c1
Counter({2: 57, 1: 49, 3: 48, 4: 46})

>>> # initialization 2: with no parameters passed acts dict-like
>>> c2 = Counter() # create an empty counter
>>> c2["a"] += 1   # increase count of an element (starting from 0)
>>> c2["b"] =  2   # set counts of an element
>>> c2["x"]        # no elements counted
0
>>> c2
Counter({'b': 2, 'a': 1})

>>> # useful methods
>>> # --------------
>>> sorted(c2.elements()) # accesso a tutti gli elementi, con ripetizione
['a', 'b', 'b']
>>> c2.total() # sum of all frequencies
3
```

```
>>> c2.most_common(2) # most common modalities
[('b', 2), ('a', 1)]

>>> # other methods from dict: c2.keys(), c2.values(), c2.items()
```

6.8.3 Enum & Flag

6.8.3.1 Enum

Sono usate per dati con un set prefissato di opzioni /categorie

- raggruppano ordinatamente stati/costanti prefissate
- aumentano leggibilita
- facilitano debugging
- single source of truth/consistency

Possono essere pensate come un binding nome valore

```
>>> from enum import Enum

>>> class State(Enum):
...     OFF: int = 0
...     ON: int = 1
...
>>> class Color(Enum):
...     RED: str = 'R'
...     GREEN: str = 'G'
...     BLUE: str = 'B'
...
>>> # Cosa contengono
>>> dir(Color)
['BLUE', 'GREEN', 'RED', '__class__', '__contains__', '__doc__', '__getitem__', '__in
>>> dir(Color.RED)
['__class__', '__doc__', '__eq__', '__hash__', '__module__', 'name', 'value']
>>> Color.RED.name    # etichetta
'RED'
>>> Color.RED.value   # valore associato
'R'
```

L'utilizzo base è chiaro dal punto di vista del codice

```
>>> print(Color.RED) # autocompletamento dopo . fornisce i colori disponibili
Color.RED
>>> print(Color("R"))
Color.RED
>>> Color("R") == Color.RED  # si può usare sia il valore che l'etichetta
True
>>> Color.RED is Color("R") # same thing direi
True
```

```
>>> def create_car(color: Color) -> None:
...     match color:
...         case Color.RED:
...             print("A red car was created")
...         case Color.BLUE:
...             print("A blue car was created")
...         case Color.GREEN:
...             print("A green car was created")
...         case _:
...             print(f"We don't have {color} in our database")
...

>>> create_car(Color.RED)
A red car was created

>>> # # in una classe
>>> # macchina.color = Color.RED
>>> # ...
>>> # if macchina.color == Color.BLUE:
>>> #     ...
```

6.8.3.2 Flag

Sono enum che permettono di combinare valori per indicare assiemi di valori (tipo set)

```
>>> from enum import Flag

>>> # Se definiamo a mano meglio usare le potenze del 2
>>> class Color(Flag):
...     RED: int = 1     # 00001
...     GREEN: int = 2   # 00010
...     BLUE: int = 4    # 00100
...     YELLOW: int = 8  # 01000
...     BLACK: int = 16  # 10000
...

>>> # Mix di colori
>>> red_and_yellow = Color.RED | Color.YELLOW

>>> # stampa del contenuto (tipo set)
>>> print(red_and_yellow)
Color.RED|YELLOW
>>> for col in red_and_yellow:
...     print(col)
...
Color.RED
Color.YELLOW
>>> # utilizzo (set/combination)
>>> cool_colors = Color.YELLOW | Color.BLACK | Color.RED # creo set colori
```

```
>>> my_car_color = Color.BLACK
>>> my_car_color in cool_colors # testo appartenenza a set di colori
True
```

Sotto la scocca una combinazione di gruppi di enum somma le relative costanti associate;

```
>>> red_and_yellow.value
9
```

nella definizione sopra non abbiamo impostato a 9 nessun colore nella definizione se no il programma si confonde e ci restituisce il colore impostato a 9. Tuttavia se impostiamo le costanti a mano potremmo fare danni

```
>>> # power of two
>>> class Wrong(Flag):
...     RED: int = 1      # 00001
...     GREEN: int = 2    # 00010
...     BLUE: int = 3     # 00100
...

>>> red_or_green = Wrong.RED | Wrong.GREEN
>>> red_or_green
<Wrong.BLUE: 3>
>>> red_or_green.value
3
```

Motivo per cui si può usare `auto` nella definizione (di `Flag` ed `enum`)

```
>>> # to prevent these errors, use auto in definition
>>> from enum import Flag, auto

>>> class Right(Flag):
...     RED: int = auto()      # auto assign a proper value
...     GREEN: int = auto()    # we don't have to worry
...     BLUE: int = auto()
...
>>> Right.RED
<Right.RED: 1>
>>> Right.GREEN
<Right.GREEN: 2>
>>> Right.BLUE
<Right.BLUE: 4>
```

Nel caso di `Enum` secche, auto non usa le potenze di due ma un progressivo numerico

```
>>> class Right2(Enum):
...     RED: int = auto()
...     GREEN: int = auto()
...     BLUE: int = auto()
...
```

```
>>> Right2.RED
<Right2.RED: 1>
>>> Right2.GREEN
<Right2.GREEN: 2>
>>> Right2.BLUE
<Right2.BLUE: 3>

>>> # btw with enums one cannot use | . errore
>>> Right2.RED | Right2.GREEN
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for |: 'Right2' and 'Right2'
```

Al termine, come informazione, Possiamo usare la combinazione anche in sede di definizione con `|`

```
>>> class Right2(Flag):
...     RED: int = auto()
...     GREEN: int = auto()
...     BLUE: int = auto()
...     ALL: int = RED | GREEN | BLUE # 1 + 2 + 4
...

>>> Right2.ALL
<Right2.ALL: 7>
```

6.8.4 Named tuples (`collections.namedtuple` e `typing.NamedTuple`)

Le named tuple rispondono alla necessità di un tipo ibrido tra

- tuple, che sono immutabili ma non si può accedere mediante nome/key (più facile rispetto a progressivo numerico)
- dict ai quali si può accedere mediante nome/key ma sono mutabili

Si ha che la definizione del tipo avviene mediante

- subclassando `typing.NamedTuple`: questo è il metodo consigliabile (a meno che il tipo di dati sia conoscibile solo a runtime) e richiede il type hint

  ```
  from typing import NamedTuple

  class Color(NamedTuple):
     red: int
     green: int
     blue: int

  c1 = Color(red = 55, green = 155, blue = 255)
  c2 = Color(55, 155, 255)

  # Accesso
  ```

```
c1
c1.red
c1.red = 123 # scrittura: errore (come tuple sono immutabili)
```

- `collections.namedtuple`, dove si specifica come primo argomento un identificatore (spesso con prima lettera maiuscola e coincidente con la variabile cui si assegna, per scimiottare una classe/tipo di dato) e secondo una lista di stringhe per i nomi degli attributi

```
>>> from collections import namedtuple

>>> Color = namedtuple("Color", ["red", "green", "blue"]) # create the class
>>> color1 = Color(blue = 55, green = 155, red = 255)     # create an instance
>>> color2 = Color(255, 255, 255)                         # another instance

>>> # Accesso
>>> color1
Color(red=255, green=155, blue=55)
>>> color1.red
255
>>> color1.red = 123 # scrittura: errore (sono immutabili)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: can't set attribute
```

6.8.5 dataclass

Il modulo `dataclasses` fornisce un decoratore da utilizzare con le classi che vogliamo battezzare come dataclass

```
from dataclasses import dataclass

@dataclass
class InventoryItem:
    """Class for keeping track of an item in inventory."""
    name: str
    unit_price: float
    quantity_on_hand: int = 0

    def total_cost(self) -> float:
        return self.unit_price * self.quantity_on_hand
```

Quello che questo decoratore fa è aggiungere un inizializzatore di default del tipo seguente, senza bisogno di specificarlo,

```
def __init__(self, name: str, unit_price: float, quantity_on_hand: int = 0):
    self.name = name
    self.unit_price = unit_price
    self.quantity_on_hand = quantity_on_hand
```

La `dataclass` aggiunge anche una `__repr__` gratuita per la stampa dei dati dell'oggetto (specificare `repr=False` nella funzione `field` esclude il dato dalla stampa)

6.8.5.1 `field`: dati mutabili, valori default, parametri

Se occorre specificare dati mutabili a livello di singola istanza (non condivisi) tra tutti gli oggetti di una determinata classe bisogna utilizzare `field` specificando la funzione utilizzata per creare l'istanza. Ad esempio se vogliamo un campo indirizzi email (lista di stringhe) che non sia condiviso tra tutti gli oggetti della classe dobbiamo programmare qualcosa del genere

```
from dataclasses import dataclass
from dataclasses import field

@dataclass
class Person:
    name: str
    address: str
    # email_addresses = []  # condiviso e sbagliato
    email_addresses: list[str] = field(default_factory=list)

l = Person(name = "Luca", address = "Via XYZ")
```

`field` e `default_factory` possono essere utilizzate per specificare una funzione che crea il valore assegnato di default se l'utente non specifica in chiamata, quindi ha anche altre applicazioni. Nel seguito la generazione di un id casuale

```
import random
import string

from dataclasses import dataclass
from dataclasses import field

# genera un id casuale
def generate_id() -> str:
    return "".join(random.choices(string.ascii_uppercase, k=12))

@dataclass
class Person:
    name: str
    address: str
    email_addresses: list[str] = field(default_factory=list)
    id: str = field(default_factory=generate_id)
```

Se si desidera che un parametro sia impostato ad un valore di default ma che non possa essere scelto dall'utente in chiamata si specifica `init=False` in field. Ad esempio per far sì che l'utente non possa scegliere l'id ma che questo sia generato da una funzione

```
@dataclass
class Person:
    name: str
    address: str
    email_addresses: list[str] = field(default_factory=list)
    id: str = field(init=False, default_factory=generate_id)
```

```
# questo da errore
l = Person(name="Luca", address="asd", id="foo")
```

Infine field accetta:

- un valore di default (utilizzato se l'utente non può o non vuole fornire in inizializzazione) con `default` (es `default=0` per un bank account saldo iniziale)
- `compare` (di default a `True`) che specifica se l'attributo è utilizzato nella comparazione (es per stabilire l'uguaglianza) o meno

6.8.5.2 Eseguire codice post inizializzazione: `post_init`

Se vogliamo eseguire del codice automaticamente post inizializzazione definiamo la funzione `__post_init__` che verrà eseguita appena dopo la init. Alcune applicazioni possono essere:

- **generare dati aggiuntivi** a partire da quelli inseriti. Ad esempio per creare una stringa per la ricerca a partire dai dati forniti in inizializzazione

  ```
  @dataclass
  class Person:
      name: str
      address: str
      email_addresses: list[str] = field(default_factory=list)
      id: str = field(init=False, default_factory=generate_id)
      search_string: str = field(init=False) # stringa che non può essere

      def __post_init__(self) -> None:
          self.search_string = f"{self.name} {self.address}"
  ```

- **validare i dati in ingresso** facendo fallire velocemente se l'inizializzazione è avvenuta con dati erronei

  ```
  @dataclass
  class Price:
      value: float

      def __post_init__(self) -> None:
          if self.value < 0:
              raise ValueError("Price must be non negative")
  ```

6.8.5.3 Freezing di una dataclass

Significa impostarla che una volta inizializzata i suoi dati non possano essere modificati (es mediante semplice assegnazione)

```
@dataclass(frozen=True)
class Person:
    name: str
    address: str
    ...
```

```
l = Person(...)
l.name = "foo" # da errore
```

Ocio che anche i `post_init` falliranno se come sopra assegnano alla classe

6.8.5.4 Altri parametri utili del decoratore

Oltre a `frozen` vi sono altri parametri interessanti per il decoratore:

- `kw_only=True` in inizializzazione bisognerà specificare per esteso nome = valore e non verrà accettato il valore associato alla posizione della chiamata
- con `match_arg=False` si disabilita il structural pattern matching (uso con `match`) abilitato di default
- `slots=True`: sotto la scocca una dataclass è un dizionario molto avanzato. Se si abilita `slot` vi è un accesso molto più rapido e di base (miglioramenti del 20% a seconda dei casi). Non si usa di default perché slots rompe tutto quando si usa ereditarietà multipla, es quanto segue non funziona perché non si possono sommare dataclass basate su slots:

```
@dataclass(slots=True)
class Person:
   name:str
   address: str
   email: str

@dataclass(slots=True)
class Employee:
    dept: str

class Worker(Person, Employee):
    pass
```

6.8.6 Queues

Le liste sono utili/efficienti se:

- i dati non sono troppi
- occorre effettuare un accesso casuale/veloce a qualsiasi elemento; tendono ad essere piu lente se si inseriscono elementi non al termine della lista

Le code sono una alternativa e sono utili per modellare situazioni in cui

- si processano informazioni nell'ordine in cui sono arrivate, togliendole dalla lista delle cose da fare (FIFO o *queues* in senso stretto)
- si processano info in ordine inverso (prima le cose arrivate per ultime, LIFO o *stacks*)
- si processano info sulla base di una priorità assegnata (*priority queues*)

Le code sono meglio anche per la memoria in quanto

- una volta che l'informazione è stata processata non serve più e viene eliminata (es pensare big data in ingresso).
- si può impostare un limite massimo di dimensione della coda; se piena non si accetteranno dati in ingresso

Le varie tipologie di code si creano mediante le classi del modulo `queues`. I metodi principali che implementano sono `put` (aggiungere un elemento in coda) e `get` (rimuovere il prossimo elemento dalla coda)

6.8.6.1 Queues (FIFO queues)

Processano l'input nell'ordine in cui è arrivato

```
>>> from queue import Queue

>>> # istanziazione
>>> work = Queue()          # senza limiti
>>> mwork = Queue(maxsize=3) # con limite

>>> # inserimento e rimozione dalla coda
>>> mwork.put("task1")  # inserimento
>>> mwork.put("task2")
>>> mwork.get()         # rimozione del primo elemento, viene ritornato
'task1'
>>> mwork.put("task3")

>>> # metodi utili
>>> mwork.qsize() # numero di elementi in coda
2
>>> mwork.queue   # stampa degli elementi in coda
deque(['task2', 'task3'])
>>> mwork.empty() # check se vuota, anche not_empty
False
>>> mwork.full()  # check se piena, anche not_full (pertinente per queue limitate)
False
```

6.8.6.2 Stacks (LIFO queues)

Processano l'input nell'ordine inverso in cui è arrivato

```
>>> from queue import LifoQueue

>>> lifo = LifoQueue()
>>> lifo.put("task1")  # inserimento
>>> lifo.put("task2")
>>> lifo.put("task3")
>>> lifo.get()         # rimozione dell'ultimo elemento, viene ritornato
'task3'
>>> lifo.queue
['task1', 'task2']
```

6.8.6.3 Heaps (priority queues)

Processano l'input

- che viene specificato mediante una tupla `(priorita, input)`
- nell'ordine di priorità dalla più bassa (più urgente) alla più alta (meno urgente)

```
>>> from queue import PriorityQueue

>>> lifo = PriorityQueue()
>>> lifo.put((2, "task1"))  # inserimento
>>> lifo.put((0, "task2"))
>>> lifo.put((3, "task3"))
>>> lifo.queue
[(0, 'task2'), (2, 'task1'), (3, 'task3')]
>>> lifo.get()         # rimozione dell'elemento più urgente
(0, 'task2')
```

6.9 Altri argomenti

6.9.1 Estensione di classi base

Supponiamo di avere un container builtin al quale vogliamo aggiungere funzionalità; abbiamo due opzioni:

- *composition*: creiamo un nuovo oggetto che contiene il container base come attributo
- *inheritance*: subclassiamo l'oggetto base e aggiungiamo o sovrascriviamo/customizziamo i suoi metodi

Solitamente la

- composizione è meglio se tutto quel che vogliamo fare è utlizzare il container per contenere oggetti e nulla più; i metodi custom della classe elaboreranno gli oggetti nel container in sequenza;
- ereditarietà è meglio se vogliamo modificare le funzionalità base del tipo base.
  Ad esempio per fare sì che ad una lista si possano aggiungere solo determinati tipi dobbiamo sovrascrivere `append`, `extend` e `__setitem__` per sollevare eccezioni qualora i dati passati non rispettino i criteri.
  Per vedere quali metodi sono disponibili in una classe (oltre a leggerne la documentazione) istanziarne un oggetto e farne `dir` (o fare `dir` su una classe utilizzata per generarlo, come sotto) dopodiché leggere la documentazione con help

  ```
  >>> dir(list)
  ['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__'
  >>> dir(dict)
  ['__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__'
  >>> # help(list.__setitem__)
  ```

Primo esempio property che aggiunge elaborazioni on the fly ad una lista (in questo caso aggiungendo la media)

```
>>> class ListWithAverage(list):
...     @property
...     def average(self):
...         return sum(self) / len(self)
...

>>> a = ListWithAverage([1, 2, 3, 4])
>>> a  # sembra una lista normale ma...
[1, 2, 3, 4]
>>> a.average
2.5
```

Secondo esempio Magari non serve una classe complessa, ma una base opportunamente modificata è sufficiente per l'applicazione.
Vogliamo creare una lista di contatti che abbia una funzionalità di ricerca: lo facciamo implementando Contact che al suo interno contiene la lista comune di tutti i contatti (istanza di ContactList) che a sua volta ha la funzione di ricerca. Quest'ultima

```
>>> class ContactList(list):
...     def search(self, name):
...         """Return all contacts that contain the search value
...         in their name."""
...         matching_contacts = []
...         for contact in self:
...             if name in contact.name:
...                 matching_contacts.append(contact)
...         return matching_contacts
...

>>> class Contact:
...     all_contacts = ContactList()
...     def __init__(self, name, email):
...         self.name = name
...         self.email = email
...         Contact.all_contacts.append(self)
...

>>> c1 = Contact("John A", "johna@example.net")
>>> c2 = Contact("John B", "johnb@sloop.net")
>>> c3 = Contact("Jenna C", "cutty@sark.io")

>>> search_result = Contact.all_contacts.search("John")
>>> [r.name for r in search_result]
['John A', 'John B']
```

6.9.2 Manager Object

Da phillips si specifica in una classe madre alcune operazioni standard e si demanda alle figlie l'implementazione delle specificità. Un esempio con un handler di file zip:

- la init della classe madre (`ZipProcessor`) prende in input i dati necessari a tutte le implementazioni. Se le figlie hanno bisogno di dati aggiuntivi (ad esempio `ZipReplace`, non `ScaleZip`) eseguono quella della madre ed aggiungono i dati necessari per i metodi specifici delle figlie
- la classe madre ha una funzione (qui `process_zip`) che orchestra l'elaborazione tipica e funzioni di supporto che sono comuni (`unzip` e `zip`), mentre lascia alle figlie l'implementazione di funzionalità specifiche (`process_files`) che può essere diversa da processing a processing
- le classi figlie debbono solo aggiungere eventualmente dati nell'init e specificare la funzione che effettua l'elaborazione peculiare

```
import sys
import shutil
import zipfile
from pathlib import Path
from PIL import Image


class ZipProcessor:
    def __init__(self, zipname):
        self.zipname = zipname
        self.temp_directory = Path(f"unzipped-{zipname[:-4]}")

    def process_zip(self):
        self.unzip_files()
        self.process_files()
        self.zip_files()

    def unzip_files(self):
        self.temp_directory.mkdir()
        with zipfile.ZipFile(self.zipname) as zip:
            zip.extractall(self.temp_directory)

    def zip_files(self):
        with zipfile.ZipFile(self.zipname, "w") as file:
            for filename in self.temp_directory.iterdir():
                file.write(filename, filename.name)
        shutil.rmtree(self.temp_directory)


class ZipReplace(ZipProcessor):
    def __init__(self, filename, search_string, replace_string):
        super().__init__(filename)
        self.search_string = search_string
```

```
        self.replace_string = replace_string

    def process_files(self):
        """perform a search and replace on all files in the
        temporary directory"""
        for filename in self.temp_directory.iterdir():
            with filename.open() as file:
                contents = file.read()
            contents = contents.replace(self.search_string, self.replace_string)
            with filename.open("w") as file:
                file.write(contents)


class ScaleZip(ZipProcessor):
    def process_files(self):
        """Scale each image in the directory to 640x480"""
        for filename in self.temp_directory.iterdir():
            im = Image.open(str(filename))
            scaled = im.resize((640, 480))
            scaled.save(filename)
```

Capitolo 7

Eccezioni

Contents

7.1 Sollevare eccezioni 62
7.2 Gestire eccezioni 62
7.2.1 `try` 62
7.2.2 Tips 63
7.2.3 Eccezioni particolari 63
7.3 Eccezioni custom 64
7.3.1 Definizione e utilizzo 64
7.4 Warnings 64
7.4.1 Sollevare warnings 65
7.4.2 Ignorare warnings 65

Le eccezioni sono classi come altre e vengono utilizzate per gestire situazioni particolari, generalmente considerate errore. Si sollevano mediante `raise` e si gestiscono mediante `try`. Servono per

- gestire il flusso della computazione
- fornire un meccanismo di comunicazione tra diverse parti del codice

Tutte le eccezioni ereditano da `BaseException`: nelle builtin si hanno 4 eccezioni principali, di cui la più importante è `Exception`, che viene subclassata da tutte le eccezioni ordinarie. Nella definizione di eccezioni proprie occorre subclassare da `Exception`.

```
BaseException
 +-- BaseExceptionGroup
 +-- GeneratorExit
 +-- KeyboardInterrupt
 +-- SystemExit
 +-- Exception
      +-- ArithmeticError
      |    +-- FloatingPointError
      |    +-- OverflowError
      |    +-- ZeroDivisionError
```

```
      +-- AssertionError
      +-- AttributeError
      +-- BufferError
      +-- EOFError
      +-- ExceptionGroup [BaseExceptionGroup]
      +-- ImportError
      ...
```

Per la lista aggiornata consultare `https://docs.python.org/3/builtins/exceptions.html`.

7.1 Sollevare eccezioni

Le eccezioni vengono sollevate mediante lo statement `raise`

```
raise NameError("Messaggio informativo")
```

Questo fa terminare l'esecuzione a meno che l'eccezione non sia gestita mediante `try`

Example 7.1.1. Come esempio creiamo una lista in append accetta solo interi dispari, ne fa il check e solo nel caso l'elemento sia valido lo aggiunge mediante il metodo standard della parent

```
>>> class ListaPari(list):
...     def append(self, integer):
...         if not isinstance(integer, int):
...             raise TypeError("Solo interi ammessi")
...         if integer % 2:
...             raise ValueError("Solo numeri pari")
...         # Qui i check sono passati
...         super().append(integer)
...

>>> l = ListaPari()
>>> l.append(2)
>>> l.append(1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 6, in append
ValueError: Solo numeri pari
>>> l.append("a")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 4, in append
TypeError: Solo interi ammessi
```

7.2 Gestire eccezioni

Supponendo che il codice sollevi una eccezione, per gestirla usiamo lo statement `try` che nella versione più generale ha la seguente struttura

```
try:
    code
except ExceptionA:
    handling_A
except (ExceptionB, ExceptionC):
    handling_BC_with_same_code
else:
    code_if_no_exception_raised
finally:
    code_done_in_every_case
```

7.2.1 `try`

La sintassi minimale prevede solo lo statement `try` e una `except`. Sono opzionali ulteriori `except`, `else` e `finally`. Nella versione più completa viene innanzitutto eseguito il codice sotto `try` e se:

- *non viene sollevata alcuna eccezione* `try` termina normalmente
- *viene sollevata una eccezione* si cerca il match della classe nell'ordine specificato dalla sequenza di `except`; le eccezioni possono essere specificate in una tupla per gestirle con lo stesso codice, se non si specifica niente (`except:` secca) viene matchata `BaseException` che tipicamente non è quello che si vuole (vedi sotto in eccezioni particolari, anch'esse matchate da `BaseException`).
  Se
  - viene trovato un match si esegue il relativo codice
  - altrimenti l'eccezione viene trasmessa a eventuali istruzioni `try` di livello superiore. ad esempio il seguente codice intercetta l'errore sollevato nel try interno mediante una eccezione gestita dal try esterno (stampando (`int(a) gone wrong`))

    ```
    >>> try:
    ...     try:
    ...         int("a")
    ...         raise TypeError("boo")
    ...     except TypeError:
    ...         print("raised boo")
    ... except ValueError:
    ...     print("int(a) gone wrong")
    ...
    int(a) gone wrong
    ```

  - se anche in `try` di livello superiore non matchano (o non esistono) l'esecuzione viene interrotta
- se `else` è presente il suo codice viene eseguito solo se la `try` va a buon fine (non solleva alcuna eccezione)

```
for arg in sys.argv[1:]:
    try:
        f = open(arg, 'r')
```

```
except IOError:
    print('cannot open', arg)
else:
    print(arg, 'has', len(f.readlines()), 'lines')
    f.close()
```

- se `finally` è presente il relativo codice viene eseguito al termine (in ogni caso sia in caso positivo che fail)

```
>>> def divide(x, y):
...     try:
...         result = x / y
...     except ZeroDivisionError:
...         print("Division by zero!")
...     else:
...         print("Result is", result)
...     finally:
...         print("Executing finally clause")
...
>>> divide(2, 1)
Result is 2.0
Executing finally clause
>>> divide(2, 0)
Division by zero!
Executing finally clause
>>> divide("2", "1")
Executing finally clause
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in divide
TypeError: unsupported operand type(s) for /: 'str' and 'str'
```

  Nelle applicazioni `finally`

  - è utile per pulizia, rilasciare risorse esterne (file, network connection) indipendentemente dal fatto che l'uso della risorsa sia andata a buon fine.
  - è utile quando, all'interno di una funzione, nel codice della `try` eseguiamo una `return`; in tal caso comunque la `finally` verrà eseguita

  Un esempio di uso di `finally` per liberare risorse (lock di un magazzino) a pag 112/113 di Python3 object oriented programming

Notare che se non vi sono eccezioni, sia `else` che `finally` vengono eseguite.

7.2.2 Tips

Per

- **ignorare una eccezione** e permettere al programma di proseguire (ma non è best practice) porre `pass` nel relativo chunk oppure `print` con un messaggio informativo. Alternativamente se il `try` è all'interno id un loop `for` si potrebbe dare `continue` per passare al prossimo elemento

- **aggiungere istruzioni ma far fallire** comunque l'eccezione, nella `except` inserire le istruzioni e poi chiudere con `raise` che solleva la medesima eccezione (o se si vuole esser più espliciti usare `as` e fare il raise dell)

```
>>> try:
...     1 / 0
... except ZeroDivisionError:
...     print("zero at denominator man")
...     raise
...
zero at denominator man
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
ZeroDivisionError: division by zero
>>> # # same
>>> # try:
>>> #     1 / 0
>>> # except ZeroDivisionError as zde:
>>> #     print("zero at denominator man")
>>> #     raise
```

- **ottenere l'eccezione sollevata** utilizzare `as` dopo l'eccezione; in questo modo si possono usare i suoi dati/metodi definiti dalla classe di appartenenza. Ad esempio le eccezioni memorizzano nell'attributo `.args` il messaggio con cui sono state sollevate

```
>>> try:
...     1 / 0
... except ZeroDivisionError as e:
...     print(dir(e)) # just to know what inside
...     print(e.args)
...     e.add_note("hello there!")
...     raise e
...
['__cause__', '__class__', '__context__', '__delattr__', '__dict__', '__dir__', '__doc__',
('division by zero',)
Traceback (most recent call last):
  File "<stdin>", line 7, in <module>
  File "<stdin>", line 2, in <module>
ZeroDivisionError: division by zero
hello there!
```

- **matchare qualsiasi eccezione** utilizzare `Exception` (che è parent di tutte le eccezioni pertinenti per questa applicazione). Best practice porla **alla fine** della sequenza di `except` per matchare tutte le eccezioni rimanenti/non gestite da codice più specifico (e ad esempio loggarle da qualche parte)

```
import logging

try:
    code
```

```
except ExceptionA:
    ...
except ExceptionB:
    ...
except Exception as e:
    logging.exception(e)
```

- bloccare il programma e uscire da Python a causa di una **eccezione grave** si può usare `sys.exit`

```
try:
    ...
except SevereError as e:
    print(e)
    sys.exit(1)
```

7.2.3 Eccezioni particolari

Di solito le eccezioni normali ereditano da `Exception`. Oltre a questa ve ne sono altre, uniche ad ereditare direttamente da `BaseException`:

- `SystemExit` è l'eccezione che viene sollevata quando si esce dal processo/chiude Python, tipicamente perché chiamiamo `sys.exit`.
  La si gestisce di rado, solamente se vogliamo eseguire codice prima di uscire; quello che si fa è catturarla, eseguire il codice di interesse e poi ri-sollevarla (altrimenti il programma non termina).

```
>>> import sys

>>> # this inhibit exiting!! no!!
>>> try:
...     sys.exit()
... except SystemExit as e:
...     print("nope you remain here")
...
nope you remain here

>>> # # this exit with message
>>> # try:
>>> #     sys.exit()
>>> # except SystemExit as e:
>>> #     print("Goodbye!")
>>> #     raise e
```

  Di solito *non* la vogliamo gestire a meno che vogliamo assicurarci che alcune azioni vengano effettuate

- `KeyboardInterrupt` è sollevata quando l'utente da Ctrl+C a tastiera (o `C-c C-c` nel sottoprocesso python di emacs) ed è tipicamente utilizzata nei comandi testuali. Dovremmo trattarla similmente a `SystemExit` quindi eseguire le operazioni necessarie e risollevarla. Quello di sotto è un utilizzo simpatico che porta l'utente ad inserire un intero con messaggi motivazionali se cerca di interrompere mediante Ctrl+C

```
# User must obey
x = "a string"
while not isinstance(x, int):
    try:
        x = int(input("Insert integer: "))  # hit ctrl+c here
    except KeyboardInterrupt as e:
        print("\nYou can't escape MF!")
    except Exception:
        # ignore anything that can go wrong (eg insert string/float etc)
        # and ask again until we get an integer
        pass

print(f"Good job: {x}")
```

7.3 Eccezioni custom

Se vogliamo sollevare una eccezione ma tra le builtin non troviamo quella pertinente/specifica possiamo definirla. Motivi per definire le proprie eccezioni è fornire maggiori info e funzionalità specifiche in caso di errore, nonché permettere comunicazione tra le varie parti del codice (quella che solleva l'eccezione e quella che la gestisce).
Le eccezioni non debbono essere considerate eccezionali ma sono incoraggiate; filosoficamente possiamo alternativamente mediante `if` annidati controllare tutte le condizioni possibili sotto le quali il codice fallisce (ed eseguirlo solo nelle condizioni migliori), oppure eseguirlo sotto una `try` e gestire tutte le possibili eccezioni:

- se queste sono molte la soluzione con `try` può essere più chiara di una serie di `if` annidati volti a eseguire il codice solo in alcuni casi
- se errori/casi particolari sono rari/meno frequenti, la soluzione con `try` è più efficiente perchè si evita di dover controllare le condizioni ogni volta.

7.3.1 Definizione e utilizzo

Si definiscono

- *ereditando da una classe base* (che in assenza può essere `Exception`) in modo da poter creare gerarchie di eccezioni.
- dando un *nome di classe* che comunica quello che è andato storto;
- a livello di *implementazione* possono essere semplicemente un `pass`

  ```
  class MyExceptionName(Exception):
      pass
  ```

  Una volta definita si potrà sollevarla mediante `raise`, ad esempio all'interno di un `try`, con un un template del genere

  ```
  try:
      some_risky_code
  ```

```
    if test:
        raise MyExceptionName("message")
except MyExceptionName as e:
    do_things
```

- essendo una classe normale possiamo sovrascrivere l'`__init__` fornendo una serie di parametri che saranno utilizzabili in sede di `raise`.
  Se si eredita da `Exception` questa salva tutti gli argomenti passati in raise nella tupla `.args` (facendo sì che si possa definire vari metodi senza dover sovrascrivere l'init, abbastanza generica)

```
>>> class InvalidDataset(Exception):
...     pass
...
>>> try:
...     raise InvalidDataset("foo", "bar", "baz")
... except InvalidDataset as e:
...     # what's inside a generic error
...     print(dir(e))
...     # what was passed during raise
...     print(e.args)
...     # ok let's fail
...     raise e
...
['__cause__', '__class__', '__context__', '__delattr__', '__dict__', '__dir__',
('foo', 'bar', 'baz')
Traceback (most recent call last):
  File "<stdin>", line 9, in <module>
  File "<stdin>", line 2, in <module>
__console__.InvalidDataset: ('foo', 'bar', 'baz')
```

Possiamo chiaramente sovrascrivere l'inizializzatore e fornire altre funzioni che potranno essere utili in sede di gestione dell'errore. Un esempio

```
>>> class InvalidWithdrawal(Exception):
...     def __init__(self, available, requested):
...         super().__init__(f"Sorry you don't have requested {requested}")
...         self.available = available
...         self.requested = requested
...     @property
...     def missing_amount(self):
...         return self.requested - self.available
...

>>> avail = 50
>>> req = 100
>>> try:
...     if avail < req:
...         raise InvalidWithdrawal(avail, req)
... except InvalidWithdrawal as e:
...     print(f"Available: {e.available}, Requested: {e.requested}")
...     print(f"Sorry, missing {e.missing_amount}")
```

```
...     raise e
...
Available: 50, Requested: 100
Sorry, missing 50
Traceback (most recent call last):
  File "<stdin>", line 7, in <module>
  File "<stdin>", line 3, in <module>
__console__.InvalidWithdrawal: Sorry you don't have requested 100
```

7.4 Warnings

Non stoppano l'esecuzione

7.4.1 Sollevare warnings

```
import warnings
warnings.warn("Warning message")
```

7.4.2 Ignorare warnings

```
with warnings.catch_warnings(action="ignore"):
    code_raising_warnings()
```

Capitolo 8

Devtools

Contents

8.1 Debugging 67
8.2 Testing 69
8.2.1 Introduzione 69
8.2.2 `pytest` 70
8.2.3 Testing con dati/input randomici 73
8.2.4 Doctest 73
8.2.5 `unittest` 74
8.3 Linting/formattazione (`ruff`) 74
8.3.1 Linting 74
8.3.2 Formattazione 75
8.4 Timing/temporizzazione 75
8.5 Profiling 75
8.6 Scrittura di plugin 76
8.7 Documentazione 76
8.7.1 Setup 77
8.7.2 Doc-writing e reStructuredText 77
8.7.3 Building 77
8.7.4 Setup di readthedocs 78

8.1 Debugging

Lo strumento è il modulo `pdb`. Alcune applicazioni:

- per emulare `browser` di R porre una chiamata a

  ```
  breakpoint()
  ```

  dove necessario. Questo fa andare in pausa l'esecuzione appena il flusso arriva nel punto, dopodiché entra in modalità debugging.
  In modalità debugging (scritta (Pdb) ad inizio linea) i comandi principali utilizzabili sono in tabella 8.1;

- per simulare il `traceback` di R, al fine di ispezionare la stack dove l'errore si è verificato utilizzare `pdb.post_mortem()` in un template del genere con try-except:

```
import pdb

def f(x):
    result = 1 / x
    return result

try:
    f(0)
except:
    pdb.post_mortem()
```

  dopodiché si è in modalità debugging e ci si può muovere con `up` e `down` nella stack;

- per eseguire uno script in modalità debugging/linea per linea (senza porre `breakpoint()` all'inizio)

```
python3 -m pdb app.py arg1 arg2
```

Command	Description
`h`	help
`h topic`	help su un topic
`h pdb`	documentazione `pdb`
`q(uit)`	chiudi debugger
`ENTER`	ripeti il comando precedente
`dir()`	listare variabili disponibili (as usual)
`p`	stampa espressione (es variabile)
`pp`	prettyprinta espressione
`display expr`	monitora variazioni ad una espressione (eg variabile)
`display`	lista le espressioni monitorate)
`undisplay expr`	togli monitoraggio
`b`	lista i breakpoint o settane uno alla linea specificata
`n(ext)`	continua l'esecuzione sino alla prossima linea della funzione corrente (eg se no scende nelle librerie)
`s(tep)`	esegui linea corrente, stoppa alla prima occasion (in funzione chiamata o funzione corrente) current (Step into a subroutine)
`c(ontinue)`	continua l'esecuzione e stoppa al prossimo breakpoint
`unt n`	continua l'esecuzione sino a linea numero `n`
`r(eturn)`	Return out of a subroutine
`w`	stampa la stack (frame più recente alla fine)
`u(p)`	sali nella stack
`d(own)`	scendi nella stack
`l(ist)`	Lista il codice attorno alla posizione corrente
`ll`	lista il codice della funzione o frame corrente

Tabella 8.1: Comandi debug

8.2 Testing

8.2.1 Introduzione

8.2.1.1 Gergo

test pezzo di codice che verifica un comportamento

test suite insieme di test del progetto

assertion specifica del comportamento atteso (eg `assert`)

fixture setup richiesto per un test

test driven development l'impostazione dei test sulle funzionalità avviene *prima* che queste vengano implementate. La filosofia del TDD è:

1. scrivi test completi che falliscono
2. scrivi codice fino a farli passare

Vantaggi

- si specificano a priori tutti i comportamenti necessari
- evita overcoding: una volta che i test passano siamo a posto
- in refactoring possiamo lavorare tranquillamente garantendoci da regressioni

8.2.1.2 Tipologie di testing

Si divide il testing in:

unit testing di singole funzionalità (es funzione/classe)

functional test di funzionalità più generali/complessive (derivante dall'interazione di più funzioni di base)

regression : test che l'output di un programma non cambi a successive versioni (a meno che ciò non sia intenzionale). Basati su output dell'esecuzione di versioni passate (validate ad occhio) o di programmi benchmark

8.2.1.3 Good testing practice

I test debbono:

- essere specifici, indipendenti e svolti in maniera automatica
- testare in caso di input corretto, l'output corretto
- testare in caso di input erroneo, la gestione corretta delle eccezioni
- dovrebbero teoricamente testare tutto l'input possibile

8.2.1.4 Fasi di un test

Sono

- *fixture setup*: setup della situazione/condizione precedente al test. È tutto ciò che rende il codice di test eseguibile
- esecuzione del test: idealmente una singola linea/cosa compatta
- verifica dei risultati
- *fixture teardown*: rimozione/cleaning delle condizioni nelle quali il test si è svolto (es liberazione risorse/file/io/etc)

Al fine di gestire setup e che teardown/pulizia siano eseguiti automaticamente indipendentemente dall'esito del test

- un ottimo modo sono i context manager con `with` (che garantiscono l'esecuzione di codice indipendentemente dall'esito delle valutazioni nel contesto)

```
with tempfile.TemporaryDirectory() as tmpdir:      # setup e teardown
   testfile = Path(tmpdir) / "test.txt"            # setup
   testfile.write_text("1\n2\n")                   # fine setup
   result = do_check(testfile)
   assert result == right_value
   # cleanup done by default
```

- alternativamente utilizzare try/finally in un template del genere

```
fixture_setup_test() # stup del test
try:
   result = do_check()          # esecuzione test
   assert result == right_value # verifica risultati
finally:
   codice_pulizia() # teardoum test
```

8.2.2 pytest

8.2.2.1 Setup

Creiamo

- una directory `tests` nella radice del progetto
- in essa diversi file dal nome `test_funzione.py` o `test_modulo.py`
- ciascun file ha
  - funzioni `test_soggetto_comportamento` dove soggetto è il nome della funzione o del modulo e comportamento è quello che viene testato. Es `test_sumfile_ignores_blanklines`
  - eventualmente classi di nome `TestXyzAsd`, senza metodo `__init__`, di cui verranno eseguiti i metodi `test_*`
- facciamo il setup di `pytest` con

```
uv add --dev pytest
```

8.2.2.2 Esecuzione dei test

Mediante uv attraverso le seguenti alternative, che verranno approfondite nel seguito

```
# esegui tutti i test
uv run pytest

# esegui in maniera quiet, utile per evidenziare solo i grossi cambiamenti
uv run pytest -q

# bloccare al primo errore o primi 3 (per stare focused e non eseguire tutto
uv run pytest -x          # exitfirst
uv run pytest --maxfail=3 # blocca al terzo errore

# per entrare in modalità debug, oltre a piazzare breakpoint qua/la si può
uv run pytest -x --pdb  # vai in modalità debug al primo errore
uv run pytest --trace file.py   # vai tipo in modalità browser di R

# verbose: fornisce test id ("file::funzione[parametrizzazione]")
uv run pytest -v

# eseguire un file specifico
uv run pytest test/test_xyz.py

# eseguire solamente un test specifico
uv run pytest file::funzione
uv run pytest "file::funzione[parametrizzazione]"

# eseguire test con un mark/tag
uv run pytest -m fast # i fast
uv run pytest -m "not slow" # tutti ad eccezione degli slow


# esegue i test filtrandoli per nome/test id (qui che hanno "filtrotestid" nel nome)
uv run pytest -k filtrotestid
uv run pytest -k "filtro1 or filtro2" # logica booleana semplice
uv run pytest -k "(api or db) and not slow" # logica booleana meno semplice
```

Per altre opzioni di run vedere Harrison, “Effective testing” a pag 74-75

8.2.2.3 Contenuto file di test

Per

- testare un singolo valore logico che deve essere `True` per non fallire utilizzare `assert`

  ```
  def test_assert():
     assert 1 == 1
  ```

- testare che una espressione sollevi una eccezione utilizzare `pytest.raises` con `with`

```
def test_exception():
   with pytest.raises(TipoEccezione):
       codice_che_solleva_TipoEccezione
```

  Importante essere ragionevolmente specifici su `TipoEccezione`

- per testare l'uguaglianza di due `pd.Series` utilizzare `pd.testing.assert_series_equal` o, nel caso di `pd.Categorical`, `pd.testing.assert_extension_array_equal`. Alcuni esempi da `pylbmisc`

```
def test_to_integer_string():
    series = pd.Series(["2001", "2011", "1999", np.nan])
    expected = pd.Series([2001, 2011, 1999, pd.NA], dtype="Int64")
    result = to_integer(series)
    pd.testing.assert_series_equal(result, expected)


def test_to_sex():
    series = pd.Series(["", "m", "f", " m", "Fm", np.nan])
    expected = pd.Categorical(
         [pd.NA, "male", "female", "male", "female", pd.NA],
         categories=["male", "female"]
    )
    result = to_sex(series)
    pd.testing.assert_extension_array_equal(result, expected)
```

8.2.2.4 Fixture

Se la preparazione per test diventa complessa o ripetitiva può essre utile inglobarla in una fixture:

- è una funzione decorata con `@pytest.fixture`

```
@pytest.fixture
def dipendenza():
    return valore
```

- il nome della funzione decorata viene passata come parametro input alla funzione di test. `pytest` eseguirà la funzione fixture è fornirà (al test in chiamata) il valore ritornato da questa

```
def test_xyz(dipendenza):
    assert dipendenza == 1
```

- è possibile mettere tutte queste fixture in un file `conftest.py`, *non* da importare nel file di test. Se i test sono organizzati in directory gerarchiche, `conftest.py` fornirà fixture nella cartella corrente e nelle figlie

Per maggiori info su fixture vedere Harrison da pag 77

8.2.2.5 Marking di test

In pratica categorizza/tagga i test per poterli eseguire selettivamente. Per fare il setup:

- in `pyproject.toml` segnalare per evitare warning

  ```
  [tool.pytest.ini_options]
  markers = [
     "fast: quick checks for busy people",
     "slow: slower test"
  ]
  ```

- per markare un test come `fast`

  ```
  @pytest.mark.fast
  def test_xyz():
      code
  ```

- utilizzare `-strict-markers` nelle opzioni di pyproject, per fare in modo che non vengano utilizzati marker che non sono validati/presenti nella lista sopra (es evitare typo tipo `pytest.mark.slowww`)

- per eseguire i test fast poi

  ```
  uv run pytest -m fast # i fast
  ```

8.2.2.6 Esecuzione condizionale di test

Si implementano con `@pytest.mark.skipif`. Ad esempio se si vuole eseguire un test solo negli ambienti dove redis è disponibile

```
def redis_available():
     try:
        network_cheks()
        return True
     except OSError:
        return False

@pytest.mark.skipif(
   not redis_available(),
   reason="Redis server not available"
)
def test_cache_limit:
    # the_test
```

8.2.2.7 Failure accettate/expected

Se si vuole che failure siano viste e riportate ma non diano errore (es in un Makefile, per automazione) markare con `xfail`

```
@pytest.mark.xfail(reason="ancora non implementato")
def test_che_fallisce():
    #...
```

Quando poi il test inizia a passare (denotato con X, unexpected pass o `xpassed`) togliere il decoratore l'`xfail` per avere un test di regressione

8.2.2.8 Parametrizzazione

Serve per validare una funzionalità su diversi input ricevuti. `pytest.mark.parametrize` prende in input

- una stringa con i nomi dei parametri (nel caso più semplice 2, input fornito e output atteso)
- una lista di tuple con corrispondenti valori

```
# funzione da testare in src
def add1(x):
   return x+1

# test in tests
@pytest.mark.parametrize("inp, expected_out", [(1, 2), (5,6)])
def test_add1(inp, expected_out):
   assert add1(inp) == expected_out
```

Si possono fornire id informativi del test svolto specificando `ids`, che nel caso possono esser filtrati mediante -k

```
@pytest.mark.parametrize("inp, expected_out", [(1, 2), (5,6)], ids=["1+1==2","5+1==6"])
def test_add1(inp, expected_out):
   assert add1(inp) == expected_out
```

`skipif` e `xfail` con parametrizzazione Nel caso si voglia skippare condizionalmente o accettare fallimento di alcuni casi forniti come parametrizzazione bisogna crearli con `pytest.param` specificando i parametri standard, eventualmente l'id e specificando il marking/decorazione con `marks`:

```
import pytest
import os

# # in the library
# def add1(x):
#    return x+1


add1_test_params = [
   (1, 2),  # test eseguito sempre
   (5, 6),  # idem
   # expected failure
   pytest.param(1, 3, id="this_will_fail",
                marks=pytest.mark.xfail(reason="well, input...")),
   # don't run on windows
   pytest.param(1, 4, id="skipif_thecondition",
                marks=pytest.mark.skipif(os.name == "posix", reason="skipped on linux...")),
```

```
]

@pytest.mark.parametrize("inp, expected_out", add1_test_params)
def test_add1(inp, expected_out):
   assert add1(inp) == expected_out
```

8.2.3 Testing con dati/input randomici

Vedere la libreria Hypothesis per stresstest vari (Harrison pag 187)

8.2.4 Doctest

Molto utilizzato da librerie scientifiche (`numpy`, `pandas`), o test si possono definire nella documentazione (docstring) di funzioni o moduli:

- si effettua qualche prova alla REPL di python di esempio di utilizzo (semplice/compatto) della funzione; una volta perfezionato ...
- si scrive l'esempio di utilizzo della funzione (sia codice che output);
- dopodiché `doctest` esegue il codice verificando che l'output prodotto combaci

Il beneficio è mantenere documentazione e testing sincronizzati e gli esempi di codice accurati/aggiornati/funzionali.
Per controllare in una directory di progetto, sia doctest che pytest:

```
uv run pytest --doctest-modules
```

8.2.4.1 Skippare test

A volte si desidera che il codice sia fornito solo come esempio e non testato (es per motivi di tempo o altro, equivalente di `dontrun` in roxygen2), nel caso porre `# doctest:  +SKIP` nella linea che si vuole skippare

```
Examples
--------
>>> import time
>>> from pylbmisc.utils import timeit
>>>
>>> @timeit()
... def test():
...     time.sleep(1)
...
>>> test()                 # doctest: +SKIP
Function test [args=() kwargs={}] took 1.00 secs.
```

8.2.4.2 Normalizzare spazi bianchi

Utilizzare a fianco delle linee che sollevano errori apparentemente esistenti (es due linee confrontate sono uguale ma differiscono per il numero di spazi bianchi)

```
# doctest: +NORMALIZE_WHITESPACE
```

8.2.5 unittest

Il modulo classico per lo unit testing in Python è `unittest`, orientato alle classi. Commentato nel sorgente, nel caso sia necessario rivedere alla luce di Harrison "Effective testing" pag 47

8.3 Linting/formattazione (ruff)

ruff serve per

- linting: ossia controllo statico di codice, senza eseguirlo, per evidenziare possibili problemi
- formatting: ossia riscrittura deterministica per seguire un certo stile

```
uv add --dev ruff
```

8.3.1 Linting

Si ottiene con

```
uv run ruff check
```

Vengono mostrate tutte le violazioni con un codice costituito da lettera e numero dove la lettera è

- `F` per le regole di pyflakes
- `E` per pycodestyle
- `I` per import etc

Per

- autofixare il possibile

  ```
  uv run ruff check --fix
  ```

- vedere a cosa si riferisce un errore

  ```
  uv run ruff rule E501
  ```

- ulteriori configurazioni (es come escludere dei check) vedere harrison pag 252

8.3.2 Formattazione

Si ottiene con

```
uv run ruff format # formattazione
```

8.4 Timing/temporizzazione

Per misurare il tempo usiamo `time.perf_counter` (posto in un decorator `lb.utils.timeit`)

```
import time
start = time.perf_counter()
# operations ...
stop = time.perf_counter()
stop - start # difference in seconds
```

8.5 Profiling

Il profiling serve per individuare le macrosezioni di codice dove si spende più tempo. Il modulo principale di profiling è `cProfile`.

Profilare codice al volo Fare qualcosa del genere

```
import cProfile
import re
cProfile.run('re.compile("foo|bar")') # ordine per cumulative time
cProfile.run('re.compile("foo|bar")', sort="tottime") # ordine per tottime
```

che da

```
      214 function calls (207 primitive calls) in 0.002 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
     1    0.000    0.000    0.002    0.002 {built-in method builtins.exec}
     1    0.000    0.000    0.001    0.001 <string>:1(<module>)
     1    0.000    0.000    0.001    0.001 __init__.py:250(compile)
     1    0.000    0.000    0.001    0.001 __init__.py:289(_compile)
     1    0.000    0.000    0.000    0.000 _compiler.py:759(compile)
...
```

La prima riga dice che 214 chiamate sono state monitorate, di cui 207 erano primitive (non ricorsive). La riga successiva specifica l'ordinamento della tabella (per tempo cumulato nella sezione di codice).
Gli header di colonna:

- `ncalls`: numero chiamate
- `tottime`: tempo speso nella funzione (escludendo le funzioni da questa chiamata)
- `percall`: `tottime/ncalls`
- `cumtime`: tempo totale speso nella funzione (includendo le funzioni chiamate)
- `percall`: `tottime` / primitive calls

- `filename:lineno(function)`: la sezione di codice

Se vi sono due numeri nella prima colonna (es 3/1), significa che la funzione è ricorrente: il secondo valore è il numero di chiamate primitive, il primo le chiamate totali (se la funzione chiamata solo una volta vi è un solo numero)

Profilare uno script uno script ordinando le sezioni di codice per tempo cumulato entro ogni posto (`cumtime`)

```
python -m cProfile cprof_example.py # ordine per cumtime
python -m cProfile -s tottime cprof_example.py
```

8.6 Scrittura di plugin

Approfondire

- `https://oneuptime.com/blog/post/2026-01-30-python-plugin-systems/view`
- `https://stackoverflow.com/questions/932069/building-a-minimal-plugin-architecture-in-pytho`
- `https://packaging.python.org/en/latest/guides/creating-and-discovering-plugins/`
- un esempio, credo, vedere il commento di `pytest-ty` di Harrison a pag 280

8.7 Documentazione

Si fa utilizzo di `sphinx` e si pubblica su `https://readthedocs.org/`. I tutorial da considerare sono `https://packaging.python.org/en/latest/tutorials/creating-documentation/` e `https://docs.python-guide.org/writing/documentation/`.

8.7.1 Setup

Iniziamo ad installare `sphinx`

```
uv add --dev sphinx
```

Creiamo la directory di documentazione e facciamo partire il tutto

```
l@m740n:~/src/pypkg$ mkdir pylb/docs
l@m740n:~/src/pypkg$ cd pylb/docs
l@m740n:~/src/pypkg/pylb/docs$ uv run sphinx-quickstart
```

Questo farà domande sul progetto per andare a creare il file `index.rst` e un `conf.py` (configurabili), più un `Makefile` di servizio.

8.7.2 Doc-writing e reStructuredText

`sphinx` converte restructured text ad altri linguaggi di markup, e utilizza reStructuredText come linguaggio di markup. Per la sintassi riferirsi a `https://www.sphinx-doc.org/en/master/usage/restructuredtext/` per le convenzioni di documentazione a quelle del progetto Numpy.

8.7.3 Building

Setup autobuilding API Per fare si che la documentazione di funzioni/classi etc, venga generata automaticamente occorre utilizzare l'estensione `autodoc`. Modificare `conf.py` come segue:

- aggiungere il path della libreria nel `sys.path` all'inizio di `conf.py`. Il package deve essere importabile per poter essere elaborato:

```
import os
import sys
sys.path.insert(0, os.path.abspath('../src'))
```

- modificare per aggiungere le seguenti estensioni sotto general configuration:

```
extensions = [
    'sphinx.ext.autodoc',    # generazione automatica api
    'sphinx.ext.viewcode',   # aggiungi link ipertestuali al codice
    'sphinx.ext.napoleon'    # supporta sintassi a-la numpy
]
```

- volendo si può cambiare il tema html installandolo

```
pip install --user sphinx_rtd_theme
```

  e modificando la linea del tema html (sostituendo `alabaster` di default)

```
html_theme = 'sphinx_rtd_theme'
```

Al termine del setup per preparare la documentazione di funzioni etc:

```
l@m740n:~/src/pypkg/pylb/docs$ sphinx-apidoc -f ../src/pylb/ -o .
Creating file ./pylb.rst.
Creating file ./pylb.experiments.rst.
Creating file ./modules.rst.
```

Aggiungere `modules.rst` in `index.rst`

```
Welcome to pylb's documentation!
================================

.. toctree::
   :maxdepth: 2
   :caption: Contents:

   modules
```

Build definitivo Per buildare definitivamente la documentazione complessivamente si usa `make` con il formato di interesse (nella cartella `docs`). Per l'html

```
l@m740n:~/src/pypkg/pylb/docs$ make html
The HTML pages are in _build/html.
l@m740n:~/src/pypkg/pylb/docs$ firefox _build/html/index.html
```

Per il latex

```
l@m740n:~/src/pypkg/pylb/docs$ make latex
The LaTeX files are in _build/latex.
Run 'make' in that directory to run these through (pdf)latex
(use `make latexpdf' here to do that automatically).
l@m740n:~/src/pypkg/pylb/docs$ make latexpdf
l@m740n:~/src/pypkg/pylb/docs$ okular _build/latex/pylb.pdf
```

8.7.4 Setup di readthedocs

Seguire il tutorial per importare il progetto e generare/hostare automaticamente la documentazione

Parte II

Scientific Stack

Capitolo 9

Numpy

Contents

9.1 L'ndarray 81
9.1.1 Creazione e copia 81
9.1.2 Tipi (dtype): coercizione e testing 81
9.1.3 Forma, dimensioni e reshape 82
9.2 Indexing 83
9.2.1 Array unidimensionali 83
9.2.2 Array multidimensionali 83
9.2.3 Subarray come viste vs copie 84
9.2.4 Assegnazione e unicità degli indici 84
9.3 Elaborazioni di array 84
9.3.1 Inserimento/rimozione elementi (insert, delete) . . 84
9.3.2 Aritmetica vettorizzata 84
9.3.3 Operazioni insiemistiche 84
9.3.4 Concatenazione (concatenate, vstack, hstack) . . . 84
9.3.5 Splitting (split, vsplit, hsplit) 85
9.3.6 Ripetizione/binding (repeat, tile) 85
9.3.7 Sorting/ordine (sort, argsort) 85
9.4 Universal functions 85
9.4.1 Introduzione 85
9.4.2 Metodi delle ufunction (reduce, accumulate, outer) 86
9.4.3 Creazione di ufunctions 87
9.5 Broadcasting 87
9.6 Altri argomenti 89
9.6.1 Lavorare con booleani 89
9.6.2 Array di stringhe 89

Il template per l'importazione è:

```
>>> import numpy as np
```

Fornisce:

- l'oggetto `ndarray`, un array multidimensionale efficiente, costruito come puntatore a dati in memoria;
- *universal functions*: funzioni per operare su tutti gli elementi di un array;
- strumenti per integrare codice scritto in C, C++ e Fortran

9.1 L'ndarray

L'`ndarray` è un array ad n dimensioni di dati omogenei composto da:

- un puntatore a dati in memoria;
- un attributo `dtype` che definisce il tipo di dato;
- un attribito `shape`, tuple che definisce la struttura dell'array.

9.1.1 Creazione e copia

Si usa la funzione `array` passando dati di tipo sequenza (liste, tuple ecc)

```
>>> # creazione a partire da una lista, in varie shape
>>> np.array([ 1,  2,  3])   # vector
array([1, 2, 3])
>>> x = np.array([[1, 2, 3], [4, 5, 6]]) # matrix
>>> x
array([[1, 2, 3],
       [4, 5, 6]])
```

Altre funzioni di convenienza per la generazione rapida di array:

```
>>> # Creazione rapida di dati
>>> np.arange(0, 20, 2)    # simile a range(), da 0 a 20 a step di 2
array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])
>>> np.linspace(0, 1, 5)   # interpolazione lineare
array([0.  , 0.25, 0.5 , 0.75, 1.  ])
>>> np.full(5, 2)          # vettore riempito di 2
array([2, 2, 2, 2, 2])
>>> np.zeros(10)           # array di zero
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
>>> np.ones((3, 5))        # array 3x5 di uno
array([[1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.]])
>>> np.eye(3)              # matrice identita 3x3
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])

>>> # creazione rapida sfruttando altre shape
>>> np.zeros_like(x)  # array di 0 della stessa shape di x (np.ones_like)
array([[0, 0, 0],
       [0, 0, 0]])
```

```
>>> # Generazione casuale
>>> np.random.random((3, 3))     # array 3x3 con valori casuali uniformi 0-1
array([[0.88072895, 0.10752208, 0.2736826 ],
       [0.03390858, 0.38858767, 0.38820146],
       [0.86640197, 0.7845211 , 0.39084423]])
>>> np.random.randint(0, 10, (3, 3)) # matrice 3x3 con interi nell'intervallo [0,10 )
array([[4, 4, 5],
       [2, 5, 4],
       [5, 8, 3]])
>>> np.random.normal(0, 1, 5)    # da normale mu=0, sd=1
array([ 0.24509254,  0.43482567, -0.55597278, -1.61022226,  0.10791801])
```

Referenza vs copia di array Se si assegna una variabile array ad un'altra variabile si crea un nuovo riferimento, non una copia: ossia non viene effettuata una copia di dati ma si creano due nomi che puntano alla stessa memoria:

```
>>> x = np.arange(10)
>>> y = x
>>> x[2] = -9999
>>> y
array([    0,     1, -9999,     3,     4,     5,     6,     7,     8,
           9])
>>> y[0] = -111
>>> x
array([ -111,     1, -9999,     3,     4,     5,     6,     7,     8,
           9])
```

Se necessario creare una copia per evitare di modificare l'originale utilizzare il metodo `copy`:

```
>>> x = np.arange(3)
>>> x_copy = x.copy()
>>> x_copy[0] = -999
>>> x
array([0, 1, 2])
```

9.1.2 Tipi (`dtype`): coercizione e testing

I tipi di dato, chiamati `dtype`, sono riportati in tabella 9.1: per utilizzarli in creazione dell'array specificare l'omonimo parametro in uno di questi due modi possibili:

```
>>> np.zeros(10, dtype = 'int16')
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int16)
>>> np.zeros(10, dtype = np.float64)
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

Coercizione del `dtype` Si usa il metodo `astype` sull'array da convertire

dtype	Descrizione
bool	Boolean (True or False) stored as a byte
intc	Identical to C int (normally int32 or int64)
intp	Integer used for indexing (same as C ssize_t; normally either int32 or int64)
int8	Byte (-128 to 127)
int16	Integer (-32768 to 32767)
int32	Integer (-2147483648 to 2147483647)
int64	Integer (-9223372036854775808 to 9223372036854775807)
uint8	Unsigned integer (0 to 255)
uint16	Unsigned integer (0 to 65535)
uint32	Unsigned integer (0 to 4294967295)
uint64	Unsigned integer (0 to 18446744073709551615)
float16	Half precision float: sign bit, 5 bits exponent, 10 bits mantissa
float32	Single precision float: sign bit, 8 bits exponent, 23 bits mantissa
float64	Double precision float: sign bit, 11 bits exponent, 52 bits mantissa
complex64	Complex number, represented by two 32-bit floats (real and imaginary components)
complex128	Complex number, represented by two 64-bit floats (real and imaginary components)

Tabella 9.1: dtypes di numpy

```
>>> a = np.zeros(10, dtype = np.float64)
>>> b = a.astype(np.int8)
>>> b
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int8)
```

Test del tipo I tipi numpy hanno una gerarchia per la quale ad esempio gli interi derivano dalla classe genitrice np.integer mentre i numeri a virgola mobile da np.floating.
Per il testing si possono usare queste classi in np.issubdtype.

```
>>> np.issubdtype(a.dtype, np.integer)
False
>>> np.issubdtype(a.dtype, np.floating)
True
```

9.1.3 Forma, dimensioni e reshape

9.1.3.1 Forma e dimensioni

Ogni array ha attributo:

- ndim: numero di dimensioni
- shape: numero di elementi per ciascuna dimensione
- size: numero di elementi complessivi
- nbytes: memoria occupata in bytes (dipendente dal dtype adottato, investigabile con itemsize)

```
>>> z1  = np.zeros(shape = 3) # vector
>>> z2i = np.zeros(shape = (3,4), dtype = np.int8)     # matrix
```

```
>>> z2f = np.zeros(shape = (3,4), dtype = np.float32)
>>> z3  = np.zeros(shape = (3,4,5)) # geneneral array

>>> z2i.ndim
2
>>> z2i.shape
(3, 4)
>>> z2i.size
12
>>> z2i.nbytes
12
>>> z2f.nbytes
48
>>> z2i.itemsize
1
>>> z2f.itemsize
4
```

9.1.3.2 Reshaping

Per modificarla la forma si può assegnare all'attributo `shape` o utilizzare il metodo `reshape` (ricordandosi di salvare); `reshape` ritorna una vista sui dati e non effettua copie, se non necessario.

```
>>> a = np.arange(4) ## assegnare all'attributo shape (C mode/row major di default)
>>> a
array([0, 1, 2, 3])
>>> a.shape = (2,2)
>>> a
array([[0, 1],
       [2, 3]])

>>> b = a.reshape((4,1)) ## uso di reshape
>>> b
array([[0],
       [1],
       [2],
       [3]])

>>> c = a.reshape((4,)) ## tornare ad un array a una dimensione di 4
>>> c
array([0, 1, 2, 3])

>>> d = a.reshape(-1) ## stessa cosa ma ancora più comodo
>>> d
array([0, 1, 2, 3])
```

9.1.3.3 Ravelling/flattening

Altri due metodi per tornare ad una struttura dati unidimensionale sono:

- `flatten` fa il flattening producendo sempre una copia dell'array di partenza
- `ravel` fa il flattening ma non produce una copia dei dati, bensì una vista dell'array originale; se si modifica l'array ritornato da ravel si potrebbero modificare gli elementi dell'array originale.
  `ravel` è spesso più veloce ma bisogna essere più attenti con le modifiche.

```
>>> b.flatten()
array([0, 1, 2, 3])
>>> b.ravel()
array([0, 1, 2, 3])
```

9.2 Indexing

Serve per accedere in lettura e scrittura agli elementi di un array.

9.2.1 Array unidimensionali

Si può specificare tra quadre alternativamente:

- lo slicing `start:stop:step` (con default rispettivamente a, 0, dimensione e 1) similmente ai dati builtin

  ```
  >>> x = np.arange(10)
  >>> x[1]
  np.int64(1)
  >>> x[1] = -1
  >>> x[3:5] = [5, 4]
  >>> x[:4:2] = 0
  >>> x[-2] = 1
  >>> x
  array([ 0, -1,  0,  5,  4,  5,  6,  7,  1,  9])
  ```

  Se `step` è negativo si procede in senso inverso (i default di `start`/`stop` vengono invertiti: per `start` la dimensione dell'array, per `stop` 0)

  ```
  >>> x[::-1]   # reverse an array
  array([ 9,  1,  7,  6,  5,  4,  5,  0, -1,  0])
  >>> x[5::-2]  # reversed from index 5 backward
  array([ 5,  5, -1])
  ```

- **liste e array numerici**; nel secondo caso la struttura ritornata ha lo stesso `shape` dell'indice (non del dato indicizzato):

  ```
  >>> # lista
  >>> select = [1, 2, 4, 5]
  >>> x[select] # x[np.array(select)] alternativamente
  array([-1,  0,  4,  5])

  >>> # array con shape (2x2)
  >>> ind = np.array([[2, 1],
  ```

```
...                 [4, 7]])
>>> x[ind]
array([[ 0, -1],
       [ 4,  7]])
```

- **liste/array logici**

```
>>> x = np.array([1, 2, 3])
>>> sell = [True, False, True]
>>> sela = np.array(sell)
>>> x[sell]
array([1, 3])
>>> x[sela]
array([1, 3])
```

9.2.2 Array multidimensionali

Tra quadre, separate da virgola, porre qualcosa di utilizzabile (slice, liste, array):

- **interi**: permettono di selezionare un singolo elemento

```
>>> # bidimensionale: [righe, colonne]
>>> x = np.arange(12).reshape(3,4)
>>> x
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])

>>> x[0, 0]                    # singolo elemento
np.int64(0)
```

- **slicing**

```
>>> x[ :2, 1: ]                # slicing: sino riga 2 esclusa, colonna 1 inclusa e sgg
array([[1, 2, 3],
       [5, 6, 7]])
>>> x[ ::-1, ::-1 ]            # reverse
array([[11, 10,  9,  8],
       [ 7,  6,  5,  4],
       [ 3,  2,  1,  0]])
```

L'**accesso a righe/colonne intere** avviene combinando indici numerici/logici e slicing vuota (`:`). Se un indice non è specificato, si prendono tutti i dati su quella dimensione:

```
>>> # se si fornisce un solo indice alla struttura multidimensionale
>>> # viene interpretato nel primo asse/dimensione
>>> x[0] # prima riga
array([0, 1, 2, 3])

>>> # prima riga the proper way
>>> # x[0, ]   # this works
>>> x[0, :]
```

```
array([0, 1, 2, 3])
>>> x[0, ::]
array([0, 1, 2, 3])

>>> # prima colonna
>>> # x[, 0] # this gives error
>>> x[:, 0]
array([0, 4, 8])
>>> x[::, 0]
array([0, 4, 8])
```

- **liste e array** in questo caso per l'accoppiamento degli indici funziona il *broadcasting*:
  - se utilizziamo due array unidimensionali verrà restituita la selezione in parallelo
  - se un indice è un array multidimensionale (vettore riga o colonna) otteniamo una struttura a due dimensioni (ogni valore di riga è matchato con ogni vettore colonna così come avviene nel broadcasting delle operazioni aritmetiche)

```
>>> x = np.arange(12).reshape(3,4)
>>> x
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])

>>> x[[0,1], [1,2]]           # liste: elemento (0, 1) ed elemento (1, 2) della
array([1, 6])


>>> row = np.array([0, 1, 2]) # array: elementi (0, 2), (1,1) e (2,3)  (broadcas
>>> col = np.array([2, 1, 3])
>>> x[row, col]
array([ 2,  5, 11])

>>> rowt = row.reshape(3,1) # array2
>>> rowt
array([[0],
       [1],
       [2]])
>>> col
array([2, 1, 3])
>>> (rowt * col).shape    # struttura degli indici post broadcast?
(3, 3)
>>> x[rowt, col]
array([[ 2,  1,  3],
       [ 6,  5,  7],
       [10,  9, 11]])
```

Nell'indexing di liste e array la struttura ritornata riflette la shape degli indici post broadcast, non la shape dell'array indicizzato.

- **indexing logico** (*boolean masking*): la selezione di righe/colonne mediante liste/array logici si fa normalmente

```
>>> x = np.arange(12).reshape(3,4)
>>> x
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])

>>> # selezione colonne (prima e seconda)
>>> cl = [True, True, False, False]
>>> ca = np.array(cl)
>>> x[:, cl]
array([[0, 1],
       [4, 5],
       [8, 9]])
>>> x[:, ca]
array([[0, 1],
       [4, 5],
       [8, 9]])

>>> # selezione righe (prima e terza)
>>> rl = [True, False, True]
>>> ra = np.array(rl)
>>> x[rl, :]
array([[ 0,  1,  2,  3],
       [ 8,  9, 10, 11]])
>>> x[ra, :]
array([[ 0,  1,  2,  3],
       [ 8,  9, 10, 11]])
```

Se il `ndim` dell'oggetto passato come indice coincide con quello indicizzato, viene ritornato un 1-dimensional array riempito con gli elementi corrispondenti a `True`.
Questo può essere usato per masking. Ad esempio:

```
>>> # esempio 1: masking
>>> b = np.arange(9).reshape(3, 3)
>>> b
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
>>> b > 4
array([[False, False, False],
       [False, False,  True],
       [ True,  True,  True]])
>>> b[b > 4]
array([5, 6, 7, 8])

>>> # esempio 2: selezionare intersezione di righe e colonne
>>> rar = ra.reshape(3,1)
>>> rar
```

```
array([[ True],
       [False],
       [ True]])
>>> ca
array([ True,  True, False, False])

>>> # x[rar, ca] # doesnt work
>>> # x[ra, ca]  # doesnt work

>>> rar * ca          # nel caso di logici questo è utile nell'indicare la selez
array([[ True,  True, False, False],
       [False, False, False, False],
       [ True,  True, False, False]])
>>> x[rar * ca]       # black magic come in b>4, ma non mantiene la struttura :(
array([0, 1, 8, 9])
>>> x[np.ix_(ra, ca)] # per mantenere la struttura usare quanto ritornato da np.
array([[0, 1],
       [8, 9]])
```

9.2.3 Subarray come viste vs copie

I subarray generati

- con *slicing*/*tuple* sono **viste**: pertanto eventuali modifiche si ripercuoteranno sull'array/struttura di base indipendentemente da dove sono state effettuate;

- con *array*/liste sono **copie**

```
>>> x = np.arange(10)      # subarray con slice sono una vista
>>> x_slice = x[5:8]
>>> x_slice[2] = 999
>>> x
array([  0,   1,   2,   3,   4,   5,   6, 999,   8,   9])

>>> x = np.arange(10)      # subarray con lista sono copia
>>> x_slice = x[[5, 6, 7]]
>>> x_slice[2] = 999
>>> x
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

>>> x = np.arange(4)       # subarray con array sono copia
>>> ind = np.array([True, True, False, False])
>>> x_slice = x[ind]
>>> x_slice[:] = 999
>>> x
array([0, 1, 2, 3])
```

9.2.4 Assegnazione e unicità degli indici

Se non si desiderano comportamenti inattesi, prestare attenzione all'unicità degli indici, in quanto se un indice è ripetuto l'assegnazione sarà effettuata molteplici volte, come i seguenti esempi mostrano

```
>>> x = np.zeros(10, dtype = np.int_)

>>> # primo esempio
>>> x[[0,0]] = [4, 6]
>>> # qui si ha x[0] = 4 e poi x[0] = 6
>>> x
array([6, 0, 0, 0, 0, 0, 0, 0, 0, 0])

>>> # secondo esempio
>>> i = [2,2,4,4,4,6,6,6,6,6,6]
>>> x[i] += 1
>>> x
array([6, 0, 1, 0, 1, 0, 1, 0, 0, 0])
>>> # anche questo non intuitivo, si pensava la posizione 6 fosse
>>> # aumentato tante volte, ma cosi non è per cazzi suoi
>>> # (vedi vanderplas fancy indexing nel caso, ma anche chi se ne ciava)
```

9.3 Elaborazioni di array

9.3.1 Inserimento/rimozione elementi (`insert`, `delete`)

```
>>> a = np.array([1, 2, 3]) # any change need to be assigned to be saved
>>> np.insert(a, 1, 5)      # insert element in an array
array([1, 5, 2, 3])
>>> np.delete(a, [1])       # remove item from an array
array([1, 3])
```

9.3.2 Aritmetica vettorizzata

Le operazioni aritmetiche sono vettorizzate per cui è possibile fare

```
>>> x = np.arange(4)
>>> x
array([0, 1, 2, 3])

>>> ## Operazioni aritmetiche
>>> -x     # unary negation
array([ 0, -1, -2, -3])
>>> x + 1  # addition
array([1, 2, 3, 4])
>>> x - 5  # subtraction
array([-5, -4, -3, -2])
>>> x * 2  # multiplication
array([0, 2, 4, 6])
>>> x ** 2 # power
```

Funzione	Descrizione
`unique(x)`	sorted unique elements in x
`intersect1d(x, y)`	sorted common elements in x and y
`union1d(x, y)`	sorted union of elements
`in1d(x, y)`	boolean array if each element of x is contained in y
`setdiff1d(x, y)`	set difference (elements in x that are not in y)
`setxor1d(x, y)`	set symmetric differences (elements in one of the arrays, but not both)

Tabella 9.2: Operazioni insiemistiche tra array

```
array([0, 1, 4, 9])
>>> x / 2  # division
array([0. , 0.5, 1. , 1.5])
>>> x // 2 # floor division (e.g., 3 // 2 = 1)
array([0, 0, 1, 1])
>>> x % 2  # modulus remainder
array([0, 1, 0, 1])
>>> -(0.5*x + 1) ** 2 # misc expression
array([-1.  , -2.25, -4.  , -6.25])

>>> ## Comparazione
>>> x == 2
array([False, False,  True, False])
>>> x != 2
array([ True,  True, False,  True])
>>> x > 2
array([False, False, False,  True])
>>> x >= 2
array([False, False,  True,  True])
>>> x < 2
array([ True,  True, False, False])
>>> x <= 2
array([ True,  True,  True, False])
```

9.3.3 Operazioni insiemistiche

Per effettuare operazioni di tipo insiemistico le funzioni sono quelle riportate in tabella 9.2.

9.3.4 Concatenazione (`concatenate`, `vstack`, `hstack`)

`numpy.concatenate` prende una tupla o una lista di array e li unisce

```
>>> x = np.array([1, 2, 3])           # unidimensionale
>>> y = np.array([3, 2, 1])
>>> z = [99, 99, 99]
>>> np.concatenate([x, y, z])
array([ 1,  2,  3,  3,  2,  1, 99, 99, 99])

>>> x = np.array([[1, 2, 3], [4, 5, 6]])     # bidimensionale
>>> y = np.array([[7, 8, 9], [10, 11, 12]])
```

```
>>> np.concatenate([x, y])                    # axis=0 default
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [ 7,  8,  9],
       [10, 11, 12]])
>>> np.concatenate([x, y], axis=1)
array([[ 1,  2,  3,  7,  8,  9],
       [ 4,  5,  6, 10, 11, 12]])
```

Per con array di dimensioni miste `numpy.hstack` e `numpy.vstack` tornano comode (per fare stack sulla prima o seconda dimensione)[1]:

```
>>> x = np.array([1, 2, 3])
>>> grid = np.array([[9, 8, 7],
...                  [6, 5, 4]])
>>> y = np.array([[99],
...               [99]])

>>> np.hstack([grid, y])     # horizontally stack the arrays
array([[ 9,  8,  7, 99],
       [ 6,  5,  4, 99]])
>>> np.vstack([x, grid])     # vertically stack the arrays
array([[1, 2, 3],
       [9, 8, 7],
       [6, 5, 4]])
```

9.3.5 Splitting (`split`, `vsplit`, `hsplit`)

`numpy.split` suddivide un array molteplici array. Si usano passando una lista di indici per indicare dove fare i tagli:

```
>>> x = [1, 2, 3, 99, 99, 3, 2, 1]     # split di array
>>> x1, x2, x3 = np.split(x, [3, 5])
>>> x1
array([1, 2, 3])
>>> x2
array([99, 99])
>>> x3
array([3, 2, 1])

>>> x = np.arange(10).reshape(5,2)     # di matrice
>>> x
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7],
       [8, 9]])
>>> x1, x2, x3 = np.split(x, [1, 3])
>>> x1
array([[0, 1]])
```

[1] Analogamente `np.dstack` effettuerà lo stack sulla terza dimensione.

```
>>> x2
array([[2, 3],
       [4, 5]])
>>> x3
array([[6, 7],
       [8, 9]])
```

`hsplit`/`vsplit` sono funzioni di convenienza per splittare sulla prima o seconda dimensione, specularmente a `vstack`/`hstack`[2]

```
>>> x = np.arange(16).reshape((4, 4))
>>> x
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15]])

>>> upper, lower = np.vsplit(x, [2])   # vertical split
>>> upper
array([[0, 1, 2, 3],
       [4, 5, 6, 7]])
>>> lower
array([[ 8,  9, 10, 11],
       [12, 13, 14, 15]])

>>> left, right = np.hsplit(grid, [2]) # horizontal split
>>> left
array([[9, 8],
       [6, 5]])
>>> right
array([[7],
       [4]])
```

9.3.6 Ripetizione/binding (`repeat`, `tile`)

Per ripetere:

- i singoli argomenti di un array, ciascuno n volte, si usa `repeat`; nel caso di array multidimensionali si può selezionare l'asse della ripetizione
- un array nel suo complesso usare `tile`: se il secondo argomento è un intero viene effettuata una copia per riga, con una tuple si specifica la struttura finale

```
>>> x = np.array([1,2,3])  # unidimensional repeat
>>> x.repeat(2)            # a) each element 2 time
array([1, 1, 2, 2, 3, 3])
>>> x.repeat([3,2,1])      # b) different times
array([1, 1, 1, 2, 2, 3])

>>> np.tile(x, 2)          # unidimensional tile
```

[2] Analogamente `numpy.dsplit` effettua il taglio sul terzo asse.

```
array([1, 2, 3, 1, 2, 3])

>>> x = np.arange(4).reshape(2,2) # multidimensional repeat
>>> x.repeat(2, axis = 0)         # a) by row, common number
array([[0, 1],
       [0, 1],
       [2, 3],
       [2, 3]])
>>> x.repeat([2,3], axis = 1)     # b) by col, different number
array([[0, 0, 1, 1, 1],
       [2, 2, 3, 3, 3]])

>>> np.tile(x, 2)                 # multidimensional tile (default by col)
array([[0, 1, 0, 1],
       [2, 3, 2, 3]])
>>> np.tile(x, (2, 3))            # specify the building repetition
array([[0, 1, 0, 1, 0, 1],
       [2, 3, 2, 3, 2, 3],
       [0, 1, 0, 1, 0, 1],
       [2, 3, 2, 3, 2, 3]])
>>> np.tile(x, (2, 1))            # by col
array([[0, 1],
       [2, 3],
       [0, 1],
       [2, 3]])
```

9.3.7 Sorting/ordine (`sort`, `argsort`)

Per

- ordinare un array usare il metodo `sort`. Nel caso di array multidimensionale, specificare `axis` per la dimensione di sorting:

```
>>> x = np.array([2,1,3,4])           # unidimensional sorting
>>> x.sort()
>>> x
array([1, 2, 3, 4])

>>> rand = np.random.RandomState(42) # bidimensional
>>> X = rand.randint(0, 10, (4, 6))
>>> np.sort(X, axis=0)               # sorting di ogni colonna
array([[2, 1, 4, 0, 1, 5],
       [5, 2, 5, 4, 3, 7],
       [6, 3, 7, 4, 6, 7],
       [7, 6, 7, 4, 9, 9]])
>>> np.sort(X, axis=1)               # sort di ogni riga
array([[3, 4, 6, 6, 7, 9],
       [2, 3, 4, 6, 7, 7],
       [1, 2, 4, 5, 7, 7],
       [0, 1, 4, 5, 5, 9]])
```

Da notare che negli ultimi due casi si perdono eventuali relazioni tra righe e colonne.

- ottenere gli indici dell'ordine si usa `argsort`

```
>>> x = np.array([2, 1, 4, 3, 5])
>>> i = np.argsort(x)
>>> i
array([1, 0, 3, 2, 4])
>>> x[i]
array([1, 2, 3, 4, 5])
```

9.4 Universal functions

9.4.1 Introduzione

Le universal functions (reference qui), o *ufunctions*, sono funzioni chiamate come

```
np.ufunc()
```

ed eseguite su tutti gli elementi di array in maniera vettorizzata.
Vi sono funzioni *unarie* (si applicano ad un array separatamente, le più importanti in tabella 9.3) e *binarie* (si applicando a più array, le più importanti in tab 9.4).
Gli operatori aritmetici e di comparazione (es `>=`) funzionano sotto la scocca, come *ufuncs* binarie di tabella (es `a+b` chiama `np.add(a,b)`).
Altre ufuncs si trovano in `scipy.special`

9.4.2 Metodi delle ufunction (`reduce`, `accumulate`, `outer`)

A quanto pare le ufunction in realtà assomigliano a classi aventi propri metodi (`reduce`, `accumulate`, `reduceat`, `outer`, `at`).
La chiamata di questi metodi è utile soprattutto per funzioni che prendono in input due argomenti e ne ritornano uno singolo in output (eg binarie):

- `reduce` applica una ufuncs agli elementi di un array sino a che un singolo risultato rimane
- `accumulate` fa l'operazione cumulata
- `outer` applica una funzione al prodotto cartesiano degli elementi di due array

Un esempio con `np.add`

```
>>> x = np.arange(1, 6)

>>> np.add.reduce(x)       # come np.sum(x)
np.int64(15)
>>> np.add.accumulate(x)  # come np.cumsum(x)
array([ 1,  3,  6, 10, 15])
>>> np.add.outer(x, x)
```

Funzione	Descrizione
`abs`, `fabs`	absolute value
`sqrt`	square root
`exp`, `expm1`	esponenziale, $\exp(x) - 1$
`log`, `log10`, `log2`, `log1p`	Natural log, log base 10, log base 2, and $log(1 + x)$, respectively
`sin`, `cos`, `tan`	trigonometriche
`arcsin`, `arccos`, `arctan`	trigonometriche inverse
`sign`	funzione segno: 1 se positivo, 0 se zero, o -1 se negativo
`ceil`	smallest integer $\geq$ to each element
`floor`	largest integer $\leq$ to each element
`rint`	round to nearest integer (preserving the dtype)
`isnan`	boolean indicating whether each value is NaN (Not a Number)
`isfinite`, `isinf`	each element is finite (non-inf , non-NaN) or infinite, respectively
`any`	per array booleani, testa se alcuni sono veri
`all`	per array booleani, testa se tutti sono veri
`logical_not`, `bitwise_not` (`~`)	logical not element-wise
`sum`	Sum of all the elements in the array or along an axis.
`prod`	Produttoria
`mean`	Arithmetic mean. Zero-length arrays have NaN mean.
`median`	Mediana
`percentile`	Percentile
`std`, `var`	Standard deviation and variance.
`min`, `max`	Minimum and maximum.
`argmin`, `argmax`	Indices of minimum and maximum elements, respectively.
`cumsum`	Cumulative sum of elements starting from 0
`cumprod`	Cumulative product of elements starting from 1

Tabella 9.3: *ufuncs* unarie

Funzione	Descrizione
`add` (+)	addition
`subtract` (-)	subtraction
`negative` (-)	unary negation
`multiply` (*)	multiplication
`divide` (/)	division
`floor_divide` (//)	resto divisione
`power` (**)	power
`mod` (%)	modulus remainder
`less` (<)	less than
`greater` (>)	
`less_equal` (<=)	
`greater_equal` (>=)	
`not_equal` (!=)	
`equal` (==)	
`maximum`, `fmax`	Element-wise maximum. `fmax` ignores `NaN`
`minimum`, `fmin`	Element-wise minimum. `fmin` ignores `NaN`
`logical_and`, `bitwise_and` (&)	and
`logical_or`, `bitwise_or` (|)	or
`logical_xor`, `bitwise_xor` (^)	xor
`matmul` (@)	Matrix product of two arrays
`cov`	covariance coefficient/matrix
`corrcoef`	correlation coefficient/matrix

Tabella 9.4: *ufuncs* binarie

```
array([[ 2,  3,  4,  5,  6],
       [ 3,  4,  5,  6,  7],
       [ 4,  5,  6,  7,  8],
       [ 5,  6,  7,  8,  9],
       [ 6,  7,  8,  9, 10]])
```

9.4.3 Creazione di ufunctions

Per la creazione di funzioni vettorizzate possiamo:

- scrivere funzioni in puro python e vettorizzarle con `np.frompyfunc` (o il wrapper `vectorize`).
  `numpy.frompyfunc` prende in input una funzione, il numero di inputs presi e il numero di outputs forniti; la funzione creata/ritornata restituisce sempre un array:

```
>>> def add1_worker(x):    # 1 input 1 output
...     return x + 1
...
>>> add1 = np.frompyfunc(add1_worker, 1, 1)
>>> add1(np.arange(10))
array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], dtype=object)

>>> def add2_worker(x, y): # 2 input, 1 output
```

```
...     return x + y
...
>>> add2 = np.frompyfunc(add2_worker, 2, 1)
>>> add2(np.arange(10), np.arange(10))
array([0, 2, 4, 6, 8, 10, 12, 14, 16, 18], dtype=object)

>>> def max_min_worker(x, y): # 2 input 2 output
...     return max(x, y), min(x, y) # pure python here
...
>>> maxmin = np.frompyfunc(max_min_worker, 2, 2)
>>> maxmin_wrong = np.frompyfunc(max_min_worker, 2, 1) # wrong in returned

>>> x = np.random.random(3)
>>> y = np.random.random(3)
>>> x
array([0.56325623, 0.90696264, 0.55838448])
>>> y
array([0.95635375, 0.15588456, 0.7161438 ])
>>> maxmin(x, y)
(array([0.9563537492958271, 0.9069626386644869, 0.7161438036280894],
      dtype=object), array([0.563256225684888, 0.15588455640579102, 0.5583844813402609],
      dtype=object))
>>> maxmin_wrong(x, y)
array([(0.9563537492958271, 0.563256225684888),
       (0.9069626386644869, 0.15588455640579102),
       (0.7161438036280894, 0.5583844813402609)], dtype=object)
```

`numpy.vectorize` è una alternativa meno generale/più lenta che permette di specificare il tipo ritornato la funzione.

- utilizzare compilatori LLVM con `numba` a partire da funzioni Python. Mediante Numba si creano funzioni veloci mediante il progetto LLVM (che traduce codice python in codice macchina eseguibile da CPU o GPU). Vedere `https://wesmckinney.com/book/advanced-numpy.html#numpy_numba`
- utilizzare l'interfaccia C (modo più generale)

9.5 Broadcasting

Per

- array della stessa dimensionalità le operazioni aritmetiche vengono effettuate elemento per elemento

```
>>> x = np.array([0, 1, 2])
>>> y = np.array([5, 5, 5])
>>> x + y
array([5, 6, 7])
```

- array di diverse dimensioni opera il *broadcasting*, ossia un set di regole per applicare ufuncs binarie ad array di dimensioni non uguale. Alcuni esempi:

```
>>> # aggiunta di costante ad array: il valore 5 viene espanso a [5 5 5] prima d
>>> x + 5
array([5, 6, 7])

>>> # aggiunta di vettore a matrice il vettore viene stretchato sulla
>>> # seconda dimensione per matchare la forma della matrice
>>> M = np.ones((3, 3))
>>> M
array([[1., 1., 1.],
       [1., 1., 1.],
       [1., 1., 1.]])
>>> M + x
array([[1., 2., 3.],
       [1., 2., 3.],
       [1., 2., 3.]])

>>> # somma di vettore 1 x 3 con sua trasposta 3 x 1: entrambi gli array
>>> # sono stretchati fino a raggiungere una dimensione comune (due
>>> # matrici $3 x 3$) dopodiché viene effettuata la somma.
>>> x = np.arange(3)
>>> y = x.reshape(3,1).copy()
>>> x
array([0, 1, 2])
>>> y
array([[0],
       [1],
       [2]])
>>> x + y
array([[0, 1, 2],
       [1, 2, 3],
       [2, 3, 4]])
```

La geometria di esempi del genere è visualizzata in figura **??**; non vi è una vera e propria espansione in memoria per ragioni di efficienza ma è utile tenere a mente come modello.

Regole di funzionamento Informalmente funziona abbastanza similmente al recycling di R: l'array più piccolo viene replicato per matchare quello più grande[3]. Formalmente le regole applicate in sequenza sono:

1. se gli array differiscono nel numero di dimensioni (il numero di elementi di `shape`), la forma di quello con un numero inferiori di dimensioni è riempita a sinistra con 1;

2. una volta uniformati `numpy` confronta gli `shape`, elemento (dimensione) per elemento, partendo dall'ultimo. Due dimensioni sono *compatibili* quando sono alternativamente uguali o una di esse è 1: se

   - *tutte le dimensioni sono compatibili* verrà eseguita l'operazione

[3] Il broadcasting è lievemente più sicuro/meno flessibile perché funziona solamente se la struttura matcha perfettamente o se si ha un array di un elemento (è questo che di fatto viene riciclato).

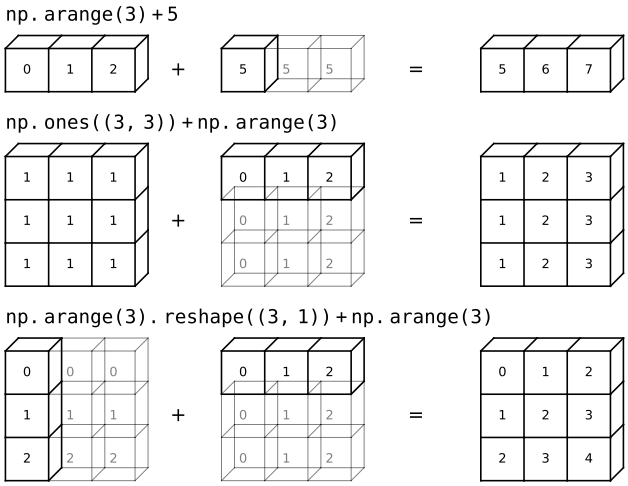


Figura 9.1: Broadcasting geometrics.

- *non tutte sono compatibili* viene sollevata l'eccezione e l'operazione termina

Nel caso di dimensioni compatibili, se due dimensioni non sono uguali, l'array con shape 1 in quella dimensione viene stretchato per matchare l'altra dimensione;

Example 9.5.1 (Somma di un array bidimensionale a monodimensionale)**.** Abbiamo:

```
>>> M = np.ones((2, 3))
>>> a = np.arange(3)
>>> M
array([[1., 1., 1.],
       [1., 1., 1.]])
>>> a
array([0, 1, 2])

>>> ## le shape sono
>>> M.shape
(2, 3)
>>> a.shape
(3,)
```

Applicando le regole del broadcasting:

1. per la regola 1 l'array `a` ha meno dimensioni quindi viene riempito sulla sinistra

   ```
   M.shape -> (2, 3)
   a.shape -> (1, 3)
   ```

2. vediamo quali dimensioni non sono in agreement, queste verranno stretchate per matchare

   ```
   M.shape -> (2, 3)
   a.shape -> (2, 3)
   ```

   Ora le shape matchano e la shape del risultato finale sarà `(2, 3)`

```
>>> M+a
array([[1., 2., 3.],
       [1., 2., 3.]])
```

Example 9.5.2 (Entrambi gli array necessitano di broadcasting)**.** Abbiamo:

```
>>> a = np.arange(3).reshape((3, 1))
>>> b = np.arange(3)
>>> a
array([[0],
       [1],
       [2]])
>>> b
array([0, 1, 2])
>>> a.shape
(3, 1)
>>> b.shape
(3,)
```

Si ha:

1. Per regola 1 `b` viene riempito sulla sinistra con 1

   ```
   a.shape -> (3, 1)
   b.shape -> (1, 3)
   ```

2. per regola 2 facciamo l'upgrade degli 1 per matchare la dimensione dell'array

   ```
   a.shape -> (3, 3)
   b.shape -> (3, 3)
   ```

3. visto che matchano il risultato sarà un array 3×3

Example 9.5.3 (Array incompatibili)**.** Un caso lievemente diverso dal primo dove `M` è trasposta

```
>>> M = np.ones((3, 2))
>>> a = np.arange(3)
>>> M
array([[1., 1.],
       [1., 1.],
       [1., 1.]])
>>> a
array([0, 1, 2])
```

```
>>> M.shape
(3, 2)
>>> a.shape
(3,)
```

Per regola

1. riempiamo a sinistra `a`

   ```
   M.shape -> (3, 2)
   a.shape -> (1, 3)
   ```

2. la prima dimensione di `a` è stretchata e si ha

   ```
   M.shape -> (3, 2)
   a.shape -> (3, 3)
   ```

3. le dimensioni finali non matchano quindi viene sollevato un errore

```
>>> M + a
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: operands could not be broadcast together with shapes (3,2) (3,)
```

9.6 Altri argomenti

9.6.1 Lavorare con booleani

Vediamo un po' di applicazioni utili

```
>>> ## dati
>>> rng = np.random.RandomState(0)
>>> n = rng.randint(10, size=(3, 4)) # interi uniformi < 10
>>> x = np.array([True, False, True, False])
>>> y = np.array([True, True, False, False])

>>> ## operatori booleani vettorizzati
>>> x & y              # np.bitwise_and
array([ True, False, False, False])
>>> x | y              # np.bitwise_or
array([ True,  True,  True, False])
>>> ~x                 # np.bitwise_not
array([False,  True, False,  True])
>>> x ^ y              # np.bitwise_xor
array([False,  True,  True, False])

>>> ## any/ all
>>> np.any(n > 8)
np.True_
>>> np.all(n < 10)
np.True_
>>> np.all(n < 8, axis=1)
```

```
array([ True, False,  True])

>>> ## conta di elementi che rispettano un test
>>> np.sum(n < 6)              # overall
np.int64(8)
>>> np.sum(n < 6, axis = 1)    # per riga
array([4, 2, 2])
>>> np.sum(n > 6, axis = 0)    # per colonna
array([1, 1, 1, 0])

>>> ## if then else vettorizzato: np.where(test, iftrue, iffalse)
>>> t = np.arange(6)
>>> f = -t
>>> test = np.array([True, False] * 3)
>>> res = np.where(test, t, f)
>>> res
array([ 0, -1,  2, -3,  4, -5])

>>> # recoding con np.where
>>> c = np.where(t > 3, 3, np.where(t > 1, 2, 1))
>>> c
array([1, 1, 2, 2, 3, 3])

>>> # o con algebra e logica pura
>>> d = 1 + (t > 1) + (t > 3)
>>> d
array([1, 1, 2, 2, 3, 3])
```

9.6.2 Array di stringhe

Gli array possono immagazzinare anche stringhe, ma queste devono essere di dimensione fissata per motivi di efficienza

```
>>> names = ["luca", "bob", "joe"]
>>> x = np.array(names)
>>> x.dtype  # stringhe di 4 elementi al massimo
dtype('<U4')
>>> x[0][0:2]  # utilizzo di indici
'lu'
>>> # sono comunque normali stringhe eh ...
>>> for i in range(3):
...     x[i] = x[i].capitalize()
...
>>> x
array(['Luca', 'Bob', 'Joe'], dtype='<U4')
>>> # ... ma di lunghezza massima fissata
>>> x[2] = "asdasdasd"  # assegnazione, ocio si tronca silentemente
>>> x
array(['Luca', 'Bob', 'asda'], dtype='<U4')

>>> # per allocare più spazio, ad esempio
```

```
>>> y = np.array(names, dtype = '<U16')
>>> y[2] = "asdasdasd"
>>> y
array(['luca', 'bob', 'asdasdasd'], dtype='<U16')
```

Per creare array di *stringhe di dimensione variabili* non preventivabile a priori si può usare `dtype=object`. Così facendo si crea un array di oggetti generici al quale si può assegnare la qualunque: si perde però l'efficienza di `numpy` (che non lavora più diretto in sequenze contigue di memoria e usare oggetti python aggiunge un sacco di overhead):

```
>>> ## https://stackoverflow.com/questions/14639496
>>> x = np.array(['apples', 'foobar', 'cowboy'], dtype=object)
>>> x
array(['apples', 'foobar', 'cowboy'], dtype=object)
>>> x[2] = "asdasdasd"
>>> x
array(['apples', 'foobar', 'asdasdasd'], dtype=object)

>>> for i in range(3):
...     x[i] = x[i].capitalize()
...
>>> x
array(['Apples', 'Foobar', 'Asdasdasd'], dtype=object)

>>> # A questo array si può assegnare la qualunque ..
>>> x[1] = {1:2, 3:4}
>>> x
array(['Apples', {1: 2, 3: 4}, 'Asdasdasd'], dtype=object)
```

Capitolo 10

Sympy

Integrazione `integrate` è utilizzato per integrali sia indenfiniti che definiti

1. definire i simboli impiegati
2. definire l'espressione che ne fa uso
3.

```
from sympy import *
init_printing(use_unicode=False, wrap_line=False)
x = Symbol('x')
y = Symbol('y')

# indefinito
integrate(x**2 + x + 1, x)

# definito da 0 a +infinito
expr = exp(-x**2)
integrate(expr,(x,0,oo) )

expr = (3 + x)/(12*6)
integrate(expr,(x, -3, 9) )

# definito usando due variabili
expr=exp(-x**2 - y**2)
integrate(expr,(x,0,oo),(y,0,oo))
```

Capitolo 11

Polars

Contents

11.1 Introduzione 93
11.1.1 Tipi di dati 93
11.1.2 Espressioni e contesti 94
11.1.3 Expression expansion e selettori 94
11.2 Data I/O 95
11.2.1 Importazione dati 95
11.2.2 Esportazione dati 95
11.3 `Series` 95
11.3.1 Creazione 95
11.3.2 Attributi 95
11.3.3 Indexing e selezione 96
11.3.4 Elaborazione vettorizzata 96
11.3.5 Coercizione di tipo (`cast`) 96
11.3.6 Valori condizionali a-la `ifelse` (`pl.when`) 96
11.3.7 Applicazione di una funzione a tutti gli elementi (`map`) 96
11.3.8 Sostituzione di valori (`replace`) 96
11.3.9 Test di appartenenza (`in`, `isin`) 96
11.3.10 Dati mancanti 96
11.3.11 Duplicati 96
11.3.12 Sorting 97
11.3.13 Discretizzazione/creazione di classi (`cut`, `qcut`) . . . 97
11.3.14 Dummy variables 97
11.3.15 Stringhe 97
11.3.16 Date/ore 98
11.3.17 Dati categorici (`Enum` e `Categorical`) 98
11.3.18 Liste, array, structs 99
11.4 `DataFrame` 100
11.4.1 Creazione 100
11.4.2 Contenuto 100
11.4.3 Copia di df e di struttura (`clone`, `clear`) 101
11.4.4 Selezione 101

11.4.5 Inserimento colonna in un punto (.insert_column) . 101
11.4.6 Assegnazione (with_columns) 101
11.4.7 Rimozione righe/colonne (.filter/.drop) 102
11.4.8 Rinominare colonne 102
11.4.9 Coercizione di tipi (.cast) 102
11.4.10 Applicazione di funzioni 102
11.4.11 Ciclo su righe/colonne 103
11.4.12 Merge (.join) 103
11.4.13 Binding (pl.concat) 104
11.4.14 Aggiunta di righe parziali ad un dataframe 104
11.4.15 Reshape 104
11.4.16 Trasporre (.transpose) 105
11.4.17 Test di appartenenza (in) 105
11.4.18 Dati mancanti 105
11.4.19 Duplicati 105
11.4.20 Sorting 106
11.4.21 Id univoco di riga 106
11.4.22 Windows functions 106
11.4.23 folds (elaborazioni orizzontali) 106
11.5 Descrittive 107
11.5.1 Univariate 107
11.5.2 Bivariate 107

`Series` e DataFrame di nuova generazione, non ha il concetto di indice:

```
>>> import polars as pl
>>> pl.__version__
'1.42.1'

>>> import numpy as np
```

Per settare opzioni

- semi-definitivamente usare

  ```
  >>> pl.Config.set_ascii_tables(True) # altrimenti la stampa di df da problemi co
  <class 'polars.config.Config'>
  ```

- al volo utilizzare `with`

  ```
  # stampa tutte le righe e colonne del df in ascii art
  with pl.Config(tbl_rows=-1, tbl_cols=-1, ascii_tables=True):
      print(df)
  ```

11.1 Introduzione

11.1.1 Tipi di dati

Polars supporta vari **tipi di dati**:

- numeric data types: signed integers, unsigned integers, floating point numbers, and decimals.
- temporal: dates, datetimes, times, and time deltas
- miscellanei: strings, binary data, Booleans, categoricals, enums, and objects
- *nested* data types: lists, structs, and arrays.

Tutti i tipi supportano valori mancanti rappresentati tramite `null` (in input inserire `None`); è diverso da `NaN` rinvenibile solo nei floating point (inseribile come `np.nan` o `float("nan")`)

- `NaN` non è considerato missing (ha sue funzioni specifiche `is_nan`, `fill_nan`), non è rilevato da `null_count` ne toccato da `fill_null`
- aggregazioni numeriche su `null` ignorano i dati missing mentre `nan` fa si che vengano propagati

11.1.2 Espressioni e contesti

Le **espressioni** sono la rappresentazione di una trasformazione dati possono essere

- composte tra loro,
- salvate in una variabile
- eseguite in un *contesto*: quando si esegue una espressione Polar cerca di semplificarla e parallelizzarla se possibile

I **contesti** sono i posti dove vengono eseguite le espressioni: i più comuni sono

- `select`: applica l'espressione alle colonne, producendo nuove colonne. L'espressione passata deve produrre o una costante (che viene ripetuta per le righe) o una serie che deve matchare con la lunghezza prodotta dalle altre valutazioni all'interno della select (ma non necessariamente col il numero righe del dataframe di partenza)

```
bmi_expr = pl.col("weight") / (pl.col("height") ** 2)
result = df.select(
    bmi=bmi_expr,
    avg_bmi=bmi_expr.mean(),
    ideal_max_bmi=25,
)
# espressione con broadcasting
result = df.select(deviation=(bmi_expr - bmi_expr.mean()) / bmi_expr.std())
```

- `with_columns`: simile a `select` ma include tutto il dataframe di provenienza, assieme alle nuove colonne create. L'espressione deve creare una serie avente la stessa lunghezza delle colonne del dataframe.
- `filter`: prende in input espressioni che valutano a booleano, che vengono utilizzate per filtro (di default in and)

```
# seleziona per datanascita ed età
result = df.filter(
    pl.col("birthdate").is_between(date(1982, 12, 31), date(1996, 1, 1)),
    pl.col("height") > 1.7,
)
```

- `group_by`: in questo contesto le righe sono raggruppate secondo i valori unici prodotti dalle espressioni passate. Dopodiché si possono applicare espressioni ai gruppi risultanti (mediante `.agg`) che potrebbero ritornare risultati di lunghezza variabili (e nel caso saranno posti in liste). Se nessuna aggregazione è effettuata, semplicemente si fa l'elenco dei risultati (in una lista, se di lunghezza differente)

  ```
  # lista i nomi dei nati per decade
  result = df.group_by(
      (pl.col("birthdate").dt.year() // 10 * 10).alias("decade"),
  ).agg(pl.col("name"))
  # numero di soggetti, altezza massima ed altezze/pesi medi per decade ed altezza
  result = df.group_by(
      (pl.col("birthdate").dt.year() // 10 * 10).alias("decade"),
      (pl.col("height") < 1.7).alias("short?"),
  ).agg(
      pl.len(),
      pl.col("height").max().alias("tallest"),
      pl.col("weight", "height").mean().name.prefix("avg_"),
  )
  ```

- `List`

11.1.3 Expression expansion e selettori

Nella .agg abbiamo usato

```
pl.col("weight", "height").mean().name.prefix("avg_"),
```

che è una "expression expansion": di base applica la stessa trasformazione a diverse colonne. L'alternativa equivalente è sarebbe stata

```
[
    pl.col("weight").mean().alias("avg_weight"),
    pl.col("height").mean().alias("avg_height"),
]
```

che presenta il contro di codice ripetuto, il pro che più facilmente sarà parallelizzabile rispetto alla expression expansion.
Una expression expansion utile per il datamanagement può essere quella basata sul tipo di dato, raggiungibile come segue

```
(pl.col(pl.Float64) * 1.1).name.suffix("*1.1")
```

che moltiplica per 1.1 tutti i float e ne modifica il nome. Alcune expression expansion utili

```
# seleziona tutti i floats
df.with_columns((pl.col(pl.Float32, pl.Float64) / eur_usd_rate).round(2))

# seleziona con regex sul nome (mostra solamente)
df.select(pl.col("ticker", "^.*_high$", "^.*_low$"))

# selezionare tutte le colonne
df.select(pl.all())

# escludere colonne
df.select(pl.all().exclude("^day_.*$"))
df.select(pl.col(pl.Float64).exclude("^day_.*$"))

# rinominazione in batch con espressioni regolari
df.select(
    (pl.col("^year_.*$") / eur_usd_rate).name.prefix("in_eur_"),
    (pl.col("day_high", "day_low") / gbp_usd_rate).name.suffix("_gbp"),
    pl.col("foo", "bar", "baz").name.map(str.upper)
)
```

Per avere selezione ancor pià flessibile il submodulo `selectors` fornisce selettori per tipo di variabile e nome

```
import polars.selectors as cs

# seleziona stringhe o variabili che terminano di nome in "high"
df.select(cs.string() | cs.ends_with("_high"))

# cs.numeric()   # tutte le numeriche
# cs.temporal()  # tutte le temporali
```

i cs producono cose assimilabili ai set, pertanto possiamo usare operatori booleani

```
df.select(cs.contains("_") - cs.string())

# Operatore  Operazione
# ---------------------
# A | B      Union
# A & B      Intersection
# A - B      Difference
# A ^ B      Symmetric difference
# ~A          Complement
```

Una volta effettuata la selezione aggiungiamo espressioni come in `pl.col`

```
df.select((cs.contains("_") - cs.string()) / eur_usd_rate)
```

Modalità esecuzione Vi sono due modalità:

- **eager**: query/espressioni eseguite immediatamente; ok per l'esecuzione interattiva/esplorazione dati (e non si sa ancora quale sarà l'elaborazione da fare)

```
df = pl.read_csv("docs/assets/data/iris.csv")
df_small = df.filter(pl.col("sepal_length") > 5)
df_agg = df_small.group_by("species").agg(pl.col("sepal_width").mean())
print(df_agg)
```

- **lazy**: query/espressioni eseguite una volta che sono *collected*. Questo ha vantaggi di performance, soprattutto con dataset di grandi dimensioni, perché permette al motore di ottimizzare (es seleziona solo le colonne e righe solo necessarie necessarie)

  ```
  # definizione query
  q = (
      pl.scan_csv("docs/assets/data/iris.csv")
      .filter(pl.col("sepal_length") > 5)
      .group_by("species")
      .agg(pl.col("sepal_width").mean())
  )

  # visualizzare le operazioni effettuate
  q.explain()

  # esecuzione ed ottenimento dei risultati
  df = q.collect()
  ```

  I benefici: minore carico su memoria e cpu, permette di caricare dataset più grandi in memoria e processarli più velocemente.

11.2 Data I/O

11.2.1 Importazione dati

11.2.1.1 Da file

L'importazione di dati può avvenere

- in maniera greedy mediante i metodi `pl.read_*`

  ```
  # wrapper
  df = lb.io2.import_data("paths")

  # File testuali
  df = pl.read_csv('path.csv', separator=";", decimal_comma=True, try_parse_dates=

  # Alcuni binari notevoli
  df = pl.read_excel('path.xlsx', sheet_name = 'asd')  # file xlsx
  ```

- in maniera non greedy mediante i metodi `pl.scan_*`

  ```
  # scanning: per file di grandi dimensioni non esegue subito il codice e porta
  # tutto in memoria ma imposta una selezione e importa il necessario (non
  # fullando la ram)
  df2_query = (
      pl.scan_csv("dati.csv")
  ```

```
        .filter(pl.col("b") > 3)
    )

    df2 = df2_query.collect()
```

11.2.1.2 Da pandas

Utilizzare `pl.from_pandas`

```
df = pl.from_pandas(pandas_df)
```

11.2.2 Esportazione dati

11.2.2.1 Su file

Si usano i metodi `write_*` dei `DataFrame`

```
# # wrapper
# df = lb.io.export_data(df_or_dict_of_dfs, path)  #TODO

# File testuali
df.write_csv('path.csv')

# Binari
df.write_excel('path.xlsx') # singolo
```

11.2.2.2 Verso pandas

Si usa il metodo `to_pandas`

```
# pandas
df.to_pandas()
```

11.3 Series

11.3.1 Creazione

Il modo base per creare una `Series` è

```
>>> import datetime as dt

>>> ints = [1, 2, None, 3]
>>> floats = [4., 5., 6., None]
>>> chars = ["a", "b", "c", None]
>>> dates = [dt.date(2001,1,1), None, dt.date(2002,1,1), dt.date(2003,1,1)]

>>> # senza specificare nome della variabile (utilizzata in data.frame)
>>> i = pl.Series(ints)
>>> i
shape: (4,)
Series: '' [i64]
[
    1
```

```
    2
    null
    3
]
>>> f = pl.Series(floats)
>>> f
shape: (4,)
Series: '' [f64]
[
    4.0
    5.0
    6.0
    null
]
>>> c = pl.Series(chars)
>>> c
shape: (4,)
Series: '' [str]
[
    "a"
    "b"
    "c"
    null
]
>>> d = pl.Series(dates)
>>> d
shape: (4,)
Series: '' [date]
[
    2001-01-01
    null
    2002-01-01
    2003-01-01
]
```

Specificando nomi variabile e tipo

```
>>> i = pl.Series("some_ints", ints, dtype=pl.Int64)
>>> f = pl.Series("some_floats", floats, dtype=pl.Float64)
>>> c = pl.Series("some_chars", chars, dtype=pl.String)
>>> d = pl.Series("some_dates", dates, dtype=pl.Date)

>>> # benefit di specificare i nomi nelle serie
>>> pl.DataFrame([i,f,c,d])
shape: (4, 4)
+-----------+-------------+------------+------------+
| some_ints | some_floats | some_chars | some_dates |
| ---       | ---         | ---        | ---        |
| i64       | f64         | str        | date       |
+=========================================================+
| 1         | 4.0         | a          | 2001-01-01 |
| 2         | 5.0         | b          | null       |
```

```
| null      | 6.0         | c           | 2002-01-01 |
| 3         | null        | null        | 2003-01-01 |
+-----------+-------------+-------------+------------+
```

```
>>> # Nomi possono essere aggiunti/modificati mediante alias
>>> d.alias("very_SAME_dates")
shape: (4,)
Series: 'very_SAME_dates' [date]
[
    2001-01-01
    null
    2002-01-01
    2003-01-01
]
```

11.3.2 Attributi

```
>>> i.dtype
Int64
>>> i.name
'some_ints'
>>> i.shape
(4,)
```

11.3.3 Indexing e selezione

Con `[]` possiamo utilizzare, interi, liste, slicing; per booleani usare `filter`

```
>>> i[0]                          # un intero
1
>>> i[[0]]                        # lista di un intero
shape: (1,)
Series: 'some_ints' [i64]
[
    1
]
>>> i[[3, 0, 1]]                  # lista di interi
shape: (3,)
Series: 'some_ints' [i64]
[
    3
    1
    2
]
>>> i[:2]                         # slicing
shape: (2,)
Series: 'some_ints' [i64]
[
    1
    2
]
```

```
>>> i.filter([True, False, False, True]) # lista di booleani
shape: (2,)
Series: 'some_ints' [i64]
[
    1
    3
]
>>> i.filter(i > 2)                      # serie di booleani
shape: (1,)
Series: 'some_ints' [i64]
[
    3
]
```

11.3.4 Elaborazione vettorizzata

Similmente a numpy le elaborazioni sono vettorizzate (si applicano *ufuncs* e broadcasting similmente a pandas

```
>>> np.exp(f) + 1  # questo funziona solo con i float
shape: (4,)
Series: 'some_floats' [f64]
[
    55.59815
    149.413159
    404.428793
    null
]
```

11.3.5 Coercizione di tipo (`cast`)

Si usa il metodo `cast` fornendo un tipo builtin di Python o un tipo di polars

```
>>> i.cast(pl.Float32)
shape: (4,)
Series: 'some_ints' [f32]
[
    1.0
    2.0
    null
    3.0
]
```

11.3.6 Valori condizionali a-la `ifelse` (`pl.when`)

Da saltarci fuori

11.3.7 Applicazione di una funzione a tutti gli elementi (`map`)

Per applicare una funzione per scalare a tutti gli elementi di una `Series` utilizzare `map_elements`

```
>>> i.map_elements(lambda x: f"{x:.2f}") # esempio di formattazione
shape: (4,)
Series: 'some_ints' [str]
[
    "1.00"
    "2.00"
    null
    "3.00"
]
```

11.3.8 Sostituzione di valori (`replace`)

Se si vuole sostituire solo alcuni elementi e lasciare invariati gli altri si usa `replace`:

```
>>> i.replace({1 : 999})
shape: (4,)
Series: 'some_ints' [i64]
[
    999
    2
    null
    3
]
```

11.3.9 Test di appartenenza (`in`, `isin`)

Con

- la keyword `in` testa l'esistenza nella serie e restituisce un booleano secco

  ```
  >>> 1 in i
  True
  ```

- il metodo `isin` della serie testa l'appartenenza ad un gruppo dei vari valori;

  ```
  >>> i.is_in([1, 2])
  shape: (4,)
  Series: 'some_ints' [bool]
  [
      true
      true
      null
      false
  ]
  ```

11.3.10 Dati mancanti

Vediamo metodi/funzioni più utili a livello di `Series`

```
>>> i.is_null()          # test; z.is_not_null() sua negazione
shape: (4,)
Series: 'some_ints' [bool]
[
    false
    false
    true
    false
]
>>> i.drop_nulls()       # eliminare
shape: (3,)
Series: 'some_ints' [i64]
[
    1
    2
    3
]
>>> i.fill_null(999)     # riempimento
shape: (4,)
Series: 'some_ints' [i64]
[
    1
    2
    999
    3
]
```

11.3.11 Duplicati

I metodi più utili:

```
>>> x = pl.Series([1, 2, 2, None, None])
>>> x.is_duplicated() # marca i doppi, anche le prime manifestazioni, nonchè gli NA
shape: (5,)
Series: '' [bool]
[
    false
    true
    true
    true
    true
]
>>> x.unique(maintain_order=True)   # rende unica: altrimenti cambia ordinamento
shape: (3,)
Series: '' [i64]
[
    1
    2
```

```
    null
]
```

11.3.12 Sorting

Si usa il metodo `sort`

```
>>> i.sort()
shape: (4,)
Series: 'some_ints' [i64]
[
    null
    1
    2
    3
]
```

11.3.13 Discretizzazione/creazione di classi (`cut`, `qcut`)

Si usa la funzione `cut` sulla `Series` specificando i cut o `qcut` sulla base dei quantili.

```
>>> rng = np.random.default_rng(12093)
>>> age = pl.Series(rng.integers(1, 100, size = 5))
>>> age
shape: (5,)
Series: '' [i64]
[
    32
    2
    79
    26
    93
]

>>> # cuts specificati
>>> cuts = [0, 25, 50, 75, 100]
>>> labs = ["prima_di_0", "giovane", "medio", "senior", "anziano", "oltre_100"]

>>> age.cut(breaks = cuts) # senza labels
shape: (5,)
Series: '' [enum]
[
    "(25, 50]"
    "(0, 25]"
    "(75, 100]"
    "(25, 50]"
    "(75, 100]"
]
>>> age.cut(breaks = cuts, labels = labs) # con
shape: (5,)
```

```
Series: '' [enum]
[
    "medio"
    "giovane"
    "anziano"
    "medio"
    "anziano"
]
```

```
>>> age.qcut(5)    # quintile of age
shape: (5,)
Series: '' [cat]
[
    "(29.6, 50.8]"
    "(-inf, 21.200000000000003]"
    "(50.8, 81.8]"
    "(21.200000000000003, 29.6]"
    "(81.8, inf]"
]
```

11.3.14 Dummy variables

11.3.14.1 Utilizzo base (`get_dummies`)

Per creare un `DataFrame` di dummy a partire da una serie si usa `to_dummies`.
Qui è meglio dare il nome alla variabile

```
>>> x = pl.Series("var", ["a","a","b", None]) # occhio ai na che sono ignorati
>>> x.to_dummies()
shape: (4, 3)
+-------+-------+----------+
| var_a | var_b | var_null |
| ---   | ---   | ---      |
| u8    | u8    | u8       |
+==========================+
| 1     | 0     | 0        |
| 1     | 0     | 0        |
| 0     | 1     | 0        |
| 0     | 0     | 1        |
+-------+-------+----------+
```

11.3.14.2 Variabili a risposta multipla

`str.getdummies` crea dummy utile per specificare i separatori (es per risposte multiple)

```
>>> x = pl.Series("dummies", ["a|b", "b|c", "a", "a|b|c"])
>>> # TODOHERE
```

11.3.15 Stringhe

Vediamo alcune funzionalità utili delle `Series` di stringhe.

- Vi sono metodi direttamente accedibili dalla `Series`
- altri metodi sono accedibili da `Series.str`

Ad ogni modo vi è overlap con i metodi delle `str` Python di base.

```
>>> suits = pl.Series(["clubsaaa", "Diamondsaa", "hearts", "Spades"])
>>> rps = pl.Series(["ROck ", " paper", "sciSSORS"])
```

11.3.15.1 Paste

Di due serie

```
>>> rps + "5"           # paste con costante
shape: (3,)
Series: '' [str]
[
    "ROck 5"
    " paper5"
    "sciSSORS5"
]
>>> rps + rps[::-1]     # paste di due serie
shape: (3,)
Series: '' [str]
[
    "ROck sciSSORS"
    " paper paper"
    "sciSSORSROck "
]
```

Di variabili di un dataframe

```
>>> df = pl.DataFrame({"rps1": rps, "rps2": rps})
>>> df.select(
...   pl.concat_str(["rps1", "rps2"], separator=", ").alias("pasted")
... )
shape: (3, 1)
+--------------------+
| pasted             |
| ---                |
| str                |
+====================+
| ROck , ROck        |
|  paper,  paper     |
| sciSSORS, sciSSORS |
+--------------------+
```

11.3.15.2 Collapse

```
>>> rps.str.join("_")    # collassa
shape: (1,)
Series: '' [str]
[
    "ROck _ paper_sciSSORS"
]
```

11.3.15.3 Formattazione varia

```
>>> rps.str.strip_chars()                # elimina bianchi
shape: (3,)
Series: '' [str]
[
    "ROck"
    "paper"
    "sciSSORS"
]
>>> rps.str.pad_start(8, fill_char="_")  # aggiungi caratteri all'inizio
shape: (3,)
Series: '' [str]
[
    "___ROck "
    "__ paper"
    "sciSSORS"
]
>>> rps.str.pad_end(8, fill_char="_")    # aggiungi caratteri alla fine
shape: (3,)
Series: '' [str]
[
    "ROck ___"
    " paper__"
    "sciSSORS"
]
>>> rps.str.to_lowercase()               # tutto minuscolo
shape: (3,)
Series: '' [str]
[
    "rock "
    " paper"
    "scissors"
]
>>> rps.str.to_uppercase()               # maiuscolo
shape: (3,)
Series: '' [str]
[
    "ROCK "
    " PAPER"
    "SCISSORS"
]
```

Per incollare/formattare assieme due/più stringhe selezionarle e applicare una funzione ad hoc per riga al dataframe (se non si riesce a fare in maniera alternativa con il lessico polar che è più efficiente)

```
>>> tmp = pl.DataFrame({
...     "l": ["a", "b", "c"],
...     "n": ["1", "2", "3"]
... })
```

```
>>> tmp.map_rows(lambda row: f"{row[0]} ({row[1]})")
shape: (3, 1)
+-------+
| map   |
| ---   |
| str   |
+=======+
| a (1) |
| b (2) |
| c (3) |
+-------+
```

11.3.15.4 Lunghezza ed sottostringhe

```
>>> rps.str.len_chars()                 # lunghezza
shape: (3,)
Series: '' [u32]
[
    5
    6
    8
]
>>> rps.str.slice(2,5)                  # subsetting stringa (es)
shape: (3,)
Series: '' [str]
[
    "ck "
    "aper"
    "iSSOR"
]
```

11.3.15.5 Splitting

```
>>> agecl = pl.Series(["0-10", "11-15", "11-15"])
>>> agecl.str.split("-")      # ritorna una serie di liste
shape: (3,)
Series: '' [list[str]]
[
    ["0", "10"]
    ["11", "15"]
    ["11", "15"]
]

>>> # selezione sulle componenti
>>> agecl.str.split("-").list[0]
shape: (3,)
Series: '' [str]
[
    "0"
    "11"
    "11"
```

```
]
>>> agecl.str.split("-").list.first()
shape: (3,)
Series: '' [str]
[
    "0"
    "11"
    "11"
]
>>> agecl.str.split("-").list.last()
shape: (3,)
Series: '' [str]
[
    "10"
    "15"
    "15"
]

>>> # splittare, eseguire cose su ciascun elemento e ricomporre
>>> (
...   agecl.str.split("-")
...   .list.eval((pl.element().cast(pl.Int64)+1).cast(pl.String))
...   .list.join("-||-")
... )
shape: (3,)
Series: '' [str]
[
    "1-||-11"
    "12-||-16"
    "12-||-16"
]

>>> # splittare, filtrare e ricomporre
>>> (
...   agecl.str.split("-")
...   .list.filter((pl.element().cast(pl.Int64)>5))
...   .list.join("-||-")
... )
shape: (3,)
Series: '' [str]
[
    "10"
    "11-||-15"
    "11-||-15"
]


>>> # per espandere in verticale
>>> (
...   agecl.str.split("-").alias("estremo")
...   .to_frame()
```

```
...    .with_row_index()
...    .explode("estremo")
... )
shape: (6, 2)
+-------+---------+
| index | estremo |
| ---   | ---     |
| u32   | str     |
+=================+
| 0     | 0       |
| 0     | 10      |
| 1     | 11      |
| 1     | 15      |
| 2     | 11      |
| 2     | 15      |
+-------+---------+

>>> # per avere un dataframe
>>> (
...    agecl.str.split_exact("-",1)
...    .struct.rename_fields(["upper", "lower"])
...    .to_frame()
...    .unnest()
... )
shape: (3, 2)
+-------+-------+
| upper | lower |
| ---   | ---   |
| str   | str   |
+===============+
| 0     | 10    |
| 11    | 15    |
| 11    | 15    |
+-------+-------+
```

11.3.15.6 Ricerca/rimpiazzo

```
>>> # Ricerca
>>> # -------
>>> suits.str.contains("[ae]")  # ne ha?
shape: (4,)
Series: '' [bool]
[
    true
    true
    true
    true
]
>>> suits.str.count_matches("[ae]")     # conta quanti
shape: (4,)
Series: '' [u32]
```

```
[
    3
    3
    2
    2
]
>>> suits.str.find("e")          # trova posizione match
shape: (4,)
Series: '' [u32]
[
    null
    null
    1
    4
]

>>> # Ricerca e rimpiazzo con regex
>>> # -----------------------------
>>> suits.str.replace("a", "4") # rimpiazzare la prima occorrenza
shape: (4,)
Series: '' [str]
[
    "clubs4aa"
    "Di4mondsaa"
    "he4rts"
    "Sp4des"
]
>>> suits.str.replace_all("a", "4") # rimpiazza tutte le occorrenze
shape: (4,)
Series: '' [str]
[
    "clubs444"
    "Di4monds44"
    "he4rts"
    "Sp4des"
]
>>> suits.str.replace_all("[^a-zA-Z-/:]", "") # pulizia stringa con caratteri ammessi
shape: (4,)
Series: '' [str]
[
    "clubsaaa"
    "Diamondsaa"
    "hearts"
    "Spades"
]

>>> # Ricerca di stringhe esatte senza .str
>>> # -------------------------------------
>>> suits.replace("Diamonds", "foobar") # .replace per parole intere
shape: (4,)
Series: '' [str]
```

```
[
    "clubsaaa"
    "Diamondsaa"
    "hearts"
    "Spades"
]
>>> suits.replace({"Diamonds": "foobar", "Spades": "asdasd"})
shape: (4,)
Series: '' [str]
[
    "clubsaaa"
    "Diamondsaa"
    "hearts"
    "asdasd"
]

>>> # Ricerca ed estrazione mediante regex
>>> # -------------------------------------
>>> import re
>>> x = pl.Series(["primo incontro_PF_02", "primo incontro_PF_03"])
>>> reg = re.compile(r"^([\w ]+?)_(\w+)")
>>> x.map_elements(lambda x: reg.sub(r"\1", x))  # extract_time
shape: (2,)
Series: '' [str]
[
    "primo incontro"
    "primo incontro"
]
>>> x.map_elements(lambda x: reg.sub(r"\2", x))  # extract_pz
shape: (2,)
Series: '' [str]
[
    "PF_02"
    "PF_03"
]
```

11.3.15.7 Estrazione

```
>>> x = pl.Series(["primo incontro_PF_02", "primo incontro_PF_03"])

>>> # da inizio sino a primo _, salva solo prima occorrenza
>>> x.str.extract(r"^(.+?)_", 1)
shape: (2,)
Series: '' [str]
[
    "primo incontro"
    "primo incontro"
]

>>> # estrai le 4 parole
>>> x.str.extract_all(r"([a-zA-Z0-9]+)")
```

```
shape: (2,)
Series: '' [list[str]]
[
    ["primo", "incontro", ... "02"]
    ["primo", "incontro", ... "03"]
]
>>> ( x.str.extract_all(r"([a-zA-Z0-9]+)")
...   .list.to_struct(fields=["one","two","three","four"])
...   .to_frame()
... ).unnest()
shape: (2, 4)
+-------+----------+-------+------+
| one   | two      | three | four |
| ---   | ---      | ---   | ---  |
| str   | str      | str   | str  |
+================================+
| primo | incontro | PF    | 02   |
| primo | incontro | PF    | 03   |
+-------+----------+-------+------+
```

11.3.16 Date/ore

Vediamo alcune funzionalità utili delle `Series` di date/ore. Qui invece vi è legame col modulo `datetime`.

11.3.16.1 Parsing da stringa

```
>>> # alcune stringhe in vari formati di data/ore
>>> iso = pl.Series(["1969-07-20 20:12:40",
...                  "1969-11-19 06:54:35",
...                  "1971-02-05 09:18:11"])

>>> eu = pl.Series(["20/07/1969 20:12:40",
...                 "19/11/1969 06:54:35",
...                 "05/02/1971 09:18:11"])

>>> iso.str.to_datetime() # iso out of the box
shape: (3,)
Series: '' [datetime[s]]
[
    1969-07-20 20:12:40
    1969-11-19 06:54:35
    1971-02-05 09:18:11
]
>>> eu.str.to_datetime(format="%d/%m/%Y %H:%M:%S") # formato custom EU
shape: (3,)
Series: '' [datetime[s]]
[
    1969-07-20 20:12:40
    1969-11-19 06:54:35
    1971-02-05 09:18:11
```

```
]
```

11.3.16.2 Creazione da componenti

```
>>> df = pl.DataFrame({
...     "year" : [1969, 1969, 1971],
...     "month" : [7, 11, 2],
...     "day" : [20, 19, 5]
... })

>>> df.with_columns(pl.date(pl.col("year"), pl.col("month"), pl.col("day")))
shape: (3, 4)
+------+-------+-----+------------+
| year | month | day | date       |
| ---  | ---   | --- | ---        |
| i64  | i64   | i64 | date       |
+================================+
| 1969 | 7     | 20  | 1969-07-20 |
| 1969 | 11    | 19  | 1969-11-19 |
| 1971 | 2     | 5   | 1971-02-05 |
+------+-------+-----+------------+
```

11.3.16.3 Estrazione componenti

```
>>> isod = iso.str.to_datetime()

>>> # estrazione di anno, mese, giorno
>>> isod.dt.year()
shape: (3,)
Series: '' [i32]
[
    1969
    1969
    1971
]
>>> isod.dt.month()
shape: (3,)
Series: '' [i8]
[
    7
    11
    2
]
>>> isod.dt.day()
shape: (3,)
Series: '' [i8]
[
    20
    19
    5
]
```

```
>>> # estrazione di data/ora
>>> isod.dt.date() # da datetime a date
shape: (3,)
Series: '' [date]
[
    1969-07-20
    1969-11-19
    1971-02-05
]
>>> isod.dt.time() # da datetime a time
shape: (3,)
Series: '' [time]
[
    20:12:40
    06:54:35
    09:18:11
]
```

11.3.16.4 Esportazione a stringa

```
>>> isod.dt.strftime("%d/%m/%Y")
shape: (3,)
Series: '' [str]
[
    "20/07/1969"
    "19/11/1969"
    "05/02/1971"
]
```

11.3.16.5 Intervalli di tempo/differenze date

Per una differenza di date

```
>>> # anni
>>> (dt.date.today() - isod.dt.date()).dt.total_days() / 365.25
shape: (3,)
Series: '' [f64]
[
    57.305955
    56.971937
    55.759069
]
```

11.3.16.6 Creazione di range di date

Se vogliamo una serie con una sequenza di date utilizzare date$_r ange$

```
>>> first_date= dt.date(1900,1,1)
>>> last_date = dt.date(2000,1,1)
>>> pl.date_range(first_date, last_date, eager=True)
shape: (36_525,)
```

```
Series: 'literal' [date]
[
    1900-01-01
    1900-01-02
    1900-01-03
    1900-01-04
    1900-01-05
    ...
    1999-12-28
    1999-12-29
    1999-12-30
    1999-12-31
    2000-01-01
]
```

11.3.17 Dati categorici (`Enum` e `Categorical`)

Sono rappresentazioni a-la `factor` di R con interi linkati a label per risparmiare spazio rispetto a stringhe. Ve ne sono due tipi

- gli Enum sono categorici con *label fisse* e ordine esplicitato

```
>>> bears_enum = pl.Series(["Polar", "Panda", None, "Brown", "Polar"],
...                        dtype=pl.Enum(["Polar", "Panda", "Brown"]))
>>> bears_enum == "Polar"
shape: (5,)
Series: '' [bool]
[
    true
    false
    null
    false
    true
]
>>> bears_enum > "Panda"
shape: (5,)
Series: '' [bool]
[
    false
    false
    null
    true
    false
]
```

- i Categorical hanno label che posson cambiare e l'ordinamento è alfabetico/implicito

```
>>> bears_cat = pl.Series(["Polar", "Panda", None, "Brown", "Polar"],
...                       dtype=pl.Categorical)
>>> bears_cat == "Polar"
shape: (5,)
```

```
Series: '' [bool]
[
    true
    false
    null
    false
    true
]
>>> bears_cat >= "Panda"
shape: (5,)
Series: '' [bool]
[
    true
    true
    null
    false
    true
]
```

Per efficienza preferire gli `Enum` laddove possibile (si può convertire al `Categorical` se necessario).
Un wrapper

```
>>> # # wrapper comodo che imita factor
>>> # import pylbmisc as lb
>>> # c = lb.dm.to_categorical(
>>> #       [1, 2, 3] * 2,
>>> #       levels = [1,2,3],
>>> #       labels = ["a","b","c"]
>>> # )
```

Il contenuto low-level si puo investigare con

```
>>> c = bears_enum
>>> c.cast(pl.UInt8)          # tipo as.integer
shape: (5,)
Series: '' [u8]
[
    0
    1
    null
    2
    0
]
>>> c.cat.get_categories()    # livelli del factor
shape: (3,)
Series: '' [str]
[
    "Polar"
    "Panda"
    "Brown"
]
```

11.3.18 Liste, array, structs

Una Series può contenere al suo interno dati molteplici

- se di tipologia comune il tipo di dato è `Array` (consigliati, ma il numero di elementi deve essere lo stesso) oppure `List` (il numero di elementi può essere variabile da riga a riga)
- se di tipologia varia abbiamo `Structs`

11.3.18.1 Liste

Il caso classico è uno splitting di stringa che crea una lista

```
>>> # weather data
>>> dfw = pl.DataFrame(
...     {
...         "station": [f"Station {idx}" for idx in range(1, 6)],
...         "temperatures": [
...             "20 5 5 E1 7 13 19 9 6 20",
...             "18 8 16 11 23 E2 8 E2 E2 E2 90 70 40",
...             "19 24 E9 16 6 12 10 22",
...             "E2 E0 15 7 8 10 E1 24 17 13 6",
...             "14 8 E0 16 22 24 E1",
...         ],
...     }
... ).with_columns(
...     pl.col("temperatures").str.split(" "),
... )

>>> sw = pl.Series([
...             "20 5 5 E1 7 13 19 9 6 20",
...             "18 8 16 11 23 E2 8 E2 E2 E2 90 70 40",
...             "19 24 E9 16 6 12 10 22",
...             "E2 E0 15 7 8 10 E1 24 17 13 6",
...             "14 8 E0 16 22 24 E1",
...         ]).str.split(" ")

>>> dfw
shape: (5, 2)
+-----------+------------------------+
| station   | temperatures           |
| ---       | ---                    |
| str       | list[str]              |
+====================================+
| Station 1 | ["20", "5", ... "20"]  |
| Station 2 | ["18", "8", ... "40"]  |
| Station 3 | ["19", "24", ... "22"] |
| Station 4 | ["E2", "E0", ... "6"]  |
| Station 5 | ["14", "8", ... "E1"]  |
+-----------+------------------------+
>>> sw
```

```
shape: (5,)
Series: '' [list[str]]
[
    ["20", "5", ... "20"]
    ["18", "8", ... "40"]
    ["19", "24", ... "22"]
    ["E2", "E0", ... "6"]
    ["14", "8", ... "E1"]
]
```

Alternativamente si possono creare con l'accoppiata `pl.concat_list` per le colonne di un df da porre in una lista

```
>>> pl.DataFrame(
...     {
...         "station": [f"Station {idx}" for idx in range(1, 11)],
...         "day_1": [17, 11, 8, 22, 9, 21, 20, 8, 8, 17],
...         "day_2": [15, 11, 10, 8, 7, 14, 18, 21, 15, 13],
...         "day_3": [16, 15, 24, 24, 8, 23, 19, 23, 16, 10],
...     }
... ).with_columns(
...     # create the list of homogeneous data
...     pl.concat_list(pl.all().exclude("station")).alias("all_temps")
... )
shape: (10, 5)
+------------+-------+-------+-------+--------------+
| station    | day_1 | day_2 | day_3 | all_temps    |
| ---        | ---   | ---   | ---   | ---          |
| str        | i64   | i64   | i64   | list[i64]    |
+=============================================+
| Station 1  | 17    | 15    | 16    | [17, 15, 16] |
| Station 2  | 11    | 11    | 15    | [11, 11, 15] |
| Station 3  | 8     | 10    | 24    | [8, 10, 24]  |
| Station 4  | 22    | 8     | 24    | [22, 8, 24]  |
| Station 5  | 9     | 7     | 8     | [9, 7, 8]    |
| Station 6  | 21    | 14    | 23    | [21, 14, 23] |
| Station 7  | 20    | 18    | 19    | [20, 18, 19] |
| Station 8  | 8     | 21    | 23    | [8, 21, 23]  |
| Station 9  | 8     | 15    | 16    | [8, 15, 16]  |
| Station 10 | 17    | 13    | 10    | [17, 13, 10] |
+------------+-------+-------+-------+--------------+
```

Una volta che abbiamo a che fare con liste, per gestirle utilizziamo l'accessore alle funzioni `.list` che contiene giusto per dire

```
list.agg              list.all              list.any
list.arg_max          list.arg_min          list.concat
list.contains         list.count_matches    list.diff
list.drop_nulls       list.eval             list.explode
list.filter           list.first            list.gather
list.gather_every     list.get              list.head
list.item             list.join             list.last
```

```
list.len                list.max                          list.mean
list.median             list.min                          list.n_unique
list.reverse            list.sample                       list.set_difference
list.set_intersection   list.set_symmetric_difference     list.set_union
list.shift              list.slice                        list.sort
list.std                list.sum                          list.tail
list.to_array           list.to_struct                    list.unique
list.var
```

Selezione/slicing Si usano alcune tra

```
>>> # selezione di elementi
>>> sweath.list.head(3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'sweath' is not defined
>>> sweath.list.tail(3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'sweath' is not defined
>>> # sweath.list.slice(1, 2)
>>> # sweath.list.last()
>>> # sweath.list.first()

>>> # dataframe
>>> weather.with_columns(
...     pl.col("temperatures").list.head(3).alias("head"),
...     pl.col("temperatures").list.tail(3).alias("tail"),
...     pl.col("temperatures").list.slice(-3, 2).alias("two_next_to_last"),
... )
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'weather' is not defined
```

Elaborazione element-wise Usiamo `eval` specificando l'espressione che usi `pl.element`. `eval` ritorna sempre una lista

```
>>> # Casting
>>> dfw.with_columns(
...     pl.col("temperatures")
...     .list.eval(pl.element().cast(pl.Int64, strict=False))
... )
shape: (5, 2)
+-----------+--------------------+
| station   | temperatures       |
| ---       | ---                |
| str       | list[i64]          |
+==================================+
| Station 1 | [20, 5, ... 20]    |
| Station 2 | [18, 8, ... 40]    |
| Station 3 | [19, 24, ... 22]   |
```

```
| Station 4 | [null, null, ... 6] |
| Station 5 | [14, 8, ... null]   |
+-----------+---------------------+

>>> # trasformare il tipo di dato in intero e poi in booleano contando i
>>> # missing/errori
>>> dfw.with_columns(
...     pl.col("temperatures")
...     .list.eval(pl.element().cast(pl.Int64, strict=False).is_null())
...     .list.sum()
...     .alias("errors"),
... )
shape: (5, 3)
+-----------+-----------------------+--------+
| station   | temperatures          | errors |
| ---       | ---                   | ---    |
| str       | list[str]             | u32    |
+=========================================+
| Station 1 | ["20", "5", ... "20"] | 1      |
| Station 2 | ["18", "8", ... "40"] | 4      |
| Station 3 | ["19", "24", ... "22"] | 1      |
| Station 4 | ["E2", "E0", ... "6"] | 3      |
| Station 5 | ["14", "8", ... "E1"] | 2      |
+-----------+-----------------------+--------+

>>> # senza il casting usare una espressione regolare per controllare che la
>>> # misurazione non inizi con una lettera
>>> dfw.with_columns(
...     pl.col("temperatures")
...     .list.eval(pl.element().str.contains("(?i)[a-z]"))
...     .list.sum()
...     .alias("errors"),
... )
shape: (5, 3)
+-----------+-----------------------+--------+
| station   | temperatures          | errors |
| ---       | ---                   | ---    |
| str       | list[str]             | u32    |
+=========================================+
| Station 1 | ["20", "5", ... "20"] | 1      |
| Station 2 | ["18", "8", ... "40"] | 4      |
| Station 3 | ["19", "24", ... "22"] | 1      |
| Station 4 | ["E2", "E0", ... "6"] | 3      |
| Station 5 | ["14", "8", ... "E1"] | 2      |
+-----------+-----------------------+--------+
```

Aggregazione `agg` possono essere utilizzate per aggregare liste

```
>>> dfw.with_columns(
...     pl.col("temperatures")
...     .list.eval(pl.element().cast(pl.Int64, strict=False))
```

```
... )
shape: (5, 2)
+-----------+---------------------+
| station   | temperatures        |
| ---       | ---                 |
| str       | list[i64]           |
+=================================+
| Station 1 | [20, 5, ... 20]     |
| Station 2 | [18, 8, ... 40]     |
| Station 3 | [19, 24, ... 22]    |
| Station 4 | [null, null, ... 6] |
| Station 5 | [14, 8, ... null]   |
+-----------+---------------------+
```

Porre in long la lista Si usa explode

```
>>> # explode pone in long
>>> dfweath.explode("temperatures", empty_as_null=False)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'dfweath' is not defined
```

11.3.18.2 Array

Si accede alle funzioni mediante l'accessore `.arr`

11.3.18.3 Structs

Si usano per dati eterogenei

- può essere generata se passiamo una lista di dict ad una Series

  ```
  >>> strs = pl.Series(
  ...     [
  ...         {"Movie": "Cars", "Theatre": "NE", "Avg_Rating": 4.5},
  ...         {"Movie": "Toy Story", "Theatre": "ME", "Avg_Rating": 4.9},
  ...     ],
  ... )
  >>> strs
  shape: (2,)
  Series: '' [struct[3]]
  [
      {"Cars","NE",4.5}
      {"Toy Story","ME",4.9}
  ]
  ```

- Può essere anche generata da elaborazioni

  ```
  >>> # dati originali
  >>> df = pl.DataFrame(
  ...     {
  ...         "Movie": ["Cars", "IT", "ET", "Cars", "Up", "IT", "Cars", "ET", "Up", "Cars"],
  ...         "Theatre": ["NE", "ME", "IL", "ND", "NE", "SD", "NE", "IL", "IL", "NE"],
  ```

```
...         "Avg_Rating": [4.5, 4.4, 4.6, 4.3, 4.8, 4.7, 4.5, 4.9, 4.7, 4.6],
...         "Count": [30, 27, 26, 29, 31, 28, 28, 26, 33, 28],
...     }
... )

>>> counts = df.select(pl.col("Theatre").value_counts(sort=True))
>>> counts
shape: (5, 1)
+-----------+
| Theatre   |
| ---       |
| struct[2] |
+===========+
| {"NE",4}  |
| {"IL",3}  |
| {"ME",1}  |
| {"ND",1}  |
| {"SD",1}  |
+-----------+
```

- può essere generata in un contesto usando `pl.struct` che torna comoda per individuare doppi sulla base di due colonne

```
>>> df.filter(pl.struct("Movie", "Theatre").is_duplicated())
shape: (5, 4)
+-------+---------+------------+-------+
| Movie | Theatre | Avg_Rating | Count |
| ---   | ---     | ---        | ---   |
| str   | str     | f64        | i64   |
+=======================================+
| Cars  | NE      | 4.5        | 30    |
| ET    | IL      | 4.6        | 26    |
| Cars  | NE      | 4.5        | 28    |
| ET    | IL      | 4.9        | 26    |
| Cars  | NE      | 4.6        | 28    |
+-------+---------+------------+-------+
```

Espandere in un df Si usa `unnest` (per struct facenti parte di un dataframe)

```
>>> counts.unnest()
shape: (5, 2)
+---------+-------+
| Theatre | count |
| ---     | ---   |
| str     | u32   |
+=================+
| NE      | 4     |
| IL      | 3     |
| ME      | 1     |
| ND      | 1     |
| SD      | 1     |
+---------+-------+
```

Nominare/rinominare i campi della struct Si usa `rename_fields`

```
>>> counts.with_columns(
...   pl.col("Theatre").struct.rename_fields(["teatro", "frequenze"])
... )
shape: (5, 1)
+-----------+
| Theatre   |
| ---       |
| struct[2] |
+===========+
| {"NE",4}  |
| {"IL",3}  |
| {"ME",1}  |
| {"ND",1}  |
| {"SD",1}  |
+-----------+
```

Non sembra che sia cambiato nulla ma torna utile sotto per la selezione e se facciamo l'unnest vengono utilizzati i nuovi nomi

Estrarre elementi si usa `field`

```
>>> counts.with_columns(
...   pl.col("Theatre").struct.rename_fields(["teatro", "frequenze"])
...   .struct.field("frequenze").alias("il_numero")
... )
shape: (5, 2)
+-----------+-----------+
| Theatre   | il_numero |
| ---       | ---       |
| struct[2] | u32       |
+=======================+
| {"NE",4}  | 4         |
| {"IL",3}  | 3         |
| {"ME",1}  | 1         |
| {"ND",1}  | 1         |
| {"SD",1}  | 1         |
+-----------+-----------+
```

Applicargli funzioni che sfruttano gli elementi Se a una struct con nomi applichiamo una funzione mediante `map_elements` all'interno della definizione possiamo programmare in dict style.
Di sotto una applicazione (overkill) giusto per fare vedere la cosa

```
>>> values = pl.DataFrame(
...     {
...         "m": [0, 0, 0, 1, 1, 1, 2],
...         "n": [2, 3, 4, 1, 2, 3, 1],
...     }
... )
```

```
>>> values.with_columns(
...     pl.struct(["m", "n"])
...     .map_elements(lambda s: s["m"] * s["n"], return_dtype=pl.Int64)
...     .alias("ack")
... )
shape: (7, 3)
+-----+-----+-----+
| m   | n   | ack |
| --- | --- | --- |
| i64 | i64 | i64 |
+=================+
| 0   | 2   | 0   |
| 0   | 3   | 0   |
| 0   | 4   | 0   |
| 1   | 1   | 1   |
| 1   | 2   | 2   |
| 1   | 3   | 3   |
| 2   | 1   | 2   |
+-----+-----+-----+
```

11.4 DataFrame

11.4.1 Creazione

Si crea mediante:

```
df = pl.DataFrame(data)
```

dove `data` possono essere ad esempio un dict (contenente sequences), una lista di `pl.Series` o array numpy) un dataframe `pd.DataFrame`. Si possono specificare le conversioni

- completamente mediante il parametro `schema`
- parzialmente (solo per alcune variabili) usando `schema_overrides`

```
>>> # dict di liste
>>> data = {"state": ["Ohio", "Ohio", "Ohio", "Nevada", "Nevada", "Nevada"],
...         "year": [2000, 2001, 2002, 2001, 2002, 2003],
...         "pop": [1.5, 1.7, 3.6, 2.4, 2.9, 3.2]}
>>> states = pl.DataFrame(data)
>>> states
shape: (6, 3)
+--------+------+-----+
| state  | year | pop |
| ---    | ---  | --- |
| str    | i64  | f64 |
+====================+
| Ohio   | 2000 | 1.5 |
| Ohio   | 2001 | 1.7 |
```

```
| Ohio   | 2002 | 3.6 |
| Nevada | 2001 | 2.4 |
| Nevada | 2002 | 2.9 |
| Nevada | 2003 | 3.2 |
+--------+------+-----+
```

```
>>> # per riga (lista di dict) specificando lo schema delle coercizioni
>>> states2 = pl.DataFrame([
...     {"state": "Ohio", "year": 2000, "pop": 1.5},
...     {"state": "Ohio", "year": 2001, "pop": 1.7} # ...
...     ],
... )

>>> # per riga (lista di liste e nomi variabili specificati in schema)
>>> data = [[1, 2, 3], [4, 5, 6]]
>>> df6 = pl.DataFrame(data, schema=["a", "b", "c"], orient="row")
>>> df6
shape: (2, 3)
+-----+-----+-----+
| a   | b   | c   |
| --- | --- | --- |
| i64 | i64 | i64 |
+=================+
| 1   | 2   | 3   |
| 4   | 5   | 6   |
+-----+-----+-----+

>>> # lista di series
>>> pl.DataFrame([i, d, f])
shape: (4, 3)
+-----------+------------+-------------+
| some_ints | some_dates | some_floats |
| ---       | ---        | ---         |
| i64       | date       | f64         |
+=====================================+
| 1         | 2001-01-01 | 4.0         |
| 2         | null       | 5.0         |
| null      | 2002-01-01 | 6.0         |
| 3         | 2003-01-01 | null        |
+-----------+------------+-------------+
```

11.4.2 Contenuto

```
>>> df = states
>>> df.shape   # dimensioni
(6, 3)
>>> df.height  # nrow
6
>>> df.width   # ncol
3
```

```
>>> df.columns # nomi di colonna
['state', 'year', 'pop']
>>> df.dtypes  # tipi di variabile
[String, Int64, Float64]
>>> df.schema  # schema
Schema({'state': String, 'year': Int64, 'pop': Float64})

>>> df.describe()
shape: (9, 4)
+------------+--------+----------+----------+
| statistic  | state  | year     | pop      |
| ---        | ---    | ---      | ---      |
| str        | str    | f64      | f64      |
+=========================================+
| count      | 6      | 6.0      | 6.0      |
| null_count | 0      | 0.0      | 0.0      |
| mean       | null   | 2001.5   | 2.55     |
| std        | null   | 1.048809 | 0.836062 |
| min        | Nevada | 2000.0   | 1.5      |
| 25%        | null   | 2001.0   | 1.7      |
| 50%        | null   | 2002.0   | 2.9      |
| 75%        | null   | 2002.0   | 3.2      |
| max        | Ohio   | 2003.0   | 3.6      |
+------------+--------+----------+----------+
>>> df.glimpse()  # equivalente di str
Rows: 6
Columns: 3
$ state <str> 'Ohio', 'Ohio', 'Ohio', 'Nevada', 'Nevada', 'Nevada'
$ year  <i64> 2000, 2001, 2002, 2001, 2002, 2003
$ pop   <f64> 1.5, 1.7, 3.6, 2.4, 2.9, 3.2

>>> df.head()  # utile per dataframe piu grossi
shape: (5, 3)
+--------+------+-----+
| state  | year | pop |
| ---    | ---  | --- |
| str    | i64  | f64 |
+====================+
| Ohio   | 2000 | 1.5 |
| Ohio   | 2001 | 1.7 |
| Ohio   | 2002 | 3.6 |
| Nevada | 2001 | 2.4 |
| Nevada | 2002 | 2.9 |
+--------+------+-----+
>>> df.top_k(2, by="pop")
shape: (2, 3)
+--------+------+-----+
| state  | year | pop |
| ---    | ---  | --- |
| str    | i64  | f64 |
+====================+
```

```
| Ohio   | 2002 | 3.6 |
| Nevada | 2003 | 3.2 |
+--------+------+-----+
>>> df.bottom_k(2, by="pop")
shape: (2, 3)
+-------+------+-----+
| state | year | pop |
| ---   | ---  | --- |
| str   | i64  | f64 |
+====================+
| Ohio  | 2000 | 1.5 |
| Ohio  | 2001 | 1.7 |
+-------+------+-----+
```

11.4.3 Copia di df e di struttura (`clone`, `clear`)

`clone` crea una copia del dataframe fornito facendo si che eventuali modifiche non intacchino il dataframe di partenza.

```
>>> states_copy = df.clone()
>>> id(states)
139850498733168
>>> id(states_copy) # different
139850498732592

>>> # otherwise
>>> df = states # no copy performed
>>> id(df)      # same
139850498733168
>>> id(states)  # same
139850498733168
```

`clear` crea una copia della struttura di un dataframe con n righe vuote

```
>>> states.clear(2)
shape: (2, 3)
+-------+------+------+
| state | year | pop  |
| ---   | ---  | ---  |
| str   | i64  | f64  |
+=====================+
| null  | null | null |
| null  | null | null |
+-------+------+------+
```

11.4.4 Selezione

```
>>> df = pl.DataFrame(
...     {
...         "name": ["Alice Archer", "Ben Brown", "Chloe Cooper", "Daniel Donovan"],
...         "birthdate": [
...             dt.date(1997, 1, 10),
```

```
...             dt.date(1985, 2, 15),
...             dt.date(1983, 3, 22),
...             dt.date(1981, 4, 30),
...         ],
...         "weight": [57.9, 72.5, 53.6, 83.1],  # (kg)
...         "height": [1.56, 1.77, 1.65, None],  # (m)
...     }
... )

>>> df2  = pl.DataFrame({
...     "state": ["Ohio", "Ohio", "Ohio", "Nevada", "Nevada", "Nevada"],
...     "year": [2000, 2001, 2002, 2001, 2002, 2003],
...     "pop": [1.5, 1.7, 3.6, 2.4, 2.9, 3.2]}
... )
```

11.4.4.1 Colonne (`.select`)

La selezione può essere basata su `[]` o usare `select`

```
>>> # una colonna
>>> # -----------
>>> df["name"]                  # restituisce una Series
shape: (4,)
Series: 'name' [str]
[
    "Alice Archer"
    "Ben Brown"
    "Chloe Cooper"
    "Daniel Donovan"
]
>>> df.get_column("name")       # come sopra
shape: (4,)
Series: 'name' [str]
[
    "Alice Archer"
    "Ben Brown"
    "Chloe Cooper"
    "Daniel Donovan"
]
>>> df.get_column_index("birthdate") # restituisce l'indice di colonna
1
>>> df.select("name")           # restituisce un DataFrame
shape: (4, 1)
+----------------+
| name           |
| ---            |
| str            |
+================+
| Alice Archer   |
| Ben Brown      |
| Chloe Cooper   |
```

```
| Daniel Donovan |
+----------------+
>>> df.select(                  # restituisce un DataFrame
...     pl.col("name")
... )
shape: (4, 1)
+----------------+
| name           |
| ---            |
| str            |
+================+
| Alice Archer   |
| Ben Brown      |
| Chloe Cooper   |
| Daniel Donovan |
+----------------+

>>> # più colonne
>>> # ------------
>>> vars = ["name", "birthdate"]
>>> df[vars]              # dataframe
shape: (4, 2)
+----------------+------------+
| name           | birthdate  |
| ---            | ---        |
| str            | date       |
+============================+
| Alice Archer   | 1997-01-10 |
| Ben Brown      | 1985-02-15 |
| Chloe Cooper   | 1983-03-22 |
| Daniel Donovan | 1981-04-30 |
+----------------+------------+
>>> df.select(vars)
shape: (4, 2)
+----------------+------------+
| name           | birthdate  |
| ---            | ---        |
| str            | date       |
+============================+
| Alice Archer   | 1997-01-10 |
| Ben Brown      | 1985-02-15 |
| Chloe Cooper   | 1983-03-22 |
| Daniel Donovan | 1981-04-30 |
+----------------+------------+
>>> df.select(
...     pl.col(vars)
... )
shape: (4, 2)
+----------------+------------+
| name           | birthdate  |
| ---            | ---        |
```

```
| str            | date       |
+=============================+
| Alice Archer   | 1997-01-10 |
| Ben Brown      | 1985-02-15 |
| Chloe Cooper   | 1983-03-22 |
| Daniel Donovan | 1981-04-30 |
+---------------+------------+
>>> df.get_columns(vars) # lista di serie
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: DataFrame.get_columns() takes 1 positional argument but 2 were given


>>> # selezioni particolari con pl.col
>>> # -------------------------------
>>> df.select(pl.all().exclude("birthdate")) # tutte le colonne eccetto alcune
shape: (4, 3)
+----------------+--------+--------+
| name           | weight | height |
| ---            | ---    | ---    |
| str            | f64    | f64    |
+=================================+
| Alice Archer   | 57.9   | 1.56   |
| Ben Brown      | 72.5   | 1.77   |
| Chloe Cooper   | 53.6   | 1.65   |
| Daniel Donovan | 83.1   | null   |
+----------------+--------+--------+
>>> df.select(pl.col("^.*t$"))               # regex: variabili che finiscono con t
shape: (4, 2)
+--------+--------+
| weight | height |
| ---    | ---    |
| f64    | f64    |
+=================+
| 57.9   | 1.56   |
| 72.5   | 1.77   |
| 53.6   | 1.65   |
| 83.1   | null   |
+--------+--------+
>>> df.select(pl.col(pl.Date))               # basate su dtype
shape: (4, 1)
+------------+
| birthdate  |
| ---        |
| date       |
+============+
| 1997-01-10 |
| 1985-02-15 |
| 1983-03-22 |
| 1981-04-30 |
+------------+
```

```
>>> df.select(                                  # mischiare selettori
...     pl.col("name"),
...     pl.col(pl.Date)
... )
shape: (4, 2)
+----------------+------------+
| name           | birthdate  |
| ---            | ---        |
| str            | date       |
+============================+
| Alice Archer   | 1997-01-10 |
| Ben Brown      | 1985-02-15 |
| Chloe Cooper   | 1983-03-22 |
| Daniel Donovan | 1981-04-30 |
+----------------+------------+
```

11.4.4.2 Righe (`.filter`/`.remove`)

Per selezionare righe si usa `filter` con `pl.col`; vengono tenute le righe che danno un risultato booleano `True` (comparazioni che riguardano dati mancanti saranno mancanti e non saranno selezionate, come spesso ragionevole).
Viceversa se si vogliono eliminare le righe `True`, utilizzare `.remove`.
Alcuni esempi con filter

```
>>> df.filter(pl.col("birthdate").dt.year() < 1990)
shape: (3, 4)
+----------------+------------+--------+--------+
| name           | birthdate  | weight | height |
| ---            | ---        | ---    | ---    |
| str            | date       | f64    | f64    |
+=============================================+
| Ben Brown      | 1985-02-15 | 72.5   | 1.77   |
| Chloe Cooper   | 1983-03-22 | 53.6   | 1.65   |
| Daniel Donovan | 1981-04-30 | 83.1   | null   |
+----------------+------------+--------+--------+

>>> # selezione basata su indici: creare un progressivo di riga (chiamato index)
>>> df.with_row_index().filter(pl.col("index") > 2)
shape: (1, 5)
+-------+----------------+------------+--------+--------+
| index | name           | birthdate  | weight | height |
| ---   | ---            | ---        | ---    | ---    |
| u32   | str            | date       | f64    | f64    |
+=================================================+
| 3     | Daniel Donovan | 1981-04-30 | 83.1   | null   |
+-------+----------------+------------+--------+--------+

>>> # Per mettere in and le condizioni di selezione fornire diverse
>>> # righe. Alternativamente usare & ma aiutare con parentesi
>>> df.filter(
...     pl.col("birthdate").dt.year() < 1990,
```

```
...     pl.col("height") > 1.7
... )
shape: (1, 4)
+-----------+------------+--------+--------+
| name      | birthdate  | weight | height |
| ---       | ---        | ---    | ---    |
| str       | date       | f64    | f64    |
+=========================================+
| Ben Brown | 1985-02-15 | 72.5   | 1.77   |
+-----------+------------+--------+--------+

>>> df.filter(
...     (pl.col("birthdate").dt.year() < 1990) & (pl.col("height") > 1.7)
... )
shape: (1, 4)
+-----------+------------+--------+--------+
| name      | birthdate  | weight | height |
| ---       | ---        | ---    | ---    |
| str       | date       | f64    | f64    |
+=========================================+
| Ben Brown | 1985-02-15 | 72.5   | 1.77   |
+-----------+------------+--------+--------+

>>> # Per mettere in or le condizioni
>>> df.filter(
...     (pl.col("birthdate").dt.year() < 1990) | (pl.col("height") > 1.7)
... )
shape: (3, 4)
+----------------+------------+--------+--------+
| name           | birthdate  | weight | height |
| ---            | ---        | ---    | ---    |
| str            | date       | f64    | f64    |
+=================================================+
| Ben Brown      | 1985-02-15 | 72.5   | 1.77   |
| Chloe Cooper   | 1983-03-22 | 53.6   | 1.65   |
| Daniel Donovan | 1981-04-30 | 83.1   | null   |
+----------------+------------+--------+--------+

>>> # Per negare una condizione usare ~
>>> df.filter(
...     ~ ((pl.col("birthdate").dt.year() < 1990) | (pl.col("height") > 1.7))
... )
shape: (1, 4)
+--------------+------------+--------+--------+
| name         | birthdate  | weight | height |
| ---          | ---        | ---    | ---    |
| str          | date       | f64    | f64    |
+==============================================+
| Alice Archer | 1997-01-10 | 57.9   | 1.56   |
+--------------+------------+--------+--------+
```

11.4.4.3 Righe e colonne

Alternare `select` e `filter`.

11.4.4.4 Singoli elementi (`.item`)

Si usa `item` (equivalente di at/iat di pandas)

```
>>> df
shape: (4, 4)
+----------------+------------+--------+--------+
| name           | birthdate  | weight | height |
| ---            | ---        | ---    | ---    |
| str            | date       | f64    | f64    |
+=============================================+
| Alice Archer   | 1997-01-10 | 57.9   | 1.56   |
| Ben Brown      | 1985-02-15 | 72.5   | 1.77   |
| Chloe Cooper   | 1983-03-22 | 53.6   | 1.65   |
| Daniel Donovan | 1981-04-30 | 83.1   | null   |
+----------------+------------+--------+--------+
>>> df.item(1,1)
datetime.date(1985, 2, 15)
>>> df.item(1, "weight")
72.5
```

11.4.5 Inserimento colonna in un punto (`.insert_column`)

Si fa mediante `insert_column`, specificando indice e `Series` o espressione da inserire

11.4.6 Assegnazione (`with_columns`)

Si opera sulle colonne con `with_columns` e selezionando con `pl.col` e il tutto funziona similmente ad `assign` di pandas (ad eccezione dell'esecuzione sequenziale):

- si può assegnare (il nome della variabile) all'inizio oppure dare `alias` alla fine (ma nel caso gli alias vanno prima, come da python base)
- se la colonna non esiste verrà creata altrimenti sovrascritta
- le istruzioni non vanno intese come eseguite sequenzialmente
- costanti (soprattutto se si intende stringhe, affinché non vengano confuse con nomi variabili, vanno specificate con `pl.lit`)

```
>>> def change_sign(x):
...     return - x
...
>>> df.with_columns(
...     # creazione di costante
...     pl.lit(1).alias("one"),
...     # sovrascrittura di variabile
```

```
...     change_sign(pl.col("weight")).alias("weight"),
...     # le istruzioni NON sono svolte sequenzialmente: sotto weight è quello
...     # corretto
...     (pl.col("weight") / (pl.col("height") ** 2)).alias("bmi"),
...     # esempio di ifelse: ancora essendo non sequenziali le istruzioni il bmi
...     # deve essere ricalcolato, non si può usare quello dell'istruzione
...     # precedente; inoltre "weight" ha ancora il segno corretto
...     fat=(
...         pl.when((pl.col("weight") / (pl.col("height") ** 2)) > 25)
...         .then(pl.lit("yes"))
...         .otherwise(pl.lit("no"))
...     )
... )
shape: (4, 7)
+----------------+------------+--------+--------+-----+-----------+-----+
| name           | birthdate  | weight | height | one | bmi       | fat |
| ---            | ---        | ---    | ---    | --- | ---       | --- |
| str            | date       | f64    | f64    | i32 | f64       | str |
+=================================================================+
| Alice Archer   | 1997-01-10 | -57.9  | 1.56   | 1   | 23.791913 | no  |
| Ben Brown      | 1985-02-15 | -72.5  | 1.77   | 1   | 23.141498 | no  |
| Chloe Cooper   | 1983-03-22 | -53.6  | 1.65   | 1   | 19.687787 | no  |
| Daniel Donovan | 1981-04-30 | -83.1  | null   | 1   | null      | no  |
+----------------+------------+--------+--------+-----+-----------+-----+
```

Per la valutazione sequenziale ad ora il meglio che ho trovato è

- salvare l'espressione e richiamarla in una medesima `with_columns`, ma evitando di riscriverla per esteso. Non è sequenziale ma almeno non si duplica il codice:

  ```
  >>> bmi = (pl.col("weight") / (pl.col("height") ** 2))

  >>> df.with_columns(
  ...     bmi = bmi,          # bmi.alias("bmi"),
  ...     fat = bmi > 25,     # (bmi > 25).alias("fat")
  ... )
  shape: (4, 6)
  +----------------+------------+--------+--------+-----------+-------+
  | name           | birthdate  | weight | height | bmi       | fat   |
  | ---            | ---        | ---    | ---    | ---       | ---   |
  | str            | date       | f64    | f64    | f64       | bool  |
  +=================================================================+
  | Alice Archer   | 1997-01-10 | 57.9   | 1.56   | 23.791913 | false |
  | Ben Brown      | 1985-02-15 | 72.5   | 1.77   | 23.141498 | false |
  | Chloe Cooper   | 1983-03-22 | 53.6   | 1.65   | 19.687787 | false |
  | Daniel Donovan | 1981-04-30 | 83.1   | null   | null      | null  |
  +----------------+------------+--------+--------+-----------+-------+
  ```

- porre nella prima chiamata le variabili indipendenti, nella seconda quelle dipendenti

```
>>> df.with_columns(
...     bmi = (pl.col("weight") / (pl.col("height") ** 2))
... ).with_columns(
...     fat = (pl.col("bmi") > 25)
... )
shape: (4, 6)
+----------------+------------+--------+--------+-----------+-------+
| name           | birthdate  | weight | height | bmi       | fat   |
| ---            | ---        | ---    | ---    | ---       | ---   |
| str            | date       | f64    | f64    | f64       | bool  |
+=================================================================+
| Alice Archer   | 1997-01-10 | 57.9   | 1.56   | 23.791913 | false |
| Ben Brown      | 1985-02-15 | 72.5   | 1.77   | 23.141498 | false |
| Chloe Cooper   | 1983-03-22 | 53.6   | 1.65   | 19.687787 | false |
| Daniel Donovan | 1981-04-30 | 83.1   | null   | null      | null  |
+----------------+------------+--------+--------+-----------+-------+
```

Vi è anche un `with_columns_seq` ma non sembra essere la soluzione.

11.4.6.1 Elaborazioni particolari

Aggiungere totali di colonna Per sommare tutte le colonne (per riga) di cui si specifica il nome utilizzare in una `with_columns` l'espressione `pl.sum_horizontal`

11.4.7 Rimozione righe/colonne (`.filter`/`.drop`)

Per le righe utilizzare `filter` similmente a sezione 11.4.4.2; per le colonne utilizzare `drop`

```
>>> df.drop(["weight"])
shape: (4, 3)
+----------------+------------+--------+
| name           | birthdate  | height |
| ---            | ---        | ---    |
| str            | date       | f64    |
+=====================================+
| Alice Archer   | 1997-01-10 | 1.56   |
| Ben Brown      | 1985-02-15 | 1.77   |
| Chloe Cooper   | 1983-03-22 | 1.65   |
| Daniel Donovan | 1981-04-30 | null   |
+----------------+------------+--------+
```

11.4.8 Rinominare colonne

Si usa il metodo `rename`

```
>>> df.rename({"weight": "peso", "height": "altezza"})
shape: (4, 4)
+----------------+------------+------+---------+
| name           | birthdate  | peso | altezza |
| ---            | ---        | ---  | ---     |
| str            | date       | f64  | f64     |
```

```
+=============================================+
| Alice Archer   | 1997-01-10 | 57.9 | 1.56    |
| Ben Brown      | 1985-02-15 | 72.5 | 1.77    |
| Chloe Cooper   | 1983-03-22 | 53.6 | 1.65    |
| Daniel Donovan | 1981-04-30 | 83.1 | null    |
+----------------+------------+------+---------+
>>> df.rename(lambda x: f"{x}2")
shape: (4, 4)
+----------------+------------+---------+---------+
| name2          | birthdate2 | weight2 | height2 |
| ---            | ---        | ---     | ---     |
| str            | date       | f64     | f64     |
+=================================================+
| Alice Archer   | 1997-01-10 | 57.9    | 1.56    |
| Ben Brown      | 1985-02-15 | 72.5    | 1.77    |
| Chloe Cooper   | 1983-03-22 | 53.6    | 1.65    |
| Daniel Donovan | 1981-04-30 | 83.1    | null    |
+----------------+------------+---------+---------+
```

11.4.9 Coercizione di tipi (`.cast`)

Possiamo usare `cast`. Se si vuole castare tutto il dataframe ad un unico tipo passarlo direttamente come in `df.cast(pl.String)`; altrimenti specificare il mapping di formato in un `dict` avente come chiave:

- i nomi variabile

```
>>> df = pl.DataFrame({
...         "foo": [1, 2, 3],
...         "bar": [6.0, 7.0, 8.0],
...         "ham": [dt.date(2020, 1, 2), dt.date(2021, 3, 4), dt.date(2022, 5, 6)
... })

>>> df.cast({"foo": pl.Float32, "bar": pl.UInt8})
shape: (3, 3)
+-----+-----+------------+
| foo | bar | ham        |
| --- | --- | ---        |
| f32 | u8  | date       |
+========================+
| 1.0 | 6   | 2020-01-02 |
| 2.0 | 7   | 2021-03-04 |
| 3.0 | 8   | 2022-05-06 |
+-----+-----+------------+
```

- il tipo variabile

```
>>> df.cast({pl.Date: pl.Datetime})
shape: (3, 3)
+-----+-----+---------------------+
| foo | bar | ham                 |
| --- | --- | ---                 |
```

```
| i64 | f64 | datetime[s]         |
+================================+
| 1   | 6.0 | 2020-01-02 00:00:00 |
| 2   | 7.0 | 2021-03-04 00:00:00 |
| 3   | 8.0 | 2022-05-06 00:00:00 |
+-----+-----+---------------------+
```

- un selettore che specifica una classe di tipi

```
>>> import polars.selectors as cs
>>> df.cast({cs.numeric(): pl.UInt32, cs.temporal(): pl.String})
shape: (3, 3)
+-----+-----+------------+
| foo | bar | ham        |
| --- | --- | ---        |
| u32 | u32 | str        |
+=========================+
| 1   | 6   | 2020-01-02 |
| 2   | 7   | 2021-03-04 |
| 3   | 8   | 2022-05-06 |
+-----+-----+------------+
```

11.4.10 Applicazione di funzioni

Per applicare una funzione all'intero data.frame (es nel contesto di una cascata di chiamate) utilizzare `pipe`.
Per righe/colonne Si usano le funzioni `map` con UDF (user defined functions) anche se sono molto meno performanti rispetto alla sintassi classica (però in questo modo si possono usare funzioni proprie o di librerie terze).
Sono consigliate le `ufuncs` di numpy e scipy per cercare di evitare il Python schietto.
In caso di missing vi possono essere problemi: approfondire qui

11.4.10.1 All'intero dataframe

Nel contesto di una cascata di chiamate può tornare utile `pipe` che prende in input una funzione (che prende in input un dataframe).

```
>>> df = pl.DataFrame({"b": [1, 2, 3, 4], "a": ["10", "20", "30", "40"]})

>>> # porre in ordine alfabetico le variabili dataset
>>> df.pipe(lambda tdf: tdf.select(sorted(tdf.columns)))
shape: (4, 2)
+-----+-----+
| a   | b   |
| --- | --- |
| str | i64 |
+===========+
| 10  | 1   |
| 20  | 2   |
| 30  | 3   |
```

```
| 40  | 4   |
+-----+-----+
```

11.4.10.2 A colonne

Per

- applicare una funzione a ciascun elemento separatamente di una serie si usa `map_elements` come già visto

```
>>> import math

>>> df = pl.DataFrame(
...     {
...         "keys": ["a", "a", "b", "b"],
...         "values": [10, 7, 1, 23],
...     }
... )
>>> def my_log(value):
...     return math.log(value)
...
>>> df.select(pl.col("values").map_elements(my_log, return_dtype=pl.Float64))
shape: (4, 1)
+----------+
| values   |
| ---      |
| f64      |
+==========+
| 2.302585 |
| 1.94591  |
| 0.0      |
| 3.135494 |
+----------+
```

- applicare una funzione al contenuto intero della serie (es per avere una aggregazione) si usa `map_batches`

```
>>> # togliere la media da una serie
>>> def demean(series):
...     total = 0
...     for value in series:
...         total += value
...     mean = total / len(series)
...     return pl.Series([value - mean for value in series])
...
>>> df.select(pl.col("values").map_batches(demean, return_dtype=pl.Float64))
shape: (4, 1)
+--------+
| values |
| ---    |
| f64    |
+========+
```

```
| -0.25  |
| -3.25  |
| -9.25  |
| 12.75  |
+--------+
```

```
>>> # effettuare un recoding a una serie
>>> # questa avrebbe bisogno di specificare il tipo specificato?
>>> # df.select(pl.all().map_batches(lambda x: x.replace(NA_recoding)))
```

- `map_columns` direi faccia all'incirca la stessa cosa di `map_batches`

```
# Se la funzione prende in input una serie e restituisce una serie
df.map_columns(df.columns, lambda x: x.replace(NA_recoding))
```

11.4.10.3 A righe

`DataFrame.map_rows`

11.4.11 Ciclo su righe/colonne

Tipicamente non di interesse ma nel caso non si possa esprimere con espressioni polars ...

11.4.11.1 Righe

Si usa:

- `rows`: per ottenere tutte le righe in botta unica (equivalente di `df.to_dict(orient="records")` di pandas)
- `iter_rows`: crea un iteratore sulle righe (che non vengono caricate tutte in memoria) e ritornando una tupla (a meno che `named=True`) per ciascuna riga in sequenza

```
>>> df = pl.DataFrame({'num_legs': [4, 2], 'num_wings': [0, 2]})
```

```
>>> # tuple
>>> for row in df.iter_rows():
...     print("first_elem = ", row[0],
...           "second_elem = ", row[1])
...
first_elem =  4 second_elem =  0
first_elem =  2 second_elem =  2
>>> # dict
>>> for row in df.iter_rows(named=True):
...     print("first_named = ", row["num_legs"],
...           "second_named = ", row["num_wings"])
...
first_named =  4 second_named =  0
first_named =  2 second_named =  2
```

11.4.11.2 Colonne

Si può utilizzare

- un ciclo `for` secco restituisce la serie
- un generatore `iter_columns`

```
>>> df = pl.DataFrame({'species': ['bear', 'bear', 'marsupial'],
...                    'population': [1864, 22000, 80000]})

>>> for s in df:
...     print(s)
...
shape: (3,)
Series: 'species' [str]
[
    "bear"
    "bear"
    "marsupial"
]
shape: (3,)
Series: 'population' [i64]
[
    1864
    22000
    80000
]
>>> for s in df.iter_columns():
...     print(s)
...
shape: (3,)
Series: 'species' [str]
[
    "bear"
    "bear"
    "marsupial"
]
shape: (3,)
Series: 'population' [i64]
[
    1864
    22000
    80000
]
```

11.4.12 Merge (`.join`)

11.4.12.1 Due dataframe

Si usa il metodo `join` sul primo DataFrame:

- bisogna specificare la variabile di merge nel parametro `on` (`left_on` e `right_on` se la chiave non ha lo stesso nome nei due dataframe)

- il tipo di merge in `how`: di default ritorna le righe dove la chiave è presente in entrambi i dataset (*inner join*). `left` è quello che spesso serve
- si può specificare un criterio di validazione (stringa passata a `validate`) che darà errore se non rispettato: es `1:1`, `1:m`, `m:1`.

```
>>> df1 = pl.DataFrame({'key': ['a', 'b', 'b', 'c', 'c', 'c'],
...                     'data': range(6)})
>>> df2 = pl.DataFrame({'key': ['a', 'b', 'd'],
...                     'values': ['x', 'y', 'z']})
>>> df1
shape: (6, 2)
+-----+------+
| key | data |
| --- | ---  |
| str | i64  |
+============+
| a   | 0    |
| b   | 1    |
| b   | 2    |
| c   | 3    |
| c   | 4    |
| c   | 5    |
+-----+------+
>>> df2
shape: (3, 2)
+-----+--------+
| key | values |
| --- | ---    |
| str | str    |
+==============+
| a   | x      |
| b   | y      |
| d   | z      |
+-----+--------+

>>> # vari tipi di merge
>>> # -------------------
>>> df1.join(df2, on = "key")                 # inner di default
shape: (3, 3)
+-----+------+--------+
| key | data | values |
| --- | ---  | ---    |
| str | i64  | str    |
+======================+
| a   | 0    | x      |
| b   | 1    | y      |
| b   | 2    | y      |
+-----+------+--------+
>>> df1.join(df2, on = "key", how = 'left')  # tiene dataset di sx, integra con quello di dx
shape: (6, 3)
```

```
+-----+------+--------+
| key | data | values |
| --- | ---  | ---    |
| str | i64  | str    |
+====================+
| a   | 0    | x      |
| b   | 1    | y      |
| b   | 2    | y      |
| c   | 3    | null   |
| c   | 4    | null   |
| c   | 5    | null   |
+-----+------+--------+
>>> df1.join(df2, on = "key", how = 'right') # tiene dataset di dx, integra con quell
shape: (4, 3)
+------+-----+--------+
| data | key | values |
| ---  | --- | ---    |
| i64  | str | str    |
+====================+
| 0    | a   | x      |
| 1    | b   | y      |
| 2    | b   | y      |
| null | d   | z      |
+------+-----+--------+
>>> df1.join(df2, on = "key", how = 'full') # tieni tutto
shape: (7, 4)
+------+------+-----------+--------+
| key  | data | key_right | values |
| ---  | ---  | ---       | ---    |
| str  | i64  | str       | str    |
+==================================+
| a    | 0    | a         | x      |
| b    | 1    | b         | y      |
| b    | 2    | b         | y      |
| c    | 3    | null      | null   |
| c    | 4    | null      | null   |
| c    | 5    | null      | null   |
| null | null | d         | z      |
+------+------+-----------+--------+

>>> # validazione
>>> df2.join(df2.rename(lambda x: f"{x}2"), left_on = "key", right_on="key2",
...          how= "left", validate="1:1")
shape: (3, 3)
+-----+--------+---------+
| key | values | values2 |
| --- | ---    | ---     |
| str | str    | str     |
+=======================+
| a   | x      | x       |
| b   | y      | y       |
```

```
| d   | z      | z       |
+-----+--------+---------+
```

11.4.12.2 Molteplici dataframe

Per mergiare diversi dataset aventi lo stesso nome di chiave in uno si può usare `reduce`

```
from functools import reduce

def merger(x, y):
    return x.join(y, on=['id', "time"], how = "left")

df = reduce(merger, [demo, lav, sal, bis])
```

11.4.12.3 Merge approssimato (`.join_asof`)

Simile ad un left-join con l'eccezione che si matcha sulla chiave *più vicina*, non necessariamente la chiave esatta. Entrambi i dataframe debbono essere ordinati su `on`. Strategie possibili

- `backward` (default): per ogni riga del dataset di sinistra si prende l'ultima riga di quello di destra su cui la chiave è minore o uguale a quella del dataset di sinistra
- `forward`: maggiore o uguale
- `nearest`: (ultima) riga con la chiave più simile (in eccesso o difetto) a quello di sinistra

Può essere utile per merge approssimato su date

```
>>> from datetime import date

>>> # df sotto già ordinati per data ma male non fa
>>> gdp = pl.DataFrame({
...         "date": pl.date_range(
...             date(2016, 1, 1),
...             date(2020, 1, 1),
...             "1y",
...             eager=True,
...         ),
...         "gdp": [4164, 4411, 4566, 4696, 4827],
... }).sort("date")

>>> population = pl.DataFrame({
...         "date": [date(2016, 3, 1), date(2018, 8, 1), date(2019, 1, 1)],
...         "population": [82.19, 82.66, 83.12],
... }).sort("date")


>>> population.join_asof(gdp, on="date") # backward: default
shape: (3, 3)
```

```
+------------+------------+------+
| date       | population | gdp  |
| ---        | ---        | ---  |
| date       | f64        | i64  |
+================================+
| 2016-03-01 | 82.19      | 4164 |
| 2018-08-01 | 82.66      | 4566 |
| 2019-01-01 | 83.12      | 4696 |
+------------+------------+------+
>>> population.join_asof(gdp, on="date", strategy="forward")
shape: (3, 3)
+------------+------------+------+
| date       | population | gdp  |
| ---        | ---        | ---  |
| date       | f64        | i64  |
+================================+
| 2016-03-01 | 82.19      | 4411 |
| 2018-08-01 | 82.66      | 4696 |
| 2019-01-01 | 83.12      | 4696 |
+------------+------------+------+
>>> population.join_asof(gdp, on="date", strategy="nearest")
shape: (3, 3)
+------------+------------+------+
| date       | population | gdp  |
| ---        | ---        | ---  |
| date       | f64        | i64  |
+================================+
| 2016-03-01 | 82.19      | 4164 |
| 2018-08-01 | 82.66      | 4696 |
| 2019-01-01 | 83.12      | 4696 |
+------------+------------+------+
```

11.4.13 Binding (`pl.concat`)

Sia per binding di riga che di colonna si usa `pl.concat`

```
>>> df1 = pl.DataFrame({'key': ['a', 'b', 'd'], 'data': ['x', 'y', 'z']})

>>> pl.concat([df1, df1], how="vertical") # rbind
shape: (6, 2)
+-----+------+
| key | data |
| --- | ---  |
| str | str  |
+=============+
| a   | x    |
| b   | y    |
| d   | z    |
| a   | x    |
| b   | y    |
| d   | z    |
```

```
+-----+------+
>>> pl.concat([df1, df1.rename(lambda x: f"{x}2")],
...           how="horizontal") # cbind (ha bisogno di renaming)
shape: (3, 4)
+-----+------+------+-------+
| key | data | key2 | data2 |
| --- | ---  | ---  | ---   |
| str | str  | str  | str   |
+============================+
| a   | x    | a    | x     |
| b   | y    | b    | y     |
| d   | z    | d    | z     |
+-----+------+------+-------+
```

11.4.14 Aggiunta di righe parziali ad un dataframe

Si fa sempre mediante `pl.concat` con l'opzione diagonal che riempie le colonne opportune, lascia le mancanti a null e se mai crea nuove colonne, come sotto

```
>>> df1 = pl.DataFrame({'key': ['a', 'b', 'd'],'variable': ['x', 'y', 'z'],'value': [1,2,3]})
>>> added = [{"key": "c", "variable": "z"}]

>>> pl.concat([df1, pl.DataFrame(added)], how="diagonal")
shape: (4, 3)
+-----+----------+-------+
| key | variable | value |
| --- | ---      | ---   |
| str | str      | i64   |
+========================+
| a   | x        | 1     |
| b   | y        | 2     |
| d   | z        | 3     |
| c   | z        | null  |
+-----+----------+-------+
```

11.4.15 Reshape

11.4.15.1 Porre in long (`.unpivot`)

Se abbiamo un dataset in formato wide si pone in long con il metodo `.unpivot`. Si può

- specificare un sottoinsieme di variabili da porre in long mediante `on`
- specificare con `variable_name` nome al posto di `variable` nel dataframe creato e `value_name` per specificare il nome della variabile al posto di `value`

```
>>> df = pl.DataFrame({"key": ["foo", "bar", "baz"],
...                    "x_bl": [1, 2, 3],
...                    "x_t0": [4, 5, 6],
...                    "x_t1": [7, 8, 9]}
```

```
... )

>>> df
shape: (3, 4)
+-----+------+------+------+
| key | x_bl | x_t0 | x_t1 |
| --- | ---  | ---  | ---  |
| str | i64  | i64  | i64  |
+==========================+
| foo | 1    | 4    | 7    |
| bar | 2    | 5    | 8    |
| baz | 3    | 6    | 9    |
+-----+------+------+------+
>>> df.unpivot(index = 'key')  # poni in long tutto
shape: (9, 3)
+-----+----------+-------+
| key | variable | value |
| --- | ---      | ---   |
| str | str      | i64   |
+=======================+
| foo | x_bl     | 1     |
| bar | x_bl     | 2     |
| baz | x_bl     | 3     |
| foo | x_t0     | 4     |
| bar | x_t0     | 5     |
| baz | x_t0     | 6     |
| foo | x_t1     | 7     |
| bar | x_t1     | 8     |
| baz | x_t1     | 9     |
+-----+----------+-------+
>>> df.unpivot(index = 'key', on = ["x_bl", "x_t0"]) # in long una selezione
shape: (6, 3)
+-----+----------+-------+
| key | variable | value |
| --- | ---      | ---   |
| str | str      | i64   |
+=======================+
| foo | x_bl     | 1     |
| bar | x_bl     | 2     |
| baz | x_bl     | 3     |
| foo | x_t0     | 4     |
| bar | x_t0     | 5     |
| baz | x_t0     | 6     |
+-----+----------+-------+
```

11.4.15.2 Porre in wide (`.pivot`)

Un dataset in formato long con colonne, id, tempo e variabili assortite può essere posto in formato wide mediante il metodo `pivot`.

```
>>> df = pl.DataFrame({'idpz': ['one', 'one', 'one', 'two', 'two'],
```

```
...                     'time': ["bl", "t1", "t2", "bl", "t1"],
...                     'x': range(1,6),
...                     'y': ['x', 'y', 'z', 'q', 'w'],
...                     'z': range(7, 12),
... })
>>> df
shape: (5, 5)
+------+------+-----+-----+-----+
| idpz | time | x   | y   | z   |
| ---  | ---  | --- | --- | --- |
| str  | str  | i64 | str | i64 |
+==============================+
| one  | bl   | 1   | x   | 7   |
| one  | t1   | 2   | y   | 8   |
| one  | t2   | 3   | z   | 9   |
| two  | bl   | 4   | q   | 10  |
| two  | t1   | 5   | w   | 11  |
+------+------+-----+-----+-----+

>>> df.pivot(index="idpz", on="time", values = ["x", "y"])
shape: (2, 7)
+------+------+------+------+------+------+------+
| idpz | x_bl | x_t1 | x_t2 | y_bl | y_t1 | y_t2 |
| ---  | ---  | ---  | ---  | ---  | ---  | ---  |
| str  | i64  | i64  | i64  | str  | str  | str  |
+=============================================+
| one  | 1    | 2    | 3    | x    | y    | z    |
| two  | 4    | 5    | null | q    | w    | null |
+------+------+------+------+------+------+------+
```

11.4.16 Trasporre (.transpose)

Eventualmente selezionare le colonne da trasporre mediante `column_names`,

```
>>> df = pl.DataFrame({"a": [1, 2, 3], "b": [4, 5, 6], "c": [7, 8, 9]})
>>> df
shape: (3, 3)
+-----+-----+-----+
| a   | b   | c   |
| --- | --- | --- |
| i64 | i64 | i64 |
+=================+
| 1   | 4   | 7   |
| 2   | 5   | 8   |
| 3   | 6   | 9   |
+-----+-----+-----+
>>> df.transpose(include_header=True,
...              header_name="variabile",
...              column_names=[f"row_{x}"for x in range(3)])
shape: (3, 4)
+-----------+-------+-------+-------+
```

```
| variabile | row_0 | row_1 | row_2 |
| ---       | ---   | ---   | ---   |
| str       | i64   | i64   | i64   |
+=================================+
| a         | 1     | 2     | 3     |
| b         | 4     | 5     | 6     |
| c         | 7     | 8     | 9     |
+-----------+-------+-------+-------+
```

11.4.17 Test di appartenenza (`in`)

`in` applicato ad un DataFrame testa la presenza di una colonna avente un determinato nome

```
>>> df = pl.DataFrame({"a": [1, 2], "b": list("xz"), "c": [2,3]})

>>> "a" in df
True
>>> "weight" in df
False
```

11.4.18 Dati mancanti

```
>>> df = pl.DataFrame({"a": [1., None, 3.],
...                    "b":[1., None, None],
...                    "c":[None, None, None],
...                    "d":[None, None, 3.]})
>>> df
shape: (3, 4)
+------+------+------+------+
| a    | b    | c    | d    |
| ---  | ---  | ---  | ---  |
| f64  | f64  | null | f64  |
+===========================+
| 1.0  | 1.0  | null | null |
| null | null | null | null |
| 3.0  | null | null | 3.0  |
+------+------+------+------+
```

11.4.18.1 Identificazione

```
>>> df.null_count() # numero mancanti
shape: (1, 4)
+-----+-----+-----+-----+
| a   | b   | c   | d   |
| --- | --- | --- | --- |
| u32 | u32 | u32 | u32 |
+=======================+
| 1   | 2   | 3   | 2   |
+-----+-----+-----+-----+
>>> df.count()      # numero non mancanti
```

```
shape: (1, 4)
+-----+-----+-----+-----+
| a   | b   | c   | d   |
| --- | --- | --- | --- |
| u32 | u32 | u32 | u32 |
+=======================+
| 2   | 1   | 0   | 1   |
+-----+-----+-----+-----+
```

11.4.18.2 Rimozione

`drop_nulls` è orientato alle righe ed elimina i casi che hanno almeno una variabile null, o che hanno almeno una variabile del subset specificato null

```
>>> df.drop_nulls()       # righe con anche un solo NA
shape: (0, 4)
+-----+-----+------+-----+
| a   | b   | c    | d   |
| --- | --- | ---  | --- |
| f64 | f64 | null | f64 |
+=========================+
+-----+-----+------+-----+
>>> df.drop_nulls(["d"])  # selezione righe senza missing nelle colonne specificate
shape: (1, 4)
+-----+------+------+-----+
| a   | b    | c    | d   |
| --- | ---  | ---  | --- |
| f64 | f64  | null | f64 |
+============================+
| 3.0 | null | null | 3.0 |
+-----+------+------+-----+

>>> # rimozione righe tutte NA
>>> df.filter(~pl.all_horizontal([pl.col(v).is_null() for v in df.columns]))
shape: (2, 4)
+-----+------+------+------+
| a   | b    | c    | d    |
| --- | ---  | ---  | ---  |
| f64 | f64  | null | f64  |
+============================+
| 1.0 | 1.0  | null | null |
| 3.0 | null | null | 3.0  |
+-----+------+------+------+
```

Per eliminare le colonne completamente null selezionare in base a dtype

```
>>> df.select(
...   pl.all().exclude(pl.Null())
... )
shape: (3, 3)
+------+------+------+
| a    | b    | d    |
```

```
| ---  | ---  | ---  |
| f64  | f64  | f64  |
+====================+
| 1.0  | 1.0  | null |
| null | null | null |
| 3.0  | null | 3.0  |
+------+------+------+
```

11.4.18.3 Imputazione

Per l'imputazione dei dati mancanti si usa a livello di dataframe `fill_null`

```
>>> df
shape: (3, 4)
+------+------+------+------+
| a    | b    | c    | d    |
| ---  | ---  | ---  | ---  |
| f64  | f64  | null | f64  |
+===========================+
| 1.0  | 1.0  | null | null |
| null | null | null | null |
| 3.0  | null | null | 3.0  |
+------+------+------+------+
>>> df.fill_null(99)                  # un valore comune
shape: (3, 4)
+------+------+-----+------+
| a    | b    | c   | d    |
| ---  | ---  | --- | ---  |
| f64  | f64  | i32 | f64  |
+=========================+
| 1.0  | 1.0  | 99  | 99.0 |
| 99.0 | 99.0 | 99  | 99.0 |
| 3.0  | 99.0 | 99  | 3.0  |
+------+------+-----+------+
>>> df.fill_null(strategy="forward") # locf verticale
shape: (3, 4)
+-----+-----+------+------+
| a   | b   | c    | d    |
| --- | --- | ---  | ---  |
| f64 | f64 | null | f64  |
+=========================+
| 1.0 | 1.0 | null | null |
| 1.0 | 1.0 | null | null |
| 3.0 | 1.0 | null | 3.0  |
+-----+-----+------+------+
>>> df.transpose().fill_null(strategy="forward").transpose(column_names=list("abcd"))
shape: (3, 4)
+------+------+------+------+
| a    | b    | c    | d    |
| ---  | ---  | ---  | ---  |
| f64  | f64  | f64  | f64  |
```

```
+=========================+
| 1.0  | 1.0  | 1.0  | 1.0  |
| null | null | null | null |
| 3.0  | 3.0  | 3.0  | 3.0  |
+------+------+------+------+
```

o `interpolate` rimpiazza i mancanti con interpolazioni lineari

```
>>> df.interpolate()
shape: (3, 4)
+-----+------+------+------+
| a   | b    | c    | d    |
| --- | ---  | ---  | ---  |
| f64 | f64  | null | f64  |
+=========================+
| 1.0 | 1.0  | null | null |
| 2.0 | null | null | null |
| 3.0 | null | null | 3.0  |
+-----+------+------+------+
```

11.4.19 Duplicati

```
>>> df = pl.DataFrame({"k1": ["one", "two"] * 3 + ["two"],
...                    "k2": [1, 1, 2, 3, 3, 4, 4]})
>>> df
shape: (7, 2)
+-----+-----+
| k1  | k2  |
| --- | --- |
| str | i64 |
+===========+
| one | 1   |
| two | 1   |
| one | 2   |
| two | 3   |
| one | 3   |
| two | 4   |
| two | 4   |
+-----+-----+
```

11.4.19.1 Identificazione (`.duplicated`)

`duplicated` ritorna un vettore di booleani per indicare se una riga è duplicata. Occhio che marchia anche alla prima occorrenza

```
>>> df.is_duplicated()            # ricerca complessiva
shape: (7,)
Series: '' [bool]
[
    false
    false
    false
```

```
    false
    false
    true
    true
]
>>> df.filter(df.is_duplicated()) # visualizzazione
shape: (2, 2)
+-----+-----+
| k1  | k2  |
| --- | --- |
| str | i64 |
+===========+
| two | 4   |
| two | 4   |
+-----+-----+
```

11.4.19.2 Rimozione (`.unique`)

Utilizzare `.unique` eventualmente specificando

- `maintain_order=True` per evitare che sia cambiato rispetto alla sorgente
- `keep` per la scelta di quale riga (es prima/ultima) nel caso di doppi

```
>>> df.unique()          # tutto il dataset
shape: (6, 2)
+-----+-----+
| k1  | k2  |
| --- | --- |
| str | i64 |
+===========+
| two | 1   |
| one | 1   |
| two | 3   |
| one | 2   |
| two | 4   |
| one | 3   |
+-----+-----+
>>> df.unique(["k2"])    # unique basato solo su alcune colonne
shape: (4, 2)
+-----+-----+
| k1  | k2  |
| --- | --- |
| str | i64 |
+===========+
| one | 2   |
| two | 4   |
| one | 1   |
| two | 3   |
+-----+-----+
```

11.4.20 Sorting

Si usa: `sort_values` per ordinare sulla base di valori di colonna

```
>>> rng = np.random.default_rng(23)
>>> df = pl.DataFrame({'g': ["x", "y", "x", "y"],
...                    'y' : rng.standard_normal((4)),
...                    'z' : rng.standard_normal((4))})

>>> df.sort('y')
shape: (4, 3)
+-----+-----------+----------+
| g   | y         | z        |
| --- | ---       | ---      |
| str | f64       | f64      |
+============================+
| y   | -2.318936 | 0.605966 |
| x   | -0.05799  | 0.909921 |
| y   | 0.217601  | -2.12628 |
| x   | 0.553261  | 0.431494 |
+-----+-----------+----------+
>>> df.sort(['g', 'y'])
shape: (4, 3)
+-----+-----------+----------+
| g   | y         | z        |
| --- | ---       | ---      |
| str | f64       | f64      |
+============================+
| x   | -0.05799  | 0.909921 |
| x   | 0.553261  | 0.431494 |
| y   | -2.318936 | 0.605966 |
| y   | 0.217601  | -2.12628 |
+-----+-----------+----------+
>>> df.sort(['g', 'y'], descending = [False, True])
shape: (4, 3)
+-----+-----------+----------+
| g   | y         | z        |
| --- | ---       | ---      |
| str | f64       | f64      |
+============================+
| x   | 0.553261  | 0.431494 |
| x   | -0.05799  | 0.909921 |
| y   | 0.217601  | -2.12628 |
| y   | -2.318936 | 0.605966 |
+-----+-----------+----------+
```

11.4.21 Id univoco di riga

Utilizzare `hash_rows` (versioni di polars differenti possono condurre a risultati differenti)

```
>>> df = pl.DataFrame(
```

```
...     {
...         "foo": [1, None, 3, 4],
...         "ham": ["a", "b", None, "d"],
...     }
... )
>>> df.hash_rows(seed=42)
shape: (4,)
Series: '' [u64]
[
    1081295755015292788
    16846555029793948966
    7550900513512627554
    9298298639392481527
]
```

11.4.22 Windows functions

Applicano funzioni separatamente a parti del dataset; `group_prog_id` dovrebbe essere un esempio. Sono ideali quando vogliamo effettuare operazioni per gruppo che viene selezionata mediante `.over` (che accetta un numero arbitrario di espressioni per creare i sottogruppi)

```
>>> group_df = (
...     pl.DataFrame({'group' : ['one', 'two'] * 5, 'data':rng.random(10)})
...     .sort(["group", "data"])
... )
>>> group_df
shape: (10, 2)
+-------+----------+
| group | data     |
| ---   | ---      |
| str   | f64      |
+==================+
| one   | 0.063854 |
| one   | 0.301453 |
| one   | 0.415222 |
| one   | 0.540298 |
| one   | 0.716585 |
| two   | 0.349148 |
| two   | 0.389077 |
| two   | 0.454666 |
| two   | 0.4707   |
| two   | 0.68359  |
+-------+----------+

>>> group_df.select(
...     pl.all(),
...     pl.col("data").rank().over("group").alias("group_rank")
... )
shape: (10, 3)
+-------+----------+------------+
```

```
| group | data     | group_rank |
| ---   | ---      | ---        |
| str   | f64      | f64        |
+=============================+
| one   | 0.063854 | 1.0        |
| one   | 0.301453 | 2.0        |
| one   | 0.415222 | 3.0        |
| one   | 0.540298 | 4.0        |
| one   | 0.716585 | 5.0        |
| two   | 0.349148 | 1.0        |
| two   | 0.389077 | 2.0        |
| two   | 0.454666 | 3.0        |
| two   | 0.4707   | 4.0        |
| two   | 0.68359  | 5.0        |
+-------+----------+------------+
```

Il parametro `mapping_strategy` di `over` gestisce come i risultati delle espressioni nei gruppi sono mappate alle righe del dataframe di origine:

- di default è `group_to_rows`: il risultato dell'espressione dovrebbe essere della stessa lunghezza del gruppo di origine e sono mappate direttamente ai rispettivi elementi. Se invece il risultato è uno scalare viene mappato a tutti i membri del gruppo. Sopra un esempio del primo caso, vediamo uno del secondo

  ```
  group_df.select(
      pl.all(),
      pl.col("data").mean().over("group").alias("group_mean")
  )
  ```

- `join`: aggrega il risultato del sottogruppo in una lista e la riporta per ciascun membro del sottogruppo

  ```
  group_df.select(
      pl.all(),
      pl.col("data").sort().over("group", mapping_strategy="join").alias("sorted_values")
  )
  ```

11.4.23 folds (elaborazioni orizzontali)

Servono per creare aggregazioni custom a livello di riga se le varie funzioni `sum_horizontal`, `mean_horizontal` etc non sono abbastanza. Si specificano con `pl.fold` che prende come parametro

- `acc`: valore iniziale (es in una somma tipicamente 0)
- `function`: funzione che prende in input *due* parametri da applicare, per riga, ad ogni elemento con il risultato delle applicazioni precedenti
- `exprs`: espressioni su cui accumulare (tipicamente selezione delle colonne con `pl.col`)

```
df = pl.DataFrame(
    {
        "label": ["foo", "bar", "spam"],
        "a": [1, 2, 3],
        "b": [10, 20, 30],
    }
)

import operator
df.select(
    # equivalente di pl.sum_horizontal(pl.col("a", "b"))
    pl.fold(
        acc=pl.lit(0),
        function=operator.add,
        exprs=pl.col("a", "b"),
    ).alias("sum_fold"),
)
```

Applicazioni su trasformazioni booleane del dataset permettono di filtrare: nel seguito

- si trasformano tutte le colonne in valori booleani mediante l'espressione
- si applica la funzione che pone in and i due valori, facendo si che si selezionino le righe dove tutti gli elementi siano maggiori di 1

```
df = pl.DataFrame(
    {
        "a": [1, 2, 3],
        "b": [0, 1, 2],
    }
)

df.filter(
    pl.fold(
        acc=pl.lit(True),
        function=lambda acc, x: acc & x,
        exprs=pl.all() > 1,
    )
)
```

11.5 Descrittive

```
>>> import numpy as np
>>> import polars as pl

>>> # dataset di prova
>>> rng = np.random.default_rng(123)
>>> df2 = pl.DataFrame({
...     'name': ['Luca', 'Silvio', 'Luisa', 'Andrea', 'Giovanni'],
...     'age' : [21, 22, 23, 24, 25],
```

Metodo	Descrizione
`count`	numero di dati non mancanti
`value_counts`	frequenze
`describe`	set di statistiche descrittive
`sum`, `product`	somma/prodotto dei valori
`cum_sum`, `cum_min`, `cum_max`	cumulate varie
`diff`, `pct_change`	differenze prime e variazione percentuale
`min`, `max`	minimo e massimo
`mean`	media
`median`	mediana
`quantile`	quantili
`var`, `std`	varianza/stddev campionaria
`skew`, `kurtosis`	skewness/kurtosis campionarie
`arg_min`, `arg_max`	posizione (intera) di minimo/massimo

Tabella 11.1: Metodi per analisi descrittiva

```
...     'income' : np.arange(5),
...     'group' : list("aaabb")
... })
>>> df3 = pl.DataFrame({"x": ["a", "a", "a", "a", "a", "b", "b"],
...                     "y": ["1", "2", "2", "2", "2", "1", "2"],
...                     "g": ["trt", "ctrl", "trt", "ctrl", "trt", "ctrl", "trt"]})

>>> categ_df = pl.DataFrame({"x": ["a", "a", "a", "a", None, "b", "b"],
...                          "y": ["1", "2", "1", "2", "2", "1", "2"]})
>>> group_df = pl.DataFrame({'key1' : list('xyyzzzwwww'),
...                          'key2' : ['one', 'two'] * 5,
...                          'data1' : rng.random(10),
...                          'data2' : rng.random(10)})

>>> strata_df = pl.DataFrame({"x": np.random.randn(7),
...                           "y": np.random.randn(7),
...                           "z": np.random.randn(7),
...                           "g": ["trt", "ctrl", "trt", "ctrl", "trt", "ctrl", "trt"]})
```

11.5.1 Univariate

Per le statistiche descrittive utilizzando `pl.Series`/`pl.DataFrame` consultare le funzioni riportate in tabella 11.1.

```
>>> # categoriche (frequenze)
>>> categ_df["x"].value_counts()
shape: (3, 2)
+------+-------+
| x    | count |
| ---  | ---   |
| str  | u32   |
+==============+
| null | 1     |
```

```
| b    | 2     |
| a    | 4     |
+------+-------+

>>> # quantitative (count = non missing)
>>> df2.describe()
shape: (9, 5)
+------------+--------+----------+----------+-------+
| statistic  | name   | age      | income   | group |
| ---        | ---    | ---      | ---      | ---   |
| str        | str    | f64      | f64      | str   |
+=================================================+
| count      | 5      | 5.0      | 5.0      | 5     |
| null_count | 0      | 0.0      | 0.0      | 0     |
| mean       | null   | 23.0     | 2.0      | null  |
| std        | null   | 1.581139 | 1.581139 | null  |
| min        | Andrea | 21.0     | 0.0      | a     |
| 25%        | null   | 22.0     | 1.0      | null  |
| 50%        | null   | 23.0     | 2.0      | null  |
| 75%        | null   | 24.0     | 3.0      | null  |
| max        | Silvio | 25.0     | 4.0      | b     |
+------------+--------+----------+----------+-------+
```

11.5.2 Bivariate

11.5.2.1 Tabelle di contingenza

Sono una speciale tipo di tabella pivot dove si applica il conteggio degli elementi in ciascun gruppo

```
>>> (
...   categ_df
...   .pivot(on='x',index='y',values='x',aggregate_function='len',sort_columns=True)
...   .fill_null(0).sort('y')
... )
shape: (2, 4)
+-----+------+-----+-----+
| y   | null | a   | b   |
| --- | ---  | --- | --- |
| str | u32  | u32 | u32 |
+=========================+
| 1   | 0    | 2   | 1   |
| 2   | 1    | 2   | 1   |
+-----+------+-----+-----+
```

11.5.2.2 Covarianza e correlazione

I metodi `cov` e `corr` permettono di ottenere la covarianza e correlazione per gli elementi di un dataframe.

```
>>> df1 = strata_df[["x", "y", "z"]]
>>> df1.corr()
```

```
shape: (3, 3)
+-----------+----------+-----------+
| x         | y        | z         |
| ---       | ---      | ---       |
| f64       | f64      | f64       |
+=================================+
| 1.0       | 0.097565 | -0.037844 |
| 0.097565  | 1.0      | 0.391141  |
| -0.037844 | 0.391141 | 1.0       |
+-----------+----------+-----------+
```

11.5.2.3 Tabelle pivot

Sono tabelle bivariate con statistiche stratificate per gruppi formati da righe e colonne (per intenderci il `tapply` con doppio indice di R).
Possono essere prodotte col metodo `pivot_table` dei `DataFrame` ed eventuali funzioni custom debbono fare utilizzo di `pl.element`

```
>>> group_df
shape: (10, 4)
+------+------+----------+----------+
| key1 | key2 | data1    | data2    |
| ---  | ---  | ---      | ---      |
| str  | str  | f64      | f64      |
+====================================+
| x    | one  | 0.682352 | 0.51297  |
| y    | two  | 0.053821 | 0.244965 |
| y    | one  | 0.22036  | 0.824242 |
| z    | two  | 0.184372 | 0.213763 |
| z    | one  | 0.175906 | 0.741467 |
| z    | two  | 0.812095 | 0.62994  |
| w    | one  | 0.923345 | 0.927407 |
| w    | two  | 0.276574 | 0.231908 |
| w    | one  | 0.819755 | 0.799125 |
| w    | two  | 0.889893 | 0.518165 |
+------+------+----------+----------+
>>> group_df.pivot(on =   "key1",      # cosa porre in colonna
...                index = "key2",      # cosa porre in riga
...                values =  "data1",     # dati per sintesi
...                aggregate_function = "median")
shape: (2, 5)
+------+----------+----------+----------+----------+
| key2 | x        | y        | z        | w        |
| ---  | ---      | ---      | ---      | ---      |
| str  | f64      | f64      | f64      | f64      |
+================================================+
| one  | 0.682352 | 0.22036  | 0.175906 | 0.87155  |
| two  | null     | 0.053821 | 0.498233 | 0.583234 |
+------+----------+----------+----------+----------+

>>> # utilizzo di funzioni custom: min per controllo
```

```
>>> group_df.pivot(on = "key1",      # cosa porre in riga
...                index = "key2",      # cosa porre in colonna
...                values =  "data1",     # dati per sintesi
...                aggregate_function = "min")
shape: (2, 5)
+------+----------+----------+----------+----------+
| key2 | x        | y        | z        | w        |
| ---  | ---      | ---      | ---      | ---      |
| str  | f64      | f64      | f64      | f64      |
+=================================================+
| one  | 0.682352 | 0.22036  | 0.175906 | 0.819755 |
| two  | null     | 0.053821 | 0.184372 | 0.276574 |
+------+----------+----------+----------+----------+
>>> # replicato con pl.element
>>> group_df.pivot(on = "key1",      # cosa porre in riga
...                index = "key2",      # cosa porre in colonna
...                values =  "data1",     # dati per sintesi
...                aggregate_function = pl.element().min())
shape: (2, 5)
+------+----------+----------+----------+----------+
| key2 | x        | y        | z        | w        |
| ---  | ---      | ---      | ---      | ---      |
| str  | f64      | f64      | f64      | f64      |
+=================================================+
| one  | 0.682352 | 0.22036  | 0.175906 | 0.819755 |
| two  | null     | 0.053821 | 0.184372 | 0.276574 |
+------+----------+----------+----------+----------+
```

11.5.2.4 Stratificate

Si applica split-apply-combine ossia:

1. si splitta il `DataFrame` mediante il metodo `groupby`: a questo si può passare alternativamente stringhe (interpretate come nomi variabili) o espressioni. Questo ritorna un oggetto `GroupBy` è un iterabile sui "chunk" di dati (quindi funziona bene con i `for`) e fornisce un set di metodi già pronti da applicare;

2. si effettua l'elaborazione sui chunk di dati, alternativamente

   - l'oggetto `GroupBy` ha un set di metodi builtin già disponibili da applicare a tutti i chunk di dati.
   - si possono applicare definire funzioni custom (mediante `.agg`)
   - si possono applicare funzioni a tutto il sotto-dataframe e ritornare i risultati bindati (mediante `map_groups`)

3. viene riassemblato il tutto: il tipo di oggetto ritornato dipende dalle elaborazioni eseguite.

Splitting Vediamo vari metodi di splitting

```
>>> group_df
shape: (10, 4)
+------+------+----------+----------+
| key1 | key2 | data1    | data2    |
| ---  | ---  | ---      | ---      |
| str  | str  | f64      | f64      |
+====================================+
| x    | one  | 0.682352 | 0.51297  |
| y    | two  | 0.053821 | 0.244965 |
| y    | one  | 0.22036  | 0.824242 |
| z    | two  | 0.184372 | 0.213763 |
| z    | one  | 0.175906 | 0.741467 |
| z    | two  | 0.812095 | 0.62994  |
| w    | one  | 0.923345 | 0.927407 |
| w    | two  | 0.276574 | 0.231908 |
| w    | one  | 0.819755 | 0.799125 |
| w    | two  | 0.889893 | 0.518165 |
+------+------+----------+----------+

>>> # Singola variabile
>>> df_spl = group_df.group_by('key2')
>>> df_spl.mean() # se no essendoci key2 da errore
shape: (2, 4)
+------+------+----------+----------+
| key2 | key1 | data1    | data2    |
| ---  | ---  | ---      | ---      |
| str  | str  | f64      | f64      |
+====================================+
| two  | null | 0.443351 | 0.367748 |
| one  | null | 0.564343 | 0.761042 |
+------+------+----------+----------+

>>> # Due variabili
>>> df_spl = group_df.group_by(['key1', 'key2'])
>>> df_spl.mean()
shape: (7, 4)
+------+------+----------+----------+
| key1 | key2 | data1    | data2    |
| ---  | ---  | ---      | ---      |
| str  | str  | f64      | f64      |
+====================================+
| z    | one  | 0.175906 | 0.741467 |
| z    | two  | 0.498233 | 0.421852 |
| w    | one  | 0.87155  | 0.863266 |
| x    | one  | 0.682352 | 0.51297  |
| y    | two  | 0.053821 | 0.244965 |
| w    | two  | 0.583234 | 0.375037 |
| y    | one  | 0.22036  | 0.824242 |
+------+------+----------+----------+

>>> # Espressione
```

```
>>> df_spl = group_df.group_by(
...     'key1',
...     (pl.col("data1") > 0.5).alias("data1_geq_05")
... )
>>> df_spl.mean()
shape: (6, 5)
+------+--------------+------+----------+----------+
| key1 | data1_geq_05 | key2 | data1    | data2    |
| ---  | ---          | ---  | ---      | ---      |
| str  | bool         | str  | f64      | f64      |
+=================================================+
| z    | false        | null | 0.180139 | 0.477615 |
| w    | false        | null | 0.276574 | 0.231908 |
| w    | true         | null | 0.877664 | 0.748232 |
| z    | true         | null | 0.812095 | 0.62994  |
| x    | true         | null | 0.682352 | 0.51297  |
| y    | false        | null | 0.13709  | 0.534603 |
+------+--------------+------+----------+----------+
```

Se si vuole che nello splitting non venga modificato l'ordinamento dei dati specificare `maintain_order=True`.
Oltre a funzioni di aggregazione ovvie come `max`, `mean`, `median`, `min`, `n_unique`, `quantile`, `sum`, `len` (contano frequenze/lunghezza dei sottogruppi tipo table) altre funzioni di interesse

- `all` crea una series dei dati in ogni gruppo
- `first`/`last`, `head`/`tail` selezionano porzioni specifiche (per queste dare `maintain_order=True` in groupby può essere voluto)
- `agg`: effettua aggregazioni custom (funzioni/variabili)
- `map_groups`: applica funzioni custom definite dall'utente (UDF)
- `having`: seleziona i gruppi che presentano date caratteristiche post splitting

Applicare funzioni di aggregazione custom `.agg` È possibile applicare funzioni che sintetizzano series dati in un unico valore passandole al metodo `agg` di un oggetto `GroupBy`:

```
>>> # a custom function taking a series and returning a number
>>> def myrange(x):
...     return x.max() - x.min()
...

>>> # a custom aggregation
>>> group_df.group_by('key2').agg(
...     # di default non aggrega nulla ma fa la lista degli elementi
...     pl.col("data1"),
...     # polars style, do elaboration and add alias at the end
...     pl.col("data1").min().alias("data1_min"),
```

```
...     pl.col("data1").max().alias("data1_max"),
...     # abbreviabili mediante
...     pl.min("data1").name.suffix("_min2"),
...     # funzione custom di un parametro (e kwargs specification)
...     data1_range = myrange(pl.col("data1")),
...     # funzione custom di due parametro (e kwargs specification)
... )
shape: (2, 6)
+------+----------------------------------+-----------+-----------+------------+-------------+
| key2 | data1                            | data1_min | data1_max | data1_min2 | data1_range |
| ---  | ---                              | ---       | ---       | ---        | ---         |
| str  | list[f64]                        | f64       | f64       | f64        | f64         |
+=============================================================================================+
| two  | [0.053821, 0.184372, ... 0.889... | 0.053821  | 0.889893  | 0.053821   | 0.836072    |
| one  | [0.682352, 0.22036, ... 0.8197... | 0.175906  | 0.923345  | 0.175906   | 0.747439    |
+------+----------------------------------+-----------+-----------+------------+-------------+
```

Applicare funzioni all'intero dataframe/subset Si fa mediante `map_groups` che prende in input una funzione:

- la funzione prende in input e restituisce un dataframe
- la funzione viene applicata a tutti i dataframe
- il risultato viene bindato di riga

È una soluzione più lenta delle espressioni polars (si esce dal rust e si passa dal python) e da utilizzare solo se non si riesce ad esprimere con queste

```
>>> # Per ogni gruppo di colore, campiona due righe
>>> df = pl.DataFrame({
...         "id": [0, 1, 2, 3, 4],
...         "color": ["red", "green", "green", "red", "red"],
...         "shape": ["square", "triangle", "square", "triangle", "square"],
... })
>>> df.group_by("color").map_groups(
...     lambda group_df: group_df.sample(2)
... )
shape: (4, 3)
+-----+-------+----------+
| id  | color | shape    |
| --- | ---   | ---      |
| i64 | str   | str      |
+========================+
| 1   | green | triangle |
| 2   | green | square   |
| 0   | red   | square   |
| 4   | red   | square   |
+-----+-------+----------+

>>> # alternativamente mediante espressioni polars sarebbe stato
>>> # df.filter(
```

```
>>> #     pl.int_range(pl.len()).shuffle().over("color") < 2
>>> # )
```

Filtrare gruppi Si fa mediante `having` che è equivalente a calcolare il criterio di filtro e filtrare le righe in seguito

```
>>> df = pl.DataFrame({
...     "a": ["a", "b", "a", "b", "c"],
... })
>>> df.group_by("a").having(pl.len() > 1).agg()
shape: (2, 1)
+-----+
| a   |
| --- |
| str |
+=====+
| b   |
| a   |
+-----+
```

Iterare sui gruppi con `for` Un oggetto `GroupBy` è iterabile quindi funziona bene con i `for`: questo genera una sequenza di tuple di due elementi, contenenti nome del gruppo (chiave) e chunk di dati di dati il pezzo di dati

```
>>> # chiave singola
>>> for (group, ), data in group_df.group_by('key2'):
...   print(f"group = {group}")
...   print(data, "\n") # simple DataFrame
...
group = two
shape: (5, 4)
+------+------+----------+----------+
| key1 | key2 | data1    | data2    |
| ---  | ---  | ---      | ---      |
| str  | str  | f64      | f64      |
+==================================+
| y    | two  | 0.053821 | 0.244965 |
| z    | two  | 0.184372 | 0.213763 |
| z    | two  | 0.812095 | 0.62994  |
| w    | two  | 0.276574 | 0.231908 |
| w    | two  | 0.889893 | 0.518165 |
+------+------+----------+----------+

group = one
shape: (5, 4)
+------+------+----------+----------+
| key1 | key2 | data1    | data2    |
| ---  | ---  | ---      | ---      |
| str  | str  | f64      | f64      |
+==================================+
| x    | one  | 0.682352 | 0.51297  |
```

```
| y    | one  | 0.22036  | 0.824242 |
| z    | one  | 0.175906 | 0.741467 |
| w    | one  | 0.923345 | 0.927407 |
| w    | one  | 0.819755 | 0.799125 |
+------+------+----------+----------+
```

```
>>> # chiave multipla: il primo elemento della tupla è una
>>> # tupla con le chiavi
>>> for (key1, key2), data in group_df.group_by(['key1', 'key2']):
...   print(f"key1 = {key1}, key2 = {key2}")
...   print(data, "\n")
...
key1 = y, key2 = two
shape: (1, 4)
+------+------+----------+----------+
| key1 | key2 | data1    | data2    |
| ---  | ---  | ---      | ---      |
| str  | str  | f64      | f64      |
+==================================+
| y    | two  | 0.053821 | 0.244965 |
+------+------+----------+----------+

key1 = x, key2 = one
shape: (1, 4)
+------+------+----------+---------+
| key1 | key2 | data1    | data2   |
| ---  | ---  | ---      | ---     |
| str  | str  | f64      | f64     |
+=================================+
| x    | one  | 0.682352 | 0.51297 |
+------+------+----------+---------+

key1 = w, key2 = two
shape: (2, 4)
+------+------+----------+----------+
| key1 | key2 | data1    | data2    |
| ---  | ---  | ---      | ---      |
| str  | str  | f64      | f64      |
+==================================+
| w    | two  | 0.276574 | 0.231908 |
| w    | two  | 0.889893 | 0.518165 |
+------+------+----------+----------+

key1 = z, key2 = two
shape: (2, 4)
+------+------+----------+----------+
| key1 | key2 | data1    | data2    |
| ---  | ---  | ---      | ---      |
| str  | str  | f64      | f64      |
+==================================+
| z    | two  | 0.184372 | 0.213763 |
```

```
| z    | two  | 0.812095 | 0.62994  |
+------+------+----------+----------+

key1 = z, key2 = one
shape: (1, 4)
+------+------+----------+----------+
| key1 | key2 | data1    | data2    |
| ---  | ---  | ---      | ---      |
| str  | str  | f64      | f64      |
+==================================+
| z    | one  | 0.175906 | 0.741467 |
+------+------+----------+----------+

key1 = y, key2 = one
shape: (1, 4)
+------+------+---------+----------+
| key1 | key2 | data1   | data2    |
| ---  | ---  | ---     | ---      |
| str  | str  | f64     | f64      |
+==================================+
| y    | one  | 0.22036 | 0.824242 |
+------+------+---------+----------+

key1 = w, key2 = one
shape: (2, 4)
+------+------+----------+----------+
| key1 | key2 | data1    | data2    |
| ---  | ---  | ---      | ---      |
| str  | str  | f64      | f64      |
+==================================+
| w    | one  | 0.923345 | 0.927407 |
| w    | one  | 0.819755 | 0.799125 |
+------+------+----------+----------+
```

Memorizzare in un dict lo split Essendo il `GroupBy` un iterabile, è possibile elaborarlo, ad esempio trasformandolo in un `dict`. Di default l'indice viene restituito come tupla di un elemento, che selezioniamo per più facile accesso

```
>>> pieces = {k[0]: v for k,v in group_df.group_by('key2')}
>>> pieces['two']
shape: (5, 4)
+------+------+----------+----------+
| key1 | key2 | data1    | data2    |
| ---  | ---  | ---      | ---      |
| str  | str  | f64      | f64      |
+==================================+
| y    | two  | 0.053821 | 0.244965 |
| z    | two  | 0.184372 | 0.213763 |
| z    | two  | 0.812095 | 0.62994  |
| w    | two  | 0.276574 | 0.231908 |
| w    | two  | 0.889893 | 0.518165 |
```

+------+------+----------+----------+

Capitolo 12

Matplotlib

Contents

12.1 Introduzione 111
12.2 Salvataggio figura 111
12.3 Impostazione layout figura ed esempi 113
12.3.1 Layout standard 113
12.3.2 Layout custom 113
12.4 Fine tuning 114
12.4.1 Ticks e subticks 114
12.4.2 Spines e grid 114
12.4.3 Gestire la sovrapposizione di elementi diversi (`zorder`) 115
12.4.4 Legenda 115
12.4.5 Plot con doppio asse delle y 115
12.4.6 Padding dei subplots (spazio bianco bordi) 115
12.5 Configurazioni 116
12.5.1 Ottenimento e modifica 116
12.5.2 Ripristino impostazioni default 116
12.5.3 Cambiare stile 116
12.6 Grafici utili 116
12.6.1 Linee 117
12.6.2 Diagramma a barre 117
12.6.3 Istogramma 117
12.6.4 Scatterplot 117
12.6.5 Matrice di scatterplot 117
12.6.6 Boxplot 117
12.6.7 Correlation matrix 117


```
import numpy as np
import pylbmisc as lb
import rdatasets
rng = np.random.default_rng(123)

# template importazione matplotlib
```

```
import matplotlib.pyplot as plt

import polars as pl
pl.__version__
pl.Config.set_ascii_tables(True) # altrimenti la stampa di df da problemi
```

12.1 Introduzione

Alcuni punti di base:

- vi sono due stili di plotting in `matplotlib`: uno classico state-based che imita matlab e uno object oriented. Preferiamo questo secondo.
- una sequenza template per creare gli oggetti necessari sino a salvare la mostrare e salvare la ha il seguente template

  ```
  fig, ax = plt.subplots() # setup fig and axis
  # ...                    # do the actual plotting
  plt.show()               # show the plot
  fig.savefig("/tmp/first_plot.png", dpi=600) # save as png
  ```

- concetti fondanti:
  - una *figure* è il container di una immagine;
  - una figure può contenere al suo interno uno o più *axes*: questi sono gli effettivi plot dell'immagine

    ```
    # Figura 1 x 1: 1 figure e 1 axes
    fig, ax = plt.subplots()

    # Figura 2 x 3: 1 figure e 6 axes
    # ax è un array:(es ax[0,2] è ultimo grafico della prima riga)
    fig, ax = plt.subplots(2, 3)
    ```

    Il maggior lavoro si farà con i metodi messi a disposizione da `ax`/`axes`; alcuni metodi utili sono riportati in tabella 12.1, mentre l'anatomia di un grafico matplotlib in figura 12.1.
  - gli *axis* invece sono i veri e propri assi di ciascun plot/axes

12.2 Salvataggio figura

Per il salvataggio figura, in `fig.savefig`:

- specificare i `dpi` per i png
- se `transparent=True` la figura sarà transparent e meglio si adatta all'inclusione in documenti con sfondo di diverso colore
- se si passa `bbox_inches='tight'` si shinka la fig alle dimensioni del grafico interno, eliminando spazio bianco attorno ad esso ad eccezione di un piccolo ammontare di padding (aggiustabile con `pad_inches`)

Nel seguito si usa `lb.io.export_figure` per comodità (wrapper di `savefig`).

Metodo	Descrizione
`plot`	linee/scatterplot
`hist`	istogramma
`bar`	diagramma a barre
`scatter`	scatterplot più flessibile/lento
`errorbar`	intervalli di confidenza
`set`	imposta tutti i seguenti in una unica chiamata
`set_title`	imposta il titolo
`set_xlabel`, `set_xlabel`	imposta il titolo degli assi
`set_xticks`, `set_yticks`	imposta la posizione dei ticks sugli assi
`set_xticklabels`, `set_yticklabels`	imposta l'etichetta dei ticks
`set_xlim`, `set_ylim`	impostare i limiti degli assi/zoom figura
`legend`	aggiunta di legenda
`text`	aggiunta di testo
`spines`	config bordi area di plot
`grid`	config griglia area di plot

Tabella 12.1: Metodi utili di `ax`

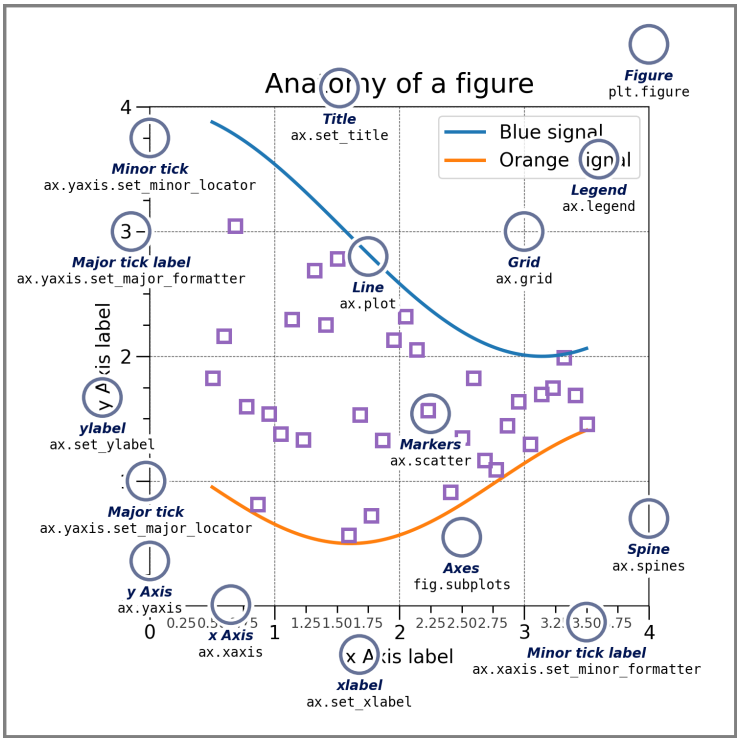


Figura 12.1: Anatomia di una figura

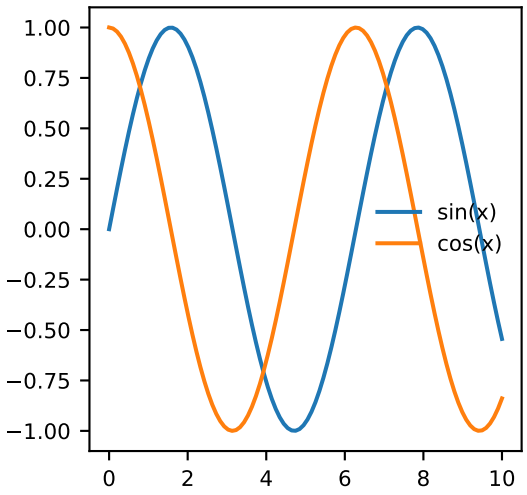


Figura 12.2: First plot

12.3 Impostazione layout figura ed esempi

12.3.1 Layout standard

Grafico singolo Esempio minimale figura **??**:

- si aggiungono funzioni plottate con chiamate successive allo stesso `ax`
- ogni funzione ha una `label` utilizzata per la `legend`

```
fig, ax = plt.subplots(figsize=(3,3))
x = np.linspace(0, 10, 100)
ax.plot(x, np.sin(x), label = 'sin(x)')
ax.plot(x, np.cos(x), label = 'cos(x)')
ax.legend()
lb.io.export_figure(fig, label = 'first_plot')
```

Vettore di grafici Un esempio con layout grafici separati, in figura **??**

- aumentiamo la dimensione di figsize
- ci riferiamo agli `ax` come ad un vettore
- interfacciamo il codice matplotlib con quello di pandas i *wrapper di pandas* in due modi
  1. prendendo in maniera grezza in indici (sull'asse delle x) e valori (sull'asse delle y) dalla serie
  2. utilizzando il metodo della serie

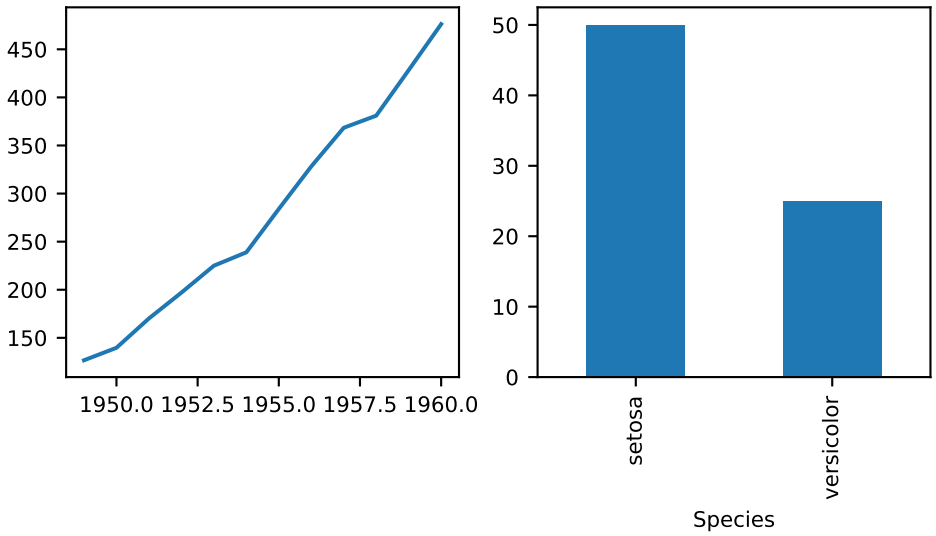


Figura 12.3: Second plot

```
fig2, ax2 = plt.subplots(nrows=1, ncols=2, figsize=(6, 2.5))

# raw matplotlib style using index and values
psng = rdatasets.data("AirPassengers").rename(columns={"value":"number"})
psng["year"] = np.floor(psng.time)
# how to select several columns: use [[]]
monthly_mean = psng.groupby("year")[["number"]].mean()
ax2[0].plot(monthly_mean["number"].index, monthly_mean["number"].values)
# otherwise quicker single column (single [])
# monthly_mean = psng.groupby("year")["number"].mean()
# ax2[0].plot(monthly_mean.index, monthly_mean.values)

# pandas methods using iris: frequency of first 75 flower types
iris = rdatasets.data("iris")
freqs = iris.iloc[:75, 5].value_counts()
freqs.plot.bar(ax = ax2[1])

lb.io.export_figure(fig2, label = 'second_plot')
```

Matrice di grafici Matrice 2×2 in cui plottiamo i subset di un grafico

- utilizziamo `constrained_layout` per aggiustare le dimensioni dei plot affinché fittino la figura (buona idea quando si hanno più plot in una singola figura) e gli axis di un plot non vadano sopra il titolo di un altro (alternativamente possiamo fare le cose a mano mediante `fig.subplot_adjust`);
- usiamo `flatten` di `numpy` per iterare sequenzialmente sugli ax
- evitiamo che l'ultimo (inutile) axes venga mostrato impostando `axis` ad `off`

- impostiamo che i range di x e y siano comuni per
- aggiungiamo un titolo complessivo di figura mediante `fig.suptitle`

```
iris = rdatasets.data("iris")

fig3, ax3 = plt.subplots(nrows=2, ncols=2, figsize = (5,5), constrained_layout=True)
flat_axes = ax3.flatten()

for ax, species in zip(flat_axes, ["setosa", "versicolor", "virginica"]):
    subs = iris.query(f"Species=='{species}'") # create subset
    ax.scatter(x=subs["Sepal.Length"], y=subs["Sepal.Width"], alpha = 0.5)
    ax.set_xlim((3, 9))
    ax.set_ylim((1, 5))
    ax.set_title(f"{species}") # single graph title
    ax.set_xlabel("length (cm)")
    ax.set_ylabel("width (cm)")

ax3[1, 1].axis("off") # non visualizzare il quarto grafico
fig3.suptitle("Sepal length and width scatterplot", fontsize=15)
lb.io.export_figure(fig3, label = 'third_plot')
```

Il risultato è in figura **??**

12.3.2 Layout custom

Griglia di axes di dimensione variabile L'uso di `plt.subplots` è comodo ma impone che tutti gli axes abbiano dimensioni comuni. Per customizzare

- si crea una figura mediante `plt.figure`
- si determina una griglia con le proporzioni degli axes inclusi nella figura mediante `plt.GridSpec`
- si creano i singoli axes mediante `add_subplot` fornendo la griglia (con slicing) di riferimento
- per un altro esempio vedere effective visualization a pagina 59
- con `plt.subplot_mosaic` si ottiene qualcosa di simile (effective visualization a pagina 67)

Sotto il codice per figura **??**;

```
fig  = plt.figure(figsize=(5,5))
grid = plt.GridSpec(nrows=2, ncols=3, figure=fig, wspace=0.4, hspace=0.3)

ax1 = fig.add_subplot(grid[0, 0])
ax1.boxplot([rng.standard_normal(50),rng.standard_normal(50) + 1])

ax2 = fig.add_subplot(grid[0, 1:])
ax2.bar(x = ["A", "B", "C", "D", "E", "F"], height = [10, 20, 40, 5, 25, 20])

ax3 = fig.add_subplot(grid[1, :2])
```

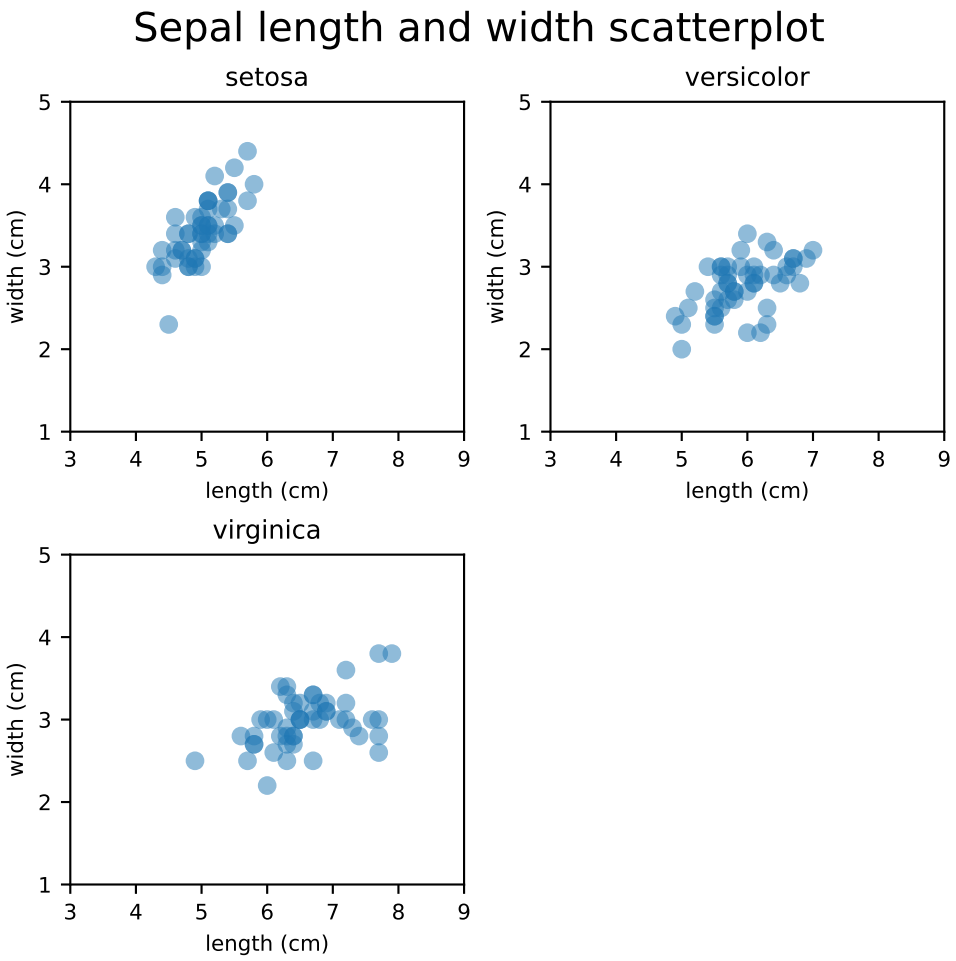


Figura 12.4: Third plot

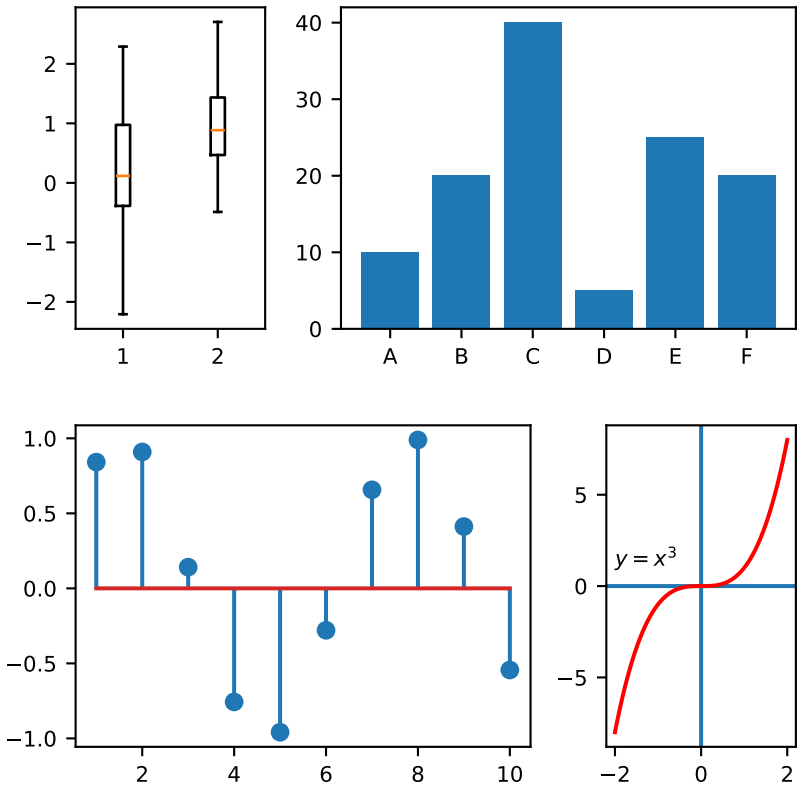


Figura 12.5: Custom grid

```
x = np.arange(1, 11)
ax3.stem(x, np.sin(x))

ax4 = fig.add_subplot(grid[1, 2])
x = np.linspace(start=-2, stop=2, num=100)
ax4.axhline(0) # riferimento
ax4.axvline(0) # cartesiano
ax4.plot(x, x**3, color="red")
ax4.text(-2, 1, '$y = x^3$')

lb.io.export_figure(fig, label = 'custom_grid')
```

Axes uno dentro all'altro Il setup di sopra pone subplot affiancati e riempie tutta la figura; maggiore controllo sul setup dei subplot può essere ottenuto con `add_axes`. Questa prende come input una lista di quattro numeri che specificano le coordinate `[left, bottom, width, height]` nel range da 0 (in basso a sinistra della figura) a 1 in alto a destra (figura **??**).

```
fig = plt.figure(figsize = (3,3))
ax1 = fig.add_axes([0,       0,    1,    1]) # assi standard
```

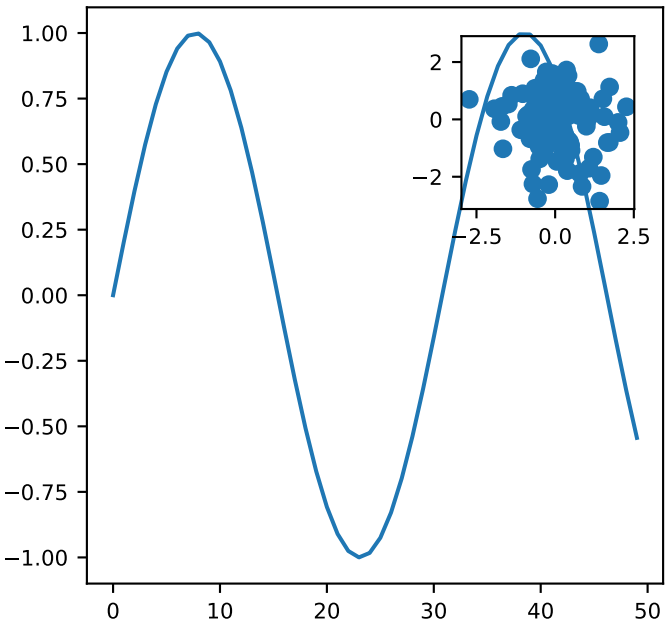


Figura 12.6: Custom subplots

```
ax2 = fig.add_axes([0.65, 0.65, 0.3, 0.3]) # assino piccolino a 0.65, 0.65
                                           # della figura ed estensione 0.3x0.3
ax1.plot(np.sin(np.linspace(0, 10)))
ax2.scatter(rng.standard_normal(100), rng.standard_normal(100))
lb.io.export_figure(fig, label = 'custom_subplots')
```

Per zoom di plot vedere anche effective visualization a pag 94

12.4 Fine tuning

12.4.1 Ticks e subticks

Qualora le configurazioni di default non vadano bene

- per impostare la *locazione*
  - per una locazione standard usare `matplotlib.ticker.MultipleLocator` nella chiamata a `ax.xaxis.set_major_locator` (ticks main) o `ax.xaxis.set_minor_locator` (ticks minor se necessari)
  - per una custom/non uniforme utilizzare `ax.set_xticks` (`ax.set_yticks`)
  - per date e time series usare locatori come `YearLocator` o `MonthLocator` cfr effective visualization pag 97)
- per scegliere il *valore mostrato* `ax.set_xticklabels` (`ax.set_yticklabels`)
- per la dimensione ticks usare `ax.tick_params`

- per aggiustare il font dell'asse ottenere ciascuna label con `ax.get_xticklabels` (`ax.get_yticklabels`) dopodiché impostare dimensione/peso/famiglia con le funzioni `set_fontsize set_weight set_family`
- se invece si vuole solo **nascondere i ticks** (ma non le label) nella chiamata `ax.tick_params` impostare `left=False` o `bottom=False`

Ad esempio in **??**

```
fig, ax = plt.subplots()
x = rng.standard_normal(50)
y = rng.standard_normal(50)
ax.scatter(x, y)
ax.set_xlim((-3, 3))
ax.set_ylim((-3, 3))

# locazione dei ticks
from matplotlib.ticker import MultipleLocator
ax.xaxis.set_major_locator(MultipleLocator(1))
ax.xaxis.set_minor_locator(MultipleLocator(0.5))
ax.yaxis.set_major_locator(MultipleLocator(1))
ax.yaxis.set_minor_locator(MultipleLocator(0.5))

# dimensione ticks
ax.tick_params(axis='x', which='major', length=10, width=2, color="red")
ax.tick_params(axis='x', which='minor', length=5,  width=1, color="orange")
ax.tick_params(axis='y', which='major', length=10, width=2, color="blue")
ax.tick_params(axis='y', which='minor', length=5,  width=1, color="cyan")

# font dell'asse
for label in ax.get_xticklabels() + ax.get_yticklabels():
    label.set_fontfamily("Roboto")
    label.set_fontsize(6)

lb.io.export_figure(fig, label = 'ticks')
```

12.4.2 Spines e grid

Le *spines* sono gli assi che racchiudono l'immagine, in due dei quali sono posizionati gli assi (a sx e sotto); possiamo gestire il formato dei 4 spines indipendentemente. Le *grid* invece sono le griglie dell'area di plotting. Un esempio di entrambe in **??** ottenuta col codice di sotto

```
fig, ax = plt.subplots()
ax.scatter(rng.standard_normal(100), rng.standard_normal(100))

# tolgo spine sopra e a dx
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

#  imposto che le grid siano sotto ai punti e inizio ad aggiungere le major
```

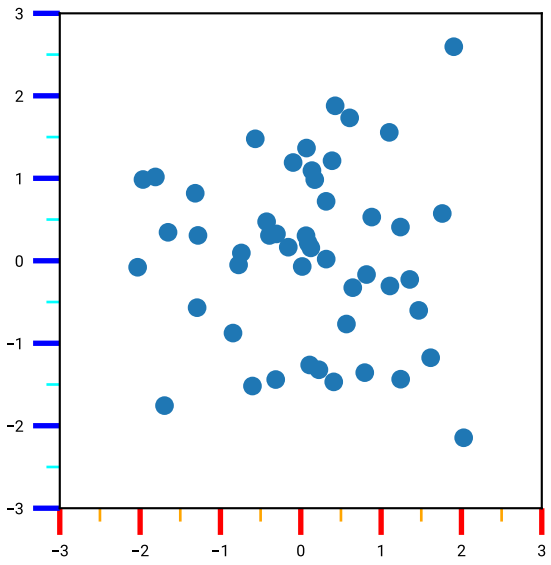


Figura 12.7: Ticks

```
ax.set_axisbelow(True)
ax.grid(which='major', linestyle = '--', color='grey', linewidth=0.5, alpha=0.5)

# aggiungo ticks minor e grid minor (ogni 0.5 sia per x che y)
ax.xaxis.set_minor_locator(MultipleLocator(0.5))
ax.yaxis.set_minor_locator(MultipleLocator(0.5))
ax.grid(which='minor', linestyle = '--', color='grey', linewidth=0.1, alpha=0.5)

lb.io.export_figure(fig, label = 'spines_grid')
```

12.4.3 Gestire la sovrapposizione di elementi diversi (`zorder`)

Il parametro `zorder` (disponibile in diversi metodi di ax come `scatter`, `plot`, etc) controlla lo stacking degli elementi del plot sull'asse z determinando quali elementi appaiano più in evidenza e quali possono essere mascherati. La regola è che valori più alti sono associati ad un piano più alto/maggiore visibilità. Un esempio con grid, nuvola di punti e retta aventi rispettivamente zorder crescente (quindi la più visibile è la retta) in figura **??**.

```
fig, ax = plt.subplots()
x = rng.standard_normal(100)
y = rng.standard_normal(100) + 1
ax.scatter(x, y,
           zorder = 2)
ax.grid(which='major', linestyle = '--', color='grey', linewidth=1, alpha=0.5,
        zorder=1)
ax.plot([-2, 2], [-1, 3], color="red", lw=1,# retta da [0, 0] a [1,1]
        zorder=3)
```

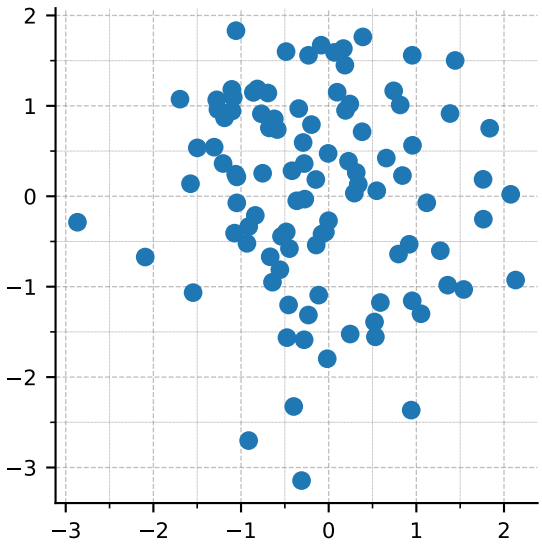


Figura 12.8: Spines grid

```
lb.io.export_figure(fig, label = 'zorder')
```

12.4.4 Legenda

Il metodo `ax.legend` genera labels dal parametro `label` specificato nel comando di plot, ma si può fornire label custom nell'argomento `labels`. Per il posizionamento

- `loc` specifica il posizionamento tipo `upper right`, ..., `lower left`; vi è anche la possibilità di specificare `best` per il migliore (interpolato)
- `bbox_to_anchor` ottimizza la posizione in relazione agli axes; es per piazzare la legenda a destra, al di fuori dell'area di plot settare `bbox_to_anchor=(1,1)`

Alcuni esempi di entrambe a seguire di legende a livello di singolo ax; per legenda a livello di figura multipla (con molteplici grafici) da fare. Maggiori info su

- `https://matplotlib.org/stable/api/_as_gen/matplotlib.axes.Axes.legend.html`
- `https://matplotlib.org/stable/users/explain/axes/legend_guide.html`

```
# ----------------------------
# loc - posizionamento dentro ax
# ----------------------------

# opzioni: "best", 'upper right', 'upper left', 'lower left', 'lower right'
```

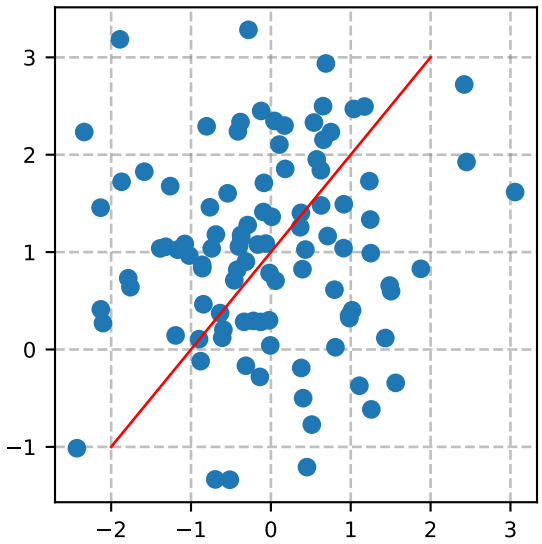


Figura 12.9: Zorder

```
# 'right' 'center left' 'center right' 'lower center' 'upper center' 'center'
# posizionamento ottimale per non overlappare (il default)

fig, ax = plt.subplots()
ax.plot([1, 2, 3], label="test1")
ax.plot([3, 2, 1], label="test2")
ax.legend(loc="best") # best è il default btw
lb.io.export_figure(fig, label="loc1")

# metti in alto a dx
fig, ax = plt.subplots()
ax.plot([1, 2, 3], label="test1")
ax.plot([3, 2, 1], label="test2")
ax.legend(loc="upper right")
lb.io.export_figure(fig, label="loc2")

# posizionamento con coordinate (percentuali di grafico x e y)
# es posiziona in mezzo per x (0.5) e a 1/4 per y (0.25)
# occhio che quello è considerato l'angolo in basso a sinistra della legenda
fig, ax = plt.subplots()
ax.plot([1, 2, 3], label="test1")
ax.plot([3, 2, 1], label="test2")
ax.legend(loc=(0.5, 0.25))
lb.io.export_figure(fig, label="loc3")

# ----------------------
# uso di bbox_to_anchor
# ----------------------
```

```
# legenda fuori dal plot in alto a dx (100% di asse x e y)
fig, ax = plt.subplots()
ax.plot([1, 2, 3], label="test1")
ax.plot([3, 2, 1], label="test2")
ax.legend(bbox_to_anchor=(1, 1), loc="upper left")
lb.io.export_figure(fig, label="testbboxtoanchor1")

# legenda fuori dal plot impostare il suo centro/sinistra della legenda al 100%
# dell'asse x e 50% dell'asse y
fig, ax = plt.subplots()
ax.plot([1, 2, 3], label="test1")
ax.plot([3, 2, 1], label="test2")
ax.legend(bbox_to_anchor=(1, 0.5), loc="center left")
lb.io.export_figure(fig, label="testbboxtoanchor2")

# legenda sopra il plot da sx:occorre indicare
# - quattro numeri, che vanno a due a due, e servono per circoscrivere il
#   posizionamento della legenda
# - il numero di colonne in cui splittare le etichette mediante ncols
fig, ax = plt.subplots()
ax.plot([1, 2, 3], label="test1")
ax.plot([3, 2, 1], label="test2")
ax.legend(bbox_to_anchor=(0, 1, 1, 1), loc="lower left", ncols=2)
lb.io.export_figure(fig, label="testbboxtoanchor3")

# legenda sopra il plot da dx
fig, ax = plt.subplots()
ax.plot([1, 2, 3], label="test1")
ax.plot([3, 2, 1], label="test2")
ax.legend(bbox_to_anchor=(0, 1, 1, 1), loc="lower right", ncols=2)
lb.io.export_figure(fig, label="testbboxtoanchor4")

# espandere la legenda per matchare i
fig, ax = plt.subplots()
ax.plot([1, 2, 3], label="test1")
ax.plot([3, 2, 1], label="test2")
ax.legend(bbox_to_anchor=(0, 1, 1, 1), loc="lower right", ncols=2, mode="expand")
lb.io.export_figure(fig, label="testbboxtoanchor5")
```

12.4.5 Plot con doppio asse delle y

Un esempio potrebbe esser il plot della relazione di due variabili (mostrate su y) con una terza comune (su x), mostrato in fig **??** (oppure l'andamento di serie storiche aventi ordini di grandezza differenti).

```
df = rdatasets.data("airquality")

fig, ax1 = plt.subplots()
ax1.scatter(x=df.Ozone.values, y=df.Wind.values, alpha = 0.3, color="blue")
```

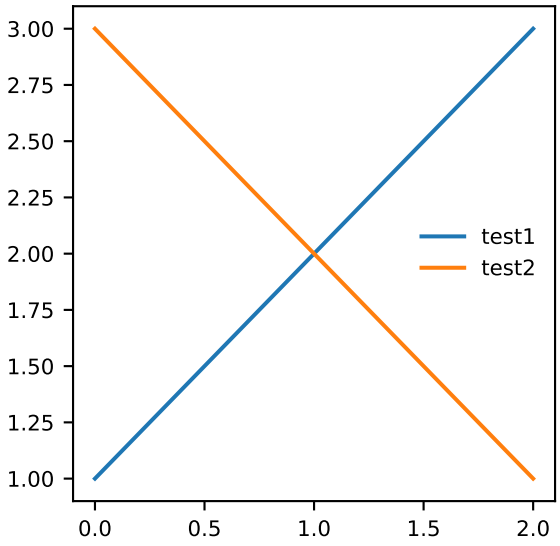


Figura 12.10: Loc1

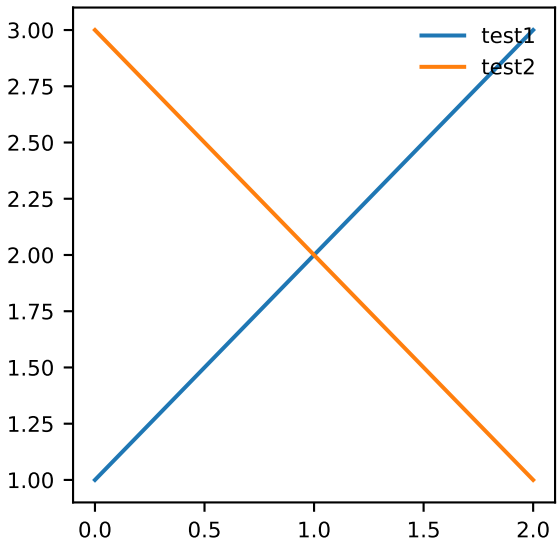


Figura 12.11: Loc2

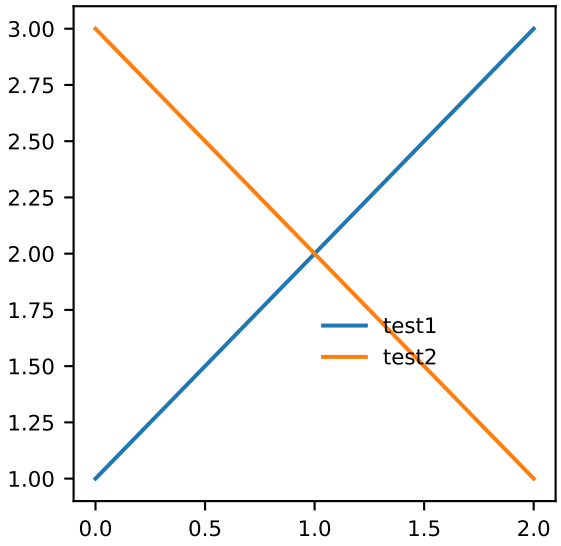


Figura 12.12: Loc3

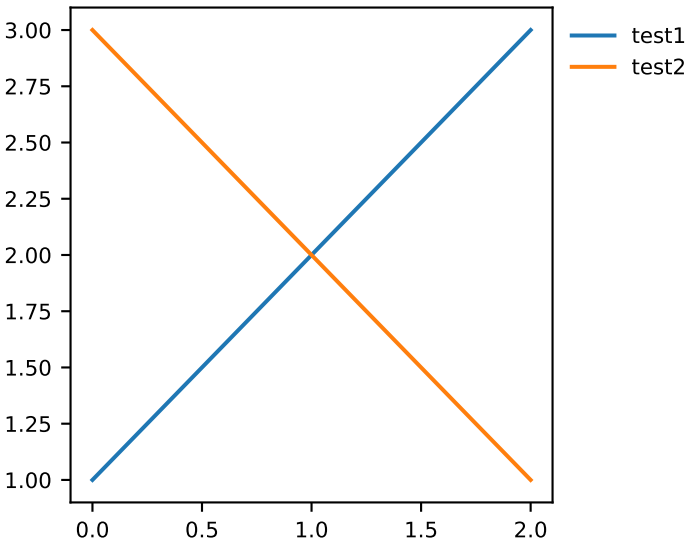


Figura 12.13: Testbboxtoanchor1

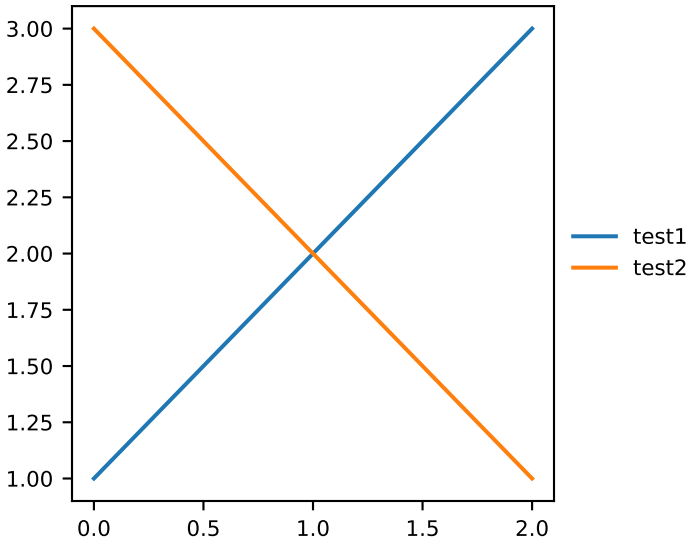


Figura 12.14: Testbboxtoanchor2

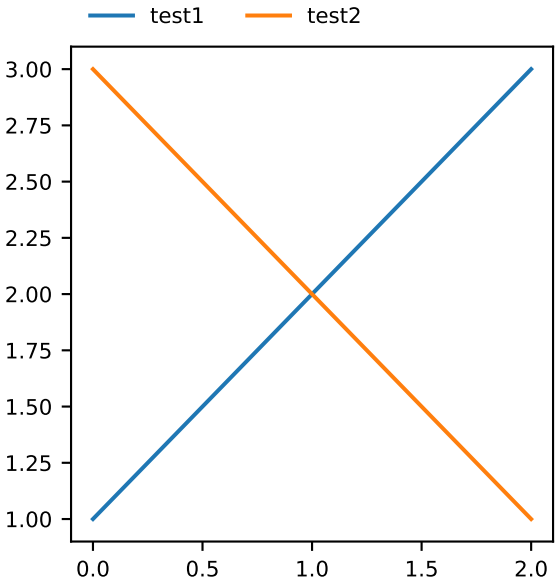


Figura 12.15: Testbboxtoanchor3

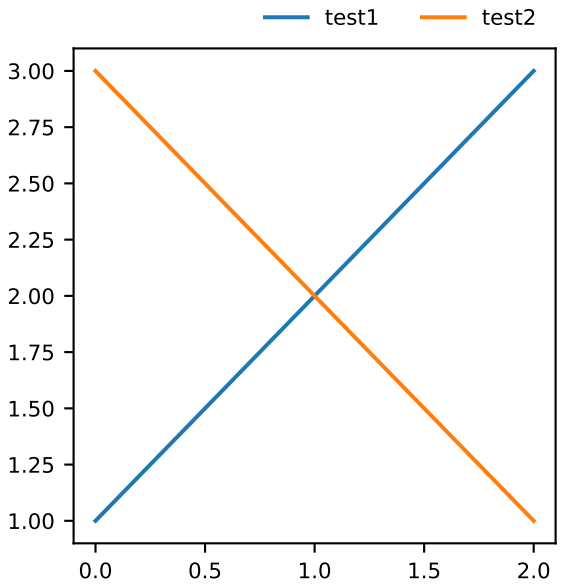


Figura 12.16: Testbboxtoanchor4

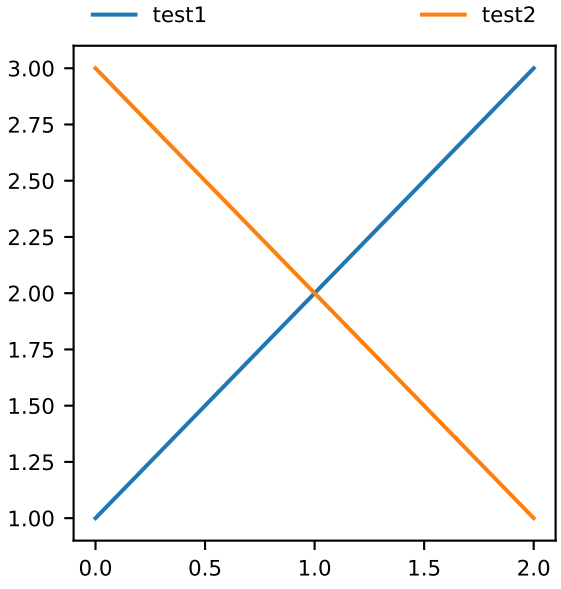


Figura 12.17: Testbboxtoanchor5

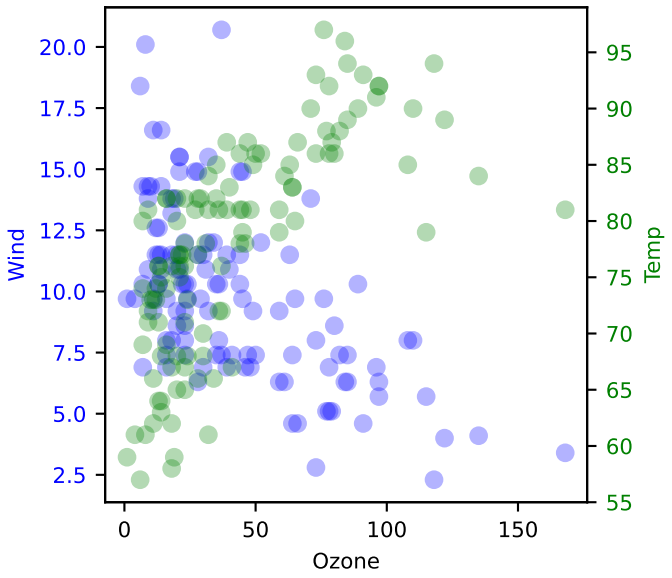


Figura 12.18: Double y axis

```
ax1.set_xlabel("Ozone")
ax1.set_ylabel("Wind", color="blue")
ax1.tick_params(axis="y", labelcolor="blue")

ax2 = ax1.twinx()
ax2.scatter(x=df.Ozone.values, y=df.Temp.values, alpha = 0.3, color="green")
ax2.set_ylabel("Temp", color="green")
ax2.tick_params(axis="y", labelcolor="green")

lb.io.export_figure(fig, label = 'double_y_axis')
```

12.4.6 Padding dei subplots (spazio bianco bordi)

Alternativamente all'automatico `constrained_layout=True` possiamo utilizzare `fig.subplots_adjust` per controllare il bianco verticale/orizzontale tra i bordi della figura e i bordi dell'axis:

- `left`, `top`, `right`, `bottom` regolano la percentuale di bianco esterna al box dell'axis (quindi se vi sono scale compresse agire su questo);
- in caso di subplot multipli `hspace` `wspace` regolano la quantità di spazio tra axis differenti

12.5 Configurazioni

Ogni volta che `matplotlib` si carica legge runtime configuration (rc) che valgono per ciascun plot creato.

12.5.1 Ottenimento e modifica

È possibile:

- listare le configurazioni indagando il dict `plt.rcParams`

  ```
  plt.rcParams # tutte le configurazioni
  plt.rcParams["figure.figsize"] # dimensioni figura
  ```

- modificarle mediante la funzione `plt.rc`

  ```
  # impostare le dimensioni delle figure a 8.5 cm (figsize prende una
  # lista di 2 misure in pollici come input)
  plt.rc("figure", figsize = [8.5/2.54] * 2)
  ```

- salvare le configurazioni in `.config/matplotlib/matplotlibrc`. Ad esempio per salvare l'impostazione come default inserire

  ```
  figure.figsize:    3.346 , 3.346  # figure size in inches
  ```

12.5.2 Ripristino impostazioni default

È possibile ripristinare i valori di default con la funzione `plt.rcdefaults()`.

12.5.3 Cambiare stile

Sono disponibili stili di grafico diversi che servono per avere un array di configurazioni già pronto:

- per listare gli stili disponibili

  ```
  plt.style.available
  ```

- per impostare lo stile dei grafici per tutto il resto della sessione

  ```
  plt.style.use('default')
  ```

- per impostare uno stile temporaneamente (es per una serie di grafici) si può usare un context manager:

  ```
  with plt.style.context('stylename'):
      make_a_plot()
      make_another_plot()
  ```

12.6 Grafici utili

Nel seguito alcuni grafici utili fatti mediante matplotlib o seaborn (che è un wrapper di più alto livello di matplotlib).
Per il plotting si farà preferibilmente uso del metodo `.plot` delle `Series` di `pandas` (che risulta particolarmente integrato con `matplotlib`).
Da ricordare che

- pandas di default plotta *l'indice* della serie/dataframe (indice di riga) sull'asse x e il valore della serie sull'asse y

- A tal proposito, a volte può essere utile trasporre il dataframe di elaborazione per invertire nomi colonne ed indici di riga (accessore `pd.DataFrame.T`) prima di procedere effettivamente a `plot`

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pylbmisc as lb
import seaborn as sns
sns.set_theme()

# dataset
iris = sns.load_dataset('iris')
cd = lb.datasets.load("compact").to_pandas()
political = lb.datasets.load("political").to_pandas()
```

```
>>> import numpy as np
>>> import pandas as pd
>>> import matplotlib.pyplot as plt
>>> import pylbmisc as lb
>>> import seaborn as sns
>>> sns.set_theme()

>>> # dataset
>>> iris = sns.load_dataset('iris')
>>> cd = lb.datasets.load("compact").to_pandas()
>>> political = lb.datasets.load("political").to_pandas()
```

12.6.1 Linee

In figura **??** la durata media dei cd di una collezione per anno di pubblicazione. Se ci fossero state altre variabili oltre a durata (a parte quella di grouping) sarebbe stato plottato anche il loro andamento

```
>>> # durata media cd per anno
>>> cd.head() # dataset
   durata  anno
0     143  1966
1     150  1967
2     241  1994
3     337  1991
4     246  1994
>>> cd.groupby("anno").mean().head() # media annua di durata
      durata
anno
1958  212.00
1960  142.00
1961  150.00
1964  162.50
1965  155.75

# plotting
```

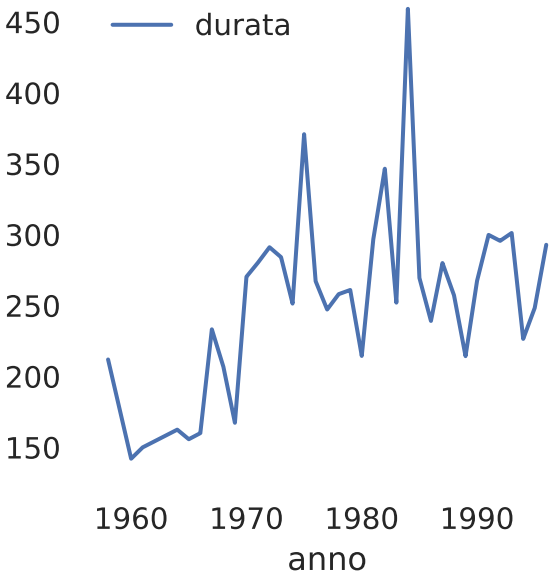


Figura 12.19: Diagramma linee

```
fig, ax = plt.subplots()
cd.groupby("anno").mean().plot.line(ax = ax)
fig = ax.get_figure()
lb.io.export_figure(fig, label="pandas_lines", caption = 'Diagramma linee')
```

12.6.2 Diagramma a barre

Si usi `bar` oppure `barh` per le barre orizzontali

```
>>> political.head()
  party_id  poid
0    3-dem     1
1    3-dem     1
2    3-dem     1
3    3-dem     1
4    3-dem     1
>>> political.party_id.astype("category").value_counts(sort = False)
party_id
1-rep     74
2-ind    111
3-dem     91
Name: count, dtype: int64
```

Per il diagramma figura **??** come farlo con pandas

```
fig, ax = plt.subplots()
political.party_id.astype("category").value_counts(sort = False).plot.bar(ax=ax)
```

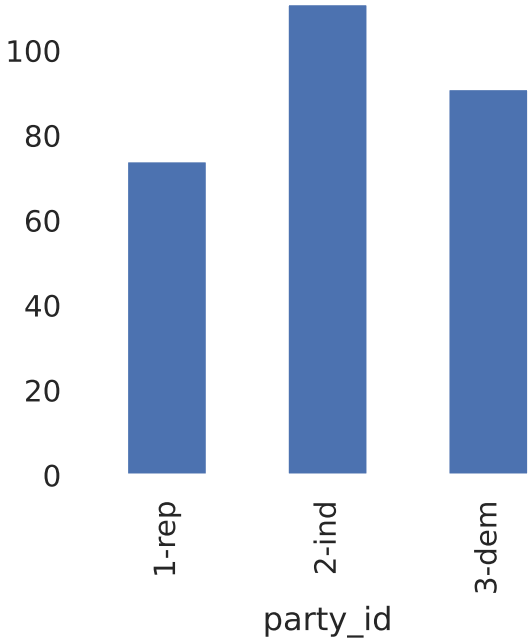


Figura 12.20: Diagramma a barre

```
fig = ax.get_figure()
lb.io.export_figure(fig, label="pandas_barre", caption = 'Diagramma a barre')
```

12.6.3 Istogramma

Utilizzando i metodi di pandas in figura **??**

```
fig, ax = plt.subplots()
ax = iris.sepal_length.plot.hist(density = True, bins = range(9), xlim = [0, 8])
iris.sepal_length.plot.density(ax = ax)
fig = ax.get_figure()
lb.io.export_figure(fig, label="pandas_histo", caption = 'Istogramma (pandas syntax)')
```

12.6.4 Scatterplot

Uno rapido con `pandas` in figura **??** con alpha shading

```
fig, ax = plt.subplots()
iris.plot.scatter(x = 'sepal_length', y = 'sepal_width', alpha = 0.5, ax=ax)
fig = ax.get_figure()
lb.io.export_figure(fig, label="pandas_scatter", caption = 'Scatterplot (pandas)')
# plt.close()
```

Uno più elaborato con colorazione condizionale, alpha shading e fatto con `seaborn`
in figura **??**

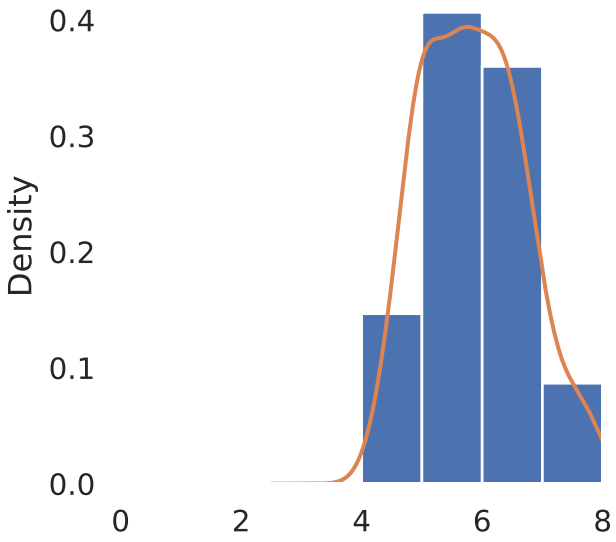


Figura 12.21: Istogramma (pandas syntax)

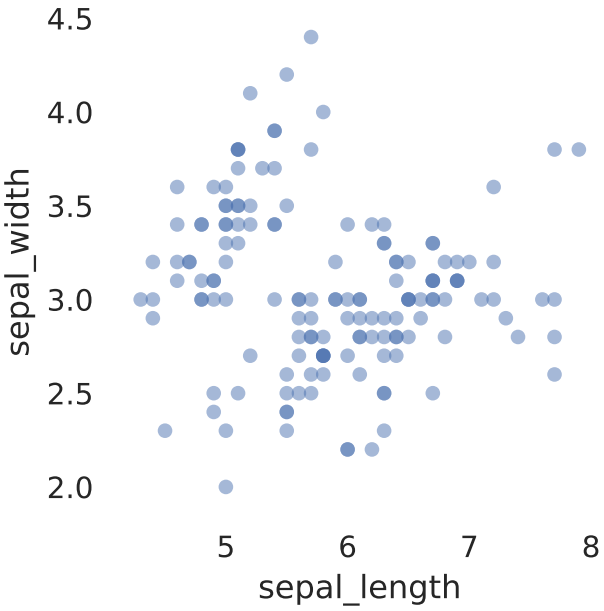


Figura 12.22: Scatterplot (pandas)

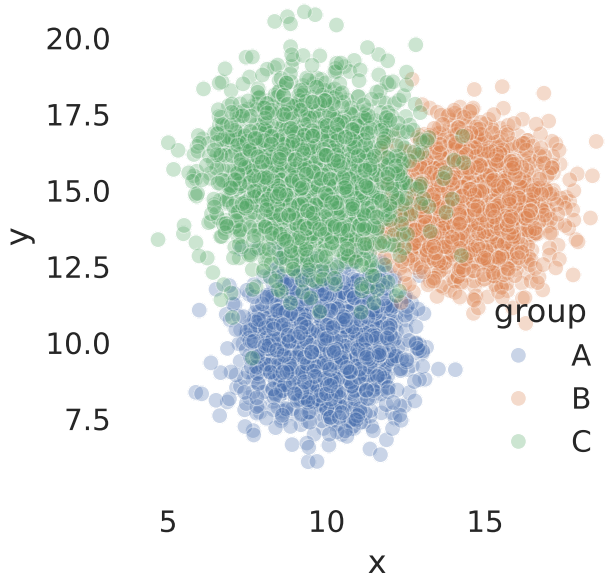


Figura 12.23: Scatterplot

```
# data
group1 = pd.DataFrame({'x': np.random.normal(10, 1.2, 2000),
                       'y': np.random.normal(10, 1.2, 2000),
                       'group': np.repeat('A',2000) })
group2 = pd.DataFrame({'x': np.random.normal(14.5, 1.2, 2000),
                       'y': np.random.normal(14.5, 1.2, 2000),
                       'group': np.repeat('B',2000) })
group3 = pd.DataFrame({'x': np.random.normal(9.5, 1.5, 2000),
                       'y': np.random.normal(15.5, 1.5, 2000),
                       'group': np.repeat('C',2000) })
df = pd.concat([group1, group2, group3])

# Plot
fig, ax = plt.subplots()
ax = sns.scatterplot(x='x', y='y', data=df, hue='group', alpha = 0.30)
# fig = ax.get_figure()
lb.io.export_figure(fig, label="sns_scatter", caption = 'Scatterplot')
```

12.6.5 Matrice di scatterplot

Invece per la matrice di scatterplot ne mettiamo senza con regressione (**??** e con colorazioni **??**.

```
# primo
fig, ax = plt.subplots()
plot = sns.pairplot(iris, kind="reg")
fig = plot.fig
```

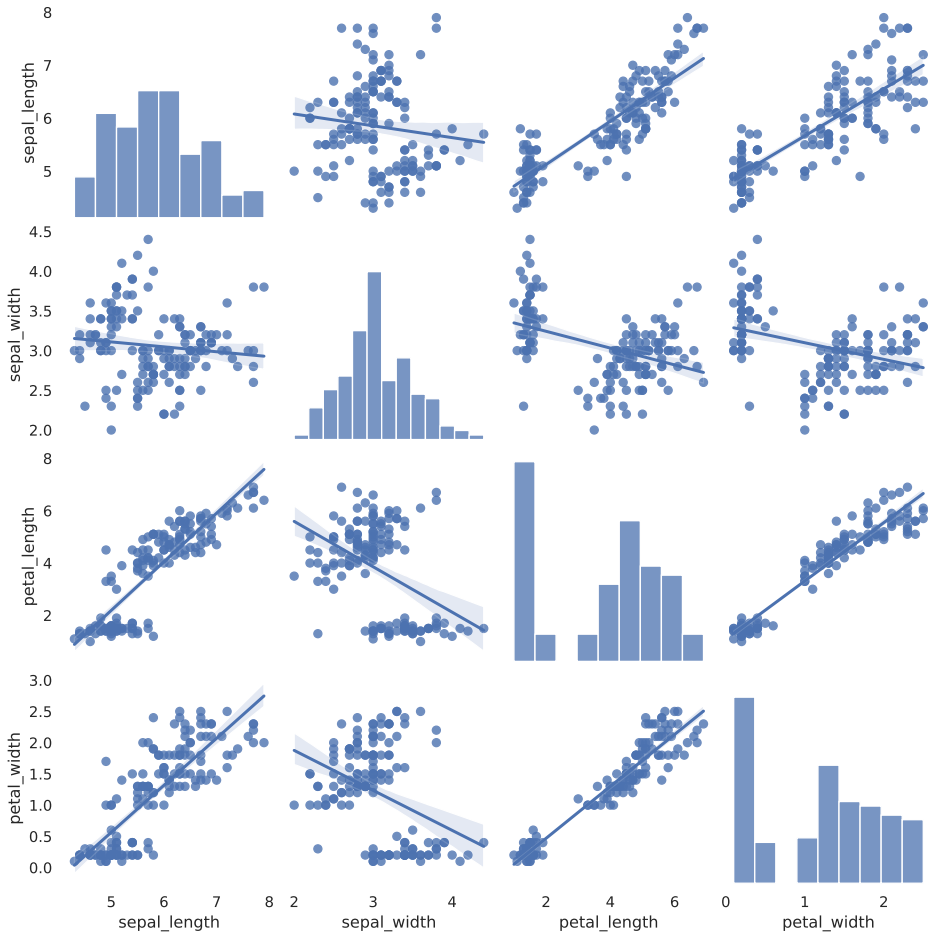


Figura 12.24: Pairplot 1

```
lb.io.export_figure(fig, label="sns_pair1", caption = 'Pairplot 1', scale = 0.5)
# plt.close()


# secondo
fig, ax = plt.subplots()
plot = sns.pairplot(iris, kind="scatter", hue="species", markers=["o", "s", "D"], pal
fig = plot.fig
lb.io.export_figure(fig, label="sns_pair2", caption = 'Pairplot 2', scale = 0.5)
# plt.close()
```

12.6.6 Boxplot

In figura ??

```
fig, ax = plt.subplots()
ax = iris.boxplot(by = "species", column="sepal_length")
```

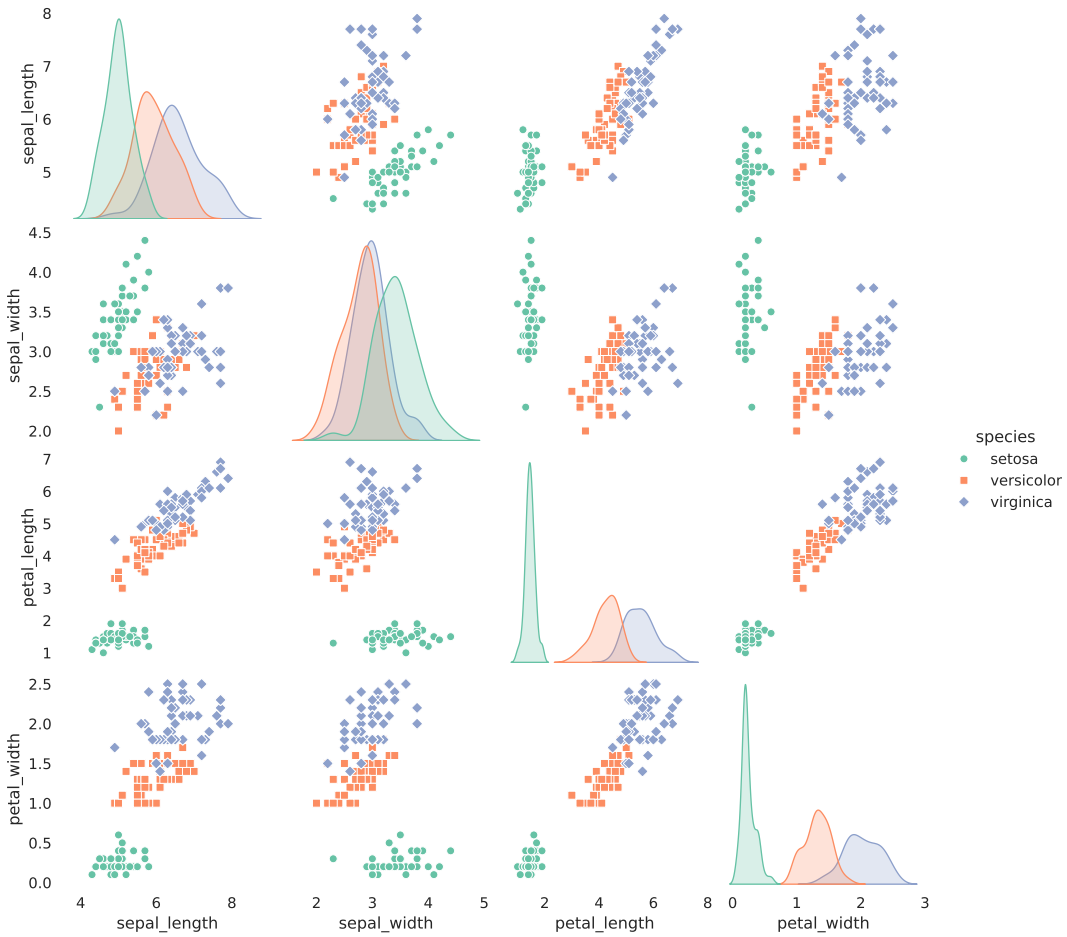


Figura 12.25: Pairplot 2

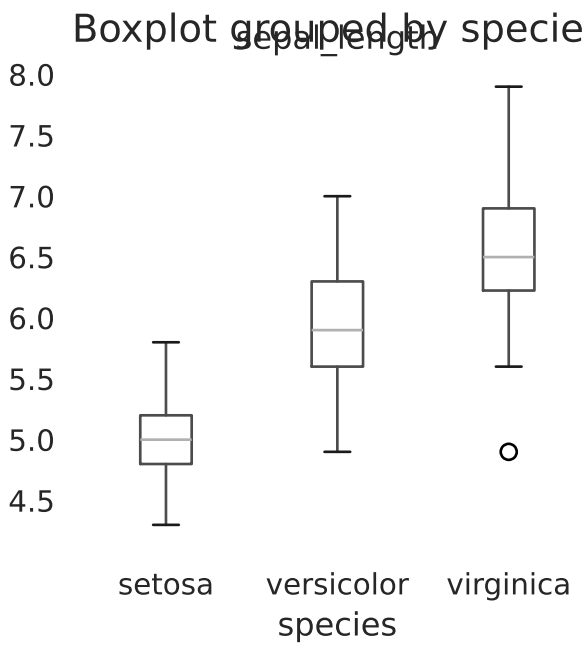


Figura 12.26: Boxplot

```
fig = ax.get_figure()
lb.io.export_figure(fig, label="pandas_boxplot", caption = 'Boxplot')
```

12.6.7 Correlation matrix

In figure **??**

```
corr = iris.select_dtypes("float").corr()

fig, ax = plt.subplots()
ax = sns.heatmap(corr,
                 cmap="RdBu",
                 vmin = -1, vmax = 1,
                 annot = True, fmt=".2f",
                 xticklabels=corr.columns.values, yticklabels=corr.columns.values)
fig = ax.get_figure()
lb.io.export_figure(fig, label="corrmatrix", caption = 'Correlation matrix')

# fig, ax = plt.subplots()
# ax = plt.matshow()
# fig = ax.get_figure()
# lb.io.export_figure(fig, label="corrmatrix", caption = 'Correlation matrix')
```

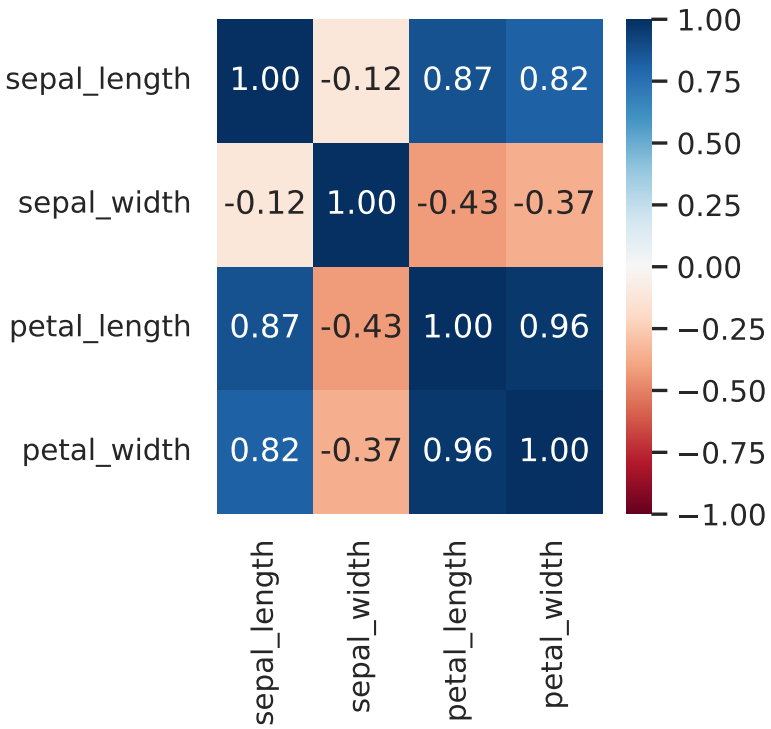


Figura 12.27: Correlation matrix

Parte III

Cookbook

Capitolo 13

Espressioni regolari

```
>>> import re

>>> # search/replace semplice (qui la gente inserisce le espressioni regolari)
>>> re.sub("à", "a", "ààestaaaà")
'aaestaaaa'
>>> re.sub("est", "a", "ààestaaaà")
'ààaaaaà'
>>> re.sub('[^0-9]', '', "sgdf352sc")
'352'
>>> # quando si usano \* meglio porre r per raw string e semplificare la sintassi
>>> re.sub(r"^(\w+) (\w+) (\w+)", r"\2", "asd foo bar")
'foo'

>>> # re.subn fa la stessa cosa ma ritorna una tupla con la stringa sostituita e il
>>> # numero di sostituzioni. Poco interessante for the moment

>>> # re.split splitta sulla base di un pattern, restituendo lista
>>> re.split(r"\d", "asd5foo66bar")
['asd', 'foo', '', 'bar']

>>> # re.findall ritorna tutti i match non overlappanti come lista di stringhe o di
>>> # tuple
>>> re.findall(r'(\w+)=(\d+)', 'set width=20 and height=10')
[('width', '20'), ('height', '10')]
>>> re.findall(r'\bf[a-z]*', 'which foot or hand fell fastest')  # parole inizianti f
['foot', 'fell', 'fastest']

>>> # re.finditer restituisce un iteratore sui match nella stringa (vedere sotto per Match)
>>> for m in re.finditer(r'(\w+)=(\d+)', 'set width=20 and height=10'):
...     print("-----")
...     print(m[0])
...     print(m[1])
...     print(m[2])
...
-----
width=20
```

```
width
20
-----
height=10
height
10
>>> # ------------------------------------------------------------------------
>>> # Ricerca/estrazione elementi
>>> # ------------------------------------------------------------------------

>>> # Scan della stringa e resitituisce un Match. Tre funzioni: match, search,
>>> # fullmatch.

>>> # re.match() cerca un singolo match all'inizio della stringa. Occhio che un
>>> # singolo match può essere composto da più elementi
>>> re.match("c", "abcdef")    # No match
>>> re.match(r"(\w+) (\w+)", "Isaac Newton, physicist")  # Match sino a virgola
<re.Match object; span=(0, 12), match='Isaac Newton'>

>>> # re.search() cerca il primo match ovunque nella the string (default di Perl)
>>> re.search("c", "abcdef")   # Match
<re.Match object; span=(2, 3), match='c'>
>>> re.search("^a", "abcdef")  # Cercare solo all'inizio con search
<re.Match object; span=(0, 1), match='a'>

>>> # re.fullmatch() verifica se l'intera stringa è match
>>> re.fullmatch("p.*n", "python")  # Match
<re.Match object; span=(0, 6), match='python'>
>>> re.fullmatch("t.*n", "python")  # No match

>>> # Il template che si usa a valle per il processing di un match è il seguente
>>> search_match = re.search("c", "abcdefabcdef")
>>> if (search_match):
...     print("Using the match.")
...
Using the match.
>>> # Match.group
>>> # -----------
>>> # restituisce sottopezzi del match: se vi sono molteplici è una tuple
>>> m1 = re.search(r"(\w+) (\w+)", "Isaac Newton, physicist")
>>> if m1:
...     m1.group(0)       # match intero
...     m1.group(1)       # contenuto prima parentesi
...     m1.group(2)       # contenuto seconda parentesi
...     m1.group(1, 2)    # selezionare elementi e porli in tupla
...     m1.group(1, 1)    # idem
...     m1[0]             # è una tupla e si può accedere anche cosi
...
'Isaac Newton'
'Isaac'
'Newton'
```

```
('Isaac', 'Newton')
('Isaac', 'Isaac')
'Isaac Newton'
>>> # è possibile assegnare nomi ai match per rendere più esplicito il codice
>>> m2 = re.search(r"(?P<name>\w+) (?P<surname>\w+)", "Isaac Newton, physicist")
>>> if m2:
...     # utilizziamo
...     print(m2["name"])       # match intero
...     print(m2["surname"])
...
Isaac
Newton

>>> # Match.groups
>>> # ------------
>>> # restituisce una tupla con tutti i sottopezzi sia che abbiamo assegnato nomi
>>> # che no
>>> m1.groups()
('Isaac', 'Newton')
>>> m2.groups()
('Isaac', 'Newton')

>>> # Match.groupdict
>>> # ------------
>>> # restituisce un dict nel caso abbiamo assegnato nomi; ocio che non da errore
>>> # se non assegnamo nomi restituisce vuoto che raffaella carrà scansati
>>> m1.groupdict()  # vuoto!
{}
>>> m2.groupdict()
{'name': 'Isaac', 'surname': 'Newton'}

>>> # Match.expand
>>> # ------------
>>> # permette di usare le sottoporzioni per generare nuove stringhe (simile a
>>> # quanto fatto da sub)
>>> m1.expand(r"\1\1\2\2")  # notare r per raw string se no avremmo dovuto usare \\1
'IsaacIsaacNewtonNewton'

>>> # --------------------------------------------------------------------------
>>> # In produzione
>>> # --------------------------------------------------------------------------
>>> #
>>> # spesso vogliamo utilizzare una stessa regex con molte stringhe. Per farlo in
>>> # maniera efficiente la compiliamo in un istanza di Pattern ed usiamo i suoi
>>> # metodi search, match, fullmatch, findall, sub etc come usate sopra on the fly

>>> np_re = re.compile(r"(\w+) (\w+)")  # occhio a r per raw string
>>> m = np_re.search("Isaac Newton, physicist")
>>> m.groups()
('Isaac', 'Newton')
```

Capitolo 14

Logging

Contents

14.1 Uso del logger di default 123
14.1.1 Setup 123
14.2 Logger custom 124
14.2.1 Setup degli handler 124
14.2.2 Impostare la formattazione di un handler 124
14.2.3 impostare livello di log generale e di un handler . . . 125
14.2.4 Filtrare log 125

Il modulo `logging` permette di utilizzare il logger di default o definirne uno personalizzato avente maggiore flessibilità.
Utilizzare il logger di default è un modo comodo di avere una prima impressione su come il logging funzioni; il contro è che la configurazione può essere non troppo flessibile (si basa su una singola chiamata a `basicConfig` data all'inizio).
Per progetti più complessi occorre maggiore flessibilità e definire logger custom.
Ad ogni modo concetto fondamentale in entrambi è il livello di logging (tab 14.1)

14.1 Uso del logger di default

14.1.1 Setup

Un template per il logging da porre a inizio file è il seguente.

Livello	Funzione (logger default)	spiegazione
`DEBUG`	`logging.debug()`	Informazioni dettagliate per sviluppatori.
`INFO`	`logging.info()`	Info di cosa sta accadendo (buon funzionamento).
`WARNING`	`logging.warning()`	Vi sono cose da controllare
`ERROR`	`logging.error()`	Un problema inatteso si è verificato.
`CRITICAL`	`logging.critical()`	Un problema grave (eg livello crash) si è verificato.

Tabella 14.1: Livelli di logging dal piu basso (`DEBUG`) al più alto (`CRITICAL`)

```
# template per il logging
import logging
logging_level = logging.DEBUG
# logging_fmt = r"[%(levelname)s %(name)s] %(asctime)s - %(message)s"
logging_fmt = r"[{levelname} {name}] {asctime} - {message}"
logging_datefmt = "%Y-%m-%d %H:%M"
logging.basicConfig(level=logging_level, format=logging_fmt, datefmt=logging_datefmt,
```

Dopodiché si può procedere con il logging. Di default:

- se si utilizzano le funzioni di tab 14.1 viene usato il "root/default" logger
- se si inizia a loggare senza avere prima impostato i parametri di log non sarà possibile farlo poi a quanto pare

Per impostare il root logger si chiama `logging.basicConfig`. Un template di setup di logging è quello sopra:

- se non si specifica `level` prima di iniziare a loggare, saranno stampati **solo i messaggi dal warning in su** (no info, no debug)
- il `format` è una stringa in formato printf ma si può impostare style con la graffa aperta per avere una stringa tipo f-string variabile che ad esempio semplifica a "levelname name message" un fomato possibile. Alcune variabili
  - `name` è il nome del logger (eg `root`)
  - `levelname` è il livello di gravita (da `DEBUG` a `ERROR`)
  - `asctime` è l'ora (formato impostabile con es `datefmt`
  - `message` è il messaggio passato in sede di chiamata.

Loggare ad un file Se si vuole evitare di mandare a console aggiungere (alle altre configurazioni di sopra)

```
logging.basicConfig(
    filename="app.log", # filepath
    encoding="utf-8",   # encoding
    filemode="a"        # appendi al file non sovrascrivere (es)
)
```

Loggare valore di variabili (eg a csv) torna comodo la f-string seguente

```
logging.debug(f"{varname=}")
```

Si può anche creare un formato di log che crea un csv. un template potrebbe essere

```
if some_problem:
   # output data to a csv log for external analysis
   logging.debug(f"{surname},{name},{age}}")
```

Catturare stack trace Con `exc_info=True` vengono stampate nel log le informazioni di debug

```
try:
    res = 5/0
except ZeroDivisionError:
    logging.error("ErroreNelCalcolo", exc_info=True)
```

Se si logga da un except handler si puo usare anche `logging.exception("ErroreNelCalcolo")` che risparmia digitazione. Questo è come `logging.error(exc_info=True)`

14.2 Logger custom

Per crearlo good practice è

```
logger = logging.getLogger(__name__)  # good practice to pass __name__ as the name parameter
```

Per configurarlo non si usa una singola chiamata a `basicConfig` ma gli *handlers*. Ciascun logger puo avere *uno o più handler*; nel secondo caso si mandano i log in vari posti (es stodut e a un file)

14.2.1 Setup degli handler

Nell'esempio di sotto impostiamo un output a file ed uno a standard error (contemporaneamente)

```
console_handler = logging.StreamHandler() # logga a console (stderr)
file_handler = logging.FileHandler("app.log", mode="a", encoding="utf-8") # logga a file
logger.addHandler(console_handler) # aggiungi l'handler al logger
logger.addHandler(file_handler)    # aggiungi l'handler al logger
```

Dopo questa configurazione, quando si logga qualcosa es con `logger.warning("Messaggio")` questo verra mandato sia a `stderr` che al file `app.log`.
Il modulo `logging` fornisce molti handler utili per specifiche applicazioni: es `RotatingFileHandler` crea un nuovo file una volta che un limite è raggiunto, o `TimedRotatingFileHandler` crea log per intervalli di tempo predefinito.
Per listare gli handler configurati

```
logger.handlers
```

14.2.2 Impostare la formattazione di un handler

Per impostare una specifica formattazione ad un handler, anche dopo che lo si è aggiunto al logger settargli un formatter come segue:

```
formatter = logging.Formatter(
    "{asctime} - {levelname} - {message}",
    style="{",
    datefmt="%Y-%m-%d %H:%M",
)

console_handler.setFormatter(formatter)
```

Questo impatta solo lo standard error, non l'output su file.
Definendo/settando differenti formatter per i vari handler si può controllare l'ammontare/formato di informazione visualizzata nei log, oppure mostrare messaggi di debug in un file ma mostrare a console solo la roba piu seria (o viceversa).
Es csv su file verboso e messaggio stretto in standard error

14.2.3 impostare livello di log generale e di un handler

Per impostare il livello di log a livello globale (per tutti gli handler)

```
logger.setLevel("INFO")
```

Qui con due handler possiamo avere un log piu verboso (con un livello inferiore) e un altro piu terso.
Si può settare il livello del singolo handler mediante il suo metodo `setLevel`

```
console_handler.setLevel("DEBUG")
```

Tuttavia il **livello generale è importante** perchè settare il livello dell'handler più basso di quello del logger fa si che i messaggi comunque non vengano stampati: il log globale definisce il livello più basso di log che verrà stampato e i singoli handler non posson mostrare log a livello piu basso del logger cui sono attaccati.
Pertanto può essere utile **settare il logger a livello** `DEBUG` e gli handler a livello desiderato (per stderr e file). Un template per l'impostazione segue

```
logger.setLevel("DEBUG") # logger generale permetti tutto
console_handler.setLevel("WARNING") # in standard error ci va la roba seria
file_handler.setLevel("DEBUG") # nel file di debug ci va di ogni
```

14.2.4 Filtrare log

Di default il livello di log è basato sull'idea che, eg, se si è interessati nei warning si è probabilmente interessati anche in error etc. Tuttavia vi pososno esser situazioni dove ha senso gestire messaggi di un dato livello a se/differentemente, e qui entrano in gioco i `Filter`.
Possono costituire un filtro

- una sottoclasse di `logging.Filter` alla quale si è sovrascritto il metodo `.filter` o comunque una classe che contiene un metodo `.filter`.
  In questi casi `.filter()` deve accettare un record di log e restituire un booleano. Dentro al corpo della funzione ha senso definire uno statement `if` che controlla il record di log fornito

- una callable (eg funzione) che si comporta come il metodo `filter` di cui sopra.
  La callable può essere una funzione con un parametro per il log record che l'handler passa al filtro per saper printare o no.
  Questo è il modo piu semplice e lo approfondimao di seguito

Ad esempio il filtro di sotto fa si che l'handler mostri solo messaggi di DEBUG (non anche i superiori). Gli altri setting come quelli dati in precedenza

```
def show_only_debug(record):
    return record.levelname == "DEBUG"


console_handler.addFilter(show_only_debug) # console printa solo il debug
```

Con questo setup qualsiasi log che non sia di livello `DEBUG` ritorna `False` e non sarà mostrato dall'handler cui è attaccato (in questo caso quello dello standard error).
Viceversa `file_handler` si comporta senza essere intaccato da questa modifica

Capitolo 15

Algebra lineare

Contents

15.1 Vettori, matrici e dimensioni 127
15.1.1 Creazione 127
15.1.2 Funzioni utilità per creazione 127
15.2 Operazioni 127
15.2.1 Somma 127
15.2.2 Trasposizione 127
15.2.3 Prodotti 128
15.3 Misc 128

Qui appunti da practical linear algebra for data science; esercizi fatti nella cartella dei test di Python.

```
>>> import numpy as np
```

15.1 Vettori, matrici e dimensioni

Con numpy si possono creare array ad una o due dimensioni; sono gli array a due dimensioni (in cui una è unitaria) ad essere assimilabili ai vettori dell'algebra lineare.

15.1.1 Creazione

Per definire un array a due dimensioni creare una lista che contiene liste delle righe dell'array

```
>>> # array ad una dimensione
>>> a_1d_array = np.array([1, 2, 3])

>>> # array a due dimensioni: vettori
>>> row_vector = np.array([      # ogni lista entro la lista più
...     [1, 2, 3]                # esterna è una riga del vettore
... ])
>>> col_vector = np.array([
```

```
...     [1],                        # ogni lista entro la lista più
...     [2],                        # esterna è una riga del vettore
...     [3]
... ])

>>> # array a due dimensioni matrici
>>> a_matrix = np.array([
...     [1, 2, 3],                      # ogni lista entro la lista più
...     [4, 5, 6],                      # esterna è una riga della matrice
...     [7, 8, 9]
... ])

>>> # non si stampano perché praticamente viene restituito
>>> # il codice inserito paro paro

>>> # differenze in termini di dimensioni e shape
>>> a_1d_array.shape    # 1d
(3,)
>>> row_vector.shape    # 2d di cui la prima è 1
(1, 3)
>>> col_vector.shape    # 2d di cui la seconda è 1
(3, 1)
>>> a_matrix.shape      # 2d con entrambe > 1
(3, 3)

>>> # len applicato a questo di fatto restituisce il numero di elementi per array
>>> # 1d e numero di righe per 2d
>>> len(a_1d_array)    # 1d (n. elementi)
3
>>> len(row_vector)    # 2d numero righe
1
>>> len(col_vector)    # 2d numero righe
3
>>> len(a_matrix)      # 2d numero righe
3
```

15.1.2 Funzioni utilità per creazione

Funzioni di convenienza per la generazione rapida di array:

```
>>> # Creazione rapida di dati
>>> np.arange(0, 20, 2)           # simile a range(), da 0 a 20 a step di 2
array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])
>>> np.arange(1, 10).reshape(3,3) # matrice
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
>>> np.linspace(0, 1, 5)          # interpolazione lineare
array([0.  , 0.25, 0.5 , 0.75, 1.  ])
>>> np.full(5, 2)                 # vettore riempito di 2
array([2, 2, 2, 2, 2])
```

```
>>> np.zeros(10)                  # array di zero
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
>>> np.ones((3, 5))               # array 3x5 di uno
array([[1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.]])
>>> np.eye(3)                     # matrice identita 3x3
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
>>> np.diag([1,2,3])              # diagonale
array([[1, 0, 0],
       [0, 2, 0],
       [0, 0, 3]])

>>> # creazione rapida sfruttando altre shape
>>> np.zeros_like(row_vector)  # array di 0 della stessa shape di x (np.ones_like)
array([[0, 0, 0]])

>>> # Generazione casuale
>>> np.random.normal(0, 1, 5)        # array 1d da normale mu=0, sd=1 di 5 elementi
array([ 7.51692446e-01, -9.45305995e-01,  4.74922399e-01,  1.74298603e-04,
       -9.30333590e-01])
>>> np.random.randint(0, 10, (1, 3)) # vettore riga (1x3) con interi nell'intervallo [0,10 )
array([[3, 9, 5]])
>>> np.random.random((3, 3))         # matrice 3x3 con valori casuali uniformi 0-1
array([[0.6049977 , 0.55339788, 0.21171995],
       [0.0862958 , 0.69179994, 0.14189359],
       [0.15555241, 0.11554883, 0.25903855]])
```

15.2 Operazioni

15.2.1 Somma

Occhio alle somme di vettori, per le quale funziona il broadcasting e spesso portano a risultati di programmazione inattesi

```
>>> a_1d_array + col_vector # dimensioni diverse (risultato inatteso)
array([[2, 3, 4],
       [3, 4, 5],
       [4, 5, 6]])
>>> row_vector + row_vector # dimensioni e shape uguali (risultato atteso)
array([[2, 4, 6]])
>>> row_vector + col_vector # dimensioni uguali, shape diverse (risultato inatteso)
array([[2, 3, 4],
       [3, 4, 5],
       [4, 5, 6]])
```

15.2.2 Trasposizione

Per trasporre si usa l'attributo `T` (più generale) o il metodo `transpose` degli array

```
a_1d_array.T # attenzione: non cambia (e non viene dato errore)
row_vector.T # diventa un col.vector
a_matrix.T   # come da programma
```

15.2.3 Prodotti

Prodotto standard Per il prodotto standard righe per colonne si usa `np.dot`, la chiocciola oppure il metodo `dot` di un array (dando l'altro come argomento):

```
>>> X = a_matrix

>>> np.dot(X.T, X)
array([[ 66,  78,  90],
       [ 78,  93, 108],
       [ 90, 108, 126]])
>>> X.T @ X
array([[ 66,  78,  90],
       [ 78,  93, 108],
       [ 90, 108, 126]])
>>> # anche X.T.dot(X)

>>> # fortunatamente i prodotti inizia ad essere schizzinoso come algebra
>>> # lineare comanda

>>> col_vector @ col_vector # errore: 3x1 non matcha con 3x1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with gufu
>>> row_vector @ row_vector # errore: 1x3 non matcha con 1x3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with gufu
>>> row_vector @ col_vector # ok: 1x3 con 3x1 (dimensioni conformi)
array([[14]])
>>> col_vector @ row_vector # ok: 3x1 con 1x3
array([[1, 2, 3],
       [2, 4, 6],
       [3, 6, 9]])
>>> a_matrix @ col_vector   # ok: 3x3 con 3x1
array([[14],
       [32],
       [50]])
>>> a_matrix @ row_vector   # errore: 3x3 non matcha con 1x3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with gufu
```

Funzione	Descrizione
`np.diag`	Return the diagonal (or off-diagonal) elements of a square matrix as a 1D array, or convert
`np.trace`	Compute the sum of the diagonal elements
`np.tril`	restituisce la matrice triangolare inferiore
`np.triu`	restituisce la matrice triangolare superiore
`np.linalg.norm`	Norma euclidea del vettore (radice della somma dei suoi quadrati)
`np.linalg.det`	Compute the matrix determinant
`np.linalg.eig`	Compute the eigenvalues and eigenvectors of a square matrix
`np.linalg.inv`	Compute the inverse of a square matrix
`np.linalg.pinv`	Compute the Moore-Penrose pseudo-inverse inverse of a square matrix
`np.linalg.qr`	Compute the QR decomposition
`np.linalg.svd`	Compute the singular value decomposition (SVD)
`np.linalg.solve`	Solve the linear system $\mathbf{Ax} = \mathbf{b}$ for $\mathbf{x}$, where $\mathbf{A}$ is a square matrix
`np.linalg.lstsq`	Compute the least-squares solution to $\mathbf{y} = \mathbf{Xb}$

Tabella 15.1: Funzioni per algebra lineare

```
>>> # attenzione che dot invece non lo è troppo
>>> np.dot(row_vector, col_vector) # expected
array([[14]])
>>> np.dot(col_vector, row_vector) # unexpected
array([[1, 2, 3],
       [2, 4, 6],
       [3, 6, 9]])
```

Prodotto di Hadamard È il prodotto elemento per elemento e si fa (su array idonei) mediante l'asterisco

```
>>> row_vector * row_vector  # ok questo è il prodotto di Hadamard
array([[1, 4, 9]])
>>> row_vector *  # no qui usa il broadcasting
  File "<stdin>", line 1
    row_vector *  # no qui usa il broadcasting
SyntaxError: invalid syntax
```

15.3 Misc

Funzioni utili riportate in tabella 15.1

Capitolo 16

Probabilità e simulazione

Contents

16.1 Funzionalità native 129
16.1.1 Combinatoria 129
16.1.2 Numeri casuali (`random`) 129
16.2 Numeri casuali (`numpy`) 129
16.2.1 Creazione del generatore 130
16.3 Variabili casuali (`scipy.stats`) 130
16.3.1 Funzioni e parametri principali 130
16.3.2 Uso interattivo rapido 132
16.3.3 Freezing di una distribuzione 132
16.3.4 Uso del generatore di `numpy` 132
16.4 Altri argomenti 132
16.4.1 Bootstrap CI 132
16.4.2 Esecuzione parallela 132

Importiamo le librerie qui usate

```
>>> import itertools
>>> import math
>>> import numpy as np
>>> import pandas as pd
>>> import scipy
>>> from scipy import stats
```

16.1 Funzionalità native

16.1.1 Combinatoria

Fattoriale

```
>>> math.factorial(4)
24
```

Coefficiente binomiale

```
>>> n = 4
>>> k = 2
>>> math.comb(n, k)
6
```

Permutazioni di un vettore

```
>>> data = [1,2,3]
>>> list(itertools.permutations(data))
[(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1)]
```

Combinazioni di elementi di un vettore

```
>>> list(itertools.combinations(data, 2))
[(1, 2), (1, 3), (2, 3)]
```

Prodotto cartesiano

```
>>> a = [1,2,3]
>>> b = [4,5,6]
>>> list(itertools.product(a,b)) # eventualmente posto in np.array
[(1, 4), (1, 5), (1, 6), (2, 4), (2, 5), (2, 6), (3, 4), (3, 5), (3, 6)]
```

16.1.2 Numeri casuali (`random`)

Per inizializzare il seed

```
>>> import random

>>> random.seed()      # se omesso viene utilizzato il system time (risultati cambiano
>>> random.seed(1983) # se specificato viene utilizzato, assicurando reproducibilita
```

Alcune funzionalità del modulo a seguire:

- per l'estrazione di interi

  ```
  >>> # intero tra min e max
  >>> d10 = random.randint(1, 10)
  >>> print(d10)
  2

  >>> # intero casuale da un range
  >>> random.randrange(1, 9, 2) # intero casuale da range(1,9,2) = [1,3,5,7]
  1
  ```

- per numeri casuali semplici

  ```
  >>> # uniforme 0-1
  >>> random.random()
  0.17028345753869512
  ```

- per estrazioni da urne o simili

```
>>> import string
>>> urn = list(string.ascii_lowercase[:10]) # una lista con prime 10 lettere

>>> # un elemento dall'urna
>>> random.choice(urn)
'h'

>>> # k elementi dall'urna, presi CON REIMMISSIONE (eventualmente con pesi)
>>> random.choices(urn, k=10)
['g', 'f', 'f', 'j', 'h', 'e', 'f', 'f', 'g', 'd']

>>> # k elementi SENZA REIMMISSIONE
>>> random.sample(urn, k=8)
['j', 'c', 'i', 'h', 'b', 'e', 'f', 'd']

>>> # shuffle inplace (modifica il dato di partenza)
>>> random.shuffle(urn)
>>> urn
['a', 'f', 'j', 'b', 'd', 'i', 'h', 'c', 'g', 'e']
```

16.2 Numeri casuali (numpy)

Per ottenerli si crea innanzitutto un generatore di numeri casuali (impostando seme e tipologia, volendo) dopodiché si utilizza questo per generare dati.

16.2.1 Creazione del generatore

Alternativamente si può

- utilizzando in **generatore di default**; il tutto è semplificato mediante `default_rng`

```
>>> rng = np.random.default_rng(seed = 6315744)
>>> rv = rng.standard_normal(5) # random vector
>>> rv
array([ 0.02015758,  2.22358201,  0.94127564, -0.75983371, -2.37769907])
>>> rm = rng.standard_normal((2, 3)) # random matrix
>>> rm
array([[ 0.25145986,  0.10362785, -1.26726019],
       [ 0.46240554, -0.14667618, -0.6394399 ]])
```

- **specificare il generatore**: occorre creare un generatore, alimentarlo con un bitgenerator (eg default o Mersenne-Twister) a cui si è fornito il seed impostato: ad esempio per un generatore standard e Mersenne-Twister (utilizzato da R) fatti a mano

```
>>> from numpy.random import Generator, PCG64, MT19937
>>> seed = 2154
>>> std = Generator(PCG64(seed = seed)) # equivalente al default
>>> std.standard_normal(5)
array([ 2.65496506, -0.20262369, -0.85302693, -0.37646063, -0.30786317])
```

Metodo	Descrizione
`integers`	Pesca interi in range
`choice`	Pesca casualmente da un array
`permutation`	Permutazione casuale di una array, restituendo una copia
`shuffle`	Permutazione in place
`random`	Estrazione da uniforme 0-1
`uniform`	Estrazione da uniform con min e max da specificare
`beta`	Estrazione da beta distribution
`binomial`	Estrazioni da binomiale
`multinomial`	Estrazioni da multinomiale
`standard_normal`	Estrazioni da una normale 0, 1
`normal`	Estrazioni da normal (con media/sd da specificare)
`multivariate_normal`	Estrazioni da una normale multivariata
tante altre ...	Vedere documentazione di `numpy.random`

Tabella 16.1: Alcuni metodi per la generazione di numeri casuali in `numpy`

```
>>> mt =  Generator(MT19937(seed = seed))
>>> mt.standard_normal(5)
array([ 0.73327039,  1.19398753, -0.66427529, -0.42821724,  0.7421379 ])
```

Metodi per generatori casuali Una volta creato il generatore, alcuni metodi di interesse per estrarre numeri casuali (o effettuare operazioni con casualità) sono in tabella 16.1 (vedere doc).

```
>>> rng = np.random.default_rng(seed = 6315744)

>>> # interi tra min e max
>>> rng.integers(5, size=20) # 20 estrazioni da 0 a 4 (5 escluso)
array([1, 1, 0, 2, 2, 4, 4, 1, 2, 4, 4, 2, 0, 2, 2, 0, 1, 3, 0, 2])
>>> rng.integers(low = 1, high=10, size = 20, endpoint=True) # endpoint per max inclu
array([ 1,  7,  2,  1,  3,  2,  8,  3,  1,  1,  3,  4,  4,  4,  1,  4,  7,
        4, 10,  2])

>>> # pescare da un array
>>> data = np.arange(10)
>>> rng.choice(data, size=10, replace=True)
array([5, 2, 8, 9, 7, 6, 2, 1, 9, 1])

>>> # permutazioni
>>> rng.permutation(data)  # copia
array([2, 4, 6, 9, 7, 5, 3, 1, 8, 0])
>>> rng.shuffle(data)      # modifica dell'array inplace
>>> data
array([9, 5, 7, 4, 6, 1, 2, 3, 8, 0])

>>> #
```

Famiglia	Funzione	Famiglia	Funzione
Bernoulli	`bernoulli`	Uniforme cont.	`uniform`
Binomiale	`binom`	Esponenziale	`expon`
Geometrica	`geom`	Normale	`norm`
Binomiale neg.	`nbinom`	Gamma	`gamma`
Ipergeometrica	`hypergeom`	Chi-quadrato	`??`
Poisson	`poisson`	Beta	`beta`
Uniforme disc.	`randint`	T di Student	`t`
		F	`??`
		Logistica	`??`
		Lognormale	`??`
		Weibull	`??`
		Pareto	`??`

Tabella 16.2: Funzioni `scipy.stats.*` per variabili casuali in Python

Metodo	Descrizione
`rvs`	generazione di numeri casuali; equivalente di `r*` di R
`pdf`, `pmf`	probability density e mass function; equivalente di `d*` di R
`cdf`	cumulative distribution function; equivalente di `p*` di R
`ppf`	percent point function (inversa di cdf); equivalente di `q*` di R
`sf`	Survival Function (1-CDF)
`isf`	Inverse Survival Function (Inverse of SF)
`stats`	media, varianza, (Fisher's) skew, or (Fisher's) kurtosis
`moment`	non-central moments of the distribution

Tabella 16.3: Metodi variabili casuali `scipy`

16.3 Variabili casuali (`scipy.stats`)

Le funzioni di `numpy.random` servono esclusivamente per la generazione di numeri casuali e applicazioni MonteCarlo-like; se si vuole disporre di più funzioni per quanto riguarda le variabili casuali (paragonabili a quelle di R) occorre utilizzare `scipy.stats`

16.3.1 Funzioni e parametri principali

In `scipy.stats` le variabili casuali di maggiore interesse sono riportate in tabella 16.2, i relativi metodi sono riportati in tabella 16.3.
I parametri di maggior interesse sono `loc` (locazione) e `scale` (dispersione); alcune quantitative anche un parametro `shape`. Nel caso della normale `loc` è la media ed è impostata di default a 0, `scale` la deviazione standard ed è impostata a 1.

16.3.2 Uso interattivo rapido

Un esempio di utilizzo interattivo rapido con normale a seguire. Per facilitare, le funzioni supportano il broadcasting

```
>>> stats.norm.rvs(size = 5) # estrazioni di casuali (come rnorm(5))
```

```
array([ 0.34973498, -1.28953405,  1.08863572,  1.06879924, -1.01351367])
>>> stats.norm.pdf(0.5)      # densità: come dnorm(0.5)
np.float64(0.3520653267642995)
>>> stats.norm.cdf(1.96)     # cdf: come pnorm(1.96)
np.float64(0.9750021048517795)
>>> stats.norm.ppf(0.5)      # quantile: come qnorm(0.5)
np.float64(0.0)

>>> # esempio con broadcasting
>>> stats.norm.pdf([0.025, 0.5, 0.975])
array([0.39881763, 0.35206533, 0.24801872])
```

16.3.3 Freezing di una distribuzione

Spesso capita di dover lavorare più volte con distribuzioni dai parametri differenti. Onde evitare di dover reinserire i parametri in ogni chiamata possiamo effettuare il freezing di una distribuzione

```
>>> n01 = stats.norm(loc = 0, scale = 1) # mu=0, sd=1 sono i valori di default
>>> n01.rvs(size = 5)
array([ 0.27895081, -0.56027842,  0.20292597,  0.06395613,  0.09064694])

>>> n25 = stats.norm(loc = 2, scale = 5) # mu=2, sd=5
>>> n25.rvs(size = 5)
array([ 1.22064367,  1.99699263, -9.10480435,  1.45113468, -3.7038789 ])
```

16.3.4 Uso del generatore di `numpy`

Passiamo il generatore di numeri casuali in `numpy` nel metodo della funzione freezata

```
>>> rng = np.random.default_rng(302132)

>>> n01 = stats.norm()                    # esempio d'uso:
>>> n01.rvs(size = 5, random_state = rng) # usato al momento
array([-0.14191574,  0.42922204, -0.33735776,  0.20712603, -0.9709426 ])
>>> n01.rvs(size = 5, random_state = rng) # dell'estrazione
array([ 0.64705971, -1.05564341,  0.94984921, -0.30577826, -0.61788831])

>>> rng2 = np.random.default_rng(302132)   # riproduzione sequenza
>>> n01.rvs(size = 5, random_state = rng2)
array([-0.14191574,  0.42922204, -0.33735776,  0.20712603, -0.9709426 ])
>>> n01.rvs(size = 5, random_state = rng2)
array([ 0.64705971, -1.05564341,  0.94984921, -0.30577826, -0.61788831])
```

16.4 Altri argomenti

16.4.1 Bootstrap CI

```
>>> from scipy.stats import bootstrap
```

```
>>> rng = np.random.default_rng(6235)
>>> data_gen = stats.norm()
>>> data = data_gen.rvs(100, random_state = rng)

>>> # 95 bca CI of median
>>> res = bootstrap(data = [data],         # ocio qui: "sequence of array-like"
...                 statistic = np.median,
...                 n_resamples = 10000,
...                 random_state = rng)

>>> res.confidence_interval      # ci
ConfidenceInterval(low=np.float64(-0.21318771294834601), high=np.float64(0.17332161369498447))
>>> res.confidence_interval.low  # ci
np.float64(-0.21318771294834601)
>>> res.confidence_interval.high # ci
np.float64(0.17332161369498447)
>>> res.bootstrap_distribution # valori delle stats nei resample
array([-0.08377   ,  0.09051194,  0.08447688, ..., -0.17939695,
       -0.13573812,  0.16842878], shape=(10000,))
```

16.4.2 Esecuzione parallela

Usiamo `multiprocessing`, che effettua una parallizzazione tipo quella di R a livello di processo e `Pool` che applica una funzione in maniera parallela cambiando il parametro di input.

```
from multiprocessing import Pool

import os
import numpy as np
import time

# a function with no particular setting (note how we don't have to
# define the imports which is automagically handled by the fork

def parallelized(s):
    """A test function which extract 500 random number from standard
    normal and calculates the mean.
    """
    rng = np.random.default_rng(seed = s)
    rv = rng.standard_normal(500) # random vector
    return {"pid" : os.getpid(),
            "seed": s,
            "mean" : rv.mean()}

def main():
    start_t = time.perf_counter()
    # Pool can be used for parallel execution of a function across
    # multiple input values, distributing the input data across processes
    # data parallelism).
    rng_seeds = [1, 1, 3]
```

```
    with Pool() as p:  # spawna 1 processo per core (usandoli tutti)
        # apply a function to different input, each in one separate process
        res = p.map(parallelized, rng_seeds)
    stop_t = time.perf_counter()

    os.getpid() # pid of the current process
    print(res)  # pid of working process are different
                # results are the same in the first two cases (same seed)
    print(f"Time elapsed {stop_t - start_t:.2f}")


main()
```

Per

- limitare il numero di processi da spawnare ad esempio fare `Pool(5)`
- i tre metodi principali che pool fornisce sono `map`, `imap` e `imap_unordered`

Capitolo 17

Test statistici

Contents

17.1 Setup 133
17.2 Medie 133
17.3 Proporzioni 133
17.3.1 McNemar 133
17.3.2 Q di Cochrane 133
17.4 Tassi 133
17.4.1 Comparazione 2 tassi 133
17.5 Varianze 133
17.5.1 Test di Bartlett 134
17.5.2 Test di Levene 134
17.5.3 Test di Fligner 134
17.6 Sopravvivenza 134
17.6.1 Logrank test 134
17.7 Agreement 134
17.7.1 Cohen's K 134
17.7.2 Fleiss K 134
17.7.3 Lin coefficient 134
17.8 Reliability/consistency 134
17.8.1 Cronbach α 134
17.8.2 ICC 134
17.9 Multiplicity 135
17.10Test simulativi 135

Sotto le cose ancora da implementare in lb.stats.tests

17.1 Setup

Importiamo le librerie qui usate

```
>>> import numpy as np
>>> import pandas as pd
>>> import pingouin as pg
>>> from scipy import stats
```

17.2 Medie

17.3 Proporzioni

17.3.1 McNemar

```
>>> data = pg.read_dataset('chi2_mcnemar')
>>> pg.chi2_mcnemar(data, 'treatment_X', 'treatment_Y')
(treatment_Y   0   1
treatment_X
0             20  40
1              8  12,                chi2  dof  p-approx   p-exact
mcnemar  20.020833    1  0.000008  0.000003)
```

17.3.2 Q di Cochrane

Mc nemar per più tempi/trattamenti su stessi soggetti

```
>>> df = pg.read_dataset('cochran')
>>> df.head()
   Subject    Time  Energetic
0        1  Monday          1
1        2  Monday          0
2        3  Monday          0
3        4  Monday          0
4        5  Monday          1
>>> df_wide = df.pivot_table(index="Subject", columns="Time", values="Energetic")
>>> pg.cochran(df_wide)
         Source  dof         Q     p-unc
cochran  Within    2  6.705882  0.034981
```

17.4 Tassi

17.4.1 Comparazione 2 tassi

Il test di poisson di python verifica che la differenza tra tassi sia nulla (quello di R che il rapporto sia unitario)

```
>>> # poisson.test(c(11, 6+8+7), c(800, 1083+1050+878))
>>> stats.poisson_means_test(11, 800, 6+8+7, 1083+1050+878)
SignificanceResult(statistic=np.float64(1.5342150126346437), pvalue=np.float64(0.1386
>>> # i risultati sono diversi ma il manuale di python dice
```

I risultati di questo test sono differenti da quelli di R ma la documentazione di python dice che ha maggior potenza del test poissoniano esatto di R.

17.5 Varianze

Vediamo le funzioni per la comparazione di k varianze sotto diverse ipotesi sempre meno restrittive

17.5.1 Test di Bartlett

Testa parametricamente la differenza di varianze ipotizzando una distribuzione normale del carattere nella popolazione. Se a 2 gruppi è il test F.

```
>>> a = [8.88, 9.12, 9.04, 8.98, 9.00, 9.08, 9.01, 8.85, 9.06, 8.99]
>>> b = [8.88, 8.95, 9.29, 9.44, 9.15, 9.58, 8.36, 9.18, 8.67, 9.05]
>>> c = [8.95, 9.12, 8.95, 8.85, 9.03, 8.84, 9.07, 8.98, 8.86, 8.98]
>>> stats.bartlett(a, b, c)
BartlettResult(statistic=np.float64(22.789434813726768), pvalue=np.float64(1.1254782518834628e-
```

17.5.2 Test di Levene

Testa parametricamente la differenza di varianze non ipotizzando distribuzioni normali

```
>>> stats.levene(a, b, c)
LeveneResult(statistic=np.float64(7.584952754501659), pvalue=np.float64(0.002431505967249681))
```

17.5.3 Test di Fligner

Equivalente non parametrico

```
>>> stats.fligner(a, b, c)
FlignerResult(statistic=np.float64(10.803687663522238), pvalue=np.float64(0.00450826080004775))
```

17.6 Sopravvivenza

Utilizziamo la libreria `lifelines`

17.6.1 Logrank test

```
>>> T1 = [1, 4, 10, 12, 12, 3, 5.4]
>>> E1 = [1, 0, 1,  0,  1,  1, 1]
>>> T2 = [4, 5, 7, 11, 14, 20, 8, 8]
>>> E2 = [1, 1, 1, 1,  1,  1,  1, 1]

>>> from lifelines.statistics import logrank_test
>>> results = logrank_test(T1, T2, event_observed_A=E1, event_observed_B=E2)
>>> results.print_summary()
<lifelines.StatisticalResult: logrank_test>
               t_0 = -1
 null_distribution = chi squared
degrees_of_freedom = 1
         test_name = logrank_test

---
 test_statistic    p  -log2(p)
           0.09 0.77      0.38
>>> # results.p_value, results.test_statistic
```

17.7 Agreement

17.7.1 Cohen's K

```
>>> from statsmodels.stats.inter_rater import cohens_kappa
>>> cohens_kappa
<function cohens_kappa at 0x7f30c731c400>
```

17.7.2 Fleiss K

```
>>> from statsmodels.stats.inter_rater import fleiss_kappa
>>> fleiss_kappa
<function fleiss_kappa at 0x7f30c731c360>
```

17.7.3 Lin coefficient

```
>>> from statsmodels.stats.inter_rater import fleiss_kappa
>>> fleiss_kappa
<function fleiss_kappa at 0x7f30c731c360>
```

17.8 Reliability/consistency

17.8.1 Cronbach α

```
>>> pg.cronbach_alpha
<function cronbach_alpha at 0x7f30c792dd00>
```

17.8.2 ICC

```
>>> pg.intraclass_corr
<function intraclass_corr at 0x7f30c792dda0>
```

17.9 Multiplicity

```
scipy.stats.tukey_hsd
scikit_posthocs.posthoc_dunn


statsmodels.stats.multitest.multipletests
scipy.stats.false_discovery_control

pg.multicomp
pg.pairwise_gameshowell
pg.pairwise_tukey
pg.pairwise_tests
pg.pairwise_corr
pg.ptests
```

17.10 Test simulativi

Importiamo le librerie qui usate

```
>>> from scipy import stats

>>> stats.bootstrap
<function bootstrap at 0x7f31284072e0>
>>> stats.permutation_test
<function permutation_test at 0x7f3128428400>
>>> stats.monte_carlo_test
<function monte_carlo_test at 0x7f3128407560>
```

Capitolo 18

Misc cookbook

Contents

18.1 Tabella trial 137
18.2 Calendario 137
18.3 Tcl/Tk 137
18.4 Telegram 137

18.1 Tabella trial

Utilizzare la libreria `tableone`.

```
tab1_df = tableone.load_dataset('pn2012')
tab1_df.info()
tab1_df.head()
ft = {0: "alive", 1: "dead"}
tab1_df["group"] = pd.Categorical(tab1_df.death.map(ft))
select = ['Age', 'SysABP', 'Height', 'Weight', 'ICU', 'group']
categ = ['ICU', 'group']
groupby = 'group'
nonnormal = ['Age']
labels={'death': 'mortality'}
tab1 = tableone.TableOne(tab1_df,
                         columns=select,
                         categorical=categ,
                         groupby=groupby,
                         nonnormal=nonnormal,
                         rename=labels, #renaming variabili di riga
                         missing=True, # conta dei missing
                         include_null=False, # evita i missing nelle var categoriche
                         pval=False)

tab1
```

18.2 Calendario

`calendar` fornisce funzionalità del calendario (utile es per trovare l'x-esimo martedì del mese tal dei tali).

```
>>> import calendar
>>> import pprint
>>> from datetime import date

>>> # Stampare il calendario del mese scelto (es del mese corrente)
>>> oggi = date.today()
>>> c = calendar.TextCalendar()
>>> c.prmonth(oggi.year, oggi.month)
   November 2026
Mo Tu We Th Fr Sa Su
                   1
 2  3  4  5  6  7  8
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28 29
30

>>> # ottenere numeri di giorni del mese come lista di liste (ogni lista è una settim
>>> m = calendar.monthcalendar(oggi.year, oggi.month)
>>> pprint.pprint(m)
[[0, 0, 0, 0, 0, 0, 1],
 [2, 3, 4, 5, 6, 7, 8],
 [9, 10, 11, 12, 13, 14, 15],
 [16, 17, 18, 19, 20, 21, 22],
 [23, 24, 25, 26, 27, 28, 29],
 [30, 0, 0, 0, 0, 0, 0]]
>>> # la struttura di dati va per ogni settimana da lunedì a domenica
>>> # 0 vuol dire che il giorno non appartiene al mese, altrimenti il numero è
>>> # il progressi

>>> # stampa di tutti i sabato e domenica di un mese con numero di giorno
>>> for week in m:
...     sab = week[calendar.SATURDAY]
...     dom = week[calendar.SUNDAY]
...     if sab: # se è diverso da 0 vi è un sabato in quella settimana
...         print("sab", sab)
...     if dom: # questo dovrebbe essere sempre vero
...         print("dom", dom)
...
dom 1
sab 7
dom 8
sab 14
dom 15
sab 21
dom 22
```

```
sab 28
dom 29
```

18.3 Tcl/Tk

```
from pathlib import Path
from tkinter.filedialog import askopenfilename

title = "Select the study protocol file to be imported"
initialdir = "/tmp"
filetypes = [("Formati", ".docx .doc .pdf")]
fpath = Path(
            askopenfilename(title=title, initialdir=initialdir, filetypes=filetypes)
)
```

18.4 Telegram

Un esempio di invio di messaggi di monitoraggio ai termini di un task (backup fatto mediante `rsync`)

```
import asyncio
import datetime as dt
import os
import telegram
from pylbmisc.tg import bot_token, user_id, group_id

# telegram message
async def send_message(start, end):
    diff = end - start
    msg ="""Backup completato. \n Inizio: {0} \n Termine: {1} \n Impiegati (min): {2}: """.form
        start.strftime("%d/%m/%Y - %H:%M:%S"),
        end.strftime("%d/%m/%Y - %H:%M:%S"),
        diff.total_seconds()/60
    )
    bot = telegram.Bot(bot_token("winston_lb_bot"))
    async with bot:
        await bot.send_message(text=msg, chat_id=user_id("lucailgarb"))


if __name__ == '__main__':
    os.system("mount usb_backup")
    start = dt.datetime.now()
    os.system("rsync -avru -L --delete doc_ricordi/ usb_backup/doc_ricordi/")
    os.system("umount usb_backup")
    end = dt.datetime.now()
    asyncio.run(send_message(start = start, end = end))
```